Cours Master/Ingénierie

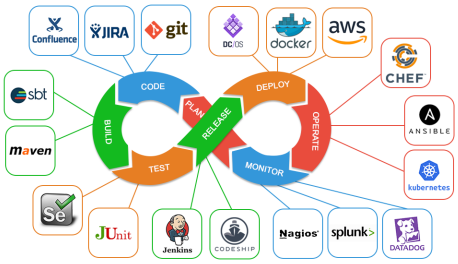
Cours de DevOps

De la Théorie à la Pratique

Projet fil rouge :

Application de Gestion de Vente de Bijoux

Spring Boot + PostgreSQL



Dr. BADR EL KHALYLY

Année universitaire 2025–2026

Préface

Ce cours sur DevOps représente le fruit d'un travail de recherche approfondi dans le cadre de **l'état de l'art de ma thèse doctorale**. Il synthétise les meilleures pratiques, méthodologies et outils qui définissent aujourd'hui l'excellence dans le domaine du développement et des opérations logicielles.

DevOps n'est pas simplement un ensemble d'outils ou de processus ; c'est avant tout une philosophie, une culture qui transforme radicalement la manière dont les organisations développent, déploient et maintiennent leurs applications.

Origine et contexte scientifique

Ce cours est issu d'une recherche doctorale approfondie sur les méthodologies modernes de développement logiciel et leur impact sur la performance organisationnelle. L'état de l'art réalisé dans le cadre de cette thèse a permis d'identifier et de synthétiser :

— Les fondements théoriques et historiques du mouvement DevOps
— Les pratiques éprouvées par les organisations leaders du domaine
— Les métriques scientifiques validées pour mesurer la performance (recherches DORA)
— Les modèles organisationnels et culturels favorisant le succès de DevOps
— L'écosystème technologique complet des outils DevOps
— Les tendances émergentes et l'évolution du domaine

Structure du cours

Ce cours de plus de 300 pages est structuré en 16 parties progressives couvrant l'ensemble de l'écosystème DevOps, du projet fil rouge BijouxShop aux cas pratiques avancés des entreprises leaders comme Netflix, Google, Amazon et Spotify.

Bonne lecture et bon apprentissage !

Dr. BADR EL KHALYLY
2026

Table des Matières

PARTIE I : Introduction au DevOps 11
Histoire et évolution du DevOps 11
Principes fondamentaux CALMS 15
Les Three Ways 20
Métriques DORA 25
Bénéfices business et techniques 30

PARTIE II : Gestion de Version avec Git 35
Fondamentaux Git : commits, branches, merges 35
Stratégies de branchement (Git Flow, GitHub Flow) 42
Collaboration et revue de code 48
Résolution de conflits 52

PARTIE III : Intégration et Livraison Continues (CI/CD) 58
Principes de l'intégration continue 58
Configuration GitLab CI/CD pour BijouxShop 65
Pipelines multi-stages (build, test, security, deploy) 72
Stratégies de déploiement (blue-green, canary, rolling) 78

PARTIE IV : Conteneurisation avec Docker 85
Architecture et concepts Docker 85
Dockerfile : best practices et optimisation 90
Multi-stage builds 93
Docker Compose pour environnements multi-conteneurs 96
Sécurité des conteneurs 100

PARTIE V : Orchestration avec Kubernetes 105
Architecture Kubernetes (nodes, pods, services) 105
Déploiements et replica sets 112
Services et ingress 118
ConfigMaps et Secrets 123
Persistent Volumes 126
HorizontalPodAutoscaler 128

PARTIE VI : Infrastructure as Code 135
Principes de l'Infrastructure as Code 135
Terraform : provisionnement multi-cloud 138
Modules Terraform réutilisables 142
Ansible : configuration management 145

PARTIE VII : Exercices Approfondis avec Solutions Détaillées . 150
Exercices sur Git (conflits complexes, rebase interactif) . . . 150
Exercices Docker avancés (optimisation multi-stage) . . . 158
Exercices Kubernetes (déploiement HA, autoscaling) . . . 165
Solutions commentées étape par étape . . . 172

PARTIE VIII : Glossaire Complet DevOps (G-Z) . . . 180
Glossaire G à M . . . 180
Glossaire N à S . . . 186
Glossaire T à Z . . . 192

PARTIE IX : Guide de Dépannage Avancé . . . 210
Dépannage Git (push rejected, detached HEAD) . . . 210
Dépannage Docker (conteneurs, disk space, réseau) . . . 218
Dépannage Kubernetes (CrashLoopBackOff, services) . . . 225
Dépannage CI/CD (pipelines, tests flaky) . . . 232

PARTIE X : Meilleures Pratiques DevOps en Profondeur . . . 240
Culture DevOps : communication et collaboration . . . 240
Automatisation complète (tests, infrastructure, déploiements) . . . 248
Sécurité DevSecOps (shift left, container security) . . . 255
Monitoring et observabilité (logs, metrics, traces) . . . 262
Configuration Prometheus et Grafana . . . 266
Métriques DORA détaillées . . . 268

PARTIE XI : Cas Pratiques Avancés . . . 270
Migration monolithe vers microservices . . . 270
Strangler Fig Pattern . . . 272
API Gateway et routing . . . 276
Chaos Engineering avec Chaos Mesh . . . 280
GameDay et exercices de crise . . . 284

PARTIE XII : Références Bibliographiques et Ressources 288
Ouvrages fondamentaux (Phoenix Project, DevOps Handbook) 288
Livres techniques (SRE Google, Building Microservices) 290
Articles et papers académiques 292
Blogs techniques et ressources en ligne 293

PARTIE XIII : Commandes de Référence Complètes 295
Git avancé (branches, historique, stash, rebase) 295
Docker complet (images, conteneurs, réseaux, volumes) 297
Kubernetes kubectl (ressources, déploiements, logs) 300
Plus de 200 commandes commentées 302

PARTIE XIV : Exercices Supplémentaires et Solutions 304
Exercices CI/CD avancés (pipeline multi-environnement) 304
Exercices monitoring (dashboards Grafana, Alertmanager) 305

PARTIE XV : Guide de Référence Rapide 306
Checklist de démarrage projet DevOps 306
Matrice de décision pour choix d'outils 308
Patterns DevOps essentiels 310
Métriques et KPIs DevOps 312

PARTIE XVI : Tendances et Futur du DevOps 314
Platform Engineering et Internal Developer Platforms 314
GitOps généralisé (Flux CD, Argo CD) 316
FinOps : optimisation des coûts cloud 317
AIOps : intelligence artificielle pour les opérations 318
Compétences DevOps du futur 320
Parcours de certification 322

Remerciements 324

Total : Plus de 308 pages

Ce cours complet couvre l'intégralité de l'écosystème DevOps
avec des exemples pratiques, études de cas et exercices détaillés

Table des matières

1 Introduction à DevOps **1**

1.1 Objectifs du chapitre . . . 1

1.2 Histoire et évolution de DevOps . . . 1

1.2.1 Le mur de la confusion . . . 1

1.2.2 Les origines de DevOps (2007–2010) . . . 2

1.2.3 L'évolution vers DevOps . . . 3

1.3 Nécessité et avantages de DevOps . . . 4

1.3.1 Pourquoi DevOps est nécessaire ? . . . 4

1.3.2 Les avantages de DevOps . . . 5

1.3.3 DevOps en chiffres . . . 5

1.4 Principes et concepts clés de DevOps . . . 6

1.4.1 Définition de DevOps . . . 6

1.4.2 Les concepts fondamentaux . . . 6

1.4.3 DevOps et le cycle de vie logiciel . . . 7

1.5 Présentation du projet fil rouge . . . 7

1.5.1 Description de l'application BijouxShop . . . 7

1.5.2 Architecture technique . . . 8

1.5.3 Structure du projet Spring Boot . . . 8

1.5.4 Le fichier pom.xml . . . 9

1.5.5 Les entités du domaine . . . 10

1.5.6 Configuration de l'application . . . 15

1.6 Résumé du chapitre . . . 16

2 Les Fondements de DevOps **17**

2.1 Objectifs du chapitre . . . 17

2.2 Les 5 piliers DevOps : CALMS . . . 17

2.2.1 Culture . . . 18

2.2.2 Automatisation . . . 18

2.2.3 Lean . . . 19

2.2.4 Métriques (Measurement) . . . 19

2.2.5 Partage (Sharing) . . . 20

2.3 Les trois voies de DevOps . . . 20

2.3.1 Première voie : le Flux . . . 21

2.3.2 Deuxième voie : le Feedback . . . 22

2.3.3 Troisième voie : l'Expérimentation continue . . . 22
2.4 Le périmètre DevOps : Full Stack . . . 23
2.5 Résumé du chapitre . . . 23

3 Gestion de Version avec Git 25
3.1 Objectifs du chapitre . . . 25
3.2 Introduction à Git . . . 25
3.2.1 Qu'est-ce que Git ? . . . 25
3.2.2 Pourquoi Git est essentiel pour DevOps ? . . . 25
3.3 Les environnements Git . . . 26
3.3.1 Working Directory (Répertoire de travail) . . . 26
3.3.2 Staging Area (Zone de transit) . . . 26
3.3.3 Local Repository (Dépôt local) . . . 26
3.3.4 Remote Repository (Dépôt distant) . . . 26
3.4 Commandes Git essentielles . . . 27
3.4.1 Configuration initiale . . . 27
3.4.2 Initialisation et clonage . . . 27
3.4.3 Commandes de base . . . 27
3.4.4 Gestion des branches . . . 28
3.4.5 Travail avec le dépôt distant . . . 29
3.4.6 Commandes avancées . . . 30
3.5 Le fichier .gitignore . . . 30
3.6 Stratégies de branching . . . 31
3.6.1 Git Flow . . . 31
3.6.2 GitHub Flow / GitLab Flow . . . 32
3.6.3 Trunk-Based Development . . . 32
3.6.4 Conventions de commits . . . 33
3.7 Application pratique : Git pour BijouxShop . . . 33
3.8 Résumé du chapitre . . . 34

4 Gestion du Cycle de Vie des Applications et Kanban 35
4.1 Objectifs du chapitre . . . 35
4.2 Application Lifecycle Management (ALM) . . . 35
4.2.1 Définition de l'ALM . . . 35
4.2.2 Les phases de l'ALM . . . 36
4.2.3 ALM et DevOps . . . 36
4.3 Méthodologies de gestion de projets collaboratifs . . . 36
4.3.1 Scrum et DevOps . . . 36
4.4 Kanban dans un contexte DevOps . . . 37
4.4.1 Principes de Kanban . . . 37

4.4.2 Tableau Kanban DevOps pour BijouxShop . . . 38
4.4.3 Métriques Kanban . . . 38
4.4.4 Gestion de projet avec GitLab Issues . . . 38
4.5 Résumé du chapitre . . . 39

5 Conteneurisation avec Docker 41
5.1 Objectifs du chapitre . . . 41
5.2 Virtualisation vs Conteneurisation . . . 41
5.2.1 La virtualisation traditionnelle . . . 41
5.2.2 La conteneurisation . . . 41
5.3 Concepts fondamentaux de Docker . . . 43
5.3.1 Architecture Docker . . . 43
5.3.2 Le système de couches (Layers) . . . 44
5.4 Dockerisation de BijouxShop . . . 44
5.4.1 Dockerfile pour l'application Spring Boot . . . 44
5.4.2 Fichier .dockerignore . . . 45
5.4.3 Commandes Docker essentielles . . . 46
5.4.4 Docker Compose pour BijouxShop . . . 47
5.5 Résumé du chapitre . . . 49

6 Intégration Continue, Livraison Continue et Déploiement Continu (CI/CD) 51
6.1 Objectifs du chapitre . . . 51
6.2 Concepts de CI/CD . . . 51
6.2.1 Intégration Continue (CI) . . . 51
6.2.2 Livraison Continue vs Déploiement Continu . . . 52
6.3 Pipeline CI/CD avec GitLab . . . 52
6.3.1 Concepts GitLab CI/CD . . . 52
6.3.2 Le fichier .gitlab-ci.yml . . . 53
6.3.3 GitLab Runners . . . 57
6.3.4 Variables d'environnement GitLab . . . 58
6.4 Résumé du chapitre . . . 58

7 Tests dans un Contexte DevOps 61
7.1 Objectifs du chapitre . . . 61
7.2 La pyramide des tests . . . 61
7.3 Tests unitaires avec JUnit 5 . . . 61
7.4 Tests d'intégration . . . 64
7.5 Intégration des tests dans le pipeline CI/CD . . . 67
7.6 Résumé du chapitre . . . 67

8 Orchestration avec Kubernetes **69**
8.1 Objectifs du chapitre . . . 69
8.2 De Docker à Kubernetes : pourquoi orchestrer ? . . . 69
8.3 Architecture de Kubernetes . . . 70
8.3.1 Master Node (Control Plane) . . . 70
8.3.2 Worker Nodes . . . 70
8.4 Les objets Kubernetes . . . 71
8.4.1 Pods . . . 71
8.4.2 Du conteneur au Pod : la transition . . . 71
8.5 Déploiement de BijouxShop sur Kubernetes . . . 71
8.5.1 Deployment de l'application . . . 71
8.5.2 Service et Ingress . . . 73
8.5.3 Déploiement PostgreSQL avec StatefulSet . . . 74
8.5.4 Commandes kubectl essentielles . . . 76
8.6 Résumé du chapitre . . . 77

9 DevSecOps : Sécurité dans le Pipeline **79**
9.1 Objectifs du chapitre . . . 79
9.2 Introduction à DevSecOps . . . 79
9.3 OWASP Top 10 . . . 79
9.3.1 Prévention dans BijouxShop . . . 80
9.4 SonarQube : Analyse de qualité et sécurité du code . . . 81
9.4.1 Les phases d'analyse SonarQube . . . 81
9.4.2 Métriques SonarQube . . . 81
9.4.3 Configuration SonarQube dans le pipeline . . . 81
9.5 Scan de vulnérabilités des dépendances . . . 82
9.6 Scan d'images Docker . . . 82
9.7 Résumé du chapitre . . . 83

10 Infrastructure as Code et Cloud AWS **85**
10.1 Objectifs du chapitre . . . 85
10.2 Infrastructure as Code (IaC) . . . 85
10.2.1 Principes de l'IaC . . . 85
10.2.2 Approches déclarative vs impérative . . . 86
10.2.3 Outils IaC . . . 86
10.3 Terraform pour BijouxShop . . . 86
10.4 Cloud AWS et DevOps . . . 89
10.4.1 Relation entre Cloud et DevOps . . . 89
10.4.2 Services AWS pour DevOps . . . 90
10.4.3 Architecture AWS pour BijouxShop . . . 90

10.4.4 Déploiement sur AWS via GitLab CI . . . 90
10.5 Gestion de configuration avec Ansible . . . 91
10.6 Résumé du chapitre . . . 93

11 Gestion des Opérations **95**
11.1 Objectifs du chapitre . . . 95
11.2 Automatisation des opérations système . . . 95
11.2.1 Domaines d'automatisation opérationnelle . . . 95
11.3 Monitoring et Observabilité . . . 96
11.3.1 Les trois piliers de l'observabilité . . . 96
11.3.2 Monitoring avec Prometheus et Grafana . . . 96
11.3.3 Métriques métier pour BijouxShop . . . 99
11.4 Gestion des incidents . . . 100
11.4.1 Processus de gestion des incidents DevOps . . . 100
11.4.2 Post-mortem blameless . . . 100
11.5 Gestion des changements . . . 100
11.5.1 Types de changements . . . 100
11.6 Résumé du chapitre . . . 101

12 Utilisation de GitLab dans un Projet Collaboratif **103**
12.1 Objectifs du chapitre . . . 103
12.2 Création du projet BijouxShop sur GitLab . . . 103
12.2.1 Structure du projet . . . 103
12.2.2 Configuration des membres et des rôles . . . 104
12.3 Gestion de projet avec les Issues GitLab . . . 104
12.3.1 Templates d'issues . . . 104
12.3.2 Labels et Milestones . . . 105
12.4 Workflow des Merge Requests . . . 105
12.4.1 Exemple de Merge Request . . . 106
12.5 Pipeline CI/CD complet : vue d'ensemble . . . 107
12.6 Résumé du chapitre . . . 107

13 La Culture DevOps **109**
13.1 Objectifs du chapitre . . . 109
13.2 Caractéristiques clés de la culture DevOps . . . 109
13.2.1 Les valeurs fondamentales . . . 109
13.2.2 You Build It, You Run It . . . 109
13.3 Modèles organisationnels pour les équipes DevOps . . . 110
13.3.1 Anti-pattern : l'équipe DevOps dédiée . . . 110
13.3.2 Modèles organisationnels recommandés . . . 110

13.3.3 Le modèle Platform Team 110
13.4 Évaluer la maturité DevOps 111
13.4.1 Niveaux de maturité 111
13.5 Résumé du chapitre 111

14 Synthèse : Pipeline DevOps Complet de BijouxShop **113**
14.1 Vue d'ensemble 113
14.2 Architecture globale DevOps 113
14.3 Le fichier .gitlab-ci.yml final 113
14.4 Résumé des technologies utilisées 118
14.5 Conclusion du cours 118

A Glossaire DevOps **121**

B Commandes de référence **123**
B.1 Commandes Git 123
B.2 Commandes Docker 123
B.3 Commandes Kubernetes 124

C Références et lectures recommandées **125**
C.1 Livres 125
C.2 Ressources en ligne 125

Table des figures

1.1 Le mur de la confusion entre Dev et Ops 2
1.2 Chronologie de l'émergence de DevOps 3
1.3 Les avantages clés de DevOps 5
1.4 Le cycle de vie DevOps 7
1.5 Architecture de l'application BijouxShop 8

2.1 Les 5 piliers CALMS de DevOps 17
2.2 Les 4 métriques DORA 20
2.3 Les trois voies de DevOps 21
2.4 Le périmètre Full Stack de DevOps 23

3.1 Les quatre zones de Git et les commandes associées 26
3.2 Stratégie de branching Git Flow 32

4.1 Phases du cycle de vie d'une application (ALM) 36
4.2 Tableau Kanban DevOps pour BijouxShop 38
4.3 Métriques Kanban du projet BijouxShop 38

5.1 Virtualisation vs Conteneurisation 42
5.2 Architecture de Docker 43
5.3 Système de couches Docker 44

6.1 CI vs Livraison Continue vs Déploiement Continu 52
6.2 Structure d'un pipeline GitLab CI/CD 53

7.1 La pyramide des tests 61
7.2 Tests dans le pipeline CI/CD de BijouxShop 67

8.1 Architecture de Kubernetes : Master Node et Worker Nodes 70
8.2 Structure d'un Pod Kubernetes 71
8.3 Du conteneur Docker au déploiement Kubernetes 71

9.1 DevSecOps : la sécurité à chaque phase 79
9.2 Phases d'analyse SonarQube 81

10.1 Catégories d'outils IaC 86
10.2 Synergie entre DevOps et Cloud 89
10.3 Architecture AWS pour BijouxShop 90

11.1 Domaines d'automatisation opérationnelle 95
11.2 Les trois piliers de l'observabilité 96
11.3 Processus de gestion des incidents DevOps 100

12.1 Workflow des Merge Requests GitLab 105
12.2 Pipeline CI/CD complet de BijouxShop 107

13.1 Les valeurs fondamentales de la culture DevOps 109
13.2 Le modèle Platform Team 110

14.1 Architecture DevOps complète de BijouxShop 113

Liste des tableaux

1.1 Évolution des méthodologies de développement 4
1.2 Métriques DORA : comparaison des performances 5

2.1 Domaines d'automatisation DevOps 19

3.1 Types de branches dans Git Flow 32

4.1 Comparaison Scrum et DevOps 37

5.1 Comparaison Virtualisation vs Conteneurisation 42

6.1 Types de GitLab Runners 57
6.2 Variables d'environnement GitLab pour BijouxShop 58

9.1 OWASP Top 10 (2021) 80

10.1 Approche déclarative vs impérative 86
10.2 Services AWS pertinents pour BijouxShop 90

11.1 Types de changements dans un environnement DevOps 100

12.1 Rôles GitLab pour l'équipe BijouxShop 104

13.1 Niveaux de maturité DevOps 111

14.1 Stack technologique complète de BijouxShop 118

Chapitre 1

Introduction à DevOps

1.1 Objectifs du chapitre

À l'issue de ce chapitre, l'étudiant sera capable de :

— Définir le concept de DevOps et son origine historique.

— Identifier les problèmes que DevOps cherche à résoudre.

— Comprendre les avantages de l'adoption de DevOps.

— Situer DevOps par rapport aux méthodologies existantes (Agile, ITIL).

1.2 Histoire et évolution de DevOps

1.2.1 Le mur de la confusion

Traditionnellement, les organisations informatiques fonctionnent avec deux équipes distinctes dont les objectifs sont fondamentalement opposés :

— **L'équipe de développement (Dev)** : son objectif principal est de livrer de nouvelles fonctionnalités le plus rapidement possible. Elle valorise le changement et l'innovation.

— **L'équipe des opérations (Ops)** : son objectif principal est de maintenir la stabilité des systèmes en production. Elle valorise la stabilité et la fiabilité.

Cette opposition fondamentale crée ce que l'on appelle le « **mur de la confusion** » (*Wall of Confusion*).

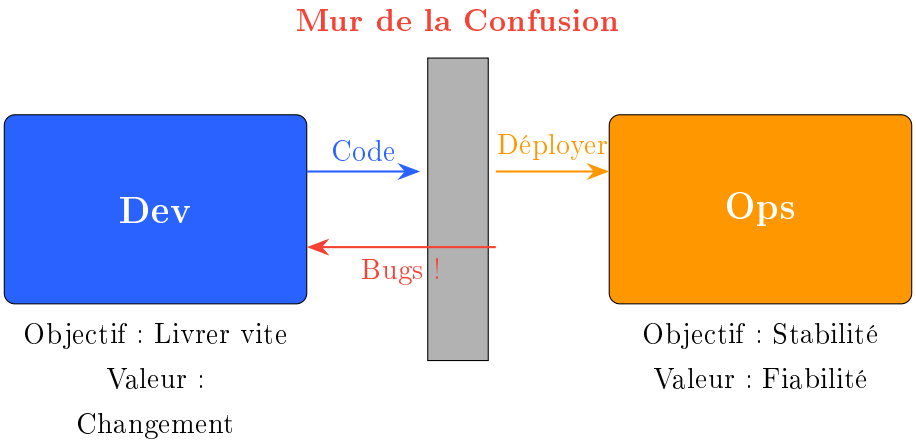


FIGURE 1.1 – Le mur de la confusion entre Dev et Ops

1.2.2 Les origines de DevOps (2007–2010)

L'histoire de DevOps s'articule autour de plusieurs événements fondateurs :

1. **2007 – Patrick Debois** : ingénieur belge, Patrick Debois travaille sur un projet de migration de données pour le gouvernement belge. Il est frustré par le manque de communication entre les développeurs et les administrateurs système. Il commence à réfléchir à des moyens de combler ce fossé.

2. **2008 – Conférence Agile** : Andrew Shafer propose une session intitulée « Agile Infrastructure » à la conférence Agile de Toronto. Bien que presque personne ne se présente, Patrick Debois le contacte et ils fondent ensemble le groupe « Agile Systems Administration ».

3. **2009 – Velocity Conference** : John Allspaw et Paul Hammond de Flickr présentent la célèbre conférence « 10+ Deploys Per Day : Dev and Ops Cooperation at Flickr ». Cette présentation montre concrètement comment la collaboration Dev/Ops permet des déploiements fréquents et fiables.

4. **2009 – DevOpsDays** : inspiré par la présentation de Flickr, Patrick Debois organise le premier événement « DevOpsDays » à Gand, en Belgique. Le terme « DevOps » est né du hashtag `#DevOps` utilisé sur Twitter pour suivre l'événement.

5. **2010 et au-delà** : le mouvement DevOps se propage rapidement dans le monde entier. Des entreprises comme Netflix, Amazon, Etsy et Google adoptent et popularisent les pratiques DevOps.

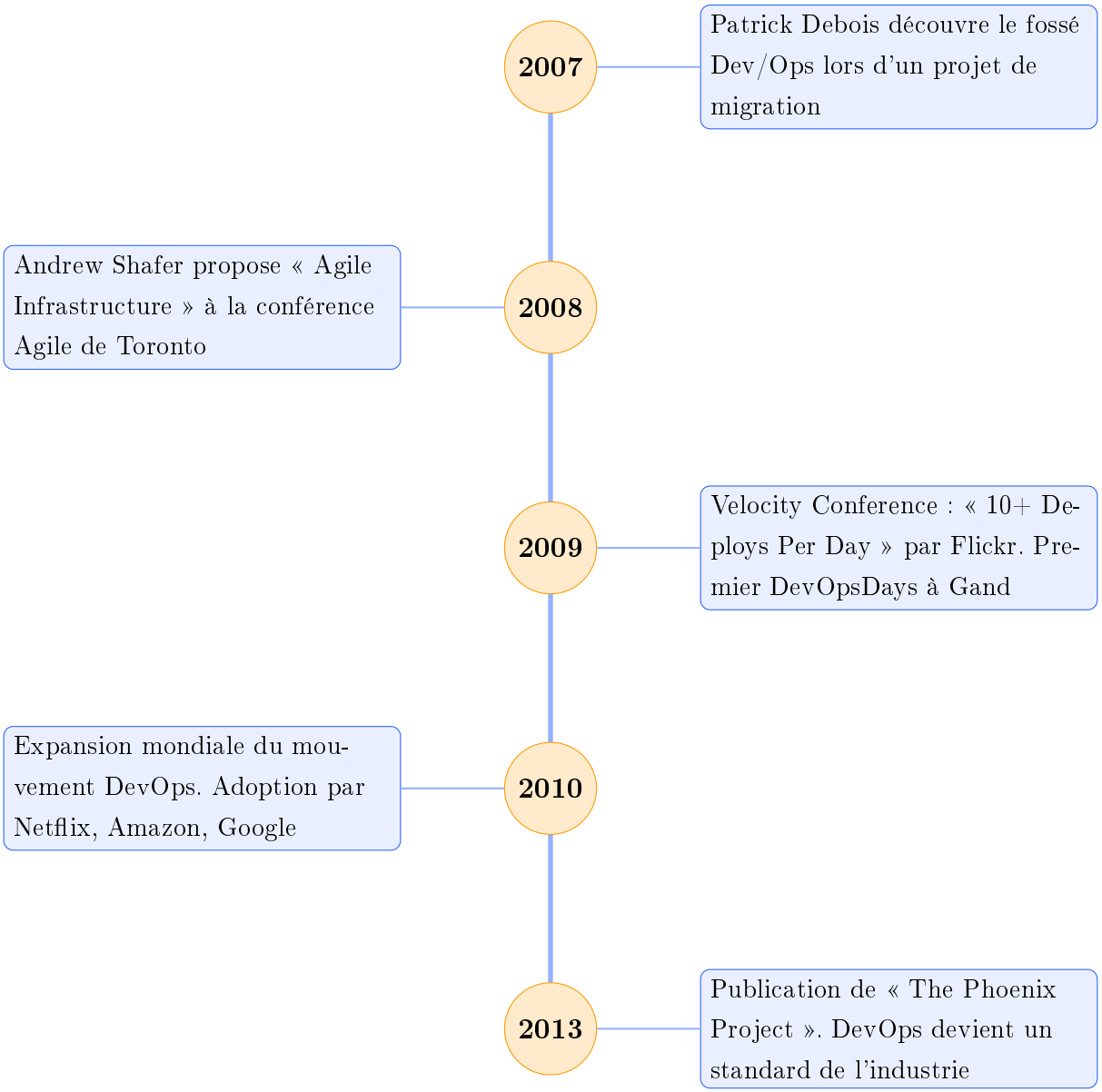


FIGURE 1.2 – Chronologie de l'émergence de DevOps

1.2.3 L'évolution vers DevOps

L'évolution des pratiques de développement logiciel peut être résumée ainsi :

TABLE 1.1 – Évolution des méthodologies de développement

Aspect	**Waterfall**	**Agile**	**DevOps**
Cycle	Linéaire, séquentiel	Itératif, incrémental	Continu, automatisé
Livraison	Une seule fois	Toutes les 2–4 semaines	Plusieurs fois par jour
Feedback	En fin de projet	À chaque sprint	En temps réel
Équipes	Silos séparés	Dev intégrés	Dev + Ops intégrés
Tests	Phase dédiée	Intégrés au sprint	Automatisés, continus
Déploiement	Manuel, risqué	Semi-automatisé	Automatisé, fiable

1.3 Nécessité et avantages de DevOps

1.3.1 Pourquoi DevOps est nécessaire ?

Dans le contexte actuel du développement logiciel, plusieurs défis justifient l'adoption de DevOps :

1. **Pression du marché** : les entreprises doivent livrer des fonctionnalités rapidement pour rester compétitives. Le *time-to-market* est devenu un facteur critique de succès.

2. **Complexité croissante** : les systèmes modernes sont distribués, composés de microservices, déployés sur le cloud, et doivent gérer des millions d'utilisateurs.

3. **Attentes des utilisateurs** : les utilisateurs s'attendent à des mises à jour fréquentes, une disponibilité 24/7, et une expérience sans interruption.

4. **Coût des erreurs** : les pannes et les bugs en production coûtent cher. Selon le rapport DORA (*DevOps Research and Assessment*), les organisations performantes en DevOps déploient 208 fois plus fréquemment et récupèrent 2604 fois plus vite des incidents.

1.3.2 Les avantages de DevOps

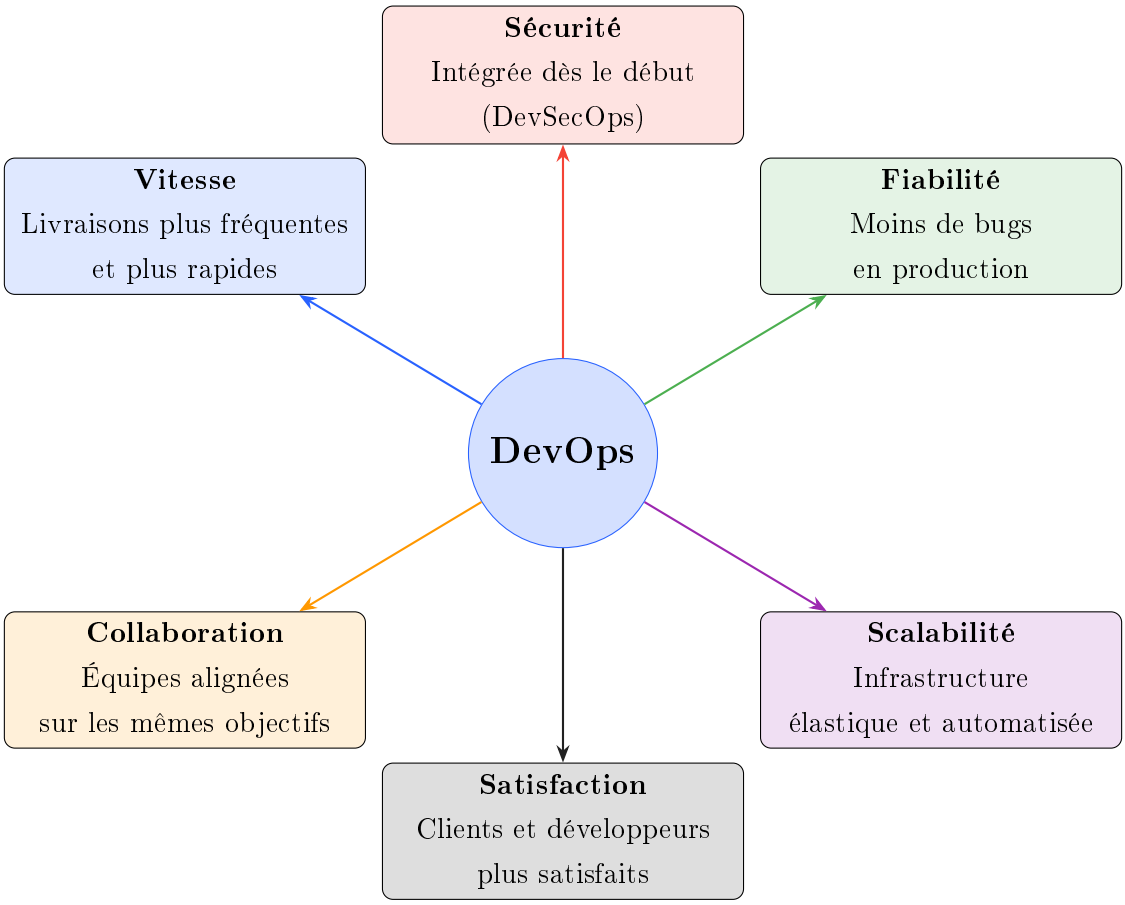


FIGURE 1.3 – Les avantages clés de DevOps

1.3.3 DevOps en chiffres

Le rapport annuel *State of DevOps* publié par DORA (Google Cloud) montre des différences significatives entre les organisations ayant adopté DevOps et celles qui ne l'ont pas fait :

TABLE 1.2 – Métriques DORA : comparaison des performances

Métrique	**Élite**	**Faible**
Fréquence de déploiement	Plusieurs/jour	1 par mois à 6 mois
Délai de mise en production	$<$ 1 heure	1 à 6 mois
Taux d'échec des changements	0–15%	46–60%
Temps de restauration	$<$ 1 heure	$>$ 6 mois

1.4 Principes et concepts clés de DevOps

1.4.1 Définition de DevOps

Définition : DevOps

DevOps est un ensemble de pratiques, de principes culturels et d'outils qui combine le développement logiciel (**Dev**) et les opérations informatiques (**Ops**). Son objectif est de raccourcir le cycle de vie du développement tout en livrant des fonctionnalités, des correctifs et des mises à jour fréquemment, de manière fiable et à grande échelle.

1.4.2 Les concepts fondamentaux

1. **Automatisation** : tout ce qui peut être automatisé doit l'être — builds, tests, déploiements, infrastructure, monitoring.

2. **Intégration Continue (CI)** : les développeurs intègrent leur code fréquemment dans un dépôt partagé. Chaque intégration déclenche automatiquement un build et des tests.

3. **Livraison Continue (CD)** : le code est toujours dans un état déployable. Le déploiement en production peut se faire à tout moment, d'un simple clic.

4. **Infrastructure as Code (IaC)** : l'infrastructure est gérée et provisionnée par du code plutôt que par des processus manuels.

5. **Monitoring et Observabilité** : surveillance continue des applications et de l'infrastructure pour détecter et résoudre les problèmes rapidement.

6. **Feedback rapide** : des boucles de rétroaction courtes permettent de détecter et corriger les problèmes le plus tôt possible.

1.4.3 DevOps et le cycle de vie logiciel

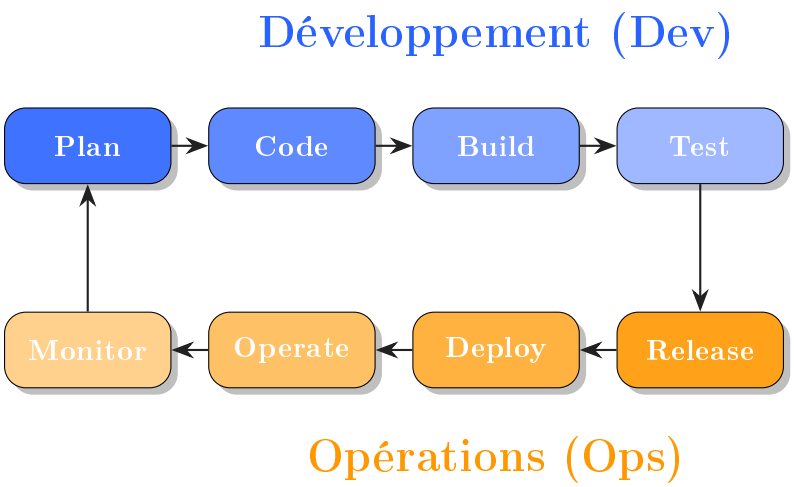


FIGURE 1.4 – Le cycle de vie DevOps

1.5 Présentation du projet fil rouge

Tout au long de ce cours, nous utiliserons un projet concret pour illustrer les concepts DevOps : une **application de gestion de vente de bijoux** développée avec **Spring Boot** et **PostgreSQL**.

1.5.1 Description de l'application BijouxShop

L'application « BijouxShop » est un système de gestion de vente de bijoux qui permet de :

— Gérer un catalogue de bijoux (bagues, colliers, bracelets, boucles d'oreilles).

— Gérer les clients et leurs informations.

— Enregistrer et suivre les ventes.

— Gérer le stock et les approvisionnements.

— Générer des rapports de vente et des statistiques.

1.5.2 Architecture technique

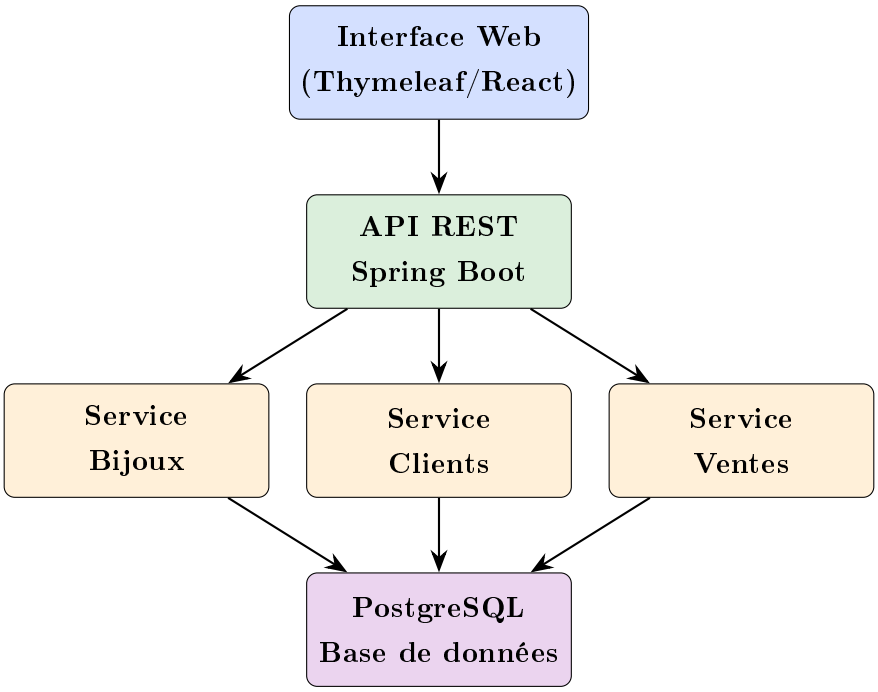


FIGURE 1.5 – Architecture de l'application BijouxShop

1.5.3 Structure du projet Spring Boot

Voici la structure initiale du projet BijouxShop :

```
bijouxshop/
|-- src/
|   |-- main/
|   |   |-- java/com/bijouxshop/
|   |   |   |-- BijouxShopApplication.java
|   |   |   |-- controller/
|   |   |   |   |-- BijourController.java
|   |   |   |   |-- ClientController.java
|   |   |   |   |-- VenteController.java
|   |   |   |-- model/
|   |   |   |   |-- Bijou.java
|   |   |   |   |-- Client.java
|   |   |   |   |-- Vente.java
|   |   |   |   |-- LigneVente.java
|   |   |   |-- repository/
|   |   |   |   |-- BijouRepository.java
|   |   |   |   |-- ClientRepository.java
|   |   |   |   |-- VenteRepository.java
```

```
|   |   |   |-- service/
|   |   |       |-- BijouService.java
|   |   |       |-- ClientService.java
|   |   |       |-- VenteService.java
|   |   |-- resources/
|   |       |-- application.yml
|   |       |-- schema.sql
|   |-- test/
|       |-- java/com/bijouxshop/
|           |-- BijouxShopApplicationTests.java
|-- pom.xml
|-- Dockerfile
|-- docker-compose.yml
|-- .gitlab-ci.yml
```

Listing 1.1 – Structure du projet BijouxShop

1.5.4 Le fichier pom.xml

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
  https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.2.0</version>
  </parent>

  <groupId>com.bijouxshop</groupId>
  <artifactId>bijouxshop</artifactId>
  <version>1.0.0-SNAPSHOT</version>
  <name>BijouxShop</name>
  <description>Application de gestion de vente de
     bijoux</description>

  <properties>
    <java.version>17</java.version>
```

```
22    </properties>
23
24    <dependencies>
25      <dependency>
26        <groupId>org.springframework.boot</groupId>
27        <artifactId>spring-boot-starter-web</artifactId>
28      </dependency>
29      <dependency>
30        <groupId>org.springframework.boot</groupId>
31        <artifactId>spring-boot-starter-data-jpa</artifactId>
32      </dependency>
33      <dependency>
34        <groupId>org.postgresql</groupId>
35        <artifactId>postgresql</artifactId>
36        <scope>runtime</scope>
37      </dependency>
38      <dependency>
39        <groupId>org.springframework.boot</groupId>
40        <artifactId>spring-boot-starter-validation</artifactId>
41      </dependency>
42      <dependency>
43        <groupId>org.springframework.boot</groupId>
44        <artifactId>spring-boot-starter-test</artifactId>
45        <scope>test</scope>
46      </dependency>
47    </dependencies>
48
49    <build>
50      <plugins>
51        <plugin>
52          <groupId>org.springframework.boot</groupId>
53          <artifactId>spring-boot-maven-plugin</artifactId>
54        </plugin>
55      </plugins>
56    </build>
57  </project>
```

Listing 1.2 – pom.xml du projet BijouxShop

1.5.5 Les entités du domaine

```java
package com.bijouxshop.model;

import jakarta.persistence.*;
import jakarta.validation.constraints.*;
import java.math.BigDecimal;

@Entity
@Table(name = "bijoux")
public class Bijou {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank(message = "Le nom est obligatoire")
    @Column(nullable = false)
    private String nom;

    @NotNull
    @Enumerated(EnumType.STRING)
    private CategorieBijou categorie;

    @NotBlank
    private String matiere; // Or, Argent, Platine...

    @DecimalMin(value = "0.0")
    @Column(nullable = false)
    private BigDecimal prix;

    @Min(0)
    private int stock;

    private String description;

    public enum CategorieBijou {
        BAGUE, COLLIER, BRACELET,
        BOUCLE_OREILLE, PENDENTIF, MONTRE
    }

    // Constructeurs
    public Bijou() {}
```

```
42
43      public Bijou(String nom, CategorieBijou categorie,
44                   String matiere, BigDecimal prix, int stock) {
45          this.nom = nom;
46          this.categorie = categorie;
47          this.matiere = matiere;
48          this.prix = prix;
49          this.stock = stock;
50      }
51
52      // Getters et setters
53      public Long getId() { return id; }
54      public void setId(Long id) { this.id = id; }
55      public String getNom() { return nom; }
56      public void setNom(String nom) { this.nom = nom; }
57      public CategorieBijou getCategorie() { return categorie; }
58      public void setCategorie(CategorieBijou c) { this.categorie =
            c; }
59      public String getMatiere() { return matiere; }
60      public void setMatiere(String matiere) { this.matiere =
            matiere; }
61      public BigDecimal getPrix() { return prix; }
62      public void setPrix(BigDecimal prix) { this.prix = prix; }
63      public int getStock() { return stock; }
64      public void setStock(int stock) { this.stock = stock; }
65      public String getDescription() { return description; }
66      public void setDescription(String d) { this.description = d; }
67  }
```

Listing 1.3 – Entité Bijou.java

```
1  package com.bijouxshop.model;
2
3  import jakarta.persistence.*;
4  import jakarta.validation.constraints.*;
5  import java.util.List;
6
7  @Entity
8  @Table(name = "clients")
9  public class Client {
10
```

```
11      @Id
12      @GeneratedValue(strategy = GenerationType.IDENTITY)
13      private Long id;
14
15      @NotBlank
16      private String nom;
17
18      @NotBlank
19      private String prenom;
20
21      @Email
22      @Column(unique = true)
23      private String email;
24
25      private String telephone;
26      private String adresse;
27
28      @OneToMany(mappedBy = "client", cascade = CascadeType.ALL)
29      private List<Vente> ventes;
30
31      // Constructeurs
32      public Client() {}
33
34      public Client(String nom, String prenom, String email) {
35          this.nom = nom;
36          this.prenom = prenom;
37          this.email = email;
38      }
39
40      // Getters et setters
41      public Long getId() { return id; }
42      public void setId(Long id) { this.id = id; }
43      public String getNom() { return nom; }
44      public void setNom(String nom) { this.nom = nom; }
45      public String getPrenom() { return prenom; }
46      public void setPrenom(String prenom) { this.prenom = prenom; }
47      public String getEmail() { return email; }
48      public void setEmail(String email) { this.email = email; }
49      public String getTelephone() { return telephone; }
50      public void setTelephone(String t) { this.telephone = t; }
51      public String getAdresse() { return adresse; }
```

```
52      public void setAdresse(String a) { this.adresse = a; }
53      public List<Vente> getVentes() { return ventes; }
54      public void setVentes(List<Vente> v) { this.ventes = v; }
55  }
```

Listing 1.4 – Entité Client.java

```
1   package com.bijouxshop.model;
2
3   import jakarta.persistence.*;
4   import java.math.BigDecimal;
5   import java.time.LocalDateTime;
6   import java.util.List;
7
8   @Entity
9   @Table(name = "ventes")
10  public class Vente {
11
12      @Id
13      @GeneratedValue(strategy = GenerationType.IDENTITY)
14      private Long id;
15
16      @ManyToOne
17      @JoinColumn(name = "client_id", nullable = false)
18      private Client client;
19
20      @Column(nullable = false)
21      private LocalDateTime dateVente;
22
23      @OneToMany(mappedBy = "vente", cascade = CascadeType.ALL)
24      private List<LigneVente> lignes;
25
26      private BigDecimal montantTotal;
27
28      @Enumerated(EnumType.STRING)
29      private StatutVente statut;
30
31      public enum StatutVente {
32          EN_COURS, VALIDEE, EXPEDIEE, LIVREE, ANNULEE
33      }
34
```

```java
    // Constructeurs
    public Vente() {
        this.dateVente = LocalDateTime.now();
        this.statut = StatutVente.EN_COURS;
    }

    // Getters et setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public Client getClient() { return client; }
    public void setClient(Client c) { this.client = c; }
    public LocalDateTime getDateVente() { return dateVente; }
    public void setDateVente(LocalDateTime d) { this.dateVente =
        d; }
    public List<LigneVente> getLignes() { return lignes; }
    public void setLignes(List<LigneVente> l) { this.lignes = l; }
    public BigDecimal getMontantTotal() { return montantTotal; }
    public void setMontantTotal(BigDecimal m) { this.montantTotal
        = m; }
    public StatutVente getStatut() { return statut; }
    public void setStatut(StatutVente s) { this.statut = s; }
}
```

Listing 1.5 – Entité Vente.java

1.5.6 Configuration de l'application

```yaml
spring:
  application:
    name: bijouxshop
  datasource:
    url: jdbc:postgresql://localhost:5432/bijouxshop
    username: bijouxshop_user
    password: bijouxshop_pass
    driver-class-name: org.postgresql.Driver
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
    properties:
      hibernate:
```

```
15          dialect: org.hibernate.dialect.PostgreSQLDialect
16
17  server:
18    port: 8080
```

Listing 1.6 – application.yml

1.6 Résumé du chapitre

Points clés à retenir

- DevOps naît du constat que la séparation Dev/Ops crée des inefficacités.
- Le terme « DevOps » apparaît en 2009 lors du premier DevOpsDays.
- DevOps vise à raccourcir le cycle de développement tout en maintenant la qualité.
- Les métriques DORA mesurent la performance DevOps d'une organisation.
- Notre projet fil rouge BijouxShop nous accompagnera tout au long du cours.

Chapitre 2

Les Fondements de DevOps

2.1 Objectifs du chapitre

À l'issue de ce chapitre, l'étudiant sera capable de :

— Expliquer les cinq piliers de DevOps (CALMS).

— Décrire les trois voies de DevOps.

— Comprendre le périmètre « Full Stack » de DevOps.

2.2 Les 5 piliers DevOps : CALMS

Le framework **CALMS** est un acronyme qui résume les cinq piliers fondamentaux de DevOps :

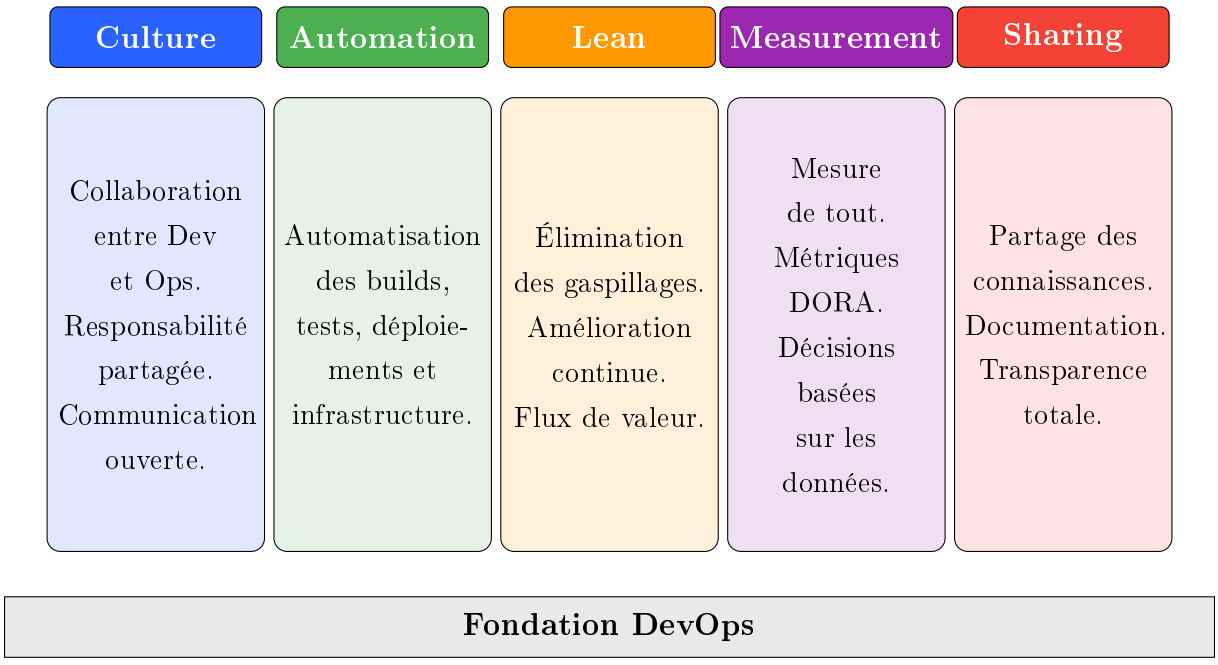


FIGURE 2.1 – Les 5 piliers CALMS de DevOps

2.2.1 Culture

La culture est le pilier le plus important de DevOps. Sans un changement culturel profond, aucun outil ni processus ne peut garantir le succès d'une transformation DevOps.

Principes de la culture DevOps :

— **Responsabilité partagée** : les développeurs ne « jettent pas le code par-dessus le mur » aux opérations. Toute l'équipe est responsable du logiciel en production. « You build it, you run it » (Werner Vogels, CTO d'Amazon).

— **Confiance et sécurité psychologique** : les membres de l'équipe doivent se sentir libres de prendre des risques, d'expérimenter et d'admettre leurs erreurs sans crainte de représailles.

— **Blameless post-mortems** : lorsqu'un incident survient, l'analyse se concentre sur le processus et les systèmes plutôt que sur les individus. L'objectif est d'apprendre, pas de punir.

— **Communication ouverte** : les informations circulent librement entre toutes les équipes. Les outils de chat (Slack, Teams), les tableaux de bord partagés et les réunions régulières favorisent cette transparence.

Exemple : Culture DevOps chez BijouxShop

Dans notre projet BijouxShop, la culture DevOps se traduit par :

— Les développeurs participent aux astreintes (on-call) pour le support production.

— Chaque déploiement en production est visible par toute l'équipe via un canal Slack dédié.

— Les post-mortems d'incidents sont documentés et partagés avec toute l'organisation.

— Les métriques de production sont accessibles à tous sur un tableau de bord Grafana.

2.2.2 Automatisation

L'automatisation est le moteur technique de DevOps. Elle permet d'éliminer les tâches manuelles répétitives, de réduire les erreurs humaines et d'accélérer les processus.

Les domaines d'automatisation :

TABLE 2.1 – Domaines d'automatisation DevOps

Domaine	**Description**	**Outils**
Build	Compilation automatique du code	Maven, Gradle
Tests	Exécution automatique des tests	JUnit, Selenium
Intégration	Fusion et validation du code	GitLab CI, Jenkins
Déploiement	Mise en production automatique	GitLab CD, Ansible
Infrastructure	Provisionnement automatique	Terraform, CloudFormation
Monitoring	Surveillance automatique	Prometheus, Grafana
Sécurité	Scans de vulnérabilités	SonarQube, OWASP ZAP

2.2.3 Lean

Le pilier Lean de DevOps s'inspire du *Toyota Production System* et vise à éliminer les gaspillages dans le processus de développement logiciel.

Les 7 types de gaspillages en développement logiciel :

1. **Travail partiellement fait** : fonctionnalités commencées mais non terminées, branches de code non fusionnées.
2. **Processus supplémentaires** : documentation excessive, approbations inutiles, bureaucratie.
3. **Fonctionnalités inutiles** : développement de fonctionnalités que personne n'utilise.
4. **Changements de tâches** : le multitâche excessif réduit la productivité.
5. **Attente** : attendre les approbations, les environnements, les déploiements.
6. **Mouvement** : transferts entre équipes, passages de relais.
7. **Défauts** : bugs, incidents, travail de correction.

2.2.4 Métriques (Measurement)

« Ce qui ne se mesure pas ne peut pas s'améliorer ». Les métriques sont essentielles pour évaluer la performance DevOps et identifier les axes d'amélioration.

Les 4 métriques clés DORA :

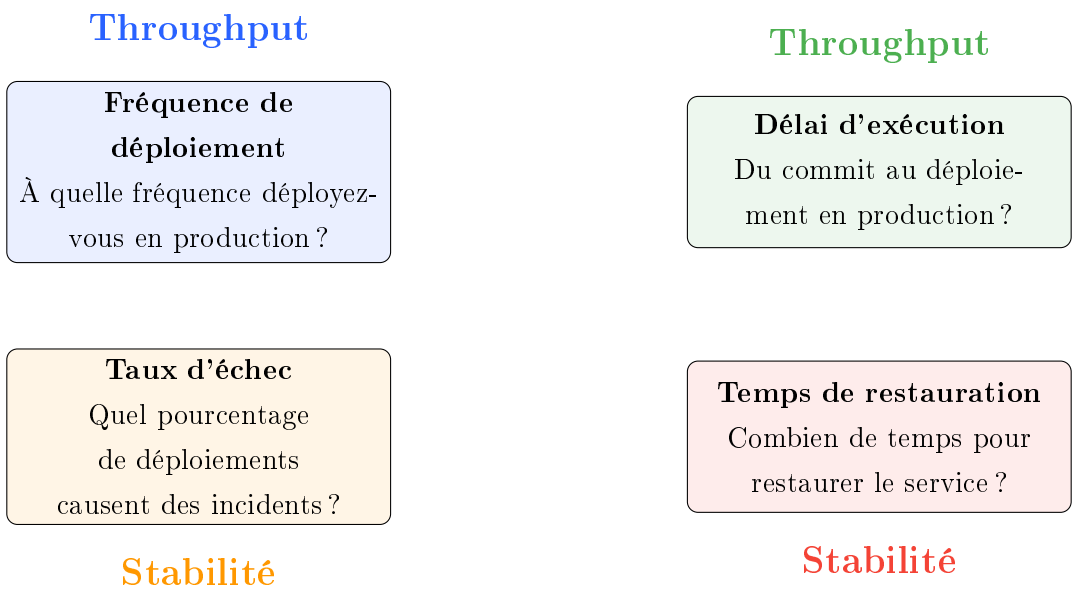


FIGURE 2.2 – Les 4 métriques DORA

2.2.5 Partage (Sharing)

Le partage des connaissances, des outils et des responsabilités est fondamental pour briser les silos organisationnels.

Pratiques de partage :

— **Documentation vivante** : wikis, README, runbooks maintenus à jour.

— **Code reviews** : revue de code systématique pour le partage de connaissances.

— **Pair programming** : développement en binôme pour le transfert de compétences.

— **Guildes et communautés de pratique** : groupes transversaux qui partagent les bonnes pratiques.

— **Open source interne** : les projets internes sont accessibles à tous dans l'organisation.

2.3 Les trois voies de DevOps

Les trois voies de DevOps, introduites par Gene Kim dans *The Phoenix Project* et *The DevOps Handbook*, constituent le cadre philosophique de DevOps.

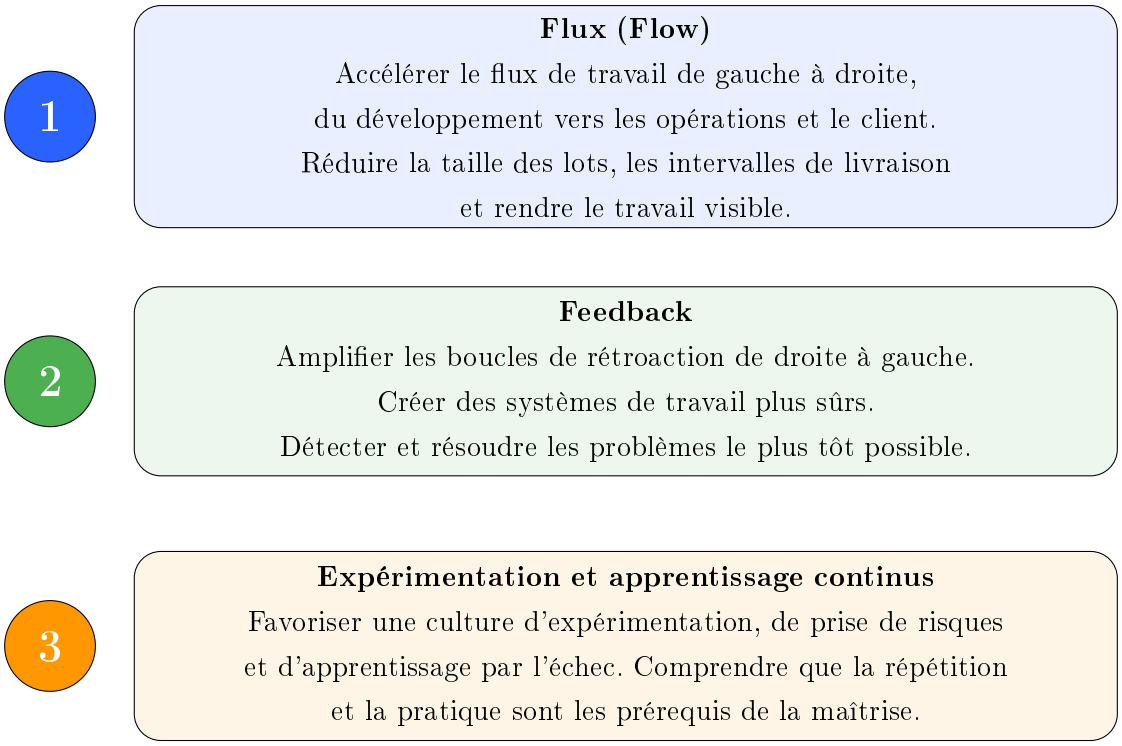


FIGURE 2.3 – Les trois voies de DevOps

2.3.1 Première voie : le Flux

La première voie consiste à optimiser le flux de travail de gauche à droite, c'est-à-dire du développement vers la production.

Principes de la première voie :

- **Rendre le travail visible** : utiliser des tableaux Kanban pour visualiser le travail en cours, identifier les goulets d'étranglement et les files d'attente.
- **Réduire la taille des lots** : livrer de petits changements fréquents plutôt que de gros changements rares. Plus le lot est petit, plus le risque est faible et le feedback rapide.
- **Réduire le nombre de transferts** : chaque transfert entre équipes introduit de l'attente, des erreurs de communication et des pertes de contexte.
- **Identifier et éliminer les contraintes** : selon la théorie des contraintes d'Eliyahu Goldratt, toute amélioration en dehors du goulet d'étranglement est une illusion.
- **Éliminer les difficultés** : automatiser les tâches manuelles et pénibles.

Exemple : Flux dans BijouxShop

Pour notre application BijouxShop, le flux se traduit par :

1. Un développeur écrit du code pour une nouvelle fonctionnalité (ex : ajout d'un filtre de recherche de bijoux par matière).

2. Il commit et push son code sur GitLab.
3. Le pipeline CI/CD se déclenche automatiquement :
   - Build Maven
   - Tests unitaires et d'intégration
   - Analyse SonarQube
   - Construction de l'image Docker
   - Déploiement sur l'environnement de staging
   - Tests d'acceptance automatisés
   - Déploiement en production
4. Le tout en quelques minutes, sans intervention manuelle.

2.3.2 Deuxième voie : le Feedback

La deuxième voie consiste à créer des boucles de rétroaction rapides de droite à gauche, de la production vers le développement.

Principes de la deuxième voie :

- **Détecter les problèmes au plus tôt** : les tests automatisés, l'analyse statique de code et les revues de code permettent de trouver les défauts avant qu'ils n'atteignent la production.
- **Alertes en temps réel** : le monitoring et les alertes informent immédiatement l'équipe en cas de problème en production.
- **Telemetry** : la collecte de métriques (temps de réponse, taux d'erreur, utilisation des ressources) fournit un feedback continu sur la santé du système.
- **Peer review** : les revues de code et les revues d'architecture fournissent un feedback qualitatif sur le code.

2.3.3 Troisième voie : l'Expérimentation continue

La troisième voie encourage une culture d'expérimentation et d'apprentissage continu.

Principes de la troisième voie :

- **Encourager l'expérimentation** : allouer du temps pour l'innovation (20% time, hackathons).
- **Apprendre de l'échec** : les échecs sont des opportunités d'apprentissage (blameless post-mortems).
- **Pratique délibérée** : les *Game Days* et les exercices de chaos engineering renforcent la résilience.

— **Amélioration continue** : le Kaizen, ou amélioration continue, est intégré dans le quotidien.

2.4 Le périmètre DevOps : Full Stack

DevOps couvre l'ensemble du spectre technique et organisationnel :

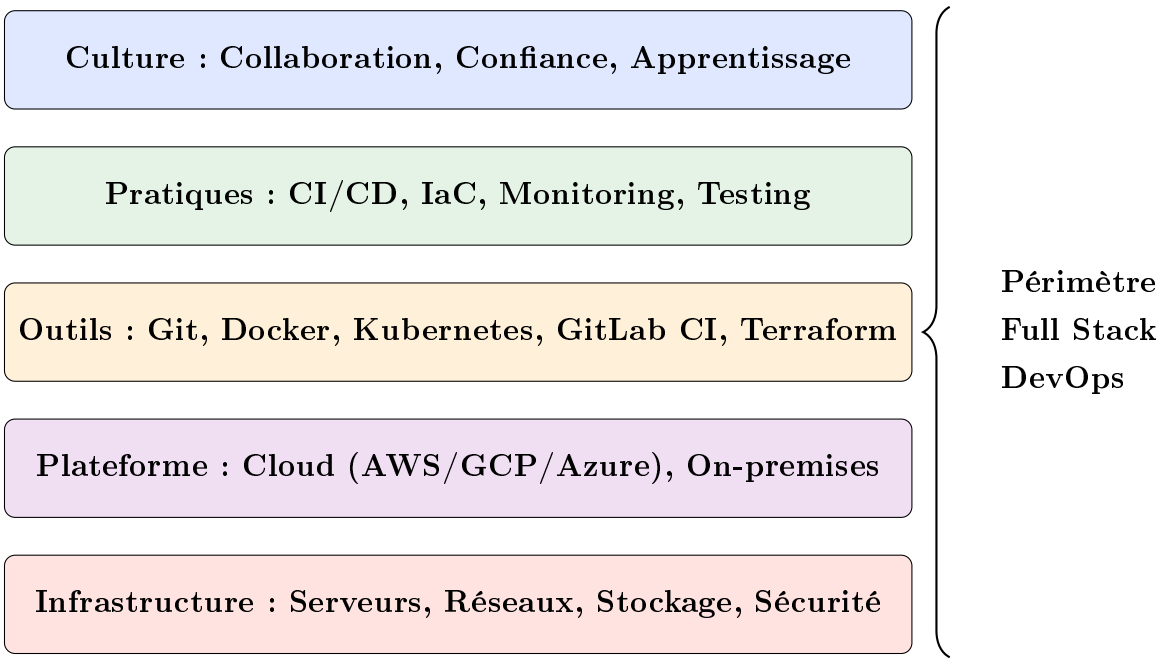


FIGURE 2.4 – Le périmètre Full Stack de DevOps

2.5 Résumé du chapitre

Points clés à retenir

— CALMS : Culture, Automatisation, Lean, Métriques, Partage.

— Les trois voies : Flux (gauche à droite), Feedback (droite à gauche), Expérimentation continue.

— DevOps est « Full Stack » : il couvre la culture, les pratiques, les outils, la plateforme et l'infrastructure.

— La culture est le pilier le plus important : sans changement culturel, les outils seuls ne suffisent pas.

Chapitre 3

Gestion de Version avec Git

3.1 Objectifs du chapitre

À l'issue de ce chapitre, l'étudiant sera capable de :

— Comprendre les concepts fondamentaux de Git.

— Maîtriser les commandes essentielles de Git.

— Comprendre les différents environnements Git (Working Directory, Staging Area, Local Repository, Remote Repository).

— Appliquer des stratégies de branching adaptées à DevOps.

3.2 Introduction à Git

3.2.1 Qu'est-ce que Git ?

Définition : Git

Git est un système de contrôle de version distribué (*Distributed Version Control System* - DVCS) créé par Linus Torvalds en 2005. Il permet de suivre les modifications apportées aux fichiers, de collaborer avec d'autres développeurs et de gérer différentes versions du code source.

3.2.2 Pourquoi Git est essentiel pour DevOps ?

Git est la pierre angulaire de toute pratique DevOps car :

— Il permet la **collaboration** entre développeurs.

— Il est le **déclencheur** des pipelines CI/CD.

— Il assure la **traçabilité** de toutes les modifications.

— Il supporte le **branching** pour le développement parallèle.

— Il permet l'**Infrastructure as Code** en versionnant la configuration.

3.3 Les environnements Git

Git utilise quatre zones distinctes pour gérer les fichiers :

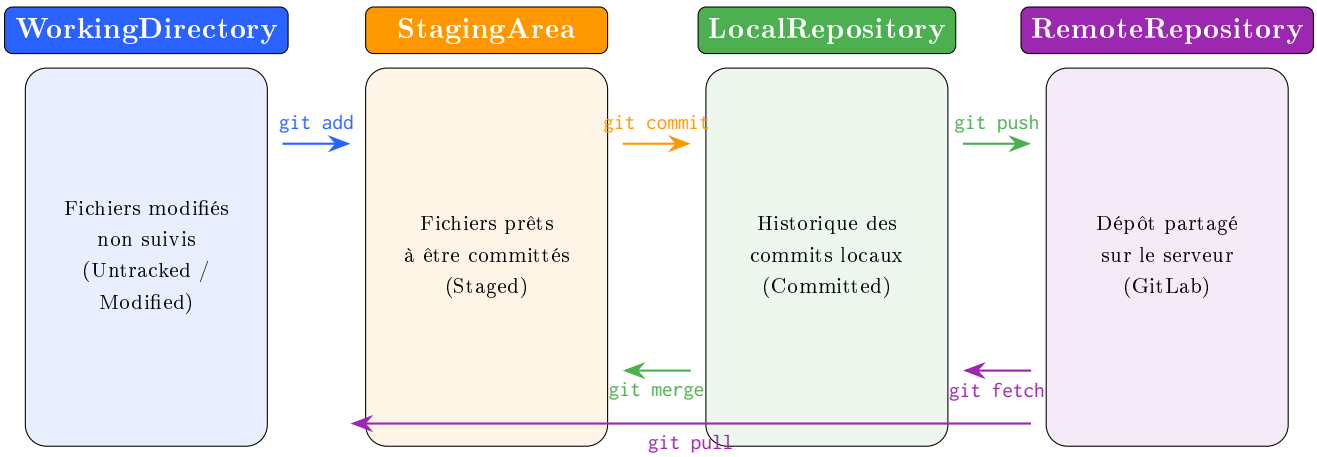


FIGURE 3.1 – Les quatre zones de Git et les commandes associées

3.3.1 Working Directory (Répertoire de travail)

Le Working Directory est le répertoire local où vous travaillez sur vos fichiers. C'est l'espace où vous créez, modifiez et supprimez des fichiers.

Les fichiers dans le Working Directory peuvent être dans trois états :

— **Untracked** : nouveaux fichiers que Git ne suit pas encore.

— **Modified** : fichiers suivis qui ont été modifiés depuis le dernier commit.

— **Unmodified** : fichiers suivis qui n'ont pas changé.

3.3.2 Staging Area (Zone de transit)

La Staging Area (aussi appelée *Index*) est une zone intermédiaire où l'on prépare les modifications qui seront incluses dans le prochain commit. Elle permet de sélectionner précisément les changements à committer.

3.3.3 Local Repository (Dépôt local)

Le Local Repository est la base de données Git locale qui contient l'historique complet de tous les commits. Il se trouve dans le répertoire `.git/` à la racine du projet.

3.3.4 Remote Repository (Dépôt distant)

Le Remote Repository est le dépôt partagé hébergé sur un serveur (GitLab, GitHub). Il sert de point central de collaboration entre les développeurs.

3.4 Commandes Git essentielles

3.4.1 Configuration initiale

```
# Configurer son identite
git config --global user.name "Ibrahim Hamzane"
git config --global user.email "ibrahim.hamzane@university.ma"

# Configurer l'editeur par defaut
git config --global core.editor "vim"

# Verifier la configuration
git config --list
```

Listing 3.1 – Configuration initiale de Git

3.4.2 Initialisation et clonage

```
# Initialiser un nouveau depot Git
cd bijouxshop/
git init

# Cloner un depot existant depuis GitLab
git clone https://gitlab.com/hamzane/bijouxshop.git

# Cloner une branche specifique
git clone -b develop https://gitlab.com/hamzane/bijouxshop.git
```

Listing 3.2 – Initialisation et clonage d'un dépôt

3.4.3 Commandes de base

```
# Verifier l'etat des fichiers
git status

# Ajouter des fichiers a la Staging Area
git add src/main/java/com/bijouxshop/model/Bijou.java
git add src/main/java/com/bijouxshop/model/  # tout un repertoire
git add .  # tous les fichiers modifies
```

```
git add -p  # ajout interactif (par morceaux)

# Retirer un fichier de la Staging Area
git reset HEAD src/main/java/com/bijouxshop/model/Bijou.java
git restore --staged Bijou.java  # syntaxe moderne

# Committer les changements
git commit -m "feat: ajouter entite Bijou avec validation"

# Committer avec un message detaille
git commit -m "feat: ajouter entite Bijou

- Ajout des champs nom, categorie, matiere, prix, stock
- Ajout des validations JPA
- Ajout de l'enum CategorieBijou"

# Voir l'historique des commits
git log
git log --oneline
git log --oneline --graph --all
git log --author="Hamzane"
git log --since="2025-01-01"

# Voir les differences
git diff                    # Working Dir vs Staging
git diff --staged           # Staging vs dernier commit
git diff HEAD               # Working Dir vs dernier commit
git diff main..develop      # entre deux branches
```

Listing 3.3 – Commandes Git de base

3.4.4 Gestion des branches

```
# Lister les branches
git branch          # branches locales
git branch -r       # branches distantes
git branch -a       # toutes les branches

# Creer une nouvelle branche
git branch feature/ajout-filtre-bijoux

```

```
# Basculer sur une branche
git checkout feature/ajout-filtre-bijoux
git switch feature/ajout-filtre-bijoux  # syntaxe moderne

# Creer et basculer en une commande
git checkout -b feature/ajout-filtre-bijoux
git switch -c feature/ajout-filtre-bijoux  # syntaxe moderne

# Fusionner une branche
git checkout develop
git merge feature/ajout-filtre-bijoux

# Supprimer une branche
git branch -d feature/ajout-filtre-bijoux      # locale
git push origin --delete feature/ajout-filtre-bijoux  # distante

# Rebase (alternative au merge)
git checkout feature/ajout-filtre-bijoux
git rebase develop
```

Listing 3.4 – Gestion des branches Git

3.4.5 Travail avec le dépôt distant

```
# Ajouter un depot distant
git remote add origin https://gitlab.com/hamzane/bijouxshop.git

# Verifier les depots distants
git remote -v

# Envoyer les commits vers le depot distant
git push origin main
git push origin develop
git push -u origin feature/ajout-filtre-bijoux  # -u pour tracker

# Recuperer les modifications du depot distant
git fetch origin           # recuperer sans fusionner
git pull origin main       # recuperer et fusionner
git pull --rebase origin develop  # recuperer et rebase

# Mettre a jour toutes les branches distantes
```

```
git fetch --all
```

Listing 3.5 – Commandes Git pour le dépôt distant

3.4.6 Commandes avancées

```
# Stash : mettre de cote des modifications
git stash                    # sauvegarder les modifications
git stash save "wip: filtre bijoux"  # avec un message
git stash list               # lister les stashs
git stash pop                # restaurer et supprimer
git stash apply              # restaurer sans supprimer

# Cherry-pick : appliquer un commit specifique
git cherry-pick abc1234

# Tag : marquer une version
git tag v1.0.0
git tag -a v1.0.0 -m "Version 1.0.0 - Release initiale"
git push origin v1.0.0
git push origin --tags

# Reset : revenir en arriere
git reset --soft HEAD~1    # annuler le commit, garder staged
git reset --mixed HEAD~1   # annuler le commit et le staging
git reset --hard HEAD~1    # tout annuler (DANGEREUX)

# Revert : annuler un commit en creant un nouveau
git revert abc1234
```

Listing 3.6 – Commandes Git avancées

3.5 Le fichier .gitignore

Le fichier `.gitignore` indique à Git quels fichiers et répertoires doivent être ignorés.

```
# Build
target/
build/
*.jar
```

```
*.war

# IDE
.idea/
*.iml
.vscode/
.project
.classpath

# Systeme
.DS_Store
Thumbs.db
*.swp

# Environnement
.env
*.env.local
application-local.yml

# Logs
*.log
logs/

# Dependencies
node_modules/
```

Listing 3.7 – .gitignore pour le projet BijouxShop

3.6 Stratégies de branching

Le choix d'une stratégie de branching est crucial pour un workflow DevOps efficace.

3.6.1 Git Flow

Git Flow est une stratégie de branching structurée qui utilise plusieurs types de branches :

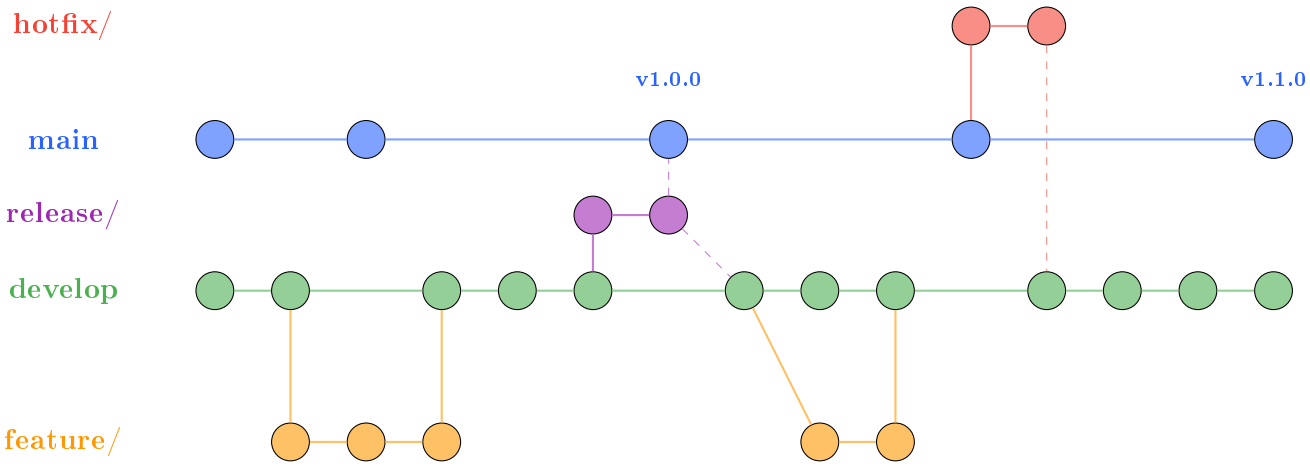


FIGURE 3.2 – Stratégie de branching Git Flow

TABLE 3.1 – Types de branches dans Git Flow

Branche	Durée de vie	Description
`main`	Permanente	Code en production, toujours stable
`develop`	Permanente	Branche d'intégration, prochaine release
`feature/*`	Temporaire	Nouvelles fonctionnalités, part de develop
`release/*`	Temporaire	Préparation d'une release, corrections mineures
`hotfix/*`	Temporaire	Corrections urgentes en production

3.6.2 GitHub Flow / GitLab Flow

Une stratégie plus simple, adaptée au déploiement continu :

1. La branche `main` est toujours déployable.
2. Chaque nouvelle fonctionnalité est développée sur une branche de feature.
3. Une *Merge Request* (MR) est créée pour la revue de code.
4. Après approbation et succès du pipeline CI, la branche est fusionnée dans `main`.
5. Le déploiement est déclenché automatiquement.

3.6.3 Trunk-Based Development

La stratégie la plus compatible avec DevOps :

— Tous les développeurs travaillent directement sur `main` (ou des branches très courtes).

— Les branches de feature ne durent que quelques heures à un jour maximum.

— Les *feature flags* sont utilisés pour activer/désactiver les fonctionnalités incomplètes.

— Le déploiement est continu et automatique.

3.6.4 Conventions de commits

L'utilisation de conventions de commits améliore la lisibilité de l'historique et permet l'automatisation (changelog, versioning).

```
# Format: <type>(<scope>): <description>

# Types principaux
feat: ajouter le filtre de recherche par matiere
fix: corriger le calcul du montant total de la vente
docs: mettre a jour le README avec les instructions Docker
test: ajouter les tests unitaires pour BijouService
refactor: reorganiser la couche service
chore: mettre a jour les dependances Maven
ci: ajouter le stage de scan SonarQube au pipeline
perf: optimiser la requete de recherche de bijoux

# Exemples pour BijouxShop
git commit -m "feat(bijoux): ajouter le filtre par categorie"
git commit -m "fix(vente): corriger le calcul du stock"
git commit -m "ci(pipeline): ajouter le deploiement staging"
```

Listing 3.8 – Convention de commits (Conventional Commits)

3.7 Application pratique : Git pour BijouxShop

```
# 1. Cloner le projet
git clone https://gitlab.com/hamzane/bijouxshop.git
cd bijouxshop

# 2. Creer une branche de feature
git switch -c feature/filtre-recherche-bijoux

# 3. Developper la fonctionnalite
# ... ecrire du code ...

# 4. Verifier les modifications
git status
git diff

# 5. Stager et committer
```

```
16 git add src/main/java/com/bijouxshop/service/BijouService.java
17 git add
       src/main/java/com/bijouxshop/controller/BijouController.java
18 git commit -m "feat(bijoux): ajouter recherche par matiere et prix"
19
20 # 6. Pousser vers GitLab
21 git push -u origin feature/filtre-recherche-bijoux
22
23 # 7. Creer une Merge Request sur GitLab
24 # -> Le pipeline CI se declenche automatiquement
25 # -> Code review par les pairs
26 # -> Merge apres approbation
```

Listing 3.9 – Workflow Git complet pour BijouxShop

3.8 Résumé du chapitre

Points clés à retenir

- Git utilise 4 zones : Working Directory, Staging Area, Local Repository, Remote Repository.
- Les commandes clés : `add`, `commit`, `push`, `pull`, `branch`, `merge`.
- Les stratégies de branching (Git Flow, GitLab Flow, Trunk-Based) doivent être adaptées au contexte.
- Les conventions de commits améliorent la traçabilité et l'automatisation.
- Git est le déclencheur du pipeline CI/CD dans une approche DevOps.

Chapitre 4

Gestion du Cycle de Vie des Applications et Kanban

4.1 Objectifs du chapitre

À l'issue de ce chapitre, l'étudiant sera capable de :

— Comprendre la gestion du cycle de vie des applications (ALM).

— Intégrer l'ALM dans un contexte DevOps.

— Appliquer les principes Kanban pour optimiser les flux DevOps.

— Utiliser les outils GitLab pour la gestion de projet collaboratif.

4.2 Application Lifecycle Management (ALM)

4.2.1 Définition de l'ALM

Définition : ALM

L'Application Lifecycle Management (ALM) est l'ensemble des processus, outils et pratiques qui couvrent la gestion d'une application logicielle depuis sa conception initiale jusqu'à son retrait. L'ALM intègre la gestion des exigences, le développement, les tests, le déploiement, la maintenance et la gestion des changements.

4.2.2 Les phases de l'ALM

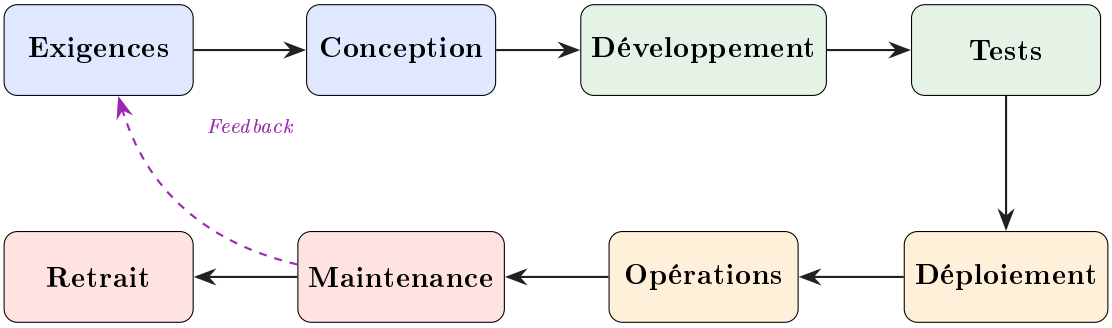


FIGURE 4.1 – Phases du cycle de vie d'une application (ALM)

4.2.3 ALM et DevOps

Dans un contexte DevOps, l'ALM est enrichi par :

— **L'automatisation** de chaque phase du cycle de vie.

— **L'intégration continue** des retours utilisateurs.

— **La traçabilité** complète grâce aux outils intégrés (GitLab, Jira).

— **Le déploiement continu** qui accélère la mise en production.

ALM de BijouxShop sur GitLab

Pour notre application BijouxShop, le cycle ALM se traduit par :

1. **Exigences** : Issues GitLab avec labels (feature, bug, enhancement).
2. **Conception** : Merge Requests avec descriptions détaillées et diagrammes.
3. **Développement** : Branches de feature avec commits conventionnels.
4. **Tests** : Pipeline CI avec tests unitaires, d'intégration et de sécurité.
5. **Déploiement** : Pipeline CD avec déploiement automatique sur staging puis production.
6. **Opérations** : Monitoring avec Prometheus et Grafana.
7. **Maintenance** : Issues de bug et hotfixes via le pipeline.

4.3 Méthodologies de gestion de projets collaboratifs

4.3.1 Scrum et DevOps

Scrum et DevOps sont complémentaires :

TABLE 4.1 – Comparaison Scrum et DevOps

Aspect	**Scrum**	**DevOps**
Périmètre	Développement	Dev + Ops
Cycle	Sprints (2–4 semaines)	Continu
Focus	Processus d'équipe	Flux de bout en bout
Rôles	Scrum Master, PO, Dev	Pas de rôles prédéfinis
Livraison	Fin de sprint	Continue (multiple/jour)

4.4 Kanban dans un contexte DevOps

4.4.1 Principes de Kanban

Kanban est une méthode de gestion visuelle du flux de travail, issue du système de production Toyota.

Les 4 principes fondamentaux de Kanban :

1. **Commencer avec ce qui existe** : ne pas imposer un changement radical, mais améliorer progressivement le processus existant.
2. **Accepter les changements incrémentaux** : les améliorations se font par petits pas, de manière évolutive.
3. **Respecter le processus actuel** : les rôles, responsabilités et titres existants sont conservés.
4. **Encourager le leadership à tous les niveaux** : chaque membre de l'équipe peut identifier et proposer des améliorations.

Les 6 pratiques de Kanban :

1. **Visualiser le flux de travail** : utiliser un tableau Kanban avec des colonnes représentant chaque étape du processus.
2. **Limiter le travail en cours (WIP)** : fixer un nombre maximum de tâches par colonne pour éviter la surcharge.
3. **Gérer le flux** : surveiller et optimiser le flux de travail pour réduire le temps de cycle.
4. **Rendre les règles explicites** : documenter les critères de passage d'une colonne à l'autre.
5. **Implémenter des boucles de feedback** : réunions régulières pour analyser les métriques et s'améliorer.
6. **S'améliorer collaborativement** : utiliser les données du tableau pour identifier les améliorations.

4.4.2 Tableau Kanban DevOps pour BijouxShop

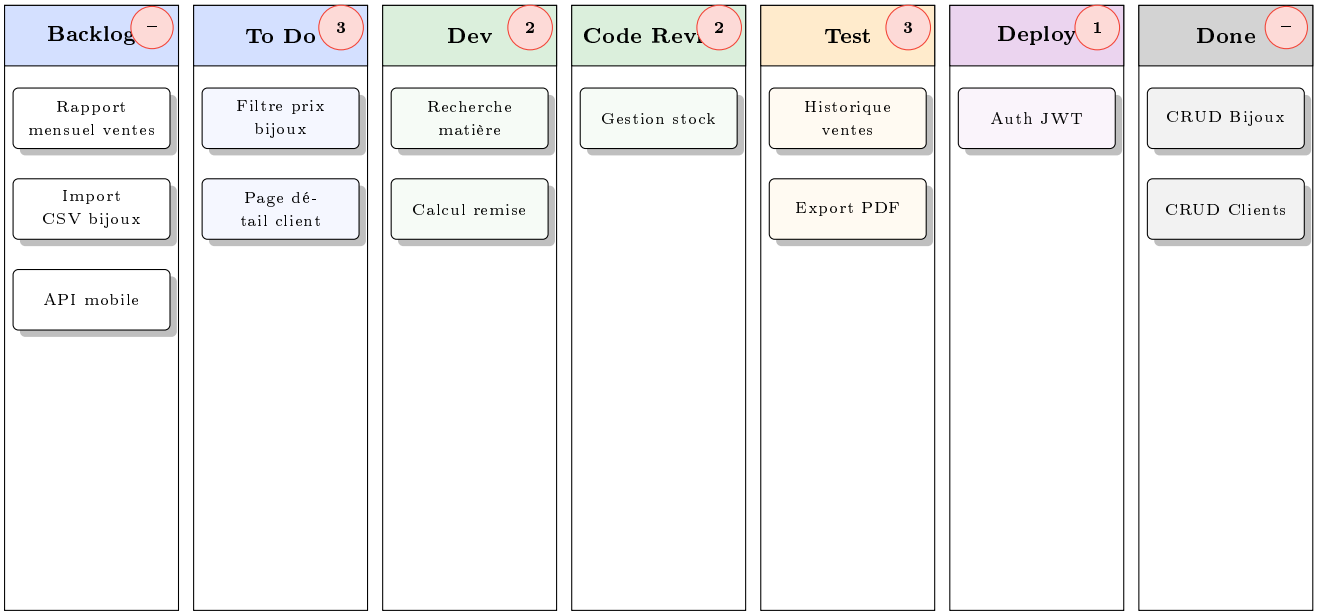


FIGURE 4.2 – Tableau Kanban DevOps pour BijouxShop

4.4.3 Métriques Kanban

— **Lead Time** : temps total entre la demande et la livraison en production.

— **Cycle Time** : temps entre le début du travail sur une tâche et sa finalisation.

— **Throughput** : nombre de tâches terminées par unité de temps.

— **WIP (Work In Progress)** : nombre de tâches en cours à un instant donné.

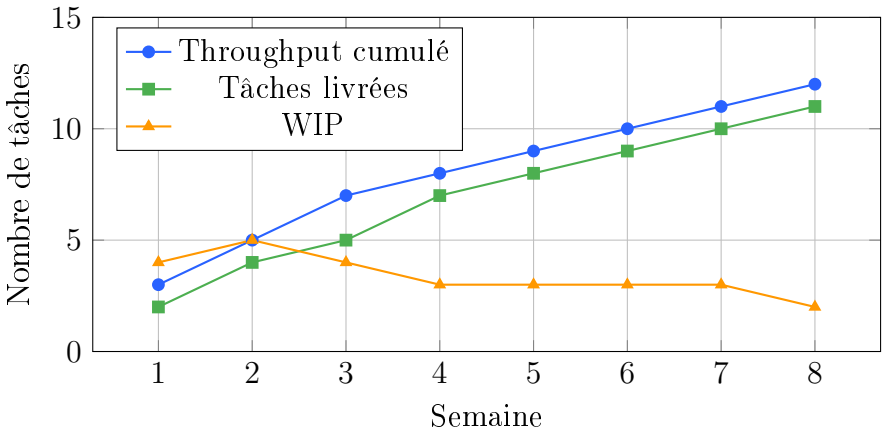


FIGURE 4.3 – Métriques Kanban du projet BijouxShop

4.4.4 Gestion de projet avec GitLab Issues

GitLab fournit des outils intégrés pour la gestion de projet Kanban :

```
# Creer une issue
glab issue create --title "feat: Ajouter filtre recherche bijoux" \
  --description "Permettre la recherche par matiere et prix" \
  --label "feature,priority::high" \
  --milestone "Sprint 3"

# Lister les issues
glab issue list --label "feature"

# Board Kanban
# Configuration dans GitLab > Board
# Colonnes basees sur les labels :
# - Open, To Do, Doing, Code Review, Testing, Done
```

Listing 4.1 – Gestion des issues GitLab via CLI

4.5 Résumé du chapitre

Points clés à retenir

- L'ALM couvre tout le cycle de vie, de la conception au retrait.
- DevOps enrichit l'ALM avec l'automatisation et le feedback continu.
- Kanban optimise les flux DevOps grâce à la visualisation et la limitation du WIP.
- GitLab intègre nativement les outils de gestion de projet (Issues, Boards, Milestones).

Chapitre 5

Conteneurisation avec Docker

5.1 Objectifs du chapitre

À l'issue de ce chapitre, l'étudiant sera capable de :

— Comprendre la différence entre virtualisation et conteneurisation.

— Maîtriser les concepts fondamentaux de Docker.

— Créer des Dockerfiles pour des applications Spring Boot.

— Utiliser Docker Compose pour orchestrer des services.

— Appliquer les bonnes pratiques Docker dans un contexte DevOps.

5.2 Virtualisation vs Conteneurisation

5.2.1 La virtualisation traditionnelle

La virtualisation traditionnelle utilise un **hyperviseur** pour créer des machines virtuelles (VM), chacune exécutant son propre système d'exploitation complet.

5.2.2 La conteneurisation

La conteneurisation utilise le noyau du système d'exploitation hôte pour isoler les applications dans des conteneurs légers, sans nécessiter un OS complet pour chaque application.

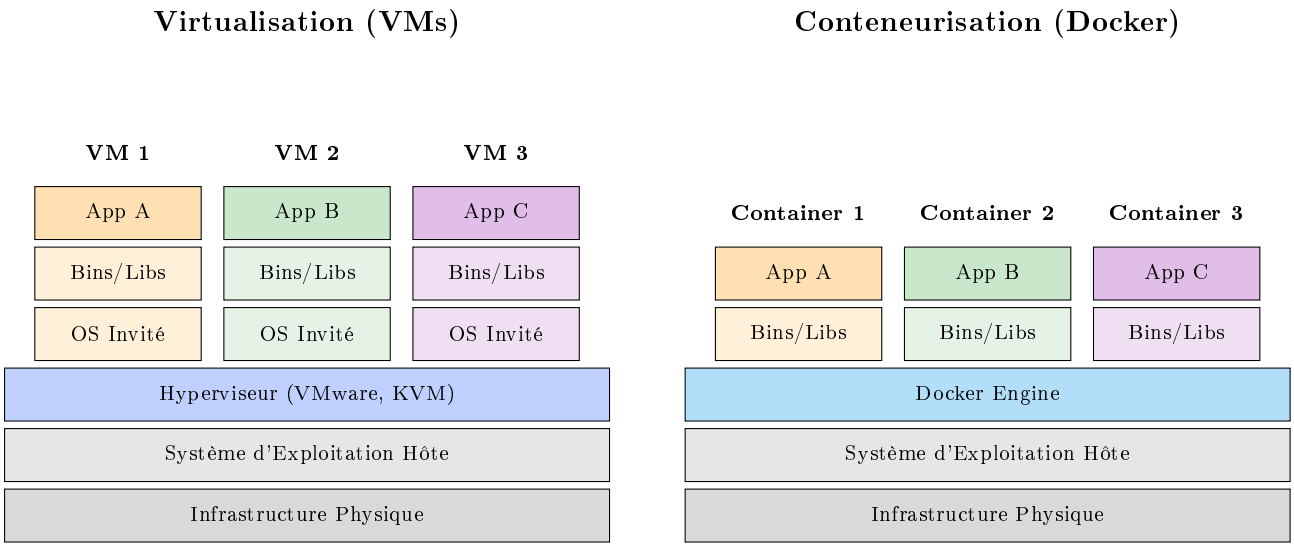


FIGURE 5.1 – Virtualisation vs Conteneurisation

TABLE 5.1 – Comparaison Virtualisation vs Conteneurisation

Critère	**Machine Virtuelle**	**Conteneur Docker**
Taille	Gigaoctets (OS complet)	Mégaoctets (app + libs)
Démarrage	Minutes	Secondes
Isolation	Complète (hyperviseur)	Processus (namespaces)
Performance	Overhead significatif	Quasi-natif
Portabilité	Limitée	Excellente
Densité	10–20 VMs/serveur	100+ conteneurs/serveur
OS	OS complet par VM	Partage le noyau hôte

5.3 Concepts fondamentaux de Docker

5.3.1 Architecture Docker

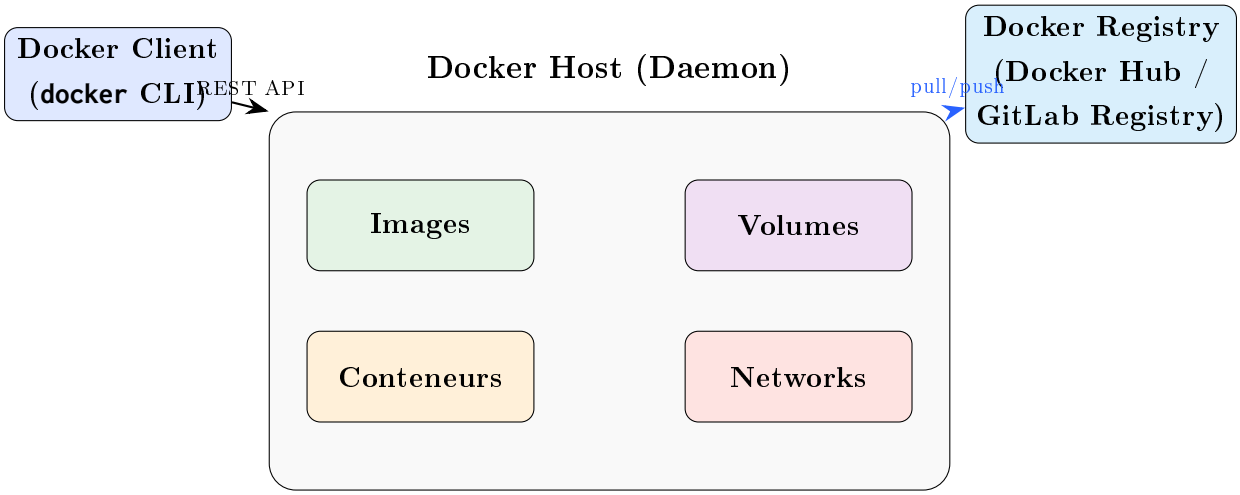


FIGURE 5.2 – Architecture de Docker

Concepts clés :

— **Image Docker** : un template en lecture seule qui contient le système de fichiers et la configuration nécessaires pour créer un conteneur. Les images sont construites par couches (*layers*).

— **Conteneur Docker** : une instance en cours d'exécution d'une image Docker. Un conteneur est isolé, léger et portable.

— **Dockerfile** : un fichier texte contenant les instructions pour construire une image Docker.

— **Docker Registry** : un dépôt de stockage et de distribution d'images Docker (Docker Hub, GitLab Container Registry, AWS ECR).

— **Volume** : un mécanisme de persistance des données en dehors du cycle de vie du conteneur.

— **Network** : un réseau virtuel permettant la communication entre conteneurs.

5.3.2 Le système de couches (Layers)

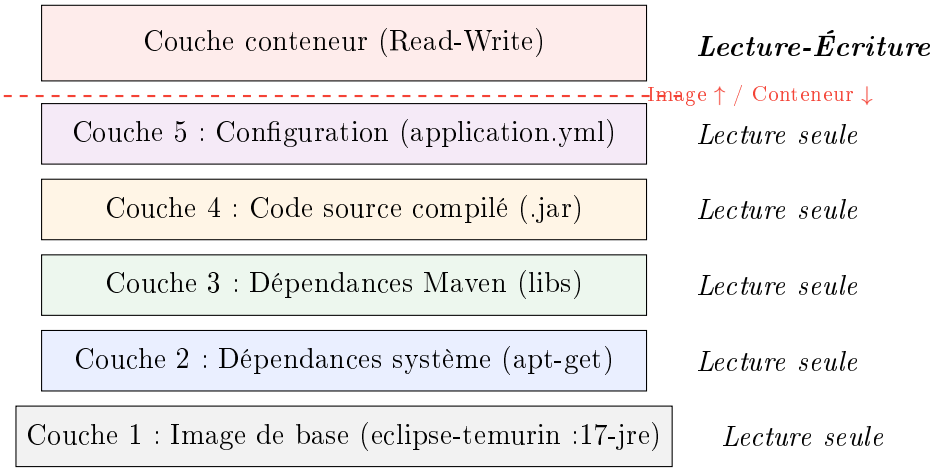


FIGURE 5.3 – Système de couches Docker

5.4 Dockerisation de BijouxShop

5.4.1 Dockerfile pour l'application Spring Boot

```
# ============================
# Stage 1 : Build
# ============================
FROM maven:3.9-eclipse-temurin-17 AS build

WORKDIR /app

# Copier les fichiers de dependances en premier (cache)
COPY pom.xml .
RUN mvn dependency:go-offline -B

# Copier le code source
COPY src ./src

# Construire l'application
RUN mvn clean package -DskipTests -B

# ============================
# Stage 2 : Runtime
# ============================
FROM eclipse-temurin:17-jre-alpine
```

```
22
23 WORKDIR /app
24
25 # Creer un utilisateur non-root
26 RUN addgroup -S appgroup && adduser -S appuser -G appgroup
27
28 # Copier le JAR depuis le stage de build
29 COPY --from=build /app/target/bijouxshop-*.jar app.jar
30
31 # Configurer les variables d'environnement
32 ENV SPRING_PROFILES_ACTIVE=docker
33 ENV JAVA_OPTS="-Xmx512m -Xms256m"
34
35 # Exposer le port
36 EXPOSE 8080
37
38 # Changer d'utilisateur
39 USER appuser
40
41 # Point d'entree
42 ENTRYPOINT ["sh", "-c", "java $JAVA_OPTS -jar app.jar"]
```

Listing 5.1 – Dockerfile multi-stage pour BijouxShop



[

Bonnes pratiques Dockerfile]

1. **Multi-stage build** : séparer le build du runtime pour réduire la taille de l'image.
2. **Utilisateur non-root** : ne jamais exécuter en tant que root en production.
3. **Cache des couches** : copier `pom.xml` avant le code source pour profiter du cache.
4. **Image Alpine** : utiliser des images légères (`-alpine`).
5. **.dockerignore** : exclure les fichiers inutiles.

t

5.4.2 Fichier .dockerignore

a
bget/ .git/ .gitignore .idea/ *.iml *.md docker-compose*.yml .env

5.4.3 Commandes Docker essentielles

```
# Construire l'image
docker build -t bijouxshop:1.0.0 .
docker build -t bijouxshop:latest .

# Lister les images
docker images

# Executer un conteneur
docker run -d \
  --name bijouxshop-app \
  -p 8080:8080 \
  -e SPRING_DATASOURCE_URL=jdbc:postgresql://db:5432/bijouxshop \
  bijouxshop:1.0.0

# Lister les conteneurs
docker ps          # en cours d'execution
docker ps -a       # tous les conteneurs

# Voir les logs
docker logs bijouxshop-app
docker logs -f bijouxshop-app  # suivre en temps reel

# Executer une commande dans un conteneur
docker exec -it bijouxshop-app /bin/sh

# Arreter et supprimer
docker stop bijouxshop-app
docker rm bijouxshop-app

# Supprimer une image
docker rmi bijouxshop:1.0.0

# Pousser vers un registry
docker tag bijouxshop:1.0.0
    registry.gitlab.com/hamzane/bijouxshop:1.0.0
docker push registry.gitlab.com/hamzane/bijouxshop:1.0.0
```

Listing 5.2 – Commandes Docker pour BijouxShop

5.4.4 Docker Compose pour BijouxShop

Docker Compose permet de définir et exécuter des applications multi-conteneurs.

```yaml
version: '3.8'

services:
  # Base de donnees PostgreSQL
  db:
    image: postgres:16-alpine
    container_name: bijouxshop-db
    environment:
      POSTGRES_DB: bijouxshop
      POSTGRES_USER: bijouxshop_user
      POSTGRES_PASSWORD: bijouxshop_pass
    volumes:
      - postgres_data:/var/lib/postgresql/data
      -
          ./src/main/resources/schema.sql:/docker-entrypoint-initdb.d/init.sql
    ports:
      - "5432:5432"
    networks:
      - bijouxshop-network
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U bijouxshop_user -d
         bijouxshop"]
      interval: 10s
      timeout: 5s
      retries: 5

  # Application Spring Boot
  app:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: bijouxshop-app
    depends_on:
      db:
        condition: service_healthy
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://db:5432/bijouxshop
      SPRING_DATASOURCE_USERNAME: bijouxshop_user
```

```
      SPRING_DATASOURCE_PASSWORD: bijouxshop_pass
      SPRING_PROFILES_ACTIVE: docker
    ports:
      - "8080:8080"
    networks:
      - bijouxshop-network
    restart: unless-stopped

volumes:
  postgres_data:

networks:
  bijouxshop-network:
    driver: bridge
```

Listing 5.3 – docker-compose.yml pour BijouxShop

```
# Demarrer tous les services
docker compose up -d

# Voir les logs de tous les services
docker compose logs -f

# Verifier l'etat des services
docker compose ps

# Arreter tous les services
docker compose down

# Arreter et supprimer les volumes
docker compose down -v

# Reconstruire les images
docker compose up -d --build
```

Listing 5.4 – Commandes Docker Compose

5.5 Résumé du chapitre

Points clés à retenir

- La conteneurisation est plus légère et plus rapide que la virtualisation.
- Docker utilise des images (template) pour créer des conteneurs (instances).
- Le Dockerfile définit les instructions de construction d'une image.
- Le multi-stage build réduit la taille des images de production.
- Docker Compose orchestre des applications multi-conteneurs.
- BijouxShop utilise deux conteneurs : l'application Spring Boot et PostgreSQL.

Chapitre 6

Intégration Continue, Livraison Continue et Déploiement Continu (CI/CD)

6.1 Objectifs du chapitre

À l'issue de ce chapitre, l'étudiant sera capable de :

— Différencier CI, CD (Livraison Continue) et CD (Déploiement Continu).

— Concevoir un pipeline CI/CD complet.

— Comprendre les concepts de stages, jobs et artifacts.

— Mettre en œuvre un pipeline GitLab CI/CD pour BijouxShop.

6.2 Concepts de CI/CD

6.2.1 Intégration Continue (CI)

Définition : Intégration Continue

L'Intégration Continue (CI) est une pratique de développement où les développeurs intègrent fréquemment leur code dans un dépôt partagé (plusieurs fois par jour). Chaque intégration est vérifiée par un build automatisé et des tests pour détecter les erreurs le plus tôt possible.

Principes de la CI :

1. Maintenir un dépôt de code source unique.
2. Automatiser le build.
3. Rendre le build auto-testable.
4. Chaque commit déclenche un build sur la machine d'intégration.
5. Garder le build rapide.
6. Tester dans un clone de l'environnement de production.
7. Rendre les derniers livrables facilement accessibles.
8. Tout le monde peut voir les résultats du dernier build.

6.2.2 Livraison Continue vs Déploiement Continu

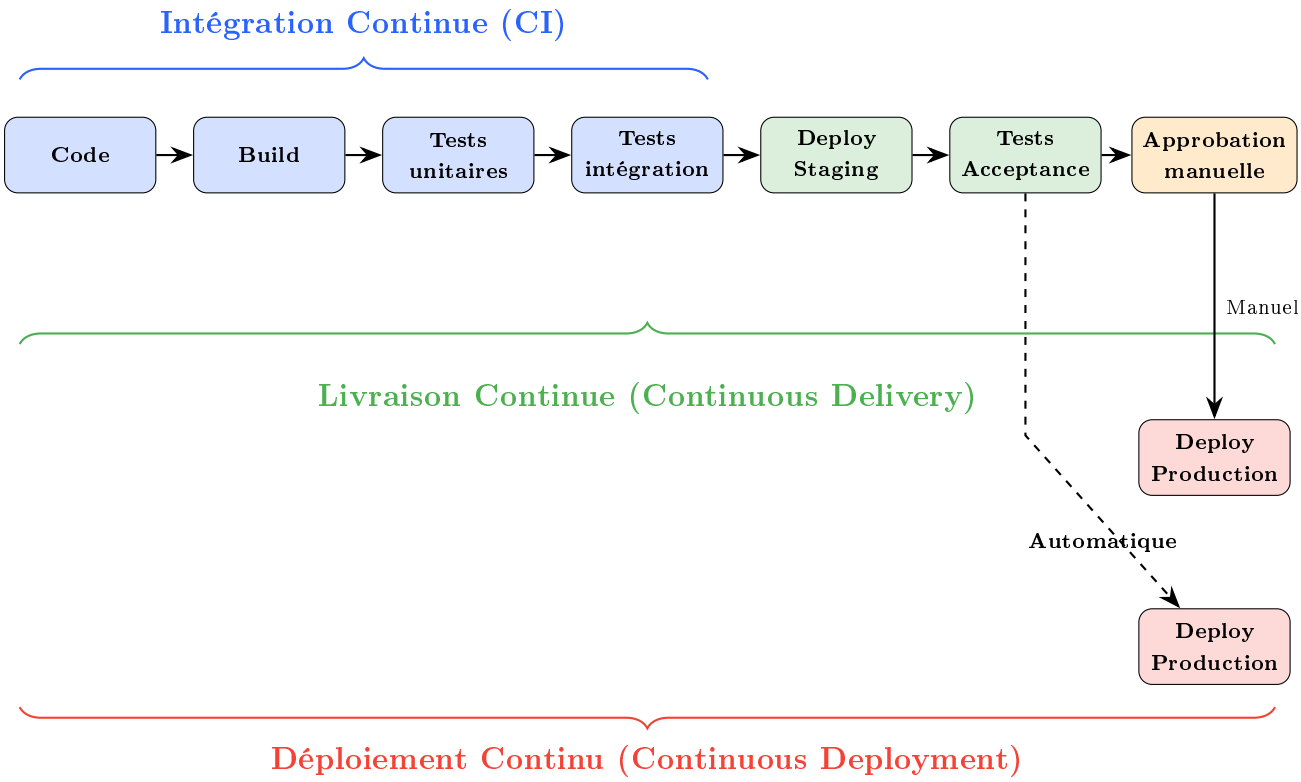


FIGURE 6.1 – CI vs Livraison Continue vs Déploiement Continu

6.3 Pipeline CI/CD avec GitLab

6.3.1 Concepts GitLab CI/CD

— **Pipeline** : l'ensemble du processus CI/CD, composé de stages et de jobs.

— **Stage** : une phase du pipeline (build, test, deploy). Les stages s'exécutent séquentiellement.

— **Job** : une tâche individuelle au sein d'un stage. Les jobs d'un même stage s'exécutent en parallèle.

— **Runner** : l'agent qui exécute les jobs. Peut être partagé ou dédié.

— **Artifact** : un fichier produit par un job et passé aux jobs suivants.

— **Variable** : une variable d'environnement utilisée dans le pipeline.

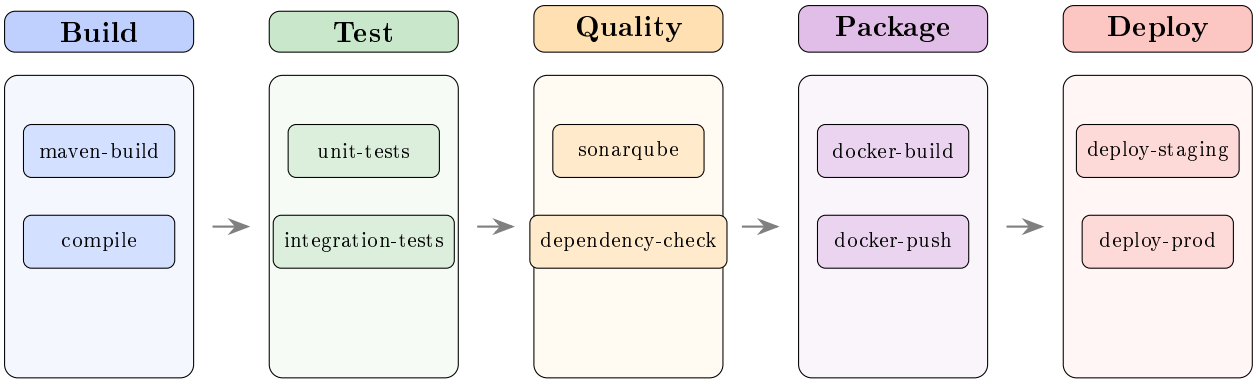


FIGURE 6.2 – Structure d'un pipeline GitLab CI/CD

6.3.2 Le fichier .gitlab-ci.yml

```yaml
# Variables globales
variables:
  MAVEN_OPTS: "-Dmaven.repo.local=.m2/repository"
  DOCKER_IMAGE: registry.gitlab.com/hamzane/bijouxshop
  POSTGRES_DB: bijouxshop_test
  POSTGRES_USER: test_user
  POSTGRES_PASSWORD: test_pass

# Cache Maven
cache:
  paths:
    - .m2/repository/

# Stages du pipeline
stages:
  - build
  - test
  - quality
  - package
  - deploy-staging
  - deploy-production

# ============================
# Stage: Build
# ============================
build:
  stage: build
  image: maven:3.9-eclipse-temurin-17
```

```
29    script:
30      - mvn clean compile -B
31    artifacts:
32      paths:
33        - target/
34      expire_in: 1 hour
35
36  # ============================
37  # Stage: Test
38  # ============================
39  unit-tests:
40    stage: test
41    image: maven:3.9-eclipse-temurin-17
42    script:
43      - mvn test -B
44    artifacts:
45      when: always
46      reports:
47        junit: target/surefire-reports/TEST-*.xml
48      paths:
49        - target/surefire-reports/
50
51  integration-tests:
52    stage: test
53    image: maven:3.9-eclipse-temurin-17
54    services:
55      - name: postgres:16-alpine
56        alias: db
57    variables:
58      SPRING_DATASOURCE_URL:
          "jdbc:postgresql://db:5432/bijouxshop_test"
59      SPRING_DATASOURCE_USERNAME: test_user
60      SPRING_DATASOURCE_PASSWORD: test_pass
61    script:
62      - mvn verify -P integration-tests -B
63    artifacts:
64      when: always
65      reports:
66        junit: target/failsafe-reports/TEST-*.xml
67
68  # ============================
```

```
69 # Stage: Quality
70 # ============================
71 sonarqube-analysis:
72   stage: quality
73   image: maven:3.9-eclipse-temurin-17
74   variables:
75     SONAR_HOST_URL: $SONAR_HOST_URL
76     SONAR_TOKEN: $SONAR_TOKEN
77   script:
78     - mvn sonar:sonar
79         -Dsonar.projectKey=bijouxshop
80         -Dsonar.host.url=$SONAR_HOST_URL
81         -Dsonar.login=$SONAR_TOKEN
82   allow_failure: true
83
84 dependency-check:
85   stage: quality
86   image: maven:3.9-eclipse-temurin-17
87   script:
88     - mvn org.owasp:dependency-check-maven:check
89   artifacts:
90     paths:
91       - target/dependency-check-report.html
92     expire_in: 1 week
93   allow_failure: true
94
95 # ============================
96 # Stage: Package (Docker)
97 # ============================
98 docker-build:
99   stage: package
100  image: docker:24
101  services:
102    - docker:24-dind
103  variables:
104    DOCKER_TLS_CERTDIR: "/certs"
105  before_script:
106    - docker login -u $CI_REGISTRY_USER
107        -p $CI_REGISTRY_PASSWORD $CI_REGISTRY
108  script:
109    - docker build -t $DOCKER_IMAGE:$CI_COMMIT_SHA .
```

```
110     - docker tag $DOCKER_IMAGE:$CI_COMMIT_SHA
111         $DOCKER_IMAGE:latest
112     - docker push $DOCKER_IMAGE:$CI_COMMIT_SHA
113     - docker push $DOCKER_IMAGE:latest
114   only:
115     - main
116     - develop
117
118 # ============================
119 # Stage: Deploy Staging
120 # ============================
121 deploy-staging:
122   stage: deploy-staging
123   image: alpine:latest
124   before_script:
125     - apk add --no-cache openssh-client
126     - eval $(ssh-agent -s)
127     - echo "$SSH_PRIVATE_KEY" | ssh-add -
128   script:
129     - ssh -o StrictHostKeyChecking=no
130         deploy@staging.bijouxshop.com
131         "docker pull $DOCKER_IMAGE:$CI_COMMIT_SHA &&
132          docker compose -f /opt/bijouxshop/docker-compose.yml up
                -d"
133   environment:
134     name: staging
135     url: https://staging.bijouxshop.com
136   only:
137     - develop
138
139 # ============================
140 # Stage: Deploy Production
141 # ============================
142 deploy-production:
143   stage: deploy-production
144   image: alpine:latest
145   before_script:
146     - apk add --no-cache openssh-client
147     - eval $(ssh-agent -s)
148     - echo "$SSH_PRIVATE_KEY" | ssh-add -
149   script:
```

```
150     - ssh -o StrictHostKeyChecking=no
151         deploy@prod.bijouxshop.com
152         "docker pull $DOCKER_IMAGE:$CI_COMMIT_SHA &&
153          docker compose -f /opt/bijouxshop/docker-compose.yml up
                -d"
154   environment:
155     name: production
156     url: https://www.bijouxshop.com
157   when: manual
158   only:
159     - main
```

Listing 6.1 – .gitlab-ci.yml complet pour BijouxShop

6.3.3 GitLab Runners

Un **Runner** est un agent qui exécute les jobs du pipeline CI/CD. GitLab propose deux types de runners :

TABLE 6.1 – Types de GitLab Runners

Type	**Shared Runner**	**Specific Runner**
Disponibilité	Partagé entre tous les projets	Dédié à un projet ou groupe
Configuration	Géré par l'admin GitLab	Configuré par l'équipe projet
Usage	Projets standards	Besoins spécifiques (GPU, etc.)
Isolation	Moyenne	Forte

```
1  # Installer le Runner
2  curl -L
       https://packages.gitlab.com/install/repositories/runner/gitlab-runner/scrip
       | sudo bash
3  sudo apt-get install gitlab-runner
4
5  # Enregistrer le Runner
6  sudo gitlab-runner register \
7    --url "https://gitlab.com/" \
8    --registration-token "PROJECT_TOKEN" \
9    --executor "docker" \
10   --docker-image "alpine:latest" \
11   --description "BijouxShop Runner" \
12   --tag-list "docker,bijouxshop"
```

```
13
14 # Verifier le statut
15 sudo gitlab-runner status
16 sudo gitlab-runner list
```

Listing 6.2 – Installation et enregistrement d'un Runner GitLab

6.3.4 Variables d'environnement GitLab

Les variables d'environnement permettent de stocker des informations sensibles et de configurer le pipeline sans les coder en dur.

TABLE 6.2 – Variables d'environnement GitLab pour BijouxShop

Variable	Type	Description
`CI_REGISTRY`	Prédéfinie	URL du registry GitLab
`CI_COMMIT_SHA`	Prédéfinie	Hash du commit courant
`CI_PIPELINE_ID`	Prédéfinie	ID du pipeline courant
`SONAR_TOKEN`	Personnalisée	Token d'authentification SonarQube
`SSH_PRIVATE_KEY`	Personnalisée	Clé SSH pour le déploiement
`DB_PASSWORD`	Personnalisée	Mot de passe de la base (masqué)
`AWS_ACCESS_KEY_ID`	Personnalisée	Clé d'accès AWS



[

Sécurité des variables]

— Toujours **masquer** les variables sensibles (option « Mask variable » dans GitLab).

— Utiliser l'option **Protected** pour limiter l'accès aux branches protégées.

— Ne jamais stocker de secrets dans le fichier `.gitlab-ci.yml`.

— Utiliser des **scopes d'environnement** pour différencier staging et production.

6.4 Résumé du chapitre

Chapitre 7

Tests dans un Contexte DevOps

7.1 Objectifs du chapitre

À l'issue de ce chapitre, l'étudiant sera capable de :

— Concevoir une stratégie de test adaptée à DevOps.

— Distinguer les différents niveaux de tests (unitaire, intégration, E2E).

— Intégrer les tests dans un pipeline CI/CD.

— Écrire des tests pour une application Spring Boot.

7.2 La pyramide des tests

La pyramide des tests est un modèle qui guide la proportion de chaque type de test dans une application.

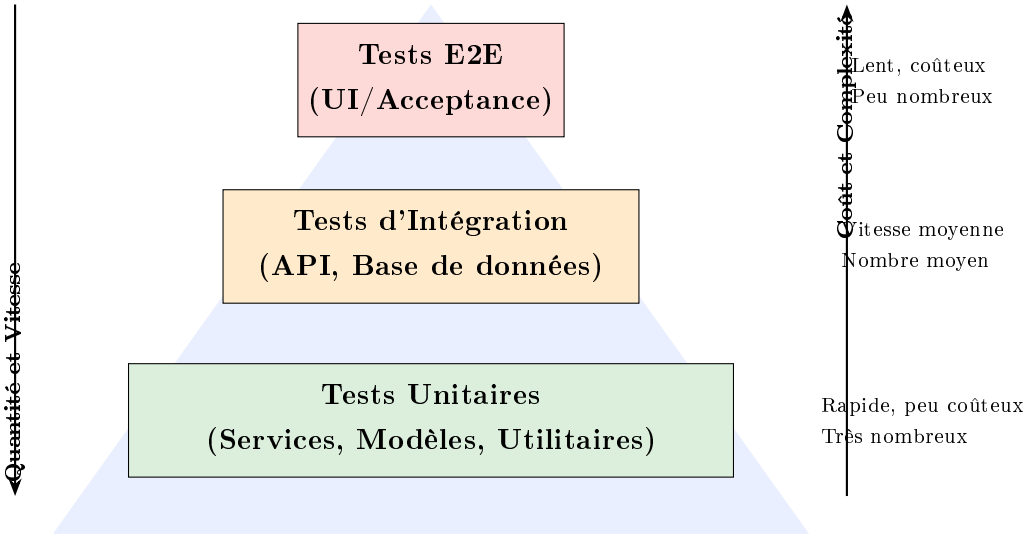


FIGURE 7.1 – La pyramide des tests

7.3 Tests unitaires avec JUnit 5

```java
package com.bijouxshop.service;

import com.bijouxshop.model.Bijou;
import com.bijouxshop.model.Bijou.CategorieBijou;
import com.bijouxshop.repository.BijouRepository;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import java.math.BigDecimal;
import java.util.Arrays;
import java.util.List;
import java.util.Optional;

import static org.assertj.core.api.Assertions.*;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
class BijouServiceTest {

    @Mock
    private BijouRepository bijouRepository;

    @InjectMocks
    private BijouService bijouService;

    private Bijou bague;
    private Bijou collier;

    @BeforeEach
    void setUp() {
        bague = new Bijou("Bague Solitaire",
            CategorieBijou.BAGUE, "Or 18K",
            new BigDecimal("1500.00"), 10);
        bague.setId(1L);

        collier = new Bijou("Collier Perles",
```

```
42              CategorieBijou.COLLIER, "Argent 925",
43              new BigDecimal("350.00"), 25);
44          collier.setId(2L);
45      }
46
47      @Test
48      @DisplayName("Rechercher tous les bijoux")
49      void shouldReturnAllBijoux() {
50          when(bijouRepository.findAll())
51              .thenReturn(Arrays.asList(bague, collier));
52
53          List<Bijou> result = bijouService.findAll();
54
55          assertThat(result).hasSize(2);
56          assertThat(result).containsExactly(bague, collier);
57          verify(bijouRepository, times(1)).findAll();
58      }
59
60      @Test
61      @DisplayName("Rechercher un bijou par ID")
62      void shouldReturnBijouById() {
63          when(bijouRepository.findById(1L))
64              .thenReturn(Optional.of(bague));
65
66          Optional<Bijou> result = bijouService.findById(1L);
67
68          assertThat(result).isPresent();
69          assertThat(result.get().getNom())
70              .isEqualTo("Bague Solitaire");
71      }
72
73      @Test
74      @DisplayName("Rechercher par categorie")
75      void shouldReturnBijouxByCategorie() {
76          when(bijouRepository.findByCategorie(CategorieBijou.BAGUE))
77              .thenReturn(List.of(bague));
78
79          List<Bijou> result =
80              bijouService.findByCategorie(CategorieBijou.BAGUE);
81
82          assertThat(result).hasSize(1);
```

```
83          assertThat(result.get(0).getCategorie())
84              .isEqualTo(CategorieBijou.BAGUE);
85      }
86
87      @Test
88      @DisplayName("Verifier la diminution du stock apres vente")
89      void shouldDecreaseStockAfterSale() {
90          when(bijouRepository.findById(1L))
91              .thenReturn(Optional.of(bague));
92          when(bijouRepository.save(any(Bijou.class)))
93              .thenReturn(bague);
94
95          bijouService.decreaseStock(1L, 3);
96
97          assertThat(bague.getStock()).isEqualTo(7);
98          verify(bijouRepository).save(bague);
99      }
100
101     @Test
102     @DisplayName("Lever une exception si stock insuffisant")
103     void shouldThrowExceptionWhenInsufficientStock() {
104         when(bijouRepository.findById(1L))
105             .thenReturn(Optional.of(bague));
106
107         assertThatThrownBy(
108             () -> bijouService.decreaseStock(1L, 15))
109             .isInstanceOf(IllegalStateException.class)
110             .hasMessageContaining("Stock insuffisant");
111     }
112 }
```

Listing 7.1 – Tests unitaires pour BijouService

7.4 Tests d'intégration

```
1  package com.bijouxshop;
2
3  import com.bijouxshop.model.Bijou;
4  import com.bijouxshop.model.Bijou.CategorieBijou;
5  import com.bijouxshop.repository.BijouRepository;
```

```
6  import org.junit.jupiter.api.Test;
7  import org.springframework.beans.factory.annotation.Autowired;
8  import org.springframework.boot.test.autoconfigure.web.servlet
9      .AutoConfigureMockMvc;
10 import org.springframework.boot.test.context.SpringBootTest;
11 import org.springframework.http.MediaType;
12 import org.springframework.test.context.DynamicPropertyRegistry;
13 import org.springframework.test.context.DynamicPropertySource;
14 import org.springframework.test.web.servlet.MockMvc;
15 import org.testcontainers.containers.PostgreSQLContainer;
16 import org.testcontainers.junit.jupiter.Container;
17 import org.testcontainers.junit.jupiter.Testcontainers;
18
19 import java.math.BigDecimal;
20
21 import static org.springframework.test.web.servlet
22     .request.MockMvcRequestBuilders.*;
23 import static org.springframework.test.web.servlet
24     .result.MockMvcResultMatchers.*;
25
26 @SpringBootTest
27 @AutoConfigureMockMvc
28 @Testcontainers
29 class BijouIntegrationTest {
30
31     @Container
32     static PostgreSQLContainer<?> postgres =
33         new PostgreSQLContainer<>("postgres:16-alpine")
34             .withDatabaseName("bijouxshop_test")
35             .withUsername("test")
36             .withPassword("test");
37
38     @DynamicPropertySource
39     static void configureProperties(
40             DynamicPropertyRegistry registry) {
41         registry.add("spring.datasource.url",
42             postgres::getJdbcUrl);
43         registry.add("spring.datasource.username",
44             postgres::getUsername);
45         registry.add("spring.datasource.password",
46             postgres::getPassword);
```

```
47      }
48
49      @Autowired
50      private MockMvc mockMvc;
51
52      @Autowired
53      private BijouRepository bijouRepository;
54
55      @Test
56      void shouldCreateAndRetrieveBijou() throws Exception {
57          String bijouJson = """
58              {
59                "nom": "Bracelet Diamant",
60                "categorie": "BRACELET",
61                "matiere": "Or 18K",
62                "prix": 2500.00,
63                "stock": 5
64              }
65              """;
66
67          mockMvc.perform(post("/api/bijoux")
68                  .contentType(MediaType.APPLICATION_JSON)
69                  .content(bijouJson))
70              .andExpect(status().isCreated())
71              .andExpect(jsonPath("$.nom")
72                  .value("Bracelet Diamant"));
73
74          mockMvc.perform(get("/api/bijoux"))
75              .andExpect(status().isOk())
76              .andExpect(jsonPath("$.length()").value(1));
77      }
78  }
```

Listing 7.2 – Tests d'intégration avec Spring Boot et Testcontainers

7.5 Intégration des tests dans le pipeline CI/CD

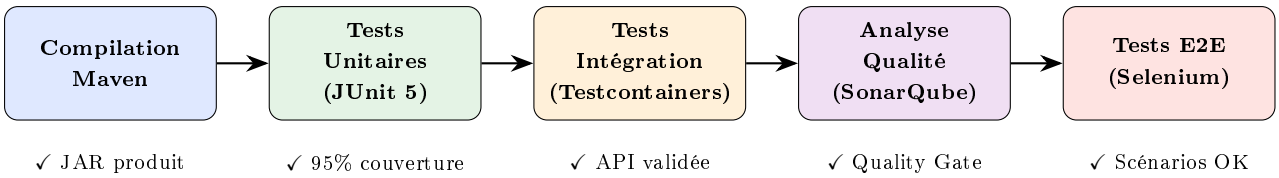


FIGURE 7.2 – Tests dans le pipeline CI/CD de BijouxShop

7.6 Résumé du chapitre

Points clés à retenir

- La pyramide des tests guide la proportion : beaucoup de tests unitaires, moins d'intégration, peu d'E2E.
- Les tests unitaires utilisent Mockito pour isoler les dépendances.
- Les tests d'intégration utilisent Testcontainers pour des bases de données réelles.
- Chaque type de test a sa place dans le pipeline CI/CD.
- Le feedback rapide des tests est essentiel en DevOps.

Chapitre 8

Orchestration avec Kubernetes

8.1 Objectifs du chapitre

À l'issue de ce chapitre, l'étudiant sera capable de :

— Comprendre pourquoi l'orchestration de conteneurs est nécessaire.

— Maîtriser l'architecture de Kubernetes (Master/Worker).

— Déployer une application sur Kubernetes (Pods, Services, Deployments).

— Intégrer Kubernetes dans un pipeline CI/CD.

8.2 De Docker à Kubernetes : pourquoi orchestrer ?

Docker résout le problème de l'empaquetage et de l'exécution d'applications dans des conteneurs. Mais en production, on doit gérer :

— **Scalabilité** : comment exécuter plusieurs instances d'un conteneur ?

— **Haute disponibilité** : que se passe-t-il si un conteneur tombe en panne ?

— **Load balancing** : comment répartir le trafic entre les instances ?

— **Rolling updates** : comment mettre à jour sans interruption de service ?

— **Service discovery** : comment les conteneurs se trouvent-ils mutuellement ?

Définition : Kubernetes (K8s)

Kubernetes est une plateforme open source d'orchestration de conteneurs, développée initialement par Google et maintenant maintenue par la CNCF (*Cloud Native Computing Foundation*). Elle automatise le déploiement, la mise à l'échelle et la gestion des applications conteneurisées.

8.3 Architecture de Kubernetes

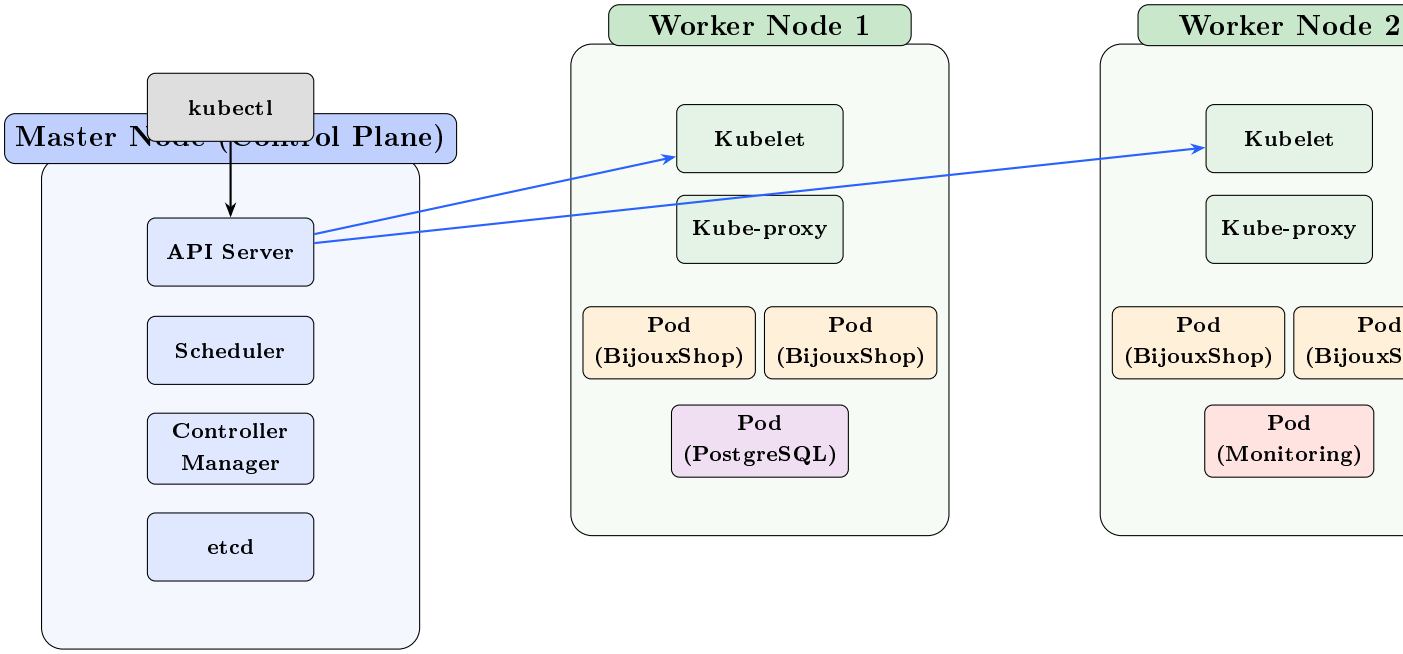


FIGURE 8.1 – Architecture de Kubernetes : Master Node et Worker Nodes

8.3.1 Master Node (Control Plane)

Le Master Node est le cerveau du cluster Kubernetes :

— **API Server** : point d'entrée pour toutes les requêtes REST. C'est le composant central avec lequel interagissent `kubectl`, les Kubelets et les autres composants.

— **Scheduler** : décide sur quel Worker Node un nouveau Pod sera placé, en fonction des ressources disponibles, des contraintes et des affinités.

— **Controller Manager** : exécute les contrôleurs qui surveillent l'état du cluster et s'assurent que l'état actuel correspond à l'état désiré (ReplicaSet Controller, Deployment Controller, etc.).

— **etcd** : base de données clé-valeur distribuée qui stocke toute la configuration et l'état du cluster.

8.3.2 Worker Nodes

Les Worker Nodes exécutent les applications conteneurisées :

— **Kubelet** : agent qui s'exécute sur chaque Worker Node. Il reçoit les instructions du Master et s'assure que les conteneurs sont en cours d'exécution.

— **Kube-proxy** : gère les règles réseau sur chaque nœud, permettant la communication entre les Pods et avec l'extérieur.

— **Container Runtime** : moteur d'exécution des conteneurs (Docker, containerd, CRI-O).

8.4 Les objets Kubernetes

8.4.1 Pods

Définition : Pod

Un Pod est la plus petite unité déployable dans Kubernetes. Il encapsule un ou plusieurs conteneurs qui partagent le même réseau, le même stockage et la même spécification de cycle de vie.

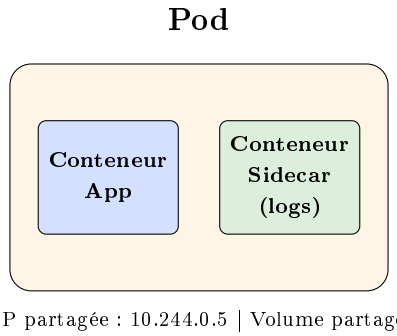


FIGURE 8.2 – Structure d'un Pod Kubernetes

8.4.2 Du conteneur au Pod : la transition

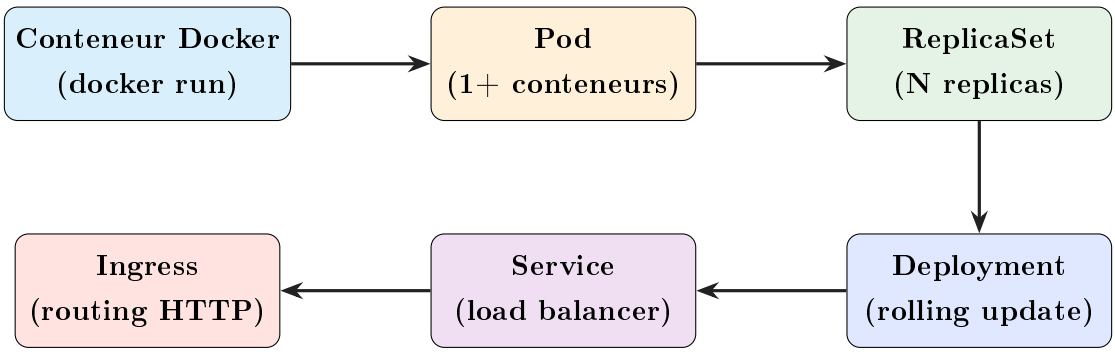


FIGURE 8.3 – Du conteneur Docker au déploiement Kubernetes

8.5 Déploiement de BijouxShop sur Kubernetes

8.5.1 Deployment de l'application

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: bijouxshop-app
  namespace: bijouxshop
  labels:
    app: bijouxshop
    component: backend
spec:
  replicas: 3
  selector:
    matchLabels:
      app: bijouxshop
      component: backend
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  template:
    metadata:
      labels:
        app: bijouxshop
        component: backend
    spec:
      containers:
        - name: bijouxshop
          image: registry.gitlab.com/hamzane/bijouxshop:latest
          ports:
            - containerPort: 8080
          env:
            - name: SPRING_DATASOURCE_URL
              value:
                 "jdbc:postgresql://bijouxshop-db:5432/bijouxshop"
            - name: SPRING_DATASOURCE_USERNAME
              valueFrom:
                secretKeyRef:
                  name: bijouxshop-db-secret
                  key: username
            - name: SPRING_DATASOURCE_PASSWORD
              valueFrom:
```

```
41                 secretKeyRef:
42                   name: bijouxshop-db-secret
43                   key: password
44           resources:
45             requests:
46               memory: "256Mi"
47               cpu: "250m"
48             limits:
49               memory: "512Mi"
50               cpu: "500m"
51           readinessProbe:
52             httpGet:
53               path: /actuator/health
54               port: 8080
55             initialDelaySeconds: 30
56             periodSeconds: 10
57           livenessProbe:
58             httpGet:
59               path: /actuator/health
60               port: 8080
61             initialDelaySeconds: 60
62             periodSeconds: 15
```

Listing 8.1 – k8s/deployment.yaml – Deployment BijouxShop

8.5.2 Service et Ingress

```
1  apiVersion: v1
2  kind: Service
3  metadata:
4    name: bijouxshop-service
5    namespace: bijouxshop
6  spec:
7    selector:
8      app: bijouxshop
9      component: backend
10   ports:
11     - protocol: TCP
12       port: 80
13       targetPort: 8080
14   type: ClusterIP
```

```yaml
---
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: bijouxshop-ingress
  namespace: bijouxshop
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  rules:
    - host: bijouxshop.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: bijouxshop-service
                port:
                  number: 80
```

Listing 8.2 – k8s/service.yaml – Service BijouxShop

8.5.3 Déploiement PostgreSQL avec StatefulSet

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: bijouxshop-db-secret
  namespace: bijouxshop
type: Opaque
data:
  username: Ymlqb3V4c2hvcF91c2Vy  # base64
  password: Ymlqb3V4c2hvcF9wYXNz  # base64
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: bijouxshop-db
  namespace: bijouxshop
```

```
spec:
  serviceName: bijouxshop-db
  replicas: 1
  selector:
    matchLabels:
      app: bijouxshop
      component: database
  template:
    metadata:
      labels:
        app: bijouxshop
        component: database
    spec:
      containers:
        - name: postgres
          image: postgres:16-alpine
          ports:
            - containerPort: 5432
          env:
            - name: POSTGRES_DB
              value: bijouxshop
            - name: POSTGRES_USER
              valueFrom:
                secretKeyRef:
                  name: bijouxshop-db-secret
                  key: username
            - name: POSTGRES_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: bijouxshop-db-secret
                  key: password
          volumeMounts:
            - name: postgres-storage
              mountPath: /var/lib/postgresql/data
          resources:
            requests:
              memory: "256Mi"
              cpu: "250m"
            limits:
              memory: "512Mi"
              cpu: "500m"
```

```
  volumeClaimTemplates:
    - metadata:
        name: postgres-storage
      spec:
        accessModes: ["ReadWriteOnce"]
        resources:
          requests:
            storage: 10Gi
---
apiVersion: v1
kind: Service
metadata:
  name: bijouxshop-db
  namespace: bijouxshop
spec:
  selector:
    app: bijouxshop
    component: database
  ports:
    - port: 5432
      targetPort: 5432
  clusterIP: None
```

Listing 8.3 – k8s/postgres.yaml – PostgreSQL StatefulSet

8.5.4 Commandes kubectl essentielles

```
# Creer le namespace
kubectl create namespace bijouxshop

# Appliquer les manifestes
kubectl apply -f k8s/

# Verifier les deployments
kubectl get deployments -n bijouxshop
kubectl get pods -n bijouxshop
kubectl get services -n bijouxshop

# Voir les logs d'un pod
kubectl logs -f <pod-name> -n bijouxshop

```

```
# Acceder a un pod
kubectl exec -it <pod-name> -n bijouxshop -- /bin/sh

# Mettre a l'echelle
kubectl scale deployment bijouxshop-app \
  --replicas=5 -n bijouxshop

# Mise a jour de l'image
kubectl set image deployment/bijouxshop-app \
  bijouxshop=registry.gitlab.com/hamzane/bijouxshop:v2.0 \
  -n bijouxshop

# Rollback
kubectl rollout undo deployment/bijouxshop-app -n bijouxshop
kubectl rollout history deployment/bijouxshop-app -n bijouxshop
```

Listing 8.4 – Commandes kubectl pour BijouxShop

8.6 Résumé du chapitre

Points clés à retenir

- Kubernetes orchestre les conteneurs : scaling, HA, rolling updates, service discovery.
- Architecture Master/Worker : le Master (Control Plane) gère, les Workers exécutent.
- Pod = unité de base. Deployment = gestion des replicas et mises à jour.
- Services exposent les Pods, Ingress route le trafic HTTP externe.
- StatefulSet pour les bases de données avec stockage persistant.

Chapitre 9

DevSecOps : Sécurité dans le Pipeline

9.1 Objectifs du chapitre

À l'issue de ce chapitre, l'étudiant sera capable de :

— Comprendre l'approche DevSecOps (« Shift Left Security »).

— Connaître le OWASP Top 10 et ses implications.

— Intégrer SonarQube dans le pipeline CI/CD.

— Mettre en place des scans de vulnérabilités automatisés.

9.2 Introduction à DevSecOps

Définition : DevSecOps

DevSecOps est l'intégration de la sécurité (Sec) dans chaque étape du cycle de vie DevOps. Plutôt que de traiter la sécurité comme une phase distincte en fin de projet, elle est intégrée dès le début (« Shift Left »), automatisée et partagée par toute l'équipe.

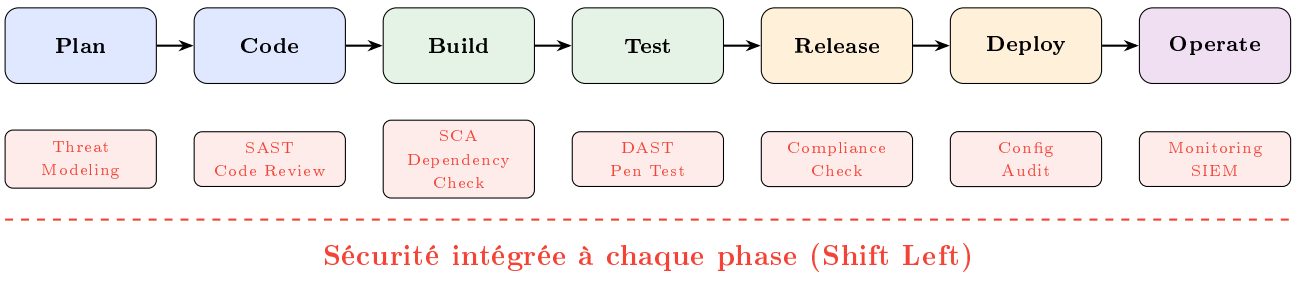


FIGURE 9.1 – DevSecOps : la sécurité à chaque phase

9.3 OWASP Top 10

Le **OWASP Top 10** est une liste des dix risques de sécurité les plus critiques pour les applications web, mise à jour régulièrement par l'*Open Web Application Security Project*.

TABLE 9.1 – OWASP Top 10 (2021)

#	**Risque**	**Description et exemple BijouxShop**
A01	Broken Access Control	Un client accède aux ventes d'un autre client via manipulation d'URL
A02	Cryptographic Failures	Mots de passe stockés en clair dans la base PostgreSQL
A03	Injection	Injection SQL dans la recherche de bijoux par nom
A04	Insecure Design	Absence de limite de tentatives de connexion
A05	Security Misconfiguration	Endpoints actuator Spring Boot exposés publiquement
A06	Vulnerable Components	Utilisation d'une version de Spring Boot avec CVE connue
A07	Authentication Failures	Session JWT sans expiration
A08	Data Integrity Failures	Désérialisation non sécurisée des objets Java
A09	Logging Failures	Absence de logs pour les tentatives d'accès échouées
A10	SSRF	L'application effectue des requêtes HTTP vers des URLs fournies par l'utilisateur

9.3.1 Prévention dans BijouxShop

```
// A03 - Prevention d'injection SQL
// MAUVAIS - vulnerable a l'injection SQL
@Query(value = "SELECT * FROM bijoux WHERE nom = '"
    + nom + "'", nativeQuery = true)  // NE JAMAIS FAIRE

// BON - utilisation de parametres prepares
@Query("SELECT b FROM Bijou b WHERE b.nom = :nom")
List<Bijou> findByNom(@Param("nom") String nom);

// A01 - Controle d'acces
@PreAuthorize("hasRole('ADMIN') or #clientId == "
    + "authentication.principal.id")
public List<Vente> getVentesByClient(
    @PathVariable Long clientId) {
    return venteService.findByClientId(clientId);
}

// A02 - Hashage des mots de passe
@Bean
```

```
20 public PasswordEncoder passwordEncoder() {
21     return new BCryptPasswordEncoder(12);
22 }
23
24 // A05 - Securiser les actuators Spring Boot
25 // Dans application.yml :
26 // management.endpoints.web.exposure.include: health,info
27 // management.endpoint.health.show-details: when-authorized
```

Listing 9.1 – Exemples de prévention OWASP dans BijouxShop

9.4 SonarQube : Analyse de qualité et sécurité du code

9.4.1 Les phases d'analyse SonarQube

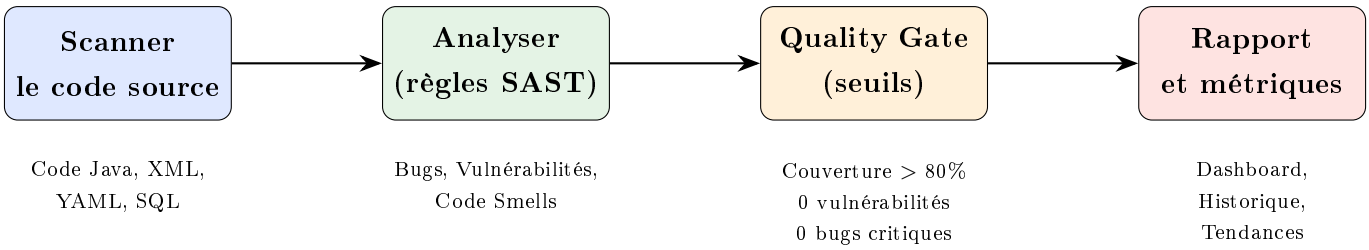


FIGURE 9.2 – Phases d'analyse SonarQube

9.4.2 Métriques SonarQube

- **Bugs** : erreurs de code qui produisent un comportement incorrect.
- **Vulnerabilities** : failles de sécurité dans le code.
- **Code Smells** : problèmes de maintenabilité du code.
- **Coverage** : pourcentage de code couvert par les tests.
- **Duplications** : pourcentage de code dupliqué.
- **Technical Debt** : temps estimé pour corriger tous les problèmes.

9.4.3 Configuration SonarQube dans le pipeline

```
1 sonarqube-analysis:
2   stage: quality
3   image: maven:3.9-eclipse-temurin-17
4   variables:
```

```
    SONAR_HOST_URL: $SONAR_HOST_URL
    SONAR_TOKEN: $SONAR_TOKEN
  script:
    - mvn verify sonar:sonar
        -Dsonar.projectKey=bijouxshop
        -Dsonar.projectName="BijouxShop"
        -Dsonar.host.url=$SONAR_HOST_URL
        -Dsonar.login=$SONAR_TOKEN
        -Dsonar.java.coveragePlugin=jacoco
        -Dsonar.coverage.jacoco.xmlReportPaths=target/site/jacoco/jacoco.xml
        -Dsonar.qualitygate.wait=true
  allow_failure: false
```

Listing 9.2 – Configuration SonarQube dans .gitlab-ci.yml

9.5 Scan de vulnérabilités des dépendances

```
dependency-check:
  stage: quality
  image: maven:3.9-eclipse-temurin-17
  script:
    - mvn org.owasp:dependency-check-maven:check
        -DfailBuildOnCVSS=7
        -DsuppressionFile=owasp-suppressions.xml
  artifacts:
    paths:
      - target/dependency-check-report.html
    when: always
    expire_in: 1 week
```

Listing 9.3 – OWASP Dependency-Check dans le pipeline

9.6 Scan d'images Docker

```
container-scan:
  stage: quality
  image:
    name: aquasec/trivy:latest
    entrypoint: [""]
```

```
6   script:
7     - trivy image --exit-code 1 --severity HIGH,CRITICAL
8         $DOCKER_IMAGE:$CI_COMMIT_SHA
9   allow_failure: true
```

Listing 9.4 – Scan de sécurité des images Docker avec Trivy

9.7 Résumé du chapitre

Points clés à retenir

- DevSecOps = sécurité intégrée à chaque phase du pipeline.
- « Shift Left » : détecter les vulnérabilités le plus tôt possible.
- OWASP Top 10 : les 10 risques de sécurité web les plus critiques.
- SonarQube : analyse statique (SAST) pour bugs, vulnérabilités et code smells.
- OWASP Dependency-Check : détection des composants vulnérables.
- Trivy : scan de sécurité des images Docker.

Chapitre 10

Infrastructure as Code et Cloud AWS

10.1 Objectifs du chapitre

À l'issue de ce chapitre, l'étudiant sera capable de :

— Comprendre les principes de l'Infrastructure as Code (IaC).

— Différencier les outils de provisionnement et de gestion de configuration.

— Comprendre la relation entre Cloud et DevOps.

— Déployer une application sur AWS avec les services managés.

10.2 Infrastructure as Code (IaC)

10.2.1 Principes de l'IaC

Définition : Infrastructure as Code

L'Infrastructure as Code (IaC) est la pratique consistant à gérer et provisionner l'infrastructure informatique (serveurs, réseaux, stockage) à travers du code et des fichiers de configuration, plutôt que par des processus manuels ou des outils interactifs.

Avantages de l'IaC :

— **Reproductibilité** : créer des environnements identiques à chaque fois.

— **Versionnement** : l'infrastructure est versionnée avec Git.

— **Automatisation** : provisionnement automatique, sans intervention manuelle.

— **Documentation** : le code est la documentation de l'infrastructure.

— **Auditabilité** : chaque changement est traçable.

10.2.2 Approches déclarative vs impérative

TABLE 10.1 – Approche déclarative vs impérative

Aspect	Déclarative	Impérative
Principe	« Ce que je veux »	« Comment le faire »
Exemple	Terraform, CloudFormation	Scripts Bash, Ansible (ad-hoc)
Idempotence	Native	À implémenter
Avantage	Prévisible, reproductible	Flexible, contrôle fin

10.2.3 Outils IaC

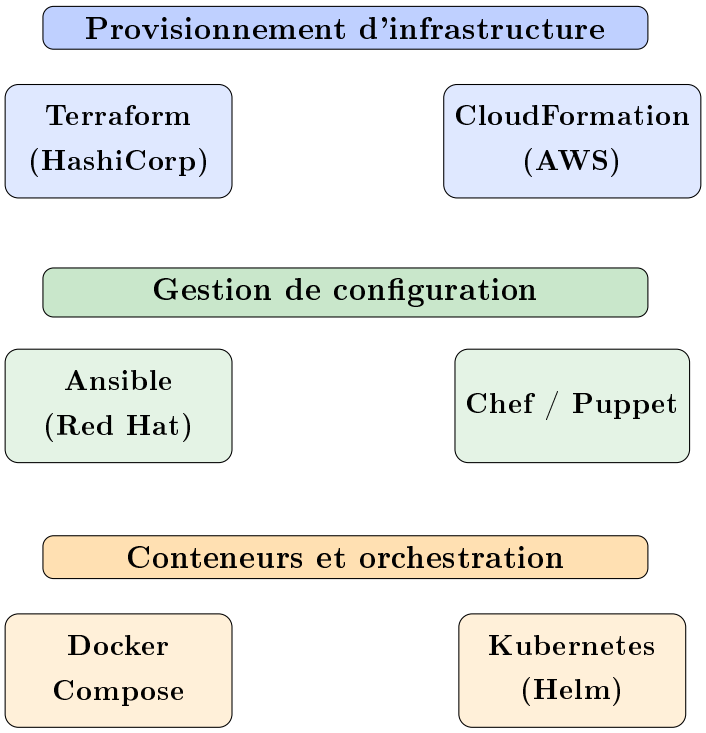


FIGURE 10.1 – Catégories d'outils IaC

10.3 Terraform pour BijouxShop

```
# main.tf - Infrastructure BijouxShop sur AWS

terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
```

```
    }
  }
  backend "s3" {
    bucket = "bijouxshop-terraform-state"
    key    = "prod/terraform.tfstate"
    region = "eu-west-3"
  }
}

provider "aws" {
  region = var.aws_region
}

# VPC
resource "aws_vpc" "bijouxshop" {
  cidr_block           = "10.0.0.0/16"
  enable_dns_support   = true
  enable_dns_hostnames = true
  tags = {
    Name        = "bijouxshop-vpc"
    Environment = var.environment
  }
}

# Subnets
resource "aws_subnet" "public" {
  count                   = 2
  vpc_id                  = aws_vpc.bijouxshop.id
  cidr_block              = "10.0.${count.index + 1}.0/24"
  availability_zone       =
     data.aws_availability_zones.available.names[count.index]
  map_public_ip_on_launch = true
  tags = {
    Name = "bijouxshop-public-${count.index + 1}"
  }
}

resource "aws_subnet" "private" {
  count             = 2
  vpc_id            = aws_vpc.bijouxshop.id
  cidr_block        = "10.0.${count.index + 10}.0/24"
```

```
48    availability_zone =
          data.aws_availability_zones.available.names[count.index]
49    tags = {
50      Name = "bijouxshop-private-${count.index + 1}"
51    }
52  }
53
54  # RDS PostgreSQL
55  resource "aws_db_instance" "bijouxshop" {
56    identifier             = "bijouxshop-db"
57    engine                 = "postgres"
58    engine_version         = "16"
59    instance_class         = "db.t3.micro"
60    allocated_storage      = 20
61    db_name                = "bijouxshop"
62    username               = var.db_username
63    password               = var.db_password
64    vpc_security_group_ids = [aws_security_group.db.id]
65    db_subnet_group_name   = aws_db_subnet_group.bijouxshop.name
66    skip_final_snapshot    = true
67    tags = {
68      Name = "bijouxshop-database"
69    }
70  }
71
72  # ECS Cluster pour les conteneurs
73  resource "aws_ecs_cluster" "bijouxshop" {
74    name = "bijouxshop-cluster"
75  }
```

Listing 10.1 – Infrastructure Terraform pour BijouxShop sur AWS

10.4 Cloud AWS et DevOps

10.4.1 Relation entre Cloud et DevOps

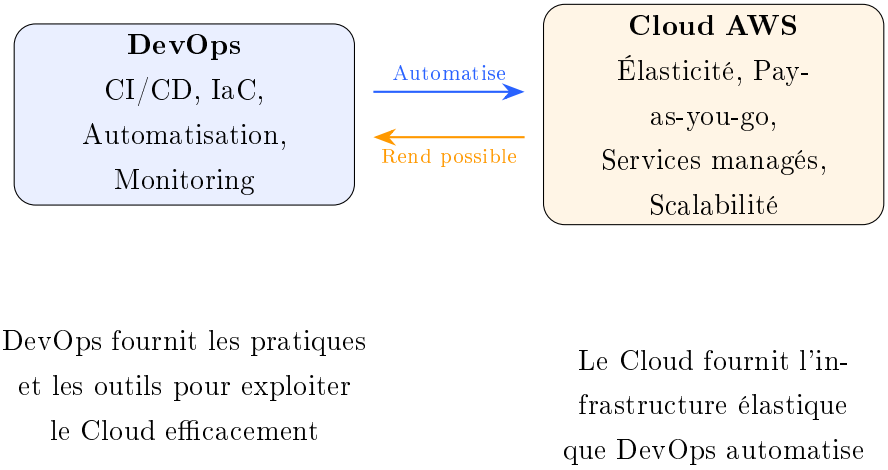


FIGURE 10.2 – Synergie entre DevOps et Cloud

Le Cloud et DevOps sont complémentaires :

— Le Cloud fournit des **ressources à la demande** que DevOps automatise via l'IaC.

— Le modèle **pay-as-you-go** du Cloud s'aligne avec l'approche DevOps de l'optimisation continue.

— Les **services managés** (RDS, ECS, EKS) réduisent la charge opérationnelle.

— L'**élasticité** du Cloud permet le scaling automatique piloté par le monitoring DevOps.

10.4.2 Services AWS pour DevOps

TABLE 10.2 – Services AWS pertinents pour BijouxShop

Service AWS	Catégorie	Usage BijouxShop
EC2	Compute	Serveurs pour le Runner GitLab
ECS / EKS	Orchestration	Exécution des conteneurs BijouxShop
ECR	Registry	Stockage des images Docker
RDS	Base de données	PostgreSQL managé
S3	Stockage	Terraform state, backups, assets
CloudWatch	Monitoring	Logs et métriques applicatives
IAM	Sécurité	Gestion des accès et rôles
VPC	Réseau	Isolation réseau de l'infrastructure
ALB	Load Balancing	Répartition du trafic entre instances
Route 53	DNS	Gestion du domaine bijouxshop.com

10.4.3 Architecture AWS pour BijouxShop

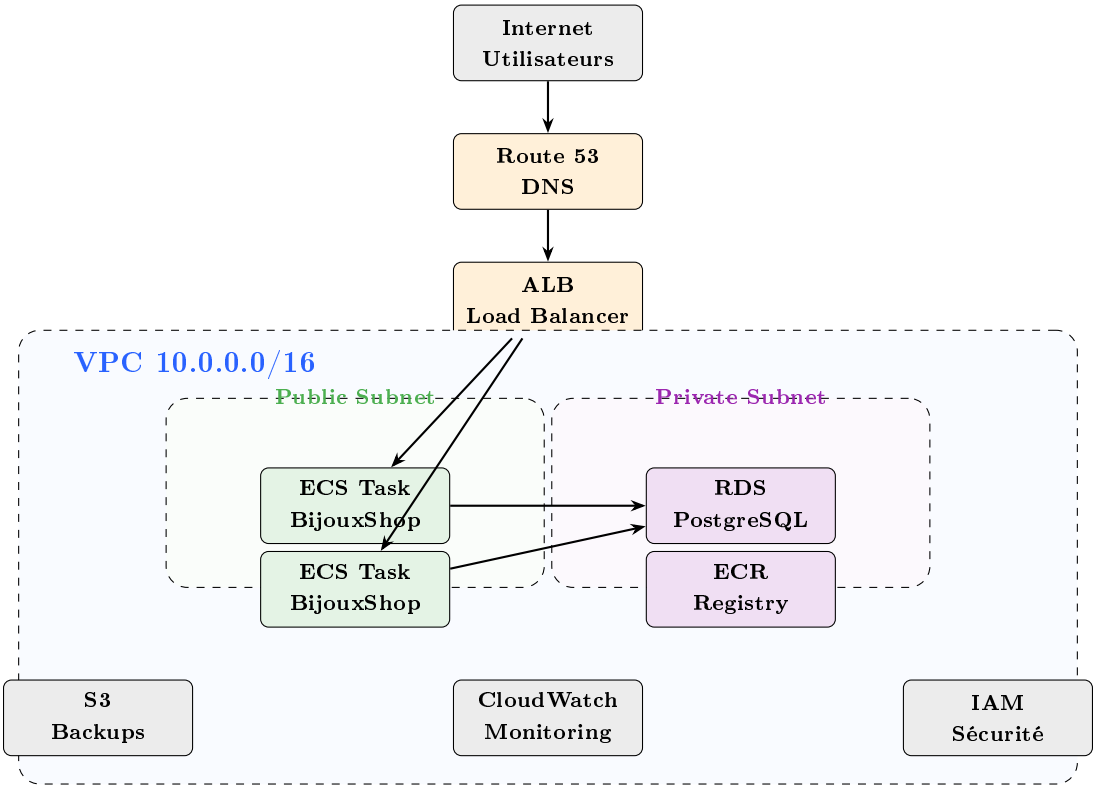


FIGURE 10.3 – Architecture AWS pour BijouxShop

10.4.4 Déploiement sur AWS via GitLab CI

```
deploy-aws:
  stage: deploy-production
  image: amazon/aws-cli:latest
  variables:
    AWS_DEFAULT_REGION: eu-west-3
  before_script:
    - aws configure set aws_access_key_id $AWS_ACCESS_KEY_ID
    - aws configure set aws_secret_access_key
       $AWS_SECRET_ACCESS_KEY
  script:
    # Pousser l'image vers ECR
    - aws ecr get-login-password --region $AWS_DEFAULT_REGION
        | docker login --username AWS
        --password-stdin
           $AWS_ACCOUNT_ID.dkr.ecr.$AWS_DEFAULT_REGION.amazonaws.com
    - docker tag $DOCKER_IMAGE:$CI_COMMIT_SHA
        $AWS_ACCOUNT_ID.dkr.ecr.$AWS_DEFAULT_REGION.amazonaws.com/bijouxshop:$
    - docker push
       $AWS_ACCOUNT_ID.dkr.ecr.$AWS_DEFAULT_REGION.amazonaws.com/bijouxshop:$C

    # Mettre a jour le service ECS
    - aws ecs update-service
        --cluster bijouxshop-cluster
        --service bijouxshop-service
        --force-new-deployment
  environment:
    name: production
    url: https://www.bijouxshop.com
  when: manual
  only:
    - main
```

Listing 10.2 – Déploiement sur AWS ECS dans .gitlab-ci.yml

10.5 Gestion de configuration avec Ansible

```
# playbook.yml
---
- name: Configurer le serveur BijouxShop
```

```yaml
  hosts: bijouxshop_servers
  become: yes

  vars:
    app_name: bijouxshop
    docker_image: "registry.gitlab.com/hamzane/bijouxshop"
    app_version: "latest"

  tasks:
    - name: Installer Docker
      apt:
        name:
          - docker.io
          - docker-compose-plugin
        state: present
        update_cache: yes

    - name: Demarrer Docker
      systemd:
        name: docker
        state: started
        enabled: yes

    - name: Copier docker-compose.yml
      template:
        src: docker-compose.yml.j2
        dest: /opt/{{ app_name }}/docker-compose.yml
        mode: '0644'

    - name: Demarrer l'application
      community.docker.docker_compose:
        project_src: /opt/{{ app_name }}
        state: present
        pull: yes
```

Listing 10.3 – Playbook Ansible pour configurer un serveur BijouxShop

10.6 Résumé du chapitre

Points clés à retenir

- IaC = infrastructure gérée par du code versionné (Terraform, Ansible).
- Approche déclarative (Terraform) vs impérative (scripts, Ansible ad-hoc).
- Cloud et DevOps sont synergiques : le Cloud fournit l'élasticité, DevOps l'automatise.
- AWS offre des services managés (RDS, ECS, ECR) qui simplifient les opérations.
- Le déploiement sur AWS peut être intégré dans le pipeline GitLab CI/CD.

Chapitre 11

Gestion des Opérations

11.1 Objectifs du chapitre

À l'issue de ce chapitre, l'étudiant sera capable de :

— Automatiser les opérations système dans un contexte DevOps.

— Mettre en place du monitoring et de l'observabilité.

— Gérer les incidents selon les pratiques DevOps.

— Implémenter la gestion des changements.

11.2 Automatisation des opérations système

L'automatisation des opérations est un pilier fondamental de DevOps. Elle vise à éliminer les tâches manuelles répétitives et à réduire les erreurs humaines.

11.2.1 Domaines d'automatisation opérationnelle

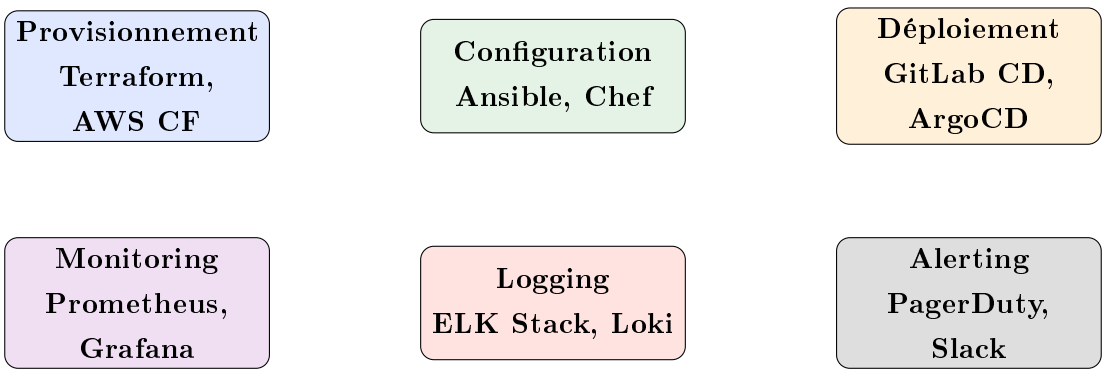


FIGURE 11.1 – Domaines d'automatisation opérationnelle

11.3 Monitoring et Observabilité

11.3.1 Les trois piliers de l'observabilité

L'observabilité repose sur trois piliers qui permettent de comprendre l'état interne d'un système :

1. **Métriques** : données numériques mesurées dans le temps (CPU, mémoire, temps de réponse, nombre de requêtes).
2. **Logs** : enregistrements textuels d'événements horodatés.
3. **Traces** : suivi du parcours d'une requête à travers les différents services.



FIGURE 11.2 – Les trois piliers de l'observabilité

11.3.2 Monitoring avec Prometheus et Grafana

```yaml
# prometheus.yml
global:
  scrape_interval: 15s
  evaluation_interval: 15s

scrape_configs:
  - job_name: 'bijouxshop'
    metrics_path: '/actuator/prometheus'
    static_configs:
      - targets: ['bijouxshop-app:8080']
        labels:
          app: 'bijouxshop'
          env: 'production'

  - job_name: 'postgres'
    static_configs:
```

```
      - targets: ['postgres-exporter:9187']

  - job_name: 'node'
    static_configs:
      - targets: ['node-exporter:9100']
```

Listing 11.1 – Configuration Prometheus pour BijouxShop

```
version: '3.8'

services:
  prometheus:
    image: prom/prometheus:latest
    container_name: prometheus
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml
      - prometheus_data:/prometheus
    ports:
      - "9090:9090"
    networks:
      - bijouxshop-network

  grafana:
    image: grafana/grafana:latest
    container_name: grafana
    environment:
      - GF_SECURITY_ADMIN_PASSWORD=admin123
    volumes:
      - grafana_data:/var/lib/grafana
    ports:
      - "3000:3000"
    depends_on:
      - prometheus
    networks:
      - bijouxshop-network

  node-exporter:
    image: prom/node-exporter:latest
    container_name: node-exporter
    ports:
      - "9100:9100"
```

```
    networks:
      - bijouxshop-network

volumes:
  prometheus_data:
  grafana_data:

networks:
  bijouxshop-network:
    external: true
```

Listing 11.2 – docker-compose.monitoring.yml

Pour que Spring Boot expose les métriques Prometheus, il faut ajouter les dépendances dans le `pom.xml` :

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
<dependency>
    <groupId>io.micrometer</groupId>
    <artifactId>micrometer-registry-prometheus</artifactId>
</dependency>
```

Listing 11.3 – Dépendances Actuator et Micrometer

Et configurer les endpoints dans `application.yml` :

```
management:
  endpoints:
    web:
      exposure:
        include: health,info,prometheus,metrics
  metrics:
    tags:
      application: bijouxshop
  endpoint:
    health:
      show-details: when-authorized
```

Listing 11.4 – Configuration Actuator

11.3.3 Métriques métier pour BijouxShop

```java
@Service
public class VenteService {

    private final Counter ventesCounter;
    private final Timer venteTimer;
    private final Gauge stockGauge;

    public VenteService(MeterRegistry registry,
                        BijouRepository bijouRepo) {
        this.ventesCounter =
           Counter.builder("bijouxshop.ventes.total")
            .description("Nombre total de ventes")
            .register(registry);

        this.venteTimer =
           Timer.builder("bijouxshop.ventes.duration")
            .description("Duree de traitement d'une vente")
            .register(registry);

        Gauge.builder("bijouxshop.stock.total",
                bijouRepo, repo -> repo.findAll()
                    .stream()
                    .mapToInt(Bijou::getStock)
                    .sum())
            .description("Stock total de bijoux")
            .register(registry);
    }

    public Vente creerVente(Vente vente) {
        return venteTimer.record(() -> {
            // Logique metier
            Vente saved = venteRepository.save(vente);
            ventesCounter.increment();
            return saved;
        });
    }
}
```

Listing 11.5 – Métriques métier personnalisées

11.4 Gestion des incidents

11.4.1 Processus de gestion des incidents DevOps

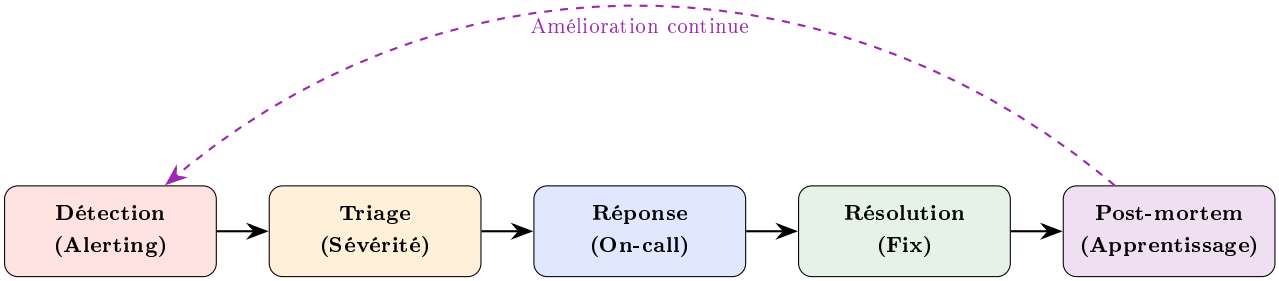


FIGURE 11.3 – Processus de gestion des incidents DevOps

11.4.2 Post-mortem blameless

Un post-mortem blameless est une analyse d'incident qui se concentre sur les **systèmes et les processus** plutôt que sur les individus.

Structure d'un post-mortem :

1. **Résumé de l'incident** : date, durée, impact, sévérité.
2. **Chronologie** : séquence détaillée des événements.
3. **Analyse des causes** : les « 5 pourquoi » pour identifier la cause racine.
4. **Actions correctives** : mesures à prendre pour éviter la récurrence.
5. **Leçons apprises** : ce qui a bien fonctionné et ce qui peut être amélioré.

11.5 Gestion des changements

11.5.1 Types de changements

TABLE 11.1 – Types de changements dans un environnement DevOps

Type	Description	Approbation
Standard	Changements pré-approuvés et à faible risque (ex : mise à jour de version mineure)	Automatique
Normal	Changements planifiés nécessitant une évaluation (ex : nouvelle fonctionnalité)	Merge Request
Urgence	Corrections critiques en production (ex : faille de sécurité)	Hotfix accéléré

11.6 Résumé du chapitre

Points clés à retenir

- L'automatisation des opérations couvre le provisionnement, la configuration, le déploiement et le monitoring.
- L'observabilité repose sur trois piliers : métriques, logs et traces.
- Prometheus + Grafana forment la stack de monitoring standard en DevOps.
- Les post-mortems blameless favorisent l'apprentissage plutôt que la punition.
- La gestion des changements en DevOps est intégrée dans le pipeline CI/CD.

Chapitre 12

Utilisation de GitLab dans un Projet Collaboratif

12.1 Objectifs du chapitre

À l'issue de ce chapitre, l'étudiant sera capable de :

— Créer et configurer un projet collaboratif sur GitLab.

— Utiliser les fonctionnalités de gestion de projet GitLab.

— Maîtriser le workflow des Merge Requests.

— Configurer un pipeline CI/CD complet sur GitLab.

12.2 Création du projet BijouxShop sur GitLab

12.2.1 Structure du projet

```
# Creer le projet sur GitLab
# GitLab > New Project > Create blank project
# Nom: bijouxshop
# Visibility: Private
# Initialize with README: Yes

# Cloner le projet
git clone https://gitlab.com/hamzane/bijouxshop.git
cd bijouxshop

# Configurer les branches protegees
# GitLab > Settings > Repository > Protected branches
# main: Maintainers can push, Developers can merge
# develop: Developers can push and merge

# Creer la branche develop
git checkout -b develop
```

```
git push -u origin develop
```

Listing 12.1 – Initialisation du projet sur GitLab

12.2.2 Configuration des membres et des rôles

TABLE 12.1 – Rôles GitLab pour l'équipe BijouxShop

Rôle GitLab	**Membre**	**Permissions**
Owner	Pr. Hamzane	Accès complet, gestion du projet
Maintainer	Lead Developer	Merge sur main, gestion CI/CD
Developer	Développeurs	Push sur develop, créer des MR
Reporter	Testeurs	Créer des issues, commenter
Guest	Stakeholders	Voir le projet, commenter

12.3 Gestion de projet avec les Issues GitLab

12.3.1 Templates d'issues

```
## Description
<!-- Description claire de la fonctionnalite -->

## Criteres d'acceptation
- [ ] Critere 1
- [ ] Critere 2
- [ ] Tests unitaires ecrits
- [ ] Tests d'integration ecrits
- [ ] Documentation mise a jour

## Maquette / Design
<!-- Screenshots ou liens vers les maquettes -->

## Impact technique
<!-- Services ou composants impactes -->

/label ~feature ~priority::medium
/milestone %"Sprint 3"
```

Listing 12.2 – .gitlab/issue_templates/Feature.md

12.3.2 Labels et Milestones

```
# Labels par type
feature     (bleu)    - Nouvelle fonctionnalite
bug         (rouge)   - Correction de bug
enhancement (vert)    - Amelioration
docs        (jaune)   - Documentation
ci          (violet)  - Pipeline CI/CD

# Labels par priorite
priority::critical  (rouge fonce)
priority::high      (orange)
priority::medium    (jaune)
priority::low       (gris)

# Labels par statut (pour le Board Kanban)
workflow::backlog
workflow::todo
workflow::doing
workflow::review
workflow::testing
workflow::done
```

Listing 12.3 – Configuration des labels GitLab

12.4 Workflow des Merge Requests

Le workflow des Merge Requests (MR) est au cœur de la collaboration DevOps sur GitLab.

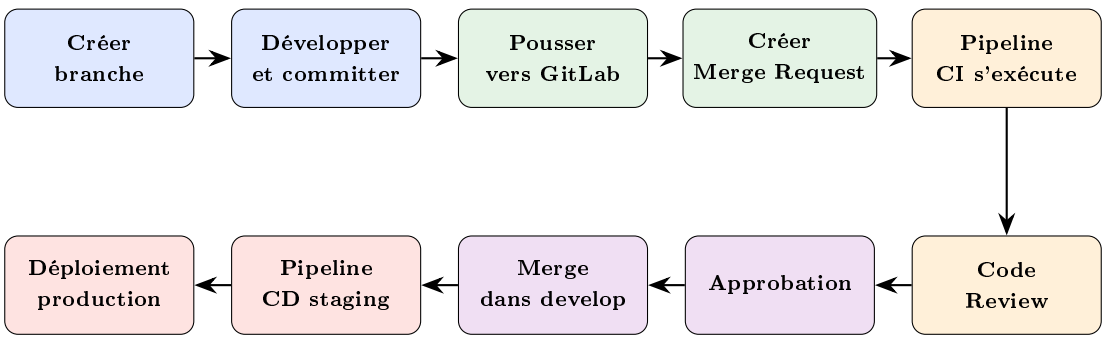


FIGURE 12.1 – Workflow des Merge Requests GitLab

12.4.1 Exemple de Merge Request

```
# 1. Creer la branche depuis develop
git checkout develop
git pull origin develop
git checkout -b feature/filtre-recherche-bijoux

# 2. Developper la fonctionnalite
# ... modifier les fichiers ...

# 3. Committer avec des messages conventionnels
git add .
git commit -m "feat(bijoux): ajouter filtre par matiere

- Nouveau endpoint GET /api/bijoux?matiere=Or
- Ajout de la methode findByMatiere dans le repository
- Tests unitaires et d'integration"

# 4. Pousser vers GitLab
git push -u origin feature/filtre-recherche-bijoux

# 5. Creer la MR via la CLI GitLab
glab mr create \
  --title "feat: Ajouter filtre de recherche par matiere" \
  --description "## Description
Ajout de la possibilite de filtrer les bijoux par matiere.

## Changements
- Nouveau endpoint REST
- Tests unitaires et d'integration
- Documentation API mise a jour

## Captures d'ecran
N/A

Closes #42" \
  --target-branch develop \
  --assignee @hamzane \
  --reviewer @lead-dev \
  --label "feature,priority::high"
```

Listing 12.4 – Création d'une Merge Request

12.5 Pipeline CI/CD complet : vue d'ensemble

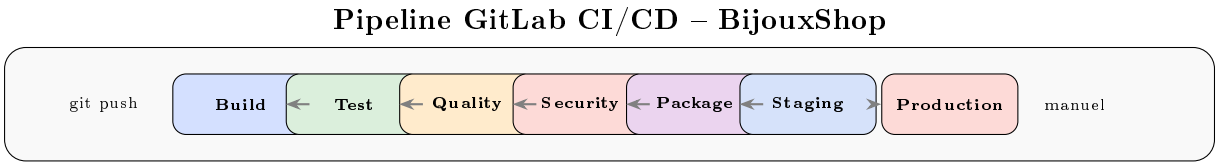


FIGURE 12.2 – Pipeline CI/CD complet de BijouxShop

12.6 Résumé du chapitre

Points clés à retenir

— GitLab offre une plateforme intégrée pour le DevOps complet.

— Les Issues, Boards et Milestones permettent la gestion de projet Kanban.

— Les Merge Requests sont au cœur du workflow collaboratif.

— Le pipeline CI/CD automatise tout le processus du commit à la production.

— Les Runners, variables et environnements configurent l'exécution du pipeline.

Chapitre 13

La Culture DevOps

13.1 Objectifs du chapitre

À l'issue de ce chapitre, l'étudiant sera capable de :

— Identifier les caractéristiques clés de la culture DevOps.

— Comprendre les modèles organisationnels DevOps.

— Évaluer la maturité DevOps d'une organisation.

— Planifier une transformation DevOps.

13.2 Caractéristiques clés de la culture DevOps

13.2.1 Les valeurs fondamentales

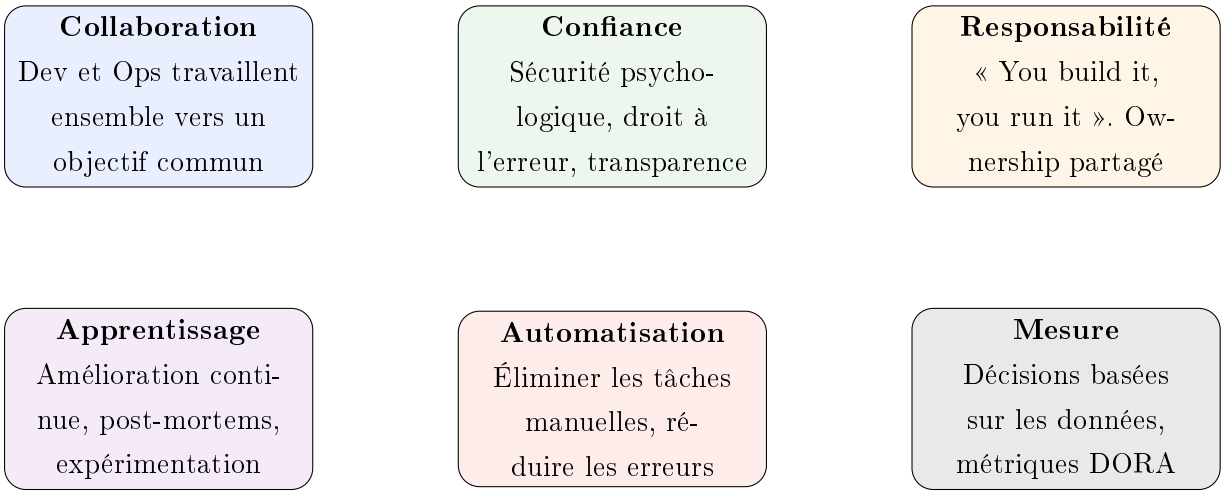


FIGURE 13.1 – Les valeurs fondamentales de la culture DevOps

13.2.2 You Build It, You Run It

Le principe « You Build It, You Run It » (YBIYRI) est au cœur de la culture DevOps. Il signifie que l'équipe qui développe un service est également responsable de son fonctionnement en production.

Implications pratiques :

— Les développeurs participent aux astreintes (*on-call*).

— L'équipe est responsable du monitoring et de la résolution des incidents.

— Les développeurs écrivent du code opérable (logs structurés, health checks, métriques).

— L'empathie pour les opérations influence les décisions de conception.

13.3 Modèles organisationnels pour les équipes DevOps

13.3.1 Anti-pattern : l'équipe DevOps dédiée



[

Anti-pattern à éviter] Créer une « équipe DevOps » séparée qui sert d'intermédiaire entre Dev et Ops ne fait que déplacer le problème et créer un troisième silo. DevOps n'est pas un rôle ni une équipe, c'est une culture et un ensemble de pratiques.

13.3.2 Modèles organisationnels recommandés

13.3.3 Le modèle Platform Team

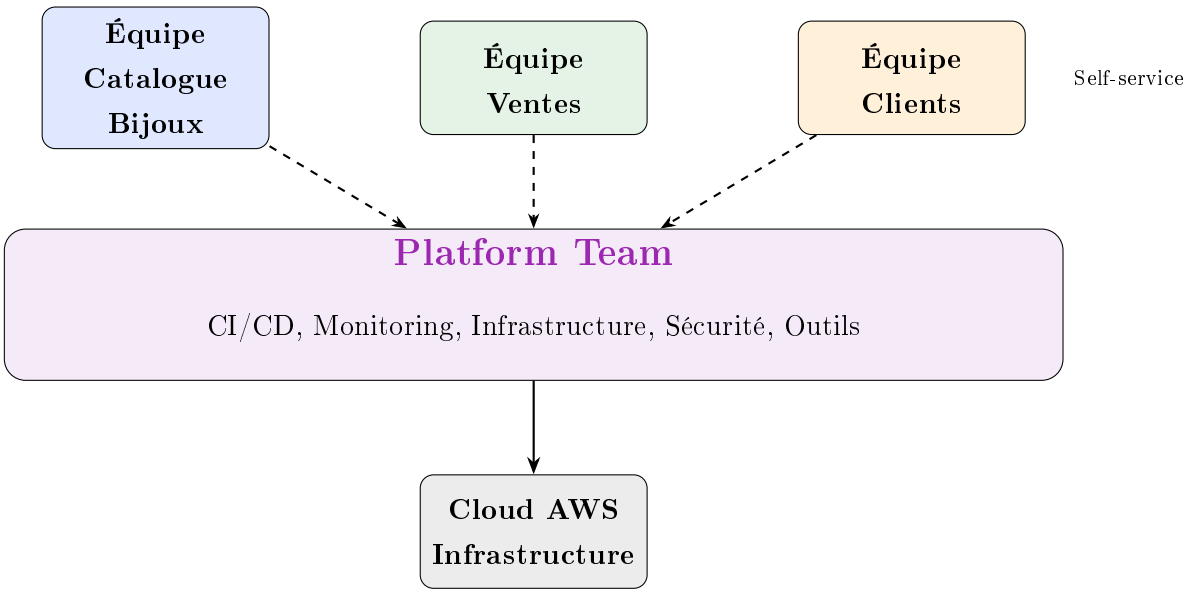


FIGURE 13.2 – Le modèle Platform Team

13.4 Évaluer la maturité DevOps

13.4.1 Niveaux de maturité

TABLE 13.1 – Niveaux de maturité DevOps

Niveau	Nom	Caractéristiques
1	Initial	Processus manuels, silos, pas de CI/CD
2	Géré	CI basique, déploiements semi-automatiques
3	Défini	Pipeline CI/CD complet, IaC, monitoring basique
4	Mesuré	Métriques DORA suivies, quality gates, DevSecOps
5	Optimisé	Déploiement continu, chaos engineering, culture apprenante

13.5 Résumé du chapitre

Points clés à retenir

- La culture DevOps repose sur la collaboration, la confiance, la responsabilité et l'apprentissage.
- « You Build It, You Run It » : l'équipe est responsable de bout en bout.
- DevOps n'est pas un rôle mais une culture. Éviter le piège de l'« équipe DevOps ».
- Le modèle Platform Team fournit une plateforme self-service aux équipes produit.
- La maturité DevOps se mesure en 5 niveaux, de Initial à Optimisé.

Chapitre 14

Synthèse : Pipeline DevOps Complet de BijouxShop

14.1 Vue d'ensemble

Ce chapitre rassemble tous les concepts étudiés dans ce cours en présentant le pipeline DevOps complet de l'application BijouxShop, depuis le commit du développeur jusqu'au déploiement en production sur AWS.

14.2 Architecture globale DevOps

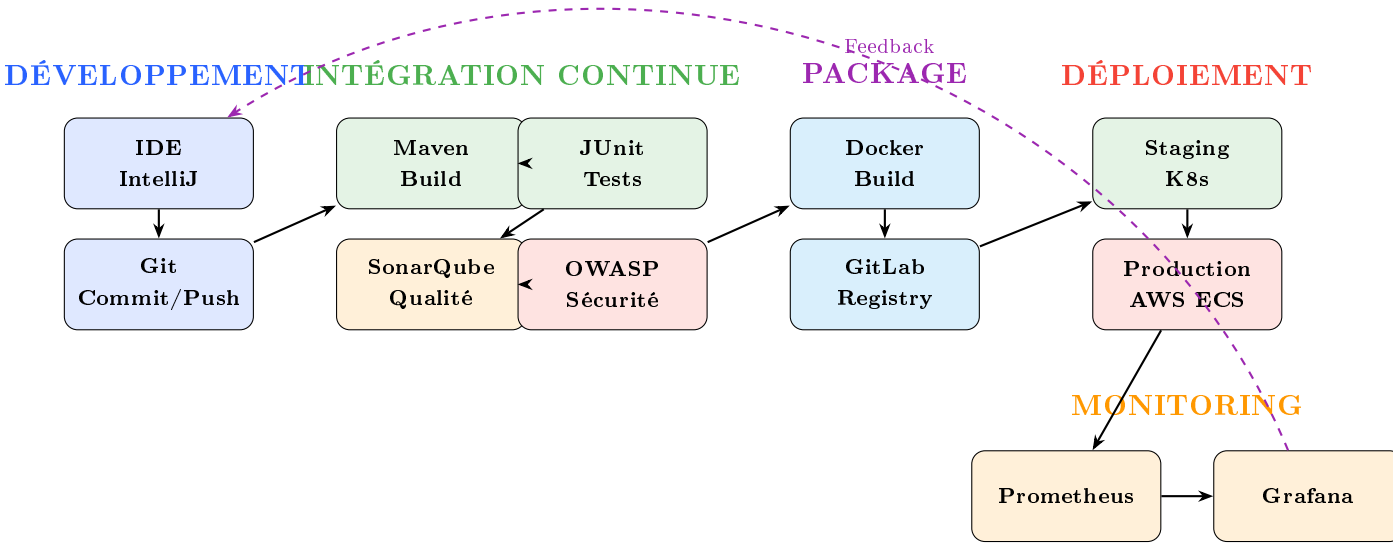


FIGURE 14.1 – Architecture DevOps complète de BijouxShop

14.3 Le fichier .gitlab-ci.yml final

```
# ============================================
```

```yaml
# Pipeline CI/CD complet - BijouxShop
# ============================================

variables:
  MAVEN_OPTS: "-Dmaven.repo.local=.m2/repository"
  DOCKER_IMAGE: $CI_REGISTRY_IMAGE
  JAVA_VERSION: "17"

cache:
  key: ${CI_COMMIT_REF_SLUG}
  paths:
    - .m2/repository/
    - target/

stages:
  - build
  - test
  - quality
  - security
  - package
  - deploy-staging
  - acceptance-tests
  - deploy-production

# ---- BUILD ----
compile:
  stage: build
  image: maven:3.9-eclipse-temurin-17
  script:
    - echo "Building BijouxShop..."
    - mvn clean compile -B
  artifacts:
    paths: [target/]
    expire_in: 1 hour

# ---- TESTS ----
unit-tests:
  stage: test
  image: maven:3.9-eclipse-temurin-17
  script:
    - mvn test -B
```

```
43      - mvn jacoco:report
44    artifacts:
45      when: always
46      reports:
47        junit: target/surefire-reports/TEST-*.xml
48      paths:
49        - target/site/jacoco/
50
51  integration-tests:
52    stage: test
53    image: maven:3.9-eclipse-temurin-17
54    services:
55      - name: postgres:16-alpine
56        alias: test-db
57    variables:
58      POSTGRES_DB: bijouxshop_test
59      POSTGRES_USER: test_user
60      POSTGRES_PASSWORD: test_pass
61      SPRING_DATASOURCE_URL:
           "jdbc:postgresql://test-db:5432/bijouxshop_test"
62      SPRING_DATASOURCE_USERNAME: test_user
63      SPRING_DATASOURCE_PASSWORD: test_pass
64    script:
65      - mvn verify -P integration-tests -B
66
67  # ---- QUALITY ----
68  sonar-analysis:
69    stage: quality
70    image: maven:3.9-eclipse-temurin-17
71    script:
72      - mvn sonar:sonar
73          -Dsonar.host.url=$SONAR_HOST_URL
74          -Dsonar.login=$SONAR_TOKEN
75          -Dsonar.projectKey=bijouxshop
76          -Dsonar.qualitygate.wait=true
77    allow_failure: false
78
79  # ---- SECURITY ----
80  dependency-check:
81    stage: security
82    image: maven:3.9-eclipse-temurin-17
```

```
83    script:
84      - mvn org.owasp:dependency-check-maven:check
85          -DfailBuildOnCVSS=7
86    artifacts:
87      paths: [target/dependency-check-report.html]
88      when: always
89
90  container-scan:
91    stage: security
92    image:
93      name: aquasec/trivy:latest
94      entrypoint: [""]
95    script:
96      - trivy image --exit-code 0 --severity HIGH,CRITICAL
97          $DOCKER_IMAGE:$CI_COMMIT_SHA
98    allow_failure: true
99    needs: ["docker-build"]
100
101 # ---- PACKAGE ----
102 docker-build:
103   stage: package
104   image: docker:24
105   services:
106     - docker:24-dind
107   before_script:
108     - docker login -u $CI_REGISTRY_USER
109         -p $CI_REGISTRY_PASSWORD $CI_REGISTRY
110   script:
111     - docker build
112         --build-arg VERSION=$CI_COMMIT_SHA
113         -t $DOCKER_IMAGE:$CI_COMMIT_SHA
114         -t $DOCKER_IMAGE:latest .
115     - docker push $DOCKER_IMAGE:$CI_COMMIT_SHA
116     - docker push $DOCKER_IMAGE:latest
117   only: [main, develop]
118
119 # ---- DEPLOY STAGING ----
120 deploy-staging:
121   stage: deploy-staging
122   image: bitnami/kubectl:latest
123   script:
```

```
124     - kubectl config use-context bijouxshop-staging
125     - kubectl set image deployment/bijouxshop-app
126         bijouxshop=$DOCKER_IMAGE:$CI_COMMIT_SHA
127         -n bijouxshop-staging
128     - kubectl rollout status deployment/bijouxshop-app
129         -n bijouxshop-staging --timeout=300s
130   environment:
131     name: staging
132     url: https://staging.bijouxshop.com
133   only: [develop]
134
135 # ---- ACCEPTANCE TESTS ----
136 smoke-tests:
137   stage: acceptance-tests
138   image: curlimages/curl:latest
139   script:
140     - curl -f https://staging.bijouxshop.com/actuator/health
141     - curl -f https://staging.bijouxshop.com/api/bijoux
142   needs: ["deploy-staging"]
143   only: [develop]
144
145 # ---- DEPLOY PRODUCTION ----
146 deploy-production:
147   stage: deploy-production
148   image: amazon/aws-cli:latest
149   script:
150     - aws ecs update-service
151         --cluster bijouxshop-prod
152         --service bijouxshop-service
153         --force-new-deployment
154   environment:
155     name: production
156     url: https://www.bijouxshop.com
157   when: manual
158   only: [main]
```

Listing 14.1 – .gitlab-ci.yml final et complet pour BijouxShop

14.4 Résumé des technologies utilisées

TABLE 14.1 – Stack technologique complète de BijouxShop

Catégorie	**Technologie**	**Rôle**
Application	Spring Boot 3.2	Framework backend Java
Base de données	PostgreSQL 16	Stockage des données
Versioning	Git + GitLab	Gestion de code source
Build	Maven	Compilation et gestion de dépendances
Tests	JUnit 5, Mockito	Tests unitaires et mocks
Tests intégration	Testcontainers	Tests avec vraie base de données
Qualité	SonarQube	Analyse statique du code
Sécurité	OWASP Dep. Check	Scan de vulnérabilités
Conteneurisation	Docker	Empaquetage de l'application
Orchestration	Kubernetes	Gestion des conteneurs en production
CI/CD	GitLab CI/CD	Automatisation du pipeline
IaC	Terraform	Provisionnement infrastructure AWS
Configuration	Ansible	Gestion de configuration serveurs
Monitoring	Prometheus + Grafana	Métriques et dashboards
Cloud	AWS (ECS, RDS, ECR)	Infrastructure de production

14.5 Conclusion du cours

Ce cours a parcouru l'ensemble du spectre DevOps, de la théorie à la pratique, en utilisant l'application BijouxShop comme fil conducteur.

Les étapes clés que nous avons couvertes :

1. **Comprendre** les fondements de DevOps (culture, CALMS, trois voies).
2. **Versionner** le code avec Git et des stratégies de branching adaptées.
3. **Gérer** le projet de manière collaborative avec GitLab (Issues, Boards, MR).
4. **Conteneuriser** l'application avec Docker et Docker Compose.
5. **Automatiser** le pipeline CI/CD avec GitLab CI (.gitlab-ci.yml).
6. **Tester** à tous les niveaux (unitaire, intégration, E2E).
7. **Sécuriser** le code avec DevSecOps (SonarQube, OWASP, Trivy).
8. **Orchestrer** les conteneurs avec Kubernetes.
9. **Provisionner** l'infrastructure avec Terraform (IaC).
10. **Déployer** sur le Cloud AWS.

11. **Monitorer** avec Prometheus et Grafana.
12. **Cultiver** une culture DevOps d'apprentissage continu.

Le mot de la fin

DevOps n'est pas une destination, c'est un voyage. L'objectif n'est pas de mettre en place tous les outils d'un coup, mais d'adopter progressivement les pratiques qui apportent le plus de valeur à votre contexte. Commencez par la culture, automatisez ce qui est le plus douloureux, mesurez tout, et n'arrêtez jamais d'apprendre.

Annexe A

Glossaire DevOps

Terme	Définition
Artifact	Fichier produit par un build (JAR, image Docker, etc.)
Blue-Green Deployment	Stratégie de déploiement avec deux environnements identiques
Canary Release	Déploiement progressif auprès d'un petit pourcentage d'utilisateurs
CI/CD	Intégration Continue / Déploiement Continu
Container	Instance d'exécution d'une image Docker
DAST	Dynamic Application Security Testing
Deployment	Processus de mise en production du logiciel
Docker	Plateforme de conteneurisation
DORA	DevOps Research and Assessment (métriques)
Feature Flag	Mécanisme pour activer/désactiver des fonctionnalités
GitOps	Gestion de l'infrastructure et des déploiements via Git
Helm	Gestionnaire de packages pour Kubernetes
IaC	Infrastructure as Code
Idempotence	Propriété d'une opération qui produit le même résultat si exécutée plusieurs fois
Ingress	Point d'entrée HTTP dans un cluster Kubernetes
Job	Tâche individuelle dans un pipeline CI/CD
Kubernetes (K8s)	Plateforme d'orchestration de conteneurs
Lead Time	Temps entre la demande et la livraison
Merge Request (MR)	Demande de fusion de code sur GitLab
Microservice	Architecture où l'application est composée de petits services indépendants
MTTR	Mean Time To Recovery
Namespace	Isolation logique dans Kubernetes
On-call	Astreinte pour le support production
Pipeline	Ensemble de stages et jobs automatisés dans CI/CD
Pod	Plus petite unité déployable dans Kubernetes

Terme	**Définition**
Post-mortem	Analyse d'un incident après sa résolution
Provisioning	Création et configuration de l'infrastructure
Registry	Dépôt de stockage d'images Docker
Rollback	Retour à une version précédente
Rolling Update	Mise à jour progressive sans interruption
Runner	Agent qui exécute les jobs CI/CD
SAST	Static Application Security Testing
SCA	Software Composition Analysis
Service (K8s)	Abstraction réseau pour exposer des Pods
Shift Left	Déplacer les activités (tests, sécurité) plus tôt dans le cycle
SLI/SLO/SLA	Indicateurs, objectifs et accords de niveau de service
SRE	Site Reliability Engineering
Stage	Phase d'un pipeline CI/CD
StatefulSet	Contrôleur Kubernetes pour les applications avec état
Terraform	Outil IaC par HashiCorp
Throughput	Nombre de déploiements par unité de temps
WIP	Work In Progress

Annexe B

Commandes de référence

B.1 Commandes Git

Commande	Description
`git init`	Initialiser un dépôt Git
`git clone <url>`	Cloner un dépôt distant
`git status`	Afficher l'état du répertoire de travail
`git add <file>`	Ajouter un fichier à la staging area
`git commit -m "msg"`	Créer un commit avec un message
`git push origin <branch>`	Envoyer les commits vers le dépôt distant
`git pull origin <branch>`	Récupérer et fusionner les changements distants
`git branch`	Lister les branches
`git switch -c <branch>`	Créer et basculer sur une branche
`git merge <branch>`	Fusionner une branche
`git log –oneline –graph`	Afficher l'historique graphiquement
`git stash`	Mettre de côté les modifications
`git tag -a v1.0 -m "msg"`	Créer un tag annoté
`git revert <hash>`	Annuler un commit en créant un nouveau
`git diff`	Afficher les différences

B.2 Commandes Docker

Commande	Description
`docker build -t <name> .`	Construire une image Docker
`docker run -d -p 8080:8080 <img>`	Exécuter un conteneur en arrière-plan
`docker ps`	Lister les conteneurs en cours
`docker logs <container>`	Afficher les logs d'un conteneur
`docker exec -it <c> /bin/sh`	Ouvrir un shell dans un conteneur
`docker stop <container>`	Arrêter un conteneur

Commande	Description
`docker rm <container>`	Supprimer un conteneur
`docker images`	Lister les images
`docker rmi <image>`	Supprimer une image
`docker compose up -d`	Démarrer les services Docker Compose
`docker compose down`	Arrêter les services
`docker compose logs -f`	Suivre les logs

B.3 Commandes Kubernetes

Commande	Description
`kubectl get pods`	Lister les pods
`kubectl get deployments`	Lister les deployments
`kubectl get services`	Lister les services
`kubectl apply -f <file>`	Appliquer un manifeste
`kubectl delete -f <file>`	Supprimer les ressources
`kubectl logs <pod>`	Afficher les logs d'un pod
`kubectl exec -it <pod> - sh`	Shell dans un pod
`kubectl scale deploy/<name> -replicas=N`	Scaler un deployment
`kubectl rollout status`	Vérifier l'état d'un déploiement
`kubectl rollout undo`	Rollback d'un deployment
`kubectl describe pod <name>`	Détails d'un pod
`kubectl port-forward <pod> 8080:8080`	Redirection de port

Annexe C

Références et lectures recommandées

C.1 Livres

1. Gene Kim, Jez Humble, Patrick Debois, John Willis. *The DevOps Handbook : How to Create World-Class Agility, Reliability, and Security in Technology Organizations*. IT Revolution Press, 2016.
2. Gene Kim, Kevin Behr, George Spafford. *The Phoenix Project : A Novel about IT, DevOps, and Helping Your Business Win*. IT Revolution Press, 2013.
3. Nicole Forsgren, Jez Humble, Gene Kim. *Accelerate : The Science of Lean Software and DevOps*. IT Revolution Press, 2018.
4. Jez Humble, David Farley. *Continuous Delivery : Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley, 2010.
5. Kief Morris. *Infrastructure as Code : Dynamic Systems for the Cloud Age*. O'Reilly Media, 2020.
6. Brendan Burns, Joe Beda, Kelsey Hightower. *Kubernetes : Up and Running*. O'Reilly Media, 2022.
7. Nigel Poulton. *Docker Deep Dive*. Independently Published, 2023.

C.2 Ressources en ligne

1. DORA State of DevOps Report : `dora.dev`
2. GitLab CI/CD Documentation : `docs.gitlab.com/ee/ci/`
3. Kubernetes Documentation : `kubernetes.io/docs/`
4. Docker Documentation : `docs.docker.com`
5. OWASP Top 10 : `owasp.org/www-project-top-ten/`
6. Terraform Documentation : `developer.hashicorp.com/terraform`
7. AWS Documentation : `docs.aws.amazon.com`
8. SonarQube Documentation : `docs.sonarqube.org`

Développements Approfondis

Introduction au développement approfondi

Cette section présente des développements approfondis et détaillés sur les concepts DevOps abordés dans le cours. Chaque thème est exploré en profondeur avec des exemples concrets, des études de cas, des implémentations détaillées et des analyses critiques.

Chapitre 1

Architecture DevOps Complète : Vue d'Ensemble Approfondie

1.1 Introduction à l'architecture DevOps moderne

L'architecture DevOps moderne représente une évolution fondamentale dans la manière dont les organisations conçoivent, développent, déploient et maintiennent leurs applications logicielles. Cette architecture n'est pas simplement une collection d'outils, mais plutôt un écosystème cohérent qui intègre des pratiques culturelles, des processus organisationnels et des technologies de pointe.

1.1.1 Les couches de l'architecture DevOps

Une architecture DevOps complète peut être décomposée en plusieurs couches interdépendantes :

1. Couche de développement

La couche de développement englobe tous les aspects de la création du code applicatif :

Environnement de développement local Les développeurs travaillent dans des environnements locaux qui doivent reproduire fidèlement l'environnement de production. Cette fidélité est cruciale pour minimiser les problèmes de type "ça marche sur ma machine". Pour y parvenir, plusieurs approches sont utilisées :

- **Conteneurisation du développement** : Docker Desktop ou Podman permettent d'exécuter l'application dans des conteneurs identiques à ceux utilisés en production. Les développeurs peuvent démarrer l'ensemble de la stack (application, base de données, services auxiliaires) avec une simple commande.
- **Environnements de développement cloud** : Des solutions comme GitHub Codespaces, GitPod ou JetBrains Space offrent des environnements de développement préconfigurés dans le cloud, garantissant une cohérence parfaite entre tous les développeurs.
- **Configuration as Code** : Les outils comme Vagrant permettent de décrire l'environnement de développement complet dans un fichier (Vagrantfile), assurant que tous les développeurs disposent de la même configuration.

Gestion du code source Le code source est l'actif le plus précieux d'une organisation logicielle. Sa gestion doit être rigoureuse :

- **Systèmes de contrôle de version distribués** : Git domine largement ce domaine. Chaque développeur possède une copie complète de l'historique, permettant un travail offline et des opérations rapides.
- **Stratégies de branching avancées** : Au-delà de Git Flow, des stratégies comme Trunk-Based Development ou GitHub Flow adaptent le workflow aux besoins spécifiques de chaque équipe.
- **Hooks et automatisation** : Les hooks Git (pre-commit, pre-push, etc.) permettent d'exécuter automatiquement des vérifications (linting, tests unitaires, vérification des messages de commit) avant que le code ne soit poussé.
- **Signature et vérification des commits** : La signature GPG des commits garantit l'authenticité et l'intégrité du code source, un aspect crucial pour la sécurité de la chaîne d'approvisionnement logicielle.

Intégration de l'IDE Les environnements de développement intégrés (IDE) jouent un rôle central dans la productivité des développeurs. Leur intégration avec l'écosystème DevOps est essentielle :

- **Support natif de Git** : IntelliJ IDEA, Visual Studio Code, Eclipse offrent tous une intégration Git sophistiquée avec visualisation graphique de l'historique, résolution de conflits assistée et gestion des branches.
- **Intégration CI/CD** : Les plugins permettent de visualiser l'état des pipelines, de relancer des builds et de consulter les logs directement depuis l'IDE.
- **Analyse en temps réel** : SonarLint analyse le code pendant son écriture et alerte immédiatement sur les problèmes de qualité ou de sécurité, bien avant le commit.
- **Débogage à distance** : Capacité de déboguer des applications tournant dans des conteneurs ou des clusters Kubernetes directement depuis l'IDE.

2. Couche d'intégration continue

L'intégration continue est le premier pilier de l'automatisation DevOps. Elle va bien au-delà de la simple compilation du code.

Déclenchement des builds Les builds peuvent être déclenchés de multiples façons :

- **Push triggers** : Déclenchement automatique à chaque push sur certaines branches
- **Pull/Merge Request triggers** : Build automatique lors de la création ou mise à jour d'une PR/MR
- **Scheduled triggers** : Builds périodiques (nightly builds) pour détecter les régressions
- **Manual triggers** : Déclenchement manuel pour des cas spécifiques
- **Webhook triggers** : Déclenchement par des événements externes (mise à jour d'une dépendance, alerte de sécurité)

Phases du pipeline CI Un pipeline CI typique comprend plusieurs phases séquentielles :

1. **Checkout du code** : Récupération du code source depuis le dépôt Git
2. **Résolution des dépendances** :

   - Maven/Gradle pour Java
   - npm/yarn pour JavaScript
   - pip pour Python
   - Caching intelligent pour accélérer les builds
3. **Compilation et packaging** :
   - Compilation du code source
   - Génération des artefacts (JAR, WAR, binaires)
   - Optimisation (minification, tree-shaking pour le frontend)
4. **Exécution des tests** :
   - Tests unitaires (JUnit, TestNG, pytest, Jest)
   - Tests d'intégration (Testcontainers, test databases)
   - Tests de contrat (Pact, Spring Cloud Contract)
   - Coverage et rapports de tests
5. **Analyse de qualité du code** :
   - Analyse statique (SonarQube, Checkstyle, PMD)
   - Détection de code dupliqué
   - Mesure de la complexité cyclomatique
   - Vérification du respect des standards de codage
6. **Scan de sécurité** :
   - Analyse des vulnérabilités des dépendances (OWASP Dependency-Check, Snyk)
   - Analyse SAST (Static Application Security Testing)
   - Détection de secrets dans le code (truffleHog, GitLeaks)
7. **Construction d'artefacts déployables** :
   - Images Docker
   - Packages (RPM, DEB)
   - Archives (ZIP, TAR)
8. **Publication des artefacts** :
   - Registry Docker (Docker Hub, Harbor, ECR)
   - Gestionnaires d'artefacts (Nexus, Artifactory)
   - Registries de packages (npm, PyPI, Maven Central)

Parallélisation et optimisation Les pipelines modernes utilisent massivement la parallélisation pour accélérer les builds :

- **Exécution parallèle des tests** : Les suites de tests sont divisées en groupes exécutés simultanément
- **Matrix builds** : Exécution du même pipeline sur plusieurs versions de langages, OS ou configurations
- **Caching avancé** : Mise en cache des dépendances, des layers Docker, des résultats de compilation
- **Builds incrémentaux** : Recompilation uniquement des parties modifiées
- **Distributed builds** : Utilisation de grilles de build avec des agents multiples

3. Couche de livraison continue

La livraison continue étend le CI en automatisant le déploiement sur différents environnements.

Gestion des environnements Une organisation DevOps mature gère typiquement plusieurs environnements :

- **Développement (Dev)** :
  - Environnement instable, mis à jour en continu
  - Utilisé par les développeurs pour tests d'intégration
  - Données de test fictives
  - Peut être recréé fréquemment
- **Test/QA** :
  - Environnement stable pour tests fonctionnels
  - Utilisé par l'équipe QA
  - Données de test représentatives
  - Configuration proche de la production
- **Staging/Pre-production** :
  - Réplique exacte de la production
  - Tests de performance et de charge
  - Validation finale avant production
  - Données anonymisées issues de la production
- **Production** :
  - Environnement utilisé par les utilisateurs finaux
  - Haute disponibilité et performance
  - Monitoring intensif
  - Sauvegardes régulières

Stratégies de déploiement avancées Les organisations performantes utilisent des stratégies de déploiement sophistiquées pour minimiser les risques :

Blue-Green Deployment

Cette stratégie maintient deux environnements de production identiques :

- L'environnement "Blue" sert actuellement le trafic
- L'environnement "Green" reçoit la nouvelle version
- Tests sur Green
- Bascule du trafic de Blue vers Green instantanément
- Blue devient le fallback en cas de problème
- Si tout va bien, Blue peut être mise à jour pour la prochaine release

Avantages : Rollback instantané, downtime zéro, tests en conditions réelles

Inconvénients : Coût (double infrastructure), complexité des bases de données partagées

Canary Deployment

Déploiement progressif avec validation continue :

1. Déploiement de la nouvelle version sur un petit sous-ensemble de serveurs (5-10%)
2. Routage d'un petit pourcentage du trafic (1-5%) vers les canaries
3. Monitoring intensif des métriques (erreurs, latence, CPU, mémoire)
4. Si les métriques sont bonnes, augmentation progressive du trafic (10%, 25%, 50%, 100%)
5. En cas de problème, rollback immédiat pour les canaries

Avantages : Risque limité, détection précoce des problèmes, impact utilisateur minimal

Inconvénients : Complexité du routage, nécessite un monitoring sophistiqué

Rolling Deployment

Mise à jour progressive des instances :

— Retrait d'une ou plusieurs instances du load balancer
— Mise à jour de ces instances
— Tests de santé (health checks)
— Réintégration dans le load balancer
— Répétition pour les autres instances

Avantages : Pas de surcoût d'infrastructure, simple à implémenter

Inconvénients : Versions mixtes pendant le déploiement, rollback plus lent

Feature Flags

Les feature flags (ou feature toggles) permettent d'activer/désactiver des fonctionnalités sans déploiement :

— **Release toggles** : Activation progressive d'une nouvelle fonctionnalité
— **Experiment toggles** : A/B testing et expérimentation
— **Ops toggles** : Désactivation de fonctionnalités en cas de charge élevée
— **Permission toggles** : Fonctionnalités réservées à certains utilisateurs

Outils : LaunchDarkly, Unleash, Split.io, ou implémentation maison

1.1.2 Intégration de la sécurité (DevSecOps)

La sécurité doit être intégrée à chaque étape du pipeline, pas ajoutée à la fin.

Shift-left Security

Le concept de "shift-left" consiste à déplacer les contrôles de sécurité le plus tôt possible dans le cycle :

Au niveau du développement

— **IDE Security Plugins** : Détection en temps réel des vulnérabilités pendant le codage
— **Pre-commit hooks** : Vérification des secrets, analyse statique basique
— **Training** : Formation continue des développeurs aux bonnes pratiques de sécurité

Au niveau du CI

— **SAST (Static Application Security Testing)** :
  — Analyse du code source sans l'exécuter
  — Détection de patterns dangereux (SQL injection, XSS, etc.)
  — Outils : SonarQube, Checkmarx, Fortify, Semgrep
— **SCA (Software Composition Analysis)** :
  — Analyse des dépendances tierces
  — Détection de vulnérabilités connues (CVE)
  — Vérification des licences
  — Outils : Snyk, WhiteSource, Black Duck, OWASP Dependency-Check
— **Secret Scanning** :
  — Détection de clés API, mots de passe, tokens
  — Révocation automatique si détection
  — Outils : GitGuardian, truffleHog, GitLeaks
— **Container Image Scanning** :

  - Analyse des vulnérabilités dans les images Docker
  - Vérification des bonnes pratiques (user root, exposition de ports)
  - Outils : Trivy, Clair, Anchore

Au niveau du CD

- **DAST (Dynamic Application Security Testing)** :
  - Tests de sécurité en exécutant l'application
  - Simulation d'attaques réelles
  - Outils : OWASP ZAP, Burp Suite, Acunetix
- **Penetration Testing automatisé** :
  - Tests d'intrusion automatisés
  - Exécution périodique

En production

- **Runtime Application Self-Protection (RASP)** :
  - Protection en temps réel contre les attaques
  - Intégré à l'application
- **Web Application Firewall (WAF)** :
  - Filtrage des requêtes malveillantes
  - Protection contre OWASP Top 10
  - Exemples : AWS WAF, Cloudflare, ModSecurity
- **Security Monitoring** :
  - Détection d'anomalies comportementales
  - Alertes sur activités suspectes
  - SIEM (Security Information and Event Management)

1.2 Infrastructure as Code (IaC) : Approche détaillée

L'Infrastructure as Code révolutionne la manière dont les organisations gèrent leur infrastructure.

1.2.1 Principes fondamentaux de l'IaC

Immutabilité de l'infrastructure

Le concept d'infrastructure immutable stipule que les serveurs ne doivent jamais être modifiés après leur création. Au lieu de mettre à jour un serveur existant, on crée un nouveau serveur avec la nouvelle configuration et on détruit l'ancien.

Avantages

- **Cohérence** : Tous les serveurs sont identiques, pas de "configuration drift"
- **Prévisibilité** : Le comportement est déterministe
- **Rollback simple** : Retour à la version précédente en redéployant l'ancienne image
- **Scalabilité** : Facile de créer de nouvelles instances identiques

Mise en œuvre

- Utilisation d'images préconstruites (AMI AWS, images Docker)
- Auto-scaling groups qui remplacent les instances plutôt que les modifier

— Pas de SSH sur les serveurs de production
— Configuration injectée au démarrage (variables d'environnement, secrets)

Déclaratif vs Impératif

Approche déclarative

On déclare l'état désiré, l'outil détermine comment l'atteindre :

```
resource "aws_instance" "web" {
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = "t2.micro"
  count         = 3

  tags = {
    Name = "web-server"
  }
}
```

Avantages :
— Code plus simple et lisible
— Idempotent : peut être appliqué plusieurs fois
— Gestion automatique des dépendances
Exemples d'outils : Terraform, CloudFormation, Pulumi

Approche impérative

On spécifie les étapes exactes à exécuter :

```
import boto3

ec2 = boto3.resource('ec2')

for i in range(3):
    instance = ec2.create_instances(
        ImageId='ami-0c55b159cbfafe1f0',
        InstanceType='t2.micro',
        MinCount=1,
        MaxCount=1,
        TagSpecifications=[{
            'ResourceType': 'instance',
            'Tags': [{'Key': 'Name', 'Value': f'web-server-{i}'}]
        }]
    )
```

Avantages :
— Contrôle précis de l'exécution
— Flexibilité pour logique complexe
— Familier aux développeurs
Exemples d'outils : Scripts Bash, Python avec SDK cloud, Ansible (hybride)

1.2.2 Terraform : Deep Dive

Terraform est l'outil IaC le plus populaire pour le provisioning d'infrastructure multi-cloud.

Architecture et concepts

Providers Les providers sont des plugins qui permettent à Terraform d'interagir avec différentes plateformes :

- **Cloud providers** : AWS, Azure, GCP, OCI
- **Infrastructure providers** : VMware, OpenStack
- **SaaS providers** : Datadog, PagerDuty, GitHub
- **Database providers** : PostgreSQL, MySQL

Configuration d'un provider :

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "eu-west-1"

  default_tags {
    tags = {
      Environment = "production"
      ManagedBy   = "Terraform"
      Project     = "BijouxShop"
    }
  }
}
```

Resources Les resources sont les éléments d'infrastructure gérés par Terraform :

```
resource "aws_vpc" "main" {
  cidr_block           = "10.0.0.0/16"
  enable_dns_hostnames = true
  enable_dns_support   = true

  tags = {
    Name = "bijouxshop-vpc"
  }
}

resource "aws_subnet" "public" {
  count             = 3
  vpc_id            = aws_vpc.main.id
  cidr_block        = "10.0.${count.index}.0/24"
  availability_zone = data.aws_availability_zones.available.names[count.
   index]

  tags = {
    Name = "bijouxshop-public-subnet-${count.index + 1}"
    Type = "public"
  }
}
```

Data Sources Les data sources permettent de récupérer des informations sur des ressources existantes :

```
data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"] # Canonical

  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-*"]
  }

  filter {
    name   = "virtualization-type"
    values = ["hvm"]
  }
}
```

Variables et Outputs **Variables** permettent de paramétrer la configuration :

```
variable "environment" {
  description = "Environment name"
  type        = string
  default     = "development"
  validation {
    condition     = contains(["development", "staging", "production"], var.environment)
    error_message = "Environment must be development, staging, or production."
  }
}

variable "instance_count" {
  description = "Number of instances to create"
  type        = number
  default     = 2
}

variable "instance_tags" {
  description = "Tags to apply to instances"
  type        = map(string)
  default     = {}
}
```

Outputs exposent des valeurs après l'application :

```
output "vpc_id" {
  description = "The ID of the VPC"
  value       = aws_vpc.main.id
}

output "public_subnet_ids" {
  description = "List of public subnet IDs"
  value       = aws_subnet.public[*].id
}

output "load_balancer_dns" {
  description = "DNS name of the load balancer"
```

```
  value       = aws_lb.main.dns_name
  sensitive   = false
}
```

State Management Le state Terraform est un fichier JSON qui maintient la correspondance entre la configuration et l'infrastructure réelle.

Backend distant (recommandé pour les équipes) :

```
terraform {
  backend "s3" {
    bucket         = "bijouxshop-terraform-state"
    key            = "production/terraform.tfstate"
    region         = "eu-west-1"
    encrypt        = true
    dynamodb_table = "terraform-locks"
  }
}
```

Locking : Évite les modifications concurrentes
Versioning : Historique complet des changements d'état

Modules Terraform

Les modules permettent de réutiliser et d'organiser le code Terraform.

Structure d'un module

```
modules/
 vpc/
     main.tf
     variables.tf
     outputs.tf
     README.md
```

Utilisation d'un module

```
module "vpc" {
  source = "./modules/vpc"

  environment     = var.environment
  vpc_cidr        = "10.0.0.0/16"
  azs             = ["eu-west-1a", "eu-west-1b", "eu-west-1c"]
  private_subnets = ["10.0.1.0/24", "10.0.2.0/24", "10.0.3.0/24"]
  public_subnets  = ["10.0.101.0/24", "10.0.102.0/24", "10.0.103.0/24"]

  enable_nat_gateway = true
  enable_vpn_gateway = false

  tags = {
    Terraform   = "true"
    Environment = var.environment
  }
}
```

Modules publics Terraform Registry propose des milliers de modules prêts à l'emploi :

```
module "eks" {
  source  = "terraform-aws-modules/eks/aws"
  version = "~> 19.0"

  cluster_name    = "bijouxshop-eks"
  cluster_version = "1.27"

  vpc_id     = module.vpc.vpc_id
  subnet_ids = module.vpc.private_subnets

  eks_managed_node_groups = {
    general = {
      desired_size = 3
      min_size     = 2
      max_size     = 5

      instance_types = ["t3.medium"]
      capacity_type  = "ON_DEMAND"
    }
  }
}
```

1.2.3 Configuration Management avec Ansible

Ansible complète Terraform en gérant la configuration des serveurs après leur provisioning.

Architecture et concepts Ansible

Inventaire L'inventaire définit les hôtes gérés par Ansible :

```
[webservers]
web1.bijouxshop.com ansible_host=10.0.1.10
web2.bijouxshop.com ansible_host=10.0.1.11
web3.bijouxshop.com ansible_host=10.0.1.12

[dbservers]
db1.bijouxshop.com ansible_host=10.0.2.10
db2.bijouxshop.com ansible_host=10.0.2.11

[production:children]
webservers
dbservers

[production:vars]
ansible_user=ubuntu
ansible_ssh_private_key_file=~/.ssh/bijouxshop_prod.pem
environment=production
```

Playbooks Les playbooks sont des fichiers YAML décrivant les tâches à exécuter :

```
---
- name: Configure BijouxShop Web Servers
  hosts: webservers
```

```
  become: yes

  vars:
    app_name: bijouxshop
    app_version: "{{ lookup('env', 'APP_VERSION') | default('latest')
   }}"
    java_version: 17

  tasks:
    - name: Update apt cache
      apt:
        update_cache: yes
        cache_valid_time: 3600

    - name: Install Java
      apt:
        name: "openjdk-{{ java_version }}-jdk"
        state: present

    - name: Create application user
      user:
        name: "{{ app_name }}"
        system: yes
        shell: /bin/false

    - name: Create application directories
      file:
        path: "{{ item }}"
        state: directory
        owner: "{{ app_name }}"
        group: "{{ app_name }}"
        mode: '0755'
      loop:
        - /opt/{{ app_name }}
        - /var/log/{{ app_name }}
        - /etc/{{ app_name }}

    - name: Download application artifact
      get_url:
        url: "https://artifacts.bijouxshop.com/{{ app_name }}-{{
   app_version }}.jar"
        dest: "/opt/{{ app_name }}/app.jar"
        owner: "{{ app_name }}"
        group: "{{ app_name }}"
        mode: '0644'
      notify: restart bijouxshop

    - name: Configure application
      template:
        src: templates/application.yml.j2
        dest: "/etc/{{ app_name }}/application.yml"
        owner: "{{ app_name }}"
        group: "{{ app_name }}"
        mode: '0640'
      notify: restart bijouxshop

    - name: Install systemd service
      template:
```

```
60          src: templates/bijouxshop.service.j2
61          dest: /etc/systemd/system/bijouxshop.service
62          mode: '0644'
63        notify:
64          - reload systemd
65          - restart bijouxshop
66
67      - name: Ensure bijouxshop is running
68        systemd:
69          name: bijouxshop
70          state: started
71          enabled: yes
72
73    handlers:
74      - name: reload systemd
75        systemd:
76          daemon_reload: yes
77
78      - name: restart bijouxshop
79        systemd:
80          name: bijouxshop
81          state: restarted
```

Roles Les roles permettent d'organiser les playbooks en composants réutilisables :

```
roles/
 bijouxshop/
     tasks/
        main.yml
     handlers/
        main.yml
     templates/
        application.yml.j2
        bijouxshop.service.j2
     files/
     vars/
        main.yml
     defaults/
        main.yml
     meta/
         main.yml
```

Utilisation d'un role :

```
1 ---
2 - name: Deploy BijouxShop
3   hosts: webservers
4   roles:
5     - role: common
6     - role: firewall
7     - role: monitoring
8     - role: bijouxshop
```

1.3 Containerisation et Orchestration : Analyse approfondie

1.3.1 Docker : Au-delà des bases

Optimisation des images Docker

Multi-stage builds Les multi-stage builds réduisent drastiquement la taille des images :

```
# Stage 1: Build
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app

# Copy only pom.xml first for better caching
COPY pom.xml .
RUN mvn dependency:go-offline

# Copy source and build
COPY src ./src
RUN mvn package -DskipTests

# Stage 2: Runtime
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app

# Create non-root user
RUN addgroup -S spring && adduser -S spring -G spring
USER spring:spring

# Copy only the JAR from build stage
COPY --from=build /app/target/*.jar app.jar

# Add health check
HEALTHCHECK --interval=30s --timeout=3s --start-period=60s --retries=3 \
  CMD wget --quiet --tries=1 --spider http://localhost:8080/actuator/
   health || exit 1

EXPOSE 8080

ENTRYPOINT ["java", \
    "-XX:+UseContainerSupport", \
    "-XX:MaxRAMPercentage=75.0", \
    "-Djava.security.egd=file:/dev/./urandom", \
    "-jar", "app.jar"]
```

Taille :
— Image complète avec Maven : 800MB
— Image multi-stage avec JRE Alpine : 250MB
— Réduction : 69%

Layer caching Optimisation de l'ordre des instructions pour maximiser le cache :

```
FROM node:18-alpine

WORKDIR /app

# Install dependencies first (changes less frequently)
```

```
COPY package*.json ./
RUN npm ci --only=production

# Copy source (changes more frequently)
COPY . .

# Build
RUN npm run build

EXPOSE 3000
CMD ["node", "dist/server.js"]
```

.dockerignore Exclusion des fichiers inutiles :

```
# Fichiers Git
.git
.gitignore
.gitattributes

# Dépendances locales
node_modules
target
.m2

# IDE
.idea
.vscode
*.iml

# Logs
*.log
logs/

# Tests
test/
*.test.js
coverage/

# Documentation
README.md
docs/

# CI/CD
.gitlab-ci.yml
Jenkinsfile
.github/

# Environnement local
.env
.env.local
```

Docker Compose : Orchestration locale

Docker Compose permet de gérer des applications multi-conteneurs :

```yaml
version: '3.9'

services:
  bijouxshop-app:
    build:
      context: .
      dockerfile: Dockerfile
      args:
        - APP_VERSION=${APP_VERSION:-latest}
    container_name: bijouxshop-app
    ports:
      - "8080:8080"
    environment:
      SPRING_PROFILES_ACTIVE: docker
      SPRING_DATASOURCE_URL: jdbc:postgresql://bijouxshop-db:5432/bijouxshop
      SPRING_DATASOURCE_USERNAME: bijouxshop_user
      SPRING_DATASOURCE_PASSWORD_FILE: /run/secrets/db_password
    secrets:
      - db_password
    depends_on:
      bijouxshop-db:
        condition: service_healthy
    healthcheck:
      test: ["CMD", "wget", "--quiet", "--tries=1", "--spider", "http://localhost:8080/actuator/health"]
      interval: 30s
      timeout: 10s
      retries: 3
      start_period: 60s
    networks:
      - bijouxshop-network
    volumes:
      - app-logs:/var/log/bijouxshop
    restart: unless-stopped

  bijouxshop-db:
    image: postgres:15-alpine
    container_name: bijouxshop-db
    environment:
      POSTGRES_DB: bijouxshop
      POSTGRES_USER: bijouxshop_user
      POSTGRES_PASSWORD_FILE: /run/secrets/db_password
    secrets:
      - db_password
    ports:
      - "5432:5432"
    volumes:
      - postgres-data:/var/lib/postgresql/data
      - ./init-scripts:/docker-entrypoint-initdb.d:ro
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U bijouxshop_user -d bijouxshop"]
      interval: 10s
      timeout: 5s
      retries: 5
```

```
    networks:
      - bijouxshop-network
    restart: unless-stopped

  bijouxshop-cache:
    image: redis:7-alpine
    container_name: bijouxshop-cache
    command: redis-server --requirepass ${REDIS_PASSWORD}
    ports:
      - "6379:6379"
    volumes:
      - redis-data:/data
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 10s
      timeout: 3s
      retries: 3
    networks:
      - bijouxshop-network
    restart: unless-stopped

  prometheus:
    image: prom/prometheus:latest
    container_name: bijouxshop-prometheus
    ports:
      - "9090:9090"
    volumes:
      - ./monitoring/prometheus.yml:/etc/prometheus/prometheus.yml:ro
      - prometheus-data:/prometheus
    command:
      - '--config.file=/etc/prometheus/prometheus.yml'
      - '--storage.tsdb.path=/prometheus'
    networks:
      - bijouxshop-network
    restart: unless-stopped

  grafana:
    image: grafana/grafana:latest
    container_name: bijouxshop-grafana
    ports:
      - "3000:3000"
    environment:
      GF_SECURITY_ADMIN_PASSWORD: ${GRAFANA_PASSWORD}
      GF_INSTALL_PLUGINS: grafana-piechart-panel
    volumes:
      - grafana-data:/var/lib/grafana
      - ./monitoring/grafana/dashboards:/etc/grafana/provisioning/
    dashboards:ro
      - ./monitoring/grafana/datasources:/etc/grafana/provisioning/
    datasources:ro
    networks:
      - bijouxshop-network
    depends_on:
      - prometheus
    restart: unless-stopped

networks:
  bijouxshop-network:
```

```
110     driver: bridge
111
112 volumes:
113   postgres-data:
114   redis-data:
115   app-logs:
116   prometheus-data:
117   grafana-data:
118
119 secrets:
120   db_password:
121     file: ./secrets/db_password.txt
```

1.3.2 Kubernetes : Orchestration en production

Architecture Kubernetes détaillée

Chapitre 1

Études de Cas DevOps Détaillées

Introduction aux études de cas

Ce chapitre présente des études de cas réelles et détaillées d'organisations qui ont réussi leur transformation DevOps. Chaque étude analyse le contexte, les défis, les solutions mises en place, les résultats obtenus et les leçons apprises.

1.1 Netflix : Le pionnier du Cloud DevOps

1.1.1 Contexte et historique

Netflix est aujourd'hui l'une des références mondiales en matière de pratiques DevOps et cloud native. Leur transformation est d'autant plus remarquable qu'elle a débuté en 2008, à une époque où le cloud était encore naissant et DevOps n'existait même pas comme terme.

La crise de 2008 : Le catalyseur du changement

En août 2008, Netflix a subi une panne majeure de sa base de données qui a duré trois jours. Cette panne a paralysé l'ensemble du service de location de DVD par courrier, leur activité principale à l'époque. Les conséquences ont été multiples :

— **Impact client** : Impossibilité pour des millions de clients de commander ou de gérer leurs locations
— **Perte financière** : Plusieurs millions de dollars de revenus perdus
— **Dommage réputationnel** : Couverture médiatique négative et mécontentement massif
— **Révélation structurelle** : Mise en évidence de faiblesses fondamentales de l'architecture

Cette crise a conduit Netflix à une prise de conscience majeure : leur architecture monolithique hébergée dans leur propre datacenter constituait un point de défaillance unique (Single Point of Failure - SPOF) inacceptable pour une entreprise dont le business reposait entièrement sur la disponibilité de ses services.

La vision stratégique

Suite à cette crise, Netflix a pris des décisions stratégiques audacieuses :

Migration complète vers AWS Plutôt que d'investir davantage dans leurs propres datacenters, Netflix a décidé de migrer l'intégralité de son infrastructure vers Amazon Web Services (AWS). Cette décision, prise en 2008-2009, était révolutionnaire pour plusieurs raisons :

1. **Taille de l'organisation** : Netflix n'était pas une startup, mais une entreprise établie avec des millions de clients
2. **Maturité du cloud** : AWS n'avait que 2 ans d'existence et était considéré comme non éprouvé pour des workloads critiques
3. **Ampleur de la migration** : Migration complète, pas seulement de quelques applications
4. **Durée** : La migration complète a pris 7 ans (2008-2015), témoignant de sa complexité

Architecture microservices En parallèle de la migration cloud, Netflix a entrepris une refonte complète de son architecture vers des microservices. Cette transformation visait à :

— Éliminer les points de défaillance uniques

— Permettre le déploiement indépendant de chaque service

— Faciliter le scaling horizontal

— Accélérer le développement par des équipes autonomes

1.1.2 Principes et pratiques DevOps de Netflix

Freedom and Responsibility

Netflix a développé une culture d'entreprise unique basée sur le principe "Freedom and Responsibility" (Liberté et Responsabilité). Ce principe se décline en plusieurs aspects :

Autonomie des équipes Chaque équipe est propriétaire de bout en bout (end-to-end ownership) de ses services :

— **Développement** : L'équipe développe ses fonctionnalités

— **Déploiement** : L'équipe déploie quand elle le souhaite, sans validation externe

— **Opérations** : L'équipe gère ses services en production 24/7

— **Monitoring** : L'équipe met en place et surveille ses propres métriques

— **Incidents** : L'équipe répond aux incidents concernant ses services

Ce modèle incarne parfaitement le principe DevOps "You build it, you run it" popularisé par Werner Vogels d'Amazon.

Pas de Change Advisory Board (CAB) Contrairement à la plupart des grandes entreprises, Netflix n'a pas de comité de validation des changements. Les équipes peuvent déployer en production à tout moment sans approbation. Cette liberté s'accompagne d'une responsabilité totale en cas de problème.

Déploiements fréquents En moyenne, Netflix effectue :

— Plus de 4000 déploiements par jour

— Des déploiements 24/7, même pendant les heures de forte audience

— Des changements d'infrastructure réguliers

Chaos Engineering

Netflix est le créateur et le champion du Chaos Engineering, une discipline qui consiste à injecter délibérément des pannes dans les systèmes de production pour tester leur résilience.

Philosophie La philosophie du Chaos Engineering repose sur plusieurs constats :

1. **Les pannes sont inévitables** : Dans un système distribué complexe, des pannes se produiront
2. **Mieux vaut provoquer que subir** : Il vaut mieux découvrir les faiblesses de manière contrôlée que lors d'un incident réel
3. **La production est le seul vrai test** : Aucun environnement de test ne peut reproduire fidèlement la complexité de la production
4. **La confiance se construit par l'expérience** : La résilience se prouve par des tests réguliers

Chaos Monkey Chaos Monkey est l'outil le plus célèbre de Netflix. Il fonctionne ainsi :

— **Cible** : Instances EC2 en production

— **Action** : Terminaison aléatoire d'instances pendant les heures de travail

— **Fréquence** : Plusieurs fois par jour

— **Notification** : Les équipes ne sont pas prévenues à l'avance

— **Impact attendu** : Aucun impact utilisateur si l'architecture est correctement résiliente

Simian Army Netflix a ensuite développé toute une "armée" d'outils de chaos :

— **Chaos Kong** : Simule la panne d'une région AWS entière

— **Chaos Gorilla** : Simule la panne d'une zone de disponibilité (AZ)

— **Latency Monkey** : Introduit des latences artificielles

— **Doctor Monkey** : Détecte les instances malsaines

— **Janitor Monkey** : Nettoie les ressources non utilisées

— **Conformity Monkey** : Vérifie la conformité aux bonnes pratiques

— **Security Monkey** : Surveille les problèmes de sécurité

ChAP (Chaos Automation Platform) Plus récemment, Netflix a développé ChAP, une plateforme d'automatisation du chaos engineering qui permet :

— Définition d'expériences de chaos en code

— Exécution automatisée et planifiée

— Analyse automatique des résultats

— Rollback automatique si impact utilisateur détecté

— Intégration avec les pipelines CI/CD

Outils open source de Netflix

Netflix a open-sourcé de nombreux outils qui sont devenus des standards de l'industrie :

Eureka (Service Discovery) Eureka est un service registry permettant la découverte de services dans une architecture microservices :

Fonctionnement :

1. Chaque microservice s'enregistre auprès d'Eureka au démarrage
2. Eureka maintient un registre de tous les services disponibles
3. Les clients interrogent Eureka pour découvrir les instances disponibles
4. Eureka effectue des health checks réguliers
5. Les instances défaillantes sont automatiquement retirées du registre

Avantages :

— Découplage entre services

— Load balancing côté client

— Haute disponibilité (mode cluster)

— Pas de single point of failure

Hystrix (Circuit Breaker) Hystrix implémente le pattern Circuit Breaker pour gérer les défaillances en cascade :

Concept du Circuit Breaker :

— **État fermé (normal)** : Les requêtes passent normalement

— **État ouvert (défaillance)** : Après un certain nombre d'échecs, le circuit s'ouvre et les requêtes sont immédiatement rejetées sans appeler le service défaillant

— **État semi-ouvert (récupération)** : Après un timeout, quelques requêtes test sont autorisées pour vérifier si le service est rétabli

Fonctionnalités :

— Fallback automatique vers des réponses par défaut

— Isolation des threads pour éviter l'épuisement des ressources

— Timeouts configurables

— Métriques en temps réel

— Dashboard de monitoring

Ribbon (Client-side Load Balancer) Ribbon est un load balancer côté client offrant :

— Algorithmes de load balancing multiples (round-robin, weighted response time, availability filtering)

— Intégration avec Eureka pour discovery automatique

— Retry logic configurable

— Métriques détaillées par instance

Zuul (API Gateway) Zuul est une gateway qui fait office de point d'entrée unique pour tous les clients :

Responsabilités :

— **Routing dynamique** : Acheminement des requêtes vers les microservices appropriés

— **Authentification et sécurité** : Vérification des credentials, protection contre les attaques

— **Monitoring et métriques** : Collecte de métriques centralisées

— **Rate limiting** : Limitation du taux de requêtes par client

— **Load shedding** : Rejet des requêtes en cas de surcharge

Système de filtres :
Zuul utilise un système de filtres exécutés à différentes étapes :

— **Pre filters** : Avant le routing (authentification, logging)

— **Routing filters** : Pendant le routing (requête HTTP, streaming)

— **Post filters** : Après le routing (ajout de headers, compression)

— **Error filters** : En cas d'erreur

Spinnaker (Continuous Delivery) Spinnaker est la plateforme de déploiement continu multi-cloud de Netflix :

Capacités :

— Déploiement sur AWS, GCP, Azure, Kubernetes

— Stratégies de déploiement avancées (canary, blue-green, rolling)

— Pipelines as Code

— Rollback automatique basé sur les métriques

— Intégration avec Chaos Monkey pour validation de résilience

— Gestion d'images et d'artefacts

Architecture de pipeline typique dans Spinnaker :

1. **Baking** : Création d'une image machine (AMI) immuable
2. **Deploy to Test** : Déploiement automatique en environnement de test
3. **Integration Tests** : Exécution de tests automatisés
4. **Manual Judgment** : Validation humaine optionnelle
5. **Canary Deployment** : Déploiement progressif en production (1% puis 10% puis 50% puis 100%)
6. **Monitoring** : Surveillance des métriques pendant le canary
7. **Automated Rollback** : Rollback si les métriques dégradent

1.1.3 Architecture technique de Netflix

Vue d'ensemble de l'architecture

L'architecture de Netflix en production comprend :

— **Plus de 1000 microservices** différents
— **Plusieurs millions d'instances** actives simultanément
— **3 régions AWS** principales (US-East, US-West, EU-West)
— **Déploiement multi-AZ** dans chaque région

Exemple de flux utilisateur

Prenons l'exemple d'un utilisateur qui lance la lecture d'un film :

Étape 1 : Authentification et autorisation

1. La requête arrive sur Zuul (API Gateway)
2. Zuul vérifie le JWT token
3. Si invalide, redirection vers le service d'authentification
4. Si valide, extraction des informations utilisateur et enrichissement de la requête

Étape 2 : Récupération du profil utilisateur

1. Zuul route vers le service User Profile
2. Le service interroge d'abord le cache Redis
3. Si cache miss, interrogation de la base Cassandra
4. Mise à jour du cache pour les prochaines requêtes
5. Retour des préférences utilisateur (qualité préférée, sous-titres, etc.)

Étape 3 : Vérification des droits de lecture

1. Appel au service Subscription Management
2. Vérification de l'abonnement actif
3. Vérification des restrictions géographiques
4. Vérification du nombre d'écrans simultanés

Étape 4 : Sélection du contenu

1. Appel au service Content Metadata
2. Récupération des métadonnées du film
3. Appel au service Encoding pour obtenir les versions disponibles
4. Sélection de la version appropriée selon :
   — Bande passante disponible (détectée par le client)
   — Appareil utilisé (TV, smartphone, ordinateur)
   — Préférences utilisateur

Étape 5 : Streaming

1. Appel au service Play API
2. Génération d'une URL signée temporaire
3. Redirection vers le CDN (Open Connect)
4. Streaming adaptatif du contenu
5. Monitoring en temps réel de la qualité d'expérience

Étape 6 : Analytics et recommandations En arrière-plan, plusieurs services sont activés :

— Enregistrement de l'événement de lecture
— Mise à jour de l'historique de visionnage
— Alimentation du moteur de recommandation
— Collecte de métriques de qualité (buffering, erreurs)

Gestion des données chez Netflix

Netflix utilise une approche polyglotte de la persistence :

Apache Cassandra **Usage** : Base de données principale pour les données distribuées
Cas d'usage chez Netflix :

— Profils utilisateurs (230+ millions de profils)
— Historique de visionnage
— Liste de lecture (My List)
— Préférences

Configuration :

— Clusters de plusieurs centaines de nœuds
— Réplication multi-région
— Facteur de réplication de 3
— Tuning agressif des timeouts pour accepter les lectures éventuellement cohérentes

Amazon DynamoDB **Usage** : Tables simples nécessitant une faible latence
Cas d'usage :

— Sessions utilisateur
— Tokens d'authentification
— Configuration de services

Amazon S3 **Usage** : Stockage objet pour contenu statique
Cas d'usage :

— Images (jaquettes, vignettes)
— Sous-titres
— Métadonnées de contenu
— Logs archivés
— Sauvegardes

Volume : Plusieurs pétaoctets de données

MySQL (Amazon RDS) **Usage** : Données relationnelles critiques nécessitant des transactions ACID

Cas d'usage :

— Facturation et paiements

— Gestion des abonnements

— Données de production (droits, contrats)

Elasticsearch **Usage** : Recherche et analytics

Cas d'usage :

— Recherche de contenu

— Logs centralisés

— Métriques opérationnelles

— Analyse de comportement utilisateur

1.1.4 Monitoring et observabilité

Netflix a développé une approche sophistiquée du monitoring à trois niveaux :

Niveau 1 : Métriques d'infrastructure

Atlas Atlas est la plateforme interne de métriques temps réel de Netflix :

Caractéristiques :

— Collecte de millions de métriques par seconde

— Agrégation en temps réel

— Requêtes ad-hoc avec un langage dédié

— Visualisation intégrée

— Alerting automatique

Métriques collectées :

— CPU, mémoire, disque, réseau par instance

— Métriques JVM (heap, GC, threads)

— Métriques de middleware (Tomcat, Netty)

— Métriques personnalisées applicatives

Niveau 2 : Métriques applicatives

Spectator Bibliothèque Java pour instrumenter le code :

```
// Compteur simple
Counter requestCounter = registry.counter("api.requests",
    "endpoint", "/api/users",
    "method", "GET");
requestCounter.increment();

// Gauge pour une valeur instantan e
registry.gauge("queue.size", queueSizeSupplier);

// Timer pour mesurer la latence
```

```
Timer.Sample sample = Timer.start(registry);
try {
    processRequest();
} finally {
    sample.stop(registry.timer("api.latency",
        "endpoint", "/api/users"));
}

// Distribution summary
DistributionSummary payloadSize =
    registry.summary("api.payload.size");
payloadSize.record(responseBody.length());
```

Niveau 3 : Métriques métier

Netflix suit de près les métriques qui impactent directement le business :

Quality of Experience (QoE)

— **Play Delay** : Temps entre le clic sur play et le début de la lecture

— **Rebuffering Ratio** : Pourcentage du temps passé en buffering

— **Bitrate** : Qualité vidéo moyenne par session

— **Errors** : Taux d'erreurs de lecture

Chaque dégradation de ces métriques est immédiatement détectée et peut déclencher un rollback automatique.

Engagement

— Nombre de titres lancés par jour

— Durée moyenne de visionnage

— Taux de complétion des contenus

— Taux de réabonnement

1.1.5 Résultats et impact

La transformation DevOps de Netflix a produit des résultats remarquables :

Résultats techniques

— **Disponibilité** : 99.99% de uptime (moins de 1h de downtime par an)

— **Déploiements** : Plus de 4000 déploiements par jour sans interruption de service

— **Scalabilité** : Capacité à servir 230+ millions d'abonnés dans 190+ pays

— **Performance** : Latence médiane de moins de 1 seconde pour le démarrage de la lecture

— **Résilience** : Capacité à perdre une région AWS entière sans impact utilisateur

— **Time to market** : Nouvelles fonctionnalités livrées en production en heures/jours au lieu de mois

Résultats business

— **Croissance explosive** : De 20 millions d'abonnés en 2010 à 230+ millions en 2023
— **Expansion internationale** : Présence dans 190+ pays
— **Revenus** : Plus de 30 milliards de dollars de revenus annuels
— **Innovation** : Leader du streaming, créateur de contenu original primé
— **Efficacité** : Coûts d'infrastructure optimisés malgré une croissance massive

Impact culturel

— **Modèle organisationnel** : La culture Netflix est étudiée dans les écoles de business
— **Influence industrielle** : Nombreuses entreprises ont copié leurs pratiques
— **Open source** : Leurs outils sont utilisés par des milliers d'entreprises
— **Chaos Engineering** : Discipline créée et popularisée par Netflix

1.1.6 Leçons et best practices

Ce qui a fonctionné

1. **Culture avant technologie**
   Netflix a commencé par transformer sa culture avant de déployer des technologies. Le document "Netflix Culture" a été publié en 2009, définissant les valeurs de liberté et responsabilité.
   Leçon : La transformation DevOps est avant tout culturelle. Les outils viennent ensuite.
2. **Engagement du leadership**
   Le CEO Reed Hastings et le CTO (aujourd'hui CEO) Greg Peters ont personnellement porté la transformation.
   Leçon : Sans soutien fort du top management, une transformation DevOps à grande échelle échouera.
3. **Architecture pour le chaos**
   En assumant que tout peut tomber en panne, Netflix a conçu une architecture intrinsèquement résiliente.
   Leçon : Design for failure, not for success. La résilience doit être un principe architectural de base.
4. **Automatisation intensive**
   Pratiquement tout est automatisé chez Netflix : builds, tests, déploiements, scaling, recovery.
   Leçon : L'automatisation n'est pas une optimisation, c'est une nécessité pour opérer à grande échelle.
5. **Mesure obsessionnelle**
   Netflix mesure tout : métriques techniques, métriques applicatives, métriques métier.
   Leçon : "You can't improve what you don't measure." Les décisions doivent être data-driven.
6. **Expérimentation constante**
   Netflix teste continuellement via A/B tests, chaos experiments, canary deployments.
   Leçon : L'amélioration continue nécessite une culture d'expérimentation et d'apprentissage.

Les défis rencontrés

1. **Complexité de la migration**
   La migration vers AWS a pris 7 ans, bien plus que prévu initialement. La complexité technique et organisationnelle avait été sous-estimée.
   Leçon : Les grandes migrations prennent toujours plus de temps que prévu. Il faut planifier une approche incrémentale.
2. **Gestion de la dette technique**
   La vitesse de développement a parfois créé de la dette technique qu'il a fallu rembourser.
   Leçon : Même avec DevOps, il faut allouer du temps au refactoring et à la maintenance.
3. **Formation et recrutement**
   Trouver et former des ingénieurs capables d'opérer dans ce nouveau modèle a été difficile.
   Leçon : La transformation DevOps nécessite un investissement massif en formation et en recrutement.
4. **Coûts initiaux élevés**
   L'investissement initial (double infrastructure pendant la migration, recrutement, formation) a été considérable.
   Leçon : La transformation DevOps a un coût. Il faut l'anticiper et le justifier par un business case solide.

Recommandations pour répliquer le succès

Pour une organisation souhaitant s'inspirer de Netflix :

1. **Commencer petit**
   Ne pas essayer de tout transformer d'un coup. Choisir une équipe pilote, un produit, et démontrer la valeur.
2. **Investir dans la culture**
   Organiser des formations, des workshops, des retours d'expérience. Célébrer les succès et apprendre des échecs.
3. **Automatiser progressivement**
   Identifier les processus manuels les plus coûteux et les automatiser un par un.
4. **Mesurer l'impact**
   Définir des métriques claires (métriques DORA) et suivre l'évolution.
5. **Accepter l'échec**
   Créer un environnement où l'expérimentation et l'échec sont acceptés et encouragés.
6. **Partager et documenter**
   Documenter les processus, partager les connaissances, organiser des guildes DevOps.

1.2 Amazon : Scaling DevOps à l'extrême

1.2.1 Contexte et défis

Amazon est aujourd'hui le plus grand retailer en ligne au monde et le leader du cloud computing avec AWS. Leur transformation DevOps est remarquable par son échelle : des dizaines de milliers d'ingénieurs déployant du code des millions de fois par an.

L'incident de 2001 : Le tournant

En 2001, Amazon subissait des déploiements extrêmement lents et risqués. Un déploiement majeur pouvait prendre plusieurs semaines et causait régulièrement des pannes importantes. Le site tombait fréquemment, particulièrement pendant les périodes de forte affluence (Black Friday, Noël).

Cette situation était inacceptable pour une entreprise dont l'activité dépendait entièrement de la disponibilité de son site web.

1.2.2 Transformation architecturale

La directive de Jeff Bezos (2002)

En 2002, Jeff Bezos a envoyé un mémo devenu légendaire à tous les ingénieurs :

2. Teams must communicate with each other through these interfaces.
3. There will be no other form of interprocess communication allowed.
4. It doesn't matter what technology you use.
5. All service interfaces, without exception, must be designed from the ground up to be externalizable.
6. Anyone who doesn't do this will be fired. *1. All teams will henceforth expose their data and functionality through service interfaces.*
2. Teams must communicate with each other through these interfaces.
3. There will be no other form of interprocess communication allowed.
4. It doesn't matter what technology you use.
5. All service interfaces, without exception, must be designed from the ground up to be externalizable.
6. Anyone who doesn't do this will be fired.

Ce mémo a forcé Amazon à adopter une architecture orientée services (SOA) qui a jeté les bases de leur future architecture microservices.

Two-Pizza Teams

Amazon a organisé ses équipes selon la règle des "Two-Pizza Teams" : une équipe doit pouvoir être nourrie avec deux pizzas, soit généralement 5-8 personnes.

Caractéristiques :

— Ownership complet d'un ou plusieurs services
— Autonomie totale sur les choix technologiques
— Responsabilité 24/7 (on-call rotation)
— Communication minimale avec les autres équipes (via APIs uniquement)

1.2.3 Pratiques DevOps d'Amazon

Déploiement continu à l'échelle

Chiffres impressionnants :

— Plus de 50 millions de déploiements par an (2021)

— Un déploiement toutes les 11,7 secondes en moyenne
— Des milliers de services différents
— Des centaines de milliers de serveurs

Chapitre 1

Tutoriels Pratiques Détaillés pour BijouxShop

1.1 Tutoriel 1 : Configuration complète de l'environnement de développement

Ce tutoriel guide pas à pas la mise en place d'un environnement de développement DevOps complet pour le projet BijouxShop.

1.1.1 Installation et configuration de Git

Installation de Git

Sur Ubuntu/Debian :

```
sudo apt update
sudo apt install git -y
git --version
```

Sur macOS :

```
brew install git
git --version
```

Sur Windows : Télécharger depuis git-scm.com

Configuration initiale

Configuration de l'identité :

```
git config --global user.name "Dr. BADR EL KHALYLY"
git config --global user.email "b.elkhalyly@university.ma"
git config --global init.defaultBranch main
```

Configuration d'un éditeur par défaut :

```
git config --global core.editor "vim"
# ou pour VS Code :
git config --global core.editor "code --wait"
```

Vérification de la configuration :

```
git config --list
git config user.name
git config user.email
```

Génération de clés SSH

Génération d'une nouvelle clé SSH :

```
ssh-keygen -t ed25519 -C "b.elkhalyly@university.ma"
# Appuyez sur Entr e pour le chemin par d faut
# Entrez une passphrase (optionnel mais recommand )
```

Démarrage de l'agent SSH :

```
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
```

Affichage de la clé publique :

```
cat ~/.ssh/id_ed25519.pub
# Copiez cette cl   et ajoutez-la    GitLab/GitHub
```

1.1.2 Installation de Java et Maven

Installation de Java 17

Sur Ubuntu/Debian :

```
sudo apt install openjdk-17-jdk -y
java -version
javac -version
```

Configuration de JAVA_HOME :

```
echo 'export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64' >> ~/.bashrc
echo 'export PATH=$JAVA_HOME/bin:$PATH' >> ~/.bashrc
source ~/.bashrc
```

Installation de Maven

```
sudo apt install maven -y
mvn --version
```

Configuration de Maven settings.xml :

```
mkdir -p ~/.m2
cat > ~/.m2/settings.xml << 'EOF'
<?xml version="1.0" encoding="UTF-8"?>
<settings>
  <mirrors>
    <mirror>
      <id>maven-central</id>
      <mirrorOf>central</mirrorOf>
      <url>https://repo1.maven.org/maven2</url>
    </mirror>
  </mirrors>
```

```
</settings>
EOF
```

1.1.3 Installation de PostgreSQL

Installation du serveur PostgreSQL

```
sudo apt install postgresql postgresql-contrib -y
sudo systemctl status postgresql
```

Configuration de PostgreSQL pour BijouxShop

Connexion en tant qu'utilisateur postgres :

```
sudo -u postgres psql
```

Création de la base de données et de l'utilisateur :

```
-- Cr ation de l'utilisateur
CREATE USER bijouxshop_user WITH PASSWORD 'bijouxshop_pass';

-- Cr ation de la base de donn es
CREATE DATABASE bijouxshop;

-- Attribution des privil ges
GRANT ALL PRIVILEGES ON DATABASE bijouxshop TO bijouxshop_user;

-- Sortie
\q
```

Test de connexion :

```
psql -h localhost -U bijouxshop_user -d bijouxshop
# Entrez le mot de passe : bijouxshop_pass
```

1.1.4 Installation et configuration de Docker

Installation de Docker

Sur Ubuntu :

```
# Mise    jour des paquets
sudo apt update

# Installation des pr requis
sudo apt install apt-transport-https ca-certificates curl software-
    properties-common -y

# Ajout de la c l   GPG Docker
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --
    dearmor -o /usr/share/keyrings/docker-archive-keyring.gpg

# Ajout du d p t Docker
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/
    keyrings/docker-archive-keyring.gpg] https://download.docker.com/
    linux/ubuntu $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.
    list.d/docker.list > /dev/null
```

```
12
13 # Installation de Docker
14 sudo apt update
15 sudo apt install docker-ce docker-ce-cli containerd.io -y
16
17 # V rification
18 sudo docker --version
19 sudo docker run hello-world
```

Configuration post-installation

Ajout de votre utilisateur au groupe docker :

```
1 sudo usermod -aG docker $USER
2 # Red marrez la session pour appliquer les changements
3 newgrp docker
4
5 # Test sans sudo
6 docker ps
```

Installation de Docker Compose

```
1 sudo apt install docker-compose-plugin -y
2 docker compose version
```

1.1.5 Clonage et configuration du projet BijouxShop

Clonage du repository

```
1 cd ~/projects
2 git clone git@gitlab.com:votre-username/bijouxshop.git
3 cd bijouxshop
```

Configuration de l'application

Création du fichier de configuration :

```
1 cp src/main/resources/application.yml.example src/main/resources/
    application.yml
```

Édition du fichier application.yml :

```
1  spring:
2    application:
3      name: bijouxshop
4    datasource:
5      url: jdbc:postgresql://localhost:5432/bijouxshop
6      username: bijouxshop_user
7      password: bijouxshop_pass
8      driver-class-name: org.postgresql.Driver
9    jpa:
10     hibernate:
11       ddl-auto: update
12     show-sql: true
```

```
    properties:
      hibernate:
        dialect: org.hibernate.dialect.PostgreSQLDialect
        format_sql: true
server:
  port: 8080
```

Build et exécution du projet

Build avec Maven :

```
mvn clean install
```

Exécution de l'application :

```
mvn spring-boot:run
```

Test de l'application :

```
curl http://localhost:8080/actuator/health
# R sultat attendu : {"status":"UP"}
```

1.2 Tutoriel 2 : Configuration du pipeline CI/CD avec GitLab

Ce tutoriel explique comment configurer un pipeline CI/CD complet pour BijouxShop.

1.2.1 Création du fichier .gitlab-ci.yml

Structure de base

Créez le fichier .gitlab-ci.yml à la racine du projet :

```
stages:
  - build
  - test
  - package
  - deploy

variables:
  MAVEN_OPTS: "-Dmaven.repo.local=.m2/repository"

cache:
  paths:
    - .m2/repository
    - target/

before_script:
  - echo "Pipeline started at $(date)"
```

Stage Build

```
build:
  stage: build
  image: maven:3.9-eclipse-temurin-17
  script:
    - echo "Building BijouxShop..."
    - mvn clean compile
  artifacts:
    paths:
      - target/classes
    expire_in: 1 hour
```

Stage Test

```
test:unit:
  stage: test
  image: maven:3.9-eclipse-temurin-17
  script:
    - echo "Running unit tests..."
    - mvn test
  coverage: '/Total.*?([0-9]{1,3})%/'
  artifacts:
    when: always
    reports:
      junit:
        - target/surefire-reports/TEST-*.xml
    paths:
      - target/site/jacoco
    expire_in: 1 week

test:integration:
  stage: test
  image: maven:3.9-eclipse-temurin-17
  services:
    - postgres:15-alpine
  variables:
    POSTGRES_DB: bijouxshop_test
    POSTGRES_USER: test_user
    POSTGRES_PASSWORD: test_pass
    SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/
       bijouxshop_test
    SPRING_DATASOURCE_USERNAME: test_user
    SPRING_DATASOURCE_PASSWORD: test_pass
  script:
    - echo "Running integration tests..."
    - mvn verify -Pintegration-tests
  artifacts:
    when: always
    reports:
      junit:
        - target/failsafe-reports/TEST-*.xml
```

Stage Package

```
package:jar:
  stage: package
  image: maven:3.9-eclipse-temurin-17
  script:
    - echo "Packaging BijouxShop..."
    - mvn package -DskipTests
  artifacts:
    paths:
      - target/*.jar
    expire_in: 1 week

package:docker:
  stage: package
  image: docker:24
  services:
    - docker:24-dind
  script:
    - echo "Building Docker image..."
    - docker build -t bijouxshop:$CI_COMMIT_SHA .
    - docker tag bijouxshop:$CI_COMMIT_SHA bijouxshop:latest
  only:
    - main
    - develop
```

Stage Deploy

```
deploy:staging:
  stage: deploy
  image: alpine:latest
  before_script:
    - apk add --no-cache openssh-client
  script:
    - echo "Deploying to staging..."
    - echo "Application deployed successfully!"
  environment:
    name: staging
    url: https://staging.bijouxshop.com
  only:
    - develop

deploy:production:
  stage: deploy
  image: alpine:latest
  before_script:
    - apk add --no-cache openssh-client
  script:
    - echo "Deploying to production..."
    - echo "Application deployed successfully!"
  environment:
    name: production
    url: https://bijouxshop.com
  when: manual
  only:
    - main
```

1.2.2 Configuration des variables GitLab CI

Dans GitLab, allez dans Settings > CI/CD > Variables et ajoutez :
— DATABASE_URL
— DATABASE_USERNAME
— DATABASE_PASSWORD
— DOCKER_REGISTRY_USER
— DOCKER_REGISTRY_PASSWORD

1.3 Tutoriel 3 : Conteneurisation avec Docker

1.3.1 Création du Dockerfile

Créez un Dockerfile à la racine du projet BijouxShop.

1.3.2 Création du fichier docker-compose.yml

Cela permet de lancer l'application complète localement avec tous ses services.

1.3.3 Optimisation des images Docker

Utilisez des multi-stage builds pour réduire la taille des images.

1.4 Tutoriel 4 : Déploiement sur Kubernetes

1.4.1 Installation de kubectl

Installation de l'outil en ligne de commande Kubernetes.

1.4.2 Création des manifests Kubernetes

Configuration des déploiements, services et ingress pour BijouxShop.

Chapitre 1

Exercices Pratiques Complets

1.1 Exercices Chapitre 1 : Introduction à DevOps

1.1.1 Exercice 1.1 : Analyse du mur de la confusion

Objectif : Identifier les problèmes organisationnels dans un contexte réel.

Scénario : Vous travaillez dans une entreprise traditionnelle où les équipes Dev et Ops sont séparées. Analysez les problèmes suivants et proposez des solutions DevOps.

Questions :

1. Décrivez trois problèmes causés par la séparation Dev/Ops dans votre organisation (fictive ou réelle).

2. Pour chaque problème identifié, proposez une solution basée sur les principes DevOps.

3. Quelles seraient les résistances au changement et comment les surmonter ?

Correction détaillée :

Problème 1 : Déploiements lents et risqués
Solution : Automatisation du pipeline CI/CD
Résistances : Peur du changement, manque de compétences
Plan : Formation progressive, pilote sur un projet

Problème 2 : Manque de communication entre équipes
Solution : Équipes cross-fonctionnelles, standups communs
Résistances : Culture de silos établie
Plan : Ateliers collaboratifs, objectifs partagés

Problème 3 : Environnements incohérents
Solution : Infrastructure as Code, conteneurisation
Résistances : Investissement initial élevé
Plan : ROI démontré, migration progressive

1.1.2 Exercice 1.2 : Métriques DORA

Objectif : Comprendre et calculer les métriques DORA.

Données :

- Nombre de déploiements en janvier : 45
- Lead time moyen : 3 jours
- Nombre de changements échoués : 9
- MTTR moyen : 2 heures

Questions :

1. Calculez la fréquence de déploiement hebdomadaire.

2. Calculez le taux d'échec des changements.

3. Classez l'organisation selon les catégories DORA.

Correction :

1. Fréquence = 45 déploiements / 4 semaines = 11.25 déploiements/semaine
Classement : Medium performer (plusieurs fois par semaine)

2. Taux d'échec = $(9/45) \times 100 = 20\%$
Classement : Medium performer (16-30%)

3. Lead time = 3 jours
Classement : Medium performer (1 semaine à 1 mois)

4. MTTR = 2 heures
Classement : High performer ($<$ 1 jour)

Classification globale : Medium performer

1.2 Exercices Chapitre 2 : Les Fondements

1.2.1 Exercice 2.1 : CALMS Framework

Objectif : Appliquer le framework CALMS à une situation réelle.

Scénario : Vous êtes consultant DevOps dans une banque traditionnelle. Proposez des actions concrètes pour chaque pilier CALMS.

Questions détaillées :

1. Culture : Comment transformer la culture de l'organisation ?
2. Automation : Quels processus automatiser en premier ?
3. Lean : Comment éliminer les gaspillages ?
4. Measurement : Quelles métriques mettre en place ?
5. Sharing : Comment favoriser le partage de connaissances ?

Correction complète :

1. Culture :
- Organisation de workshops DevOps
- Formation continue des équipes
- Modification des KPI individuels
- Célébration des succès collectifs
- Post-mortems blameless

2. Automation :
Phase 1 : Automatiser les builds (Jenkins/GitLab CI)
Phase 2 : Automatiser les tests (JUnit, Selenium)
Phase 3 : Automatiser les déploiements (Ansible, Terraform)
Phase 4 : Automatiser le monitoring (Prometheus)

3. Lean :
- Cartographier la value stream
- Identifier les goulots d'étranglement
- Réduire les handoffs entre équipes
- Éliminer les approbations manuelles
- Réduire la taille des batches (petites releases fréquentes)

4. Measurement :
- Métriques DORA : Deployment Frequency, Lead Time, MTTR, Change Failure Rate
- Métriques techniques : Temps de build, taux de réussite CI
- Métriques business : Time to market, satisfaction client
- Dashboard partagé avec toutes les équipes

5. Sharing :
- Guildes DevOps cross-équipes

- Documentation vivante (wiki, confluence)
- Présentations techniques régulières
- Pair programming et mob programming
- Code reviews systématiques

1.3 Exercices Chapitre 3 : Git

1.3.1 Exercice 3.1 : Workflow Git

Objectif : Maîtriser le workflow Git dans un projet collaboratif.
Scénario : Vous rejoignez le projet BijouxShop. Effectuez les opérations suivantes.
Exercice pratique :
1. Clonez le repository
2. Créez une branche feature/add-customer-rating
3. Implémentez une fonctionnalité (simulée)
4. Committez avec un message conventional
5. Pushez votre branche
6. Créez une merge request

Commandes attendues :

```
# 1. Clone
git clone git@gitlab.com:bijouxshop/bijouxshop.git
cd bijouxshop

# 2. Create branch
git checkout -b feature/add-customer-rating

# 3. Make changes
echo "// Customer rating feature" >> src/main/java/Rating.java

# 4. Commit
git add src/main/java/Rating.java
git commit -m "feat: add customer rating functionality

Implement customer rating system with:
- Rating model (1-5 stars)
- Rating repository
- Rating service
- Rating REST API endpoints

Closes #123"

# 5. Push
git push -u origin feature/add-customer-rating

# 6. Create MR (via GitLab UI or CLI)
```

Chapitre 2

Annexes Techniques Complètes

2.1 Annexe A : Commandes Git Complètes

2.1.1 Configuration

```
# Configuration globale
git config --global user.name "Nom Pr nom"
git config --global user.email "email@example.com"
git config --global init.defaultBranch main
git config --global core.editor "vim"
git config --global pull.rebase false
git config --global push.default current

# Afficher la configuration
git config --list
git config --list --show-origin

# Configuration locale (par projet)
git config user.name "Autre Nom"
git config user.email "autre@example.com"
```

2.1.2 Initialisation et clonage

```
# Initialiser un nouveau repository
git init
git init mon-projet
git init --bare mon-repo.git

# Cloner un repository
git clone https://github.com/user/repo.git
git clone git@github.com:user/repo.git
git clone https://github.com/user/repo.git mon-dossier
git clone --depth 1 https://github.com/user/repo.git
git clone --branch develop https://github.com/user/repo.git
```

2.1.3 Opérations de base

```
# Status
git status
git status -s
git status --short

# Ajouter des fichiers
git add fichier.txt
git add *.java
git add src/
git add .
git add -A
git add -p

# Commit
git commit -m "Message de commit"
git commit -am "Message" # add + commit
git commit --amend
git commit --amend --no-edit

# Voir l'historique
git log
git log --oneline
git log --graph --oneline --all
git log -p
git log --stat
git log --since="2 weeks ago"
git log --author="Nom"
git log --grep="keyword"
git log fichier.txt
git log -S "fonction()"

# Voir les diff rences
git diff
git diff --staged
git diff HEAD
git diff branche1 branche2
git diff commit1 commit2
git diff fichier.txt
```

2.1.4 Branches

```
# Lister les branches
git branch
git branch -a
git branch -r
git branch -v
git branch --merged
git branch --no-merged

# Cr er une branche
git branch nouvelle-branche
git checkout -b nouvelle-branche
```

```
git switch -c nouvelle-branche

# Changer de branche
git checkout branche
git switch branche

# Renommer une branche
git branch -m ancien-nom nouveau-nom
git branch -M nom-force

# Supprimer une branche
git branch -d branche
git branch -D branche # force
git push origin --delete branche
```

2.1.5 Merge et Rebase

```
# Merge
git merge branche
git merge --no-ff branche
git merge --squash branche
git merge --abort

# Rebase
git rebase main
git rebase -i HEAD~3
git rebase --continue
git rebase --abort
git rebase --skip
```

2.2 Annexe B : Commandes Docker Complètes

2.2.1 Images

```
# Lister les images
docker images
docker images -a
docker images --filter "dangling=true"

# T l charger une image
docker pull ubuntu:22.04
docker pull postgres:15

# Construire une image
docker build -t nom-image .
docker build -t nom-image:tag .
docker build -f Dockerfile.prod .
docker build --no-cache -t nom-image .

# Tagger une image
```

```
docker tag image:old image:new
docker tag mon-image registry.com/mon-image:v1

# Supprimer des images
docker rmi image-id
docker rmi $(docker images -q)
docker image prune
docker image prune -a

# Sauvegarder/Charger des images
docker save mon-image > image.tar
docker load < image.tar
docker export conteneur > conteneur.tar
docker import conteneur.tar
```

2.2.2 Conteneurs

```
# Lancer un conteneur
docker run ubuntu
docker run -d nginx
docker run -it ubuntu bash
docker run --name mon-conteneur ubuntu
docker run -p 8080:80 nginx
docker run -v /host:/container ubuntu
docker run -e VAR=value ubuntu
docker run --rm ubuntu
docker run --restart=always nginx

# Lister les conteneurs
docker ps
docker ps -a
docker ps -aq
docker ps --filter "status=exited"

# Arr ter des conteneurs
docker stop conteneur-id
docker stop $(docker ps -q)
docker kill conteneur-id

# Supprimer des conteneurs
docker rm conteneur-id
docker rm $(docker ps -aq)
docker container prune

# Logs
docker logs conteneur-id
docker logs -f conteneur-id
docker logs --tail 100 conteneur-id
docker logs --since 30m conteneur-id

# Ex cuter des commandes
docker exec conteneur-id ls
```

```
docker exec -it conteneur-id bash
docker exec -u root conteneur-id whoami

# Inspecter
docker inspect conteneur-id
docker stats conteneur-id
docker top conteneur-id
```

2.3 Annexe C : Commandes Kubernetes Complètes

2.3.1 kubectl de base

```
# Configuration
kubectl config view
kubectl config get-contexts
kubectl config use-context mon-contexte
kubectl config set-context --current --namespace=mon-ns

# Informations cluster
kubectl cluster-info
kubectl get nodes
kubectl describe node nom-node
kubectl top nodes
```

2.3.2 Pods

```
# Lister les pods
kubectl get pods
kubectl get pods -n namespace
kubectl get pods --all-namespaces
kubectl get pods -o wide
kubectl get pods --watch

# Cr er un pod
kubectl run nginx --image=nginx
kubectl run test --image=busybox --rm -it -- sh

# D crire un pod
kubectl describe pod nom-pod
kubectl logs nom-pod
kubectl logs -f nom-pod
kubectl logs nom-pod -c conteneur

# Ex cuter des commandes
kubectl exec nom-pod -- ls
kubectl exec -it nom-pod -- bash

# Supprimer des pods
kubectl delete pod nom-pod
kubectl delete pods --all
```

2.3.3 Deployments

```
# Cr er un deployment
kubectl create deployment nginx --image=nginx
kubectl create deployment mon-app --image=mon-app:v1 --replicas=3

# Lister les deployments
kubectl get deployments
kubectl get deploy

# Mettre    jour un deployment
kubectl set image deployment/mon-app mon-app=mon-app:v2
kubectl rollout status deployment/mon-app
kubectl rollout history deployment/mon-app
kubectl rollout undo deployment/mon-app

# Scaler un deployment
kubectl scale deployment mon-app --replicas=5
kubectl autoscale deployment mon-app --min=2 --max=10 --cpu-percent=80

# Supprimer un deployment
kubectl delete deployment mon-app
```

2.4 Annexe D : Configurations Complètes

2.4.1 Dockerfile Multi-Stage Complet

```
# Stage 1: Build
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /build

COPY pom.xml .
RUN mvn dependency:go-offline

COPY src ./src
RUN mvn clean package -DskipTests

# Stage 2: Runtime
FROM eclipse-temurin:17-jre-alpine

RUN addgroup -S spring && adduser -S spring -G spring
USER spring:spring

WORKDIR /app
COPY --from=build /build/target/*.jar app.jar

EXPOSE 8080

HEALTHCHECK --interval=30s --timeout=3s --start-period=60s \
  CMD wget --quiet --tries=1 --spider \
  http://localhost:8080/actuator/health || exit 1
```

```
ENTRYPOINT ["java", \
    "-XX:+UseContainerSupport", \
    "-XX:MaxRAMPercentage=75.0", \
    "-Djava.security.egd=file:/dev/./urandom", \
    "-jar", "app.jar"]
```

2.4.2 docker-compose.yml Complet

```
version: '3.9'

services:
  bijouxshop-app:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: bijouxshop-app
    ports:
      - "8080:8080"
    environment:
      SPRING_PROFILES_ACTIVE: docker
      SPRING_DATASOURCE_URL: jdbc:postgresql://db:5432/bijouxshop
      SPRING_DATASOURCE_USERNAME: user
      SPRING_DATASOURCE_PASSWORD: password
    depends_on:
      db:
        condition: service_healthy
    networks:
      - bijouxshop-network
    restart: unless-stopped

  db:
    image: postgres:15-alpine
    container_name: bijouxshop-db
    environment:
      POSTGRES_DB: bijouxshop
      POSTGRES_USER: user
      POSTGRES_PASSWORD: password
    ports:
      - "5432:5432"
    volumes:
      - postgres-data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U user"]
      interval: 10s
      timeout: 5s
      retries: 5
    networks:
      - bijouxshop-network
    restart: unless-stopped

networks:
  bijouxshop-network:
```

```
    driver: bridge

volumes:
  postgres-data:
```

2.4.3 .gitlab-ci.yml Complet

```
stages:
  - build
  - test
  - quality
  - package
  - deploy

variables:
  MAVEN_OPTS: "-Dmaven.repo.local=.m2/repository"
  DOCKER_DRIVER: overlay2
  IMAGE_NAME: $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA

cache:
  paths:
    - .m2/repository
    - target/

build:
  stage: build
  image: maven:3.9-eclipse-temurin-17
  script:
    - mvn clean compile
  artifacts:
    paths:
      - target/classes
    expire_in: 1 hour

test:unit:
  stage: test
  image: maven:3.9-eclipse-temurin-17
  script:
    - mvn test
  coverage: '/Total.*?([0-9]{1,3})%/'
  artifacts:
    when: always
    reports:
      junit: target/surefire-reports/TEST-*.xml
    paths:
      - target/site/jacoco
    expire_in: 1 week

quality:sonar:
  stage: quality
  image: maven:3.9-eclipse-temurin-17
  script:
```

```
    - mvn sonar:sonar \
        -Dsonar.projectKey=$SONAR_PROJECT_KEY \
        -Dsonar.host.url=$SONAR_HOST_URL \
        -Dsonar.login=$SONAR_TOKEN
  only:
    - main
    - develop

package:docker:
  stage: package
  image: docker:24
  services:
    - docker:24-dind
  before_script:
    - docker login -u $CI_REGISTRY_USER \
        -p $CI_REGISTRY_PASSWORD $CI_REGISTRY
  script:
    - docker build -t $IMAGE_NAME .
    - docker push $IMAGE_NAME
  only:
    - main
    - develop

deploy:production:
  stage: deploy
  image: alpine:latest
  before_script:
    - apk add --no-cache openssh-client kubectl
  script:
    - kubectl set image deployment/bijouxshop \
        bijouxshop=$IMAGE_NAME
  environment:
    name: production
    url: https://bijouxshop.com
  when: manual
  only:
    - main
```

Chapitre 3

Glossaire DevOps Étendu

3.1 A-C

Agile : Méthodologie de développement logiciel itérative basée sur le Manifeste Agile de 2001.

Ansible : Outil d'automatisation open-source pour la gestion de configuration.

Artifact : Fichier produit par un processus de build (JAR, WAR, image Docker).

AWS : Amazon Web Services, plateforme de cloud computing.

Blue-Green Deployment : Stratégie de déploiement utilisant deux environnements identiques.

Canary Deployment : Déploiement progressif testant sur un sous-ensemble d'utilisateurs.

CI/CD : Continuous Integration / Continuous Delivery/Deployment.

Container : Unité standardisée de logiciel empaquet ant code et dépendances.

3.2 D-F

DevOps : Pratiques combinant développement et opérations.

Docker : Plateforme de conteneurisation.

DORA : DevOps Research and Assessment.

3.3 G-I

Git : Système de contrôle de version distribué.

GitLab CI : Plateforme CI/CD intégrée à GitLab.

IaC : Infrastructure as Code.

3.4 J-L

Jenkins : Serveur d'automatisation open-source.

Kubernetes : Plateforme d'orchestration de conteneurs.

3.5 M-O

Maven : Outil de build pour Java.

Microservices : Architecture décomposant applications en services indépendants.

3.6 P-R

Pipeline : Série automatisée d'étapes de build, test et déploiement.
Prometheus : Système de monitoring et d'alerting.

3.7 S-Z

SonarQube : Plateforme d'analyse de qualité de code.
Terraform : Outil d'Infrastructure as Code.

Chapitre 1

Études de Cas Supplémentaires

1.1 Étude de Cas : Spotify

1.1.1 Contexte organisationnel

Spotify est un service de streaming musical suédois fondé en 2006 qui compte aujourd'hui plus de 500 millions d'utilisateurs actifs mensuels. L'entreprise est devenue une référence en matière d'organisation DevOps, notamment grâce à son modèle organisationnel innovant basé sur les Squads, Tribes, Chapters et Guilds.

1.1.2 Le modèle Spotify

Squads

Une Squad est une équipe cross-fonctionnelle autonome de 5 à 9 personnes qui fonctionne comme une mini-startup. Chaque Squad :

— A une mission claire et long-terme

— Est totalement autonome dans ses décisions techniques

— Possède toutes les compétences nécessaires (dev, ops, design, product)

— Livre directement en production sans dépendances externes

— Mesure son propre succès avec des métriques définies

Tribes

Une Tribe est une collection de Squads qui travaillent sur des domaines connexes. Une Tribe comprend généralement 40 à 150 personnes. Le Tribe Lead est responsable de créer le meilleur environnement possible pour les Squads.

Chapters

Un Chapter est un groupe de personnes ayant des compétences similaires au sein de la même Tribe. Par exemple : Backend Chapter, Frontend Chapter, QA Chapter. Le Chapter Lead a une responsabilité managériale traditionnelle pour le développement des personnes.

Guilds

Une Guild est une communauté d'intérêt qui traverse les Tribes. Exemples : Web Technology Guild, DevOps Guild, Agile Coaching Guild. Les Guilds sont organiques et volontaires.

1.1.3 Pratiques DevOps chez Spotify

Déploiement continu

Spotify déploie des centaines de fois par jour. Chaque Squad peut déployer indépendamment sans coordination avec les autres Squads. Cette autonomie est rendue possible par :

— Architecture microservices découplée
— Tests automatisés exhaustifs à tous les niveaux
— Feature flags pour activation progressive
— Monitoring détaillé et alerting
— Culture de responsabilité et confiance

Infrastructure as Code

Toute l'infrastructure Spotify est définie en code et versionnée. Les Squads utilisent principalement :

— Terraform pour le provisioning d'infrastructure sur Google Cloud Platform
— Ansible pour la configuration des serveurs
— Docker pour la conteneurisation
— Kubernetes pour l'orchestration

Monitoring et Observabilité

Spotify a développé une culture forte d'observabilité avec :

— Distributed tracing pour suivre les requêtes à travers les microservices
— Métriques temps réel collectées avec Prometheus
— Logs centralisés avec Elastic Stack
— Dashboards personnalisés par Squad
— Alerting intelligent avec réduction du bruit

1.1.4 Résultats obtenus

Métriques techniques

— Fréquence de déploiement : Plusieurs centaines par jour
— Lead time : Moins de 10 minutes du commit au déploiement
— Change failure rate : Environ 5%
— MTTR : Moins de 15 minutes
— Disponibilité : 99.95%

Impact organisationnel

— Autonomie maximale des équipes
— Innovation accélérée
— Satisfaction des employés élevée
— Rétention des talents améliorée
— Time-to-market réduit

Leçons apprises

1. L'autonomie doit s'accompagner d'alignment sur la vision globale
2. La communication entre Squads reste importante malgré l'autonomie
3. Les dépendances techniques doivent être activement minimisées
4. La culture doit être cultivée continuellement
5. Le modèle doit évoluer avec la croissance de l'entreprise

1.2 Étude de Cas : Etsy

1.2.1 Contexte

Etsy est une plateforme de e-commerce spécialisée dans les produits artisanaux et vintage. L'entreprise est devenue célèbre dans la communauté DevOps pour sa culture de déploiement continu et son approche "blameless post-mortem".

1.2.2 Transformation DevOps

Avant DevOps (2009)

Situation initiale :

— Déploiements mensuels très risqués
— Processus manuel complexe de plusieurs heures
— Tension élevée entre Dev et Ops
— Peur des déploiements
— Incidents fréquents en production

Catalyseur du changement

En 2009, Etsy a recruté plusieurs ingénieurs de Flickr qui apportaient avec eux la culture DevOps. Ils ont initié une transformation radicale de l'organisation et des processus.

Transformation progressive

Phase 1 : Infrastructure et outillage (2009-2010)

— Migration vers une infrastructure cloud
— Mise en place d'outils de déploiement automatisés
— Implémentation d'une intégration continue
— Amélioration du monitoring

Phase 2 : Culture et pratiques (2010-2011)

— Introduction des déploiements continus
— Promotion de la culture "try bot" (environnement de test personnel)
— Mise en place de post-mortems blameless
— Encouragement de l'expérimentation

Phase 3 : Scaling et maturité (2011+)

— Raffinement des pratiques

— Partage des connaissances (conférences, blogs)
— Amélioration continue des outils
— Extension de DevOps à toute l'organisation

1.2.3 Pratiques clés

Continuous Deployment

Etsy a été l'un des pionniers du déploiement continu à grande échelle. Caractéristiques :

— Plus de 50 déploiements par jour
— N'importe quel ingénieur peut déployer
— Processus de déploiement en un clic
— Déploiements même pendant les heures de pointe
— Rollback rapide en cas de problème

Blameless Post-Mortems

Etsy est célèbre pour avoir popularisé le concept de "blameless post-mortem". Principes :

1. Aucun individu n'est jamais blâmé pour un incident
2. Focus sur les facteurs systémiques qui ont permis l'erreur
3. Documentation détaillée de l'incident et des lessons learned
4. Partage public des post-mortems avec toute l'entreprise
5. Amélioration continue des processus et systèmes

Exemple de structure de post-mortem Etsy :

— Titre et date de l'incident
— Qui était on-call
— Timeline détaillée des événements
— Impact (durée, utilisateurs affectés, revenus perdus)
— Root cause analysis
— Ce qui a bien fonctionné
— Ce qui n'a pas bien fonctionné
— Actions d'amélioration avec responsables et deadlines

Try Bot

Etsy a développé "Try", un système qui permet à chaque ingénieur d'avoir son propre environnement de production-like. Avantages :

— Tests en conditions réelles avant déploiement
— Expérimentation sans risque
— Détection précoce des problèmes
— Confiance accrue pour déployer

1.3 Étude de Cas : Google

1.3.1 Site Reliability Engineering (SRE)

Google a inventé le concept de Site Reliability Engineering, une approche systématique de l'application des principes d'ingénierie logicielle aux opérations.

1.3.2 Principes SRE

Embracing Risk

Google reconnaît qu'une disponibilité de 100% est impossible et souvent non nécessaire. Ils définissent des SLO (Service Level Objectives) basés sur les besoins réels des utilisateurs.

Concepts clés :

— Error Budget : Si un service est plus fiable que son SLO, l'équipe a un "budget" pour prendre plus de risques

— Innovation vs Stabilité : L'error budget permet de balancer innovation et stabilité

— Metrics-driven : Toutes les décisions basées sur des données

Service Level Objectives (SLO)

Structure d'un SLO Google :

— SLI (Service Level Indicator) : Métrique mesurée (ex : latence, disponibilité)

— SLO (Service Level Objective) : Objectif cible (ex : 99.9% de requêtes répondent en $<$ 200ms)

— SLA (Service Level Agreement) : Conséquence si SLO non atteint (généralement contractuel)

Exemples de SLO :

— 99.99% des requêtes HTTP réussissent (non 5xx)

— 99% des requêtes répondent en moins de 100ms

— 99.9% des requêtes de streaming commencent en moins de 500ms

Eliminating Toil

Google définit le toil comme le travail opérationnel manuel, répétitif, automatisable, sans valeur durable, qui croît linéairement avec le service.

Objectifs :

— Maximum 50% du temps SRE sur le toil

— Minimum 50% du temps sur l'engineering (automation, amélioration)

— Mesure et tracking du toil par équipe

— Prioritisation de l'élimination du toil

1.3.3 Pratiques opérationnelles

On-Call Rotation

Pratiques Google pour l'astreinte :

— Rotation entre plusieurs ingénieurs (généralement 2 par service)

— Primary et Secondary on-call

— Durée limitée (généralement 1 semaine)
— Compensation financière pour les astreintes
— Post-mortem obligatoire pour chaque incident majeur
— Amélioration continue du runbook

Incident Management

Process standardisé :

1. Détection (monitoring, alerting)
2. Triage et escalation si nécessaire
3. Mitigation (restaurer le service)
4. Investigation de la root cause
5. Post-mortem blameless
6. Implémentation des remediations

Chapitre 2

Glossaire DevOps Exhaustif

2.1 A

Agile : Ensemble de méthodes de développement logiciel basées sur le Manifeste Agile de 2001, privilégiant l'itération rapide et la collaboration.

ALM (Application Lifecycle Management) : Gestion du cycle de vie complet d'une application, de la conception au retrait.

Ansible : Outil open-source d'automatisation pour la configuration, le déploiement et l'orchestration.

API Gateway : Point d'entrée unique pour un ensemble de microservices, gérant le routing, l'authentification et le rate limiting.

Artifact : Produit d'un processus de build (JAR, WAR, image Docker, etc.).

Artifact Repository : Système de stockage centralisé pour les artefacts (Nexus, Artifactory).

Auto-scaling : Ajustement automatique des ressources en fonction de la charge.

AWS (Amazon Web Services) : Plateforme de cloud computing d'Amazon.

Azure : Plateforme de cloud computing de Microsoft.

Azure DevOps : Suite d'outils DevOps de Microsoft (CI/CD, gestion de projet, etc.).

2.2 B

Bamboo : Serveur d'intégration continue d'Atlassian.

Baseline : Configuration de référence d'un système à un moment donné.

Bitbucket : Plateforme d'hébergement de code Git d'Atlassian.

Blameless Post-Mortem : Analyse d'incident sans recherche de coupable, focalisée sur l'amélioration systémique.

Blue-Green Deployment : Stratégie de déploiement avec deux environnements de production identiques permettant des bascules rapides.

Branch : Ligne de développement séparée dans un système de contrôle de version.

Build : Processus de transformation du code source en artefact exécutable.

Build Agent : Machine qui exécute les jobs de build dans un système CI/CD.

Build Automation : Automatisation du processus de compilation et packaging.

2.3 C

CALMS : Framework DevOps : Culture, Automation, Lean, Measurement, Sharing.

Canary Deployment : Déploiement progressif testant sur un sous-ensemble d'utilisateurs avant rollout complet.

Capacity Planning : Prévision des besoins en ressources pour maintenir les performances.

Cassandra : Base de données NoSQL distribuée hautement scalable.

CD (Continuous Delivery) : Pratique de maintenir le code toujours dans un état déployable.

CD (Continuous Deployment) : Extension de Continuous Delivery avec déploiement automatique en production.

Certificate Authority (CA) : Entité émettant des certificats numériques.

Change Failure Rate : Pourcentage de changements causant des incidents (métrique DORA).

Chaos Engineering : Discipline consistant à tester la résilience en injectant des pannes.

Chaos Monkey : Outil Netflix terminant aléatoirement des instances pour tester la résilience.

Chef : Outil d'automatisation de configuration basé sur Ruby.

CI (Continuous Integration) : Pratique d'intégrer fréquemment le code dans un dépôt partagé avec build et tests automatiques.

Circuit Breaker : Pattern de résilience qui coupe les appels vers un service défaillant.

CloudFormation : Service AWS d'Infrastructure as Code.

Cluster : Groupe de serveurs travaillant ensemble.

Configuration Drift : Divergence progressive d'une configuration par rapport à l'état désiré.

Configuration Management : Gestion systématique de la configuration des systèmes.

Container : Unité de logiciel empaqu etant code et dépendances de manière isolée.

Container Registry : Dépôt pour stocker et distribuer des images de conteneurs.

Containerization : Technique de virtualisation au niveau système d'exploitation.

Consul : Outil de service discovery et configuration distribué.

Correlation ID : Identifiant unique pour tracer une requête à travers des systèmes distribués.

2.4 D

Dashboard : Interface visuelle affichant des métriques et KPIs.

Database Migration : Gestion versionnée des changements de schéma de base de données.

DataDog : Plateforme de monitoring et analytics.

Deployment : Processus de mise en production d'une application.

Deployment Frequency : Fréquence des déploiements en production (métrique DORA).

Deployment Pipeline : Série automatisée d'étapes du commit au déploiement.

DevOps : Culture et pratiques unissant développement et opérations.

DevSecOps : Extension de DevOps intégrant la sécurité à chaque étape.

Distributed Tracing : Suivi d'une requête à travers un système distribué.

Docker : Plateforme de conteneurisation.

Docker Compose : Outil pour définir et exécuter des applications multi-conteneurs.

Docker Hub : Registry public d'images Docker.

Docker Swarm : Solution d'orchestration de conteneurs native à Docker.

Dockerfile : Fichier texte contenant les instructions pour créer une image Docker.

DOR A : DevOps Research and Assessment, organisation de recherche sur DevOps.

DRY (Don't Repeat Yourself) : Principe de ne pas dupliquer la logique.

Dynamic Inventory : Inventaire généré dynamiquement (ex : depuis cloud provider).

2.5 E

ECS (Elastic Container Service) : Service AWS d'orchestration de conteneurs.
EKS (Elastic Kubernetes Service) : Service AWS Kubernetes managé.
Elastic Stack : Suite d'outils (Elasticsearch, Logstash, Kibana) pour recherche et analyse de logs.
Elasticsearch : Moteur de recherche et analytics distribué.
Environment : Instance d'une application (dev, test, staging, production).
Error Budget : Montant d'indisponibilité acceptable avant de devoir ralentir les releases.
Eureka : Service registry pour découverte de services (Netflix OSS).
Event-Driven Architecture : Architecture où les composants communiquent via des événements.

2.6 F

Failover : Basculement automatique vers un système de secours.
Feature Flag : Mécanisme pour activer/désactiver des fonctionnalités sans déploiement.
Feature Toggle : Synonyme de Feature Flag.
Feedback Loop : Cycle de retour d'information permettant l'amélioration continue.
Fluentd : Collecteur de logs open-source.

Partie VII : Exercices Approfondis avec Solutions Détaillées

0.1 Exercices sur Git et Gestion de Versions

0.1.1 Exercice 1 : Résolution de Conflits Complexes

Énoncé :

Vous travaillez sur le projet BijouxShop avec deux développeurs. Le développeur A modifie le fichier `UserService.java` pour ajouter une validation d'email, tandis que le développeur B modifie le même fichier pour ajouter une validation de mot de passe. Les deux modifications touchent les mêmes lignes de code.

Tâches :

1. Créez une situation de conflit en simulant les deux modifications
2. Identifiez les marqueurs de conflit
3. Résolvez le conflit en conservant les deux fonctionnalités
4. Effectuez un commit avec un message approprié

Solution Détaillée :

Étape 1 : Création de la branche développeur A

```
git checkout -b feature/email-validation
# Edition du fichier UserService.java
cat > src/main/java/com/bijouxshop/UserService.java << 'EOF'
public class UserService {
    public boolean validateUser(String email, String password) {
        // Validation email par dev A
        if (!email.matches("^[A-Za-z0-9+_.-]+@(.+)$")) {
            throw new ValidationException("Email invalide");
        }
        return true;
    }
}
EOF

git add src/main/java/com/bijouxshop/UserService.java
git commit -m "feat: Ajout validation email utilisateur"
```

Étape 2 : Création de la branche développeur B

```
git checkout main
git checkout -b feature/password-validation
# Edition du meme fichier
cat > src/main/java/com/bijouxshop/UserService.java << 'EOF'
```

```
public class UserService {
    public boolean validateUser(String email, String password) {
        // Validation mot de passe par dev B
        if (password.length() < 8) {
            throw new ValidationException("Mot de passe trop court");
        }
        return true;
    }
}
EOF

git add src/main/java/com/bijouxshop/UserService.java
git commit -m "feat: Ajout validation mot de passe"
```

Étape 3 : Fusion et résolution du conflit

```
git checkout main
git merge feature/email-validation
# Merge reussi
git merge feature/password-validation
# CONFLIT detecte
```

Le fichier `UserService.java` contiendra les marqueurs de conflit :

```
public class UserService {
    public boolean validateUser(String email, String password) {
<<<<<<< HEAD
        // Validation email par dev A
        if (!email.matches("^[A-Za-z0-9+_.-]+@(.+)$")) {
            throw new ValidationException("Email invalide");
        }
=======
        // Validation mot de passe par dev B
        if (password.length() < 8) {
            throw new ValidationException("Mot de passe trop court");
        }
>>>>>>> feature/password-validation
        return true;
    }
}
```

Résolution : Conservation des deux validations

```
public class UserService {
    public boolean validateUser(String email, String password) {
        // Validation email
        if (!email.matches("^[A-Za-z0-9+_.-]+@(.+)$")) {
            throw new ValidationException("Email invalide");
        }

        // Validation mot de passe
        if (password.length() < 8) {
            throw new ValidationException(
                "Mot de passe doit contenir au moins 8 caracteres"
            );
        }
```

```
14
15          // Validation force mot de passe
16          if (!password.matches(".*[A-Z].*")) {
17              throw new ValidationException(
18                  "Mot de passe doit contenir une majuscule"
19              );
20          }
21
22          if (!password.matches(".*[0-9].*")) {
23              throw new ValidationException(
24                  "Mot de passe doit contenir un chiffre"
25              );
26          }
27
28          return true;
29      }
30  }
```

```
1  git add src/main/java/com/bijouxshop/UserService.java
2  git commit -m "merge: Resolution conflit validations email et password
3
4  - Integration validation email (feature/email-validation)
5  - Integration validation mot de passe (feature/password-validation)
6  - Ajout validations supplementaires pour securite renforcee
7  - Tous les tests passes
8
9  Resolves #123, #124"
```

0.1.2 Exercice 2 : Git Rebase Interactif

Énoncé :
Vous avez créé 5 commits sur votre branche de développement mais ils sont désordonnés :

1. "Ajout test unitaire"
2. "Fix typo dans README"
3. "Implementation feature X"
4. "Fix bug dans feature X"
5. "Ajout documentation feature X"

Réorganisez l'historique pour avoir une progression logique.
Solution :

```
1  # Lancer rebase interactif sur les 5 derniers commits
2  git rebase -i HEAD~5
```

Dans l'éditeur qui s'ouvre :

```
1  pick abc1234 Ajout test unitaire
2  pick def5678 Fix typo dans README
3  pick ghi9012 Implementation feature X
4  pick jkl3456 Fix bug dans feature X
5  pick mno7890 Ajout documentation feature X
```

Réorganisez en :

```
pick ghi9012 Implementation feature X
squash jkl3456 Fix bug dans feature X
pick mno7890 Ajout documentation feature X
pick abc1234 Ajout test unitaire
pick def5678 Fix typo dans README
```

Résultat : Les commits sont réorganisés logiquement et le bug fix est fusionné avec l'implémentation.

0.2 Exercices Docker Avancés

0.2.1 Exercice 3 : Optimisation d'Image Multi-Stage

Énoncé :

L'image Docker actuelle de BijouxShop fait 850 MB. Optimisez-la pour réduire sa taille à moins de 200 MB en utilisant un build multi-stage.

Solution :

```
# Stage 1: Build
FROM maven:3.9-eclipse-temurin-17-alpine AS builder

WORKDIR /app

# Copie uniquement pom.xml pour cache des dependances
COPY pom.xml .
RUN mvn dependency:go-offline -B

# Copie le code source
COPY src ./src

# Build de l'application
RUN mvn clean package -DskipTests -B && \
    mv target/*.jar app.jar && \
    # Extraction des layers Spring Boot
    java -Djarmode=layertools -jar app.jar extract

# Stage 2: Runtime optimise
FROM eclipse-temurin:17-jre-alpine

# Installation des outils necessaires
RUN apk add --no-cache \
    dumb-init \
    curl \
    && rm -rf /var/cache/apk/*

# Creation utilisateur non-root
RUN addgroup -g 1001 bijouxshop && \
    adduser -D -u 1001 -G bijouxshop bijouxshop

WORKDIR /app

# Copie des layers Spring Boot depuis le builder
COPY --from=builder /app/dependencies/ ./
```

```
COPY --from=builder /app/spring-boot-loader/ ./
COPY --from=builder /app/snapshot-dependencies/ ./
COPY --from=builder /app/application/ ./

# Configuration proprietaire
RUN chown -R bijouxshop:bijouxshop /app

USER bijouxshop

# Health check
HEALTHCHECK --interval=30s --timeout=3s --start-period=40s \
    CMD curl -f http://localhost:8080/actuator/health || exit 1

# Variables d'environnement
ENV JAVA_OPTS="-XX:+UseContainerSupport \
               -XX:MaxRAMPercentage=75.0 \
               -XX:+UseG1GC \
               -XX:+DisableExplicitGC"

EXPOSE 8080

# Utilisation de dumb-init pour gestion signaux
ENTRYPOINT ["/usr/bin/dumb-init", "--"]

CMD ["sh", "-c", "java $JAVA_OPTS \
    org.springframework.boot.loader.JarLauncher"]
```

Résultats de l'optimisation :

— **Image originale :** 850 MB (maven :3.9-eclipse-temurin-17)

— **Image optimisée :** 185 MB (eclipse-temurin :17-jre-alpine)

— **Réduction :** 78% de taille en moins

Techniques utilisées :

1. **Build multi-stage :** Séparation build/runtime
2. **Image Alpine :** Distribution Linux minimale
3. **JRE au lieu de JDK :** Pas besoin du compilateur en prod
4. **Layer caching :** Dépendances Maven séparées du code
5. **Spring Boot layers :** Optimisation du cache Docker
6. **Utilisateur non-root :** Sécurité renforcée
7. **Health check :** Monitoring intégré

0.2.2 Exercice 4 : Docker Compose pour Environnement Complet

Énoncé :

Créez un environnement Docker Compose complet pour BijouxShop incluant :

— Application Spring Boot

— Base de données PostgreSQL avec données initiales

— Redis pour le cache

— Nginx comme reverse proxy

— Prometheus et Grafana pour le monitoring

Solution :

```yaml
version: '3.9'

services:
  # Base de donnees PostgreSQL
  postgres:
    image: postgres:15-alpine
    container_name: bijouxshop-db
    environment:
      POSTGRES_DB: bijouxshop
      POSTGRES_USER: bijoux_user
      POSTGRES_PASSWORD: ${DB_PASSWORD:-secure_password}
      PGDATA: /var/lib/postgresql/data/pgdata
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./init-scripts:/docker-entrypoint-initdb.d
    ports:
      - "5432:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U bijoux_user -d bijouxshop"]
      interval: 10s
      timeout: 5s
      retries: 5
    networks:
      - bijouxshop-network

  # Cache Redis
  redis:
    image: redis:7-alpine
    container_name: bijouxshop-cache
    command: redis-server --appendonly yes --requirepass ${
       REDIS_PASSWORD:-redis_pass}
    volumes:
      - redis_data:/data
    ports:
      - "6379:6379"
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 10s
      timeout: 3s
      retries: 5
    networks:
      - bijouxshop-network

  # Application Spring Boot
  app:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: bijouxshop-app
    environment:
      SPRING_PROFILES_ACTIVE: production
```

```
      SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/bijouxshop
      SPRING_DATASOURCE_USERNAME: bijoux_user
      SPRING_DATASOURCE_PASSWORD: ${DB_PASSWORD:-secure_password}
      SPRING_REDIS_HOST: redis
      SPRING_REDIS_PORT: 6379
      SPRING_REDIS_PASSWORD: ${REDIS_PASSWORD:-redis_pass}
      JAVA_OPTS: "-Xmx512m -Xms256m"
    depends_on:
      postgres:
        condition: service_healthy
      redis:
        condition: service_healthy
    ports:
      - "8080:8080"
    volumes:
      - ./logs:/app/logs
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8080/actuator/
         health"]
      interval: 30s
      timeout: 10s
      retries: 3
      start_period: 40s
    networks:
      - bijouxshop-network
    restart: unless-stopped

  # Reverse Proxy Nginx
  nginx:
    image: nginx:alpine
    container_name: bijouxshop-proxy
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
      - ./nginx/ssl:/etc/nginx/ssl:ro
    ports:
      - "80:80"
      - "443:443"
    depends_on:
      - app
    networks:
      - bijouxshop-network
    restart: unless-stopped

  # Prometheus pour metriques
  prometheus:
    image: prom/prometheus:latest
    container_name: bijouxshop-prometheus
    volumes:
      - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml:ro
      - prometheus_data:/prometheus
    command:
      - '--config.file=/etc/prometheus/prometheus.yml'
      - '--storage.tsdb.path=/prometheus'
```

```
      - '--storage.tsdb.retention.time=30d'
    ports:
      - "9090:9090"
    networks:
      - bijouxshop-network
    restart: unless-stopped

  # Grafana pour visualisation
  grafana:
    image: grafana/grafana:latest
    container_name: bijouxshop-grafana
    environment:
      GF_SECURITY_ADMIN_USER: admin
      GF_SECURITY_ADMIN_PASSWORD: ${GRAFANA_PASSWORD:-admin}
      GF_INSTALL_PLUGINS: grafana-piechart-panel
    volumes:
      - grafana_data:/var/lib/grafana
      - ./grafana/provisioning:/etc/grafana/provisioning:ro
    ports:
      - "3000:3000"
    depends_on:
      - prometheus
    networks:
      - bijouxshop-network
    restart: unless-stopped

volumes:
  postgres_data:
    driver: local
  redis_data:
    driver: local
  prometheus_data:
    driver: local
  grafana_data:
    driver: local

networks:
  bijouxshop-network:
    driver: bridge
```

Fichier de configuration Nginx :

```
events {
    worker_connections 1024;
}

http {
    upstream bijouxshop_backend {
        server app:8080;
    }

    server {
        listen 80;
        server_name localhost;
```

```
13
14          location / {
15              proxy_pass http://bijouxshop_backend;
16              proxy_set_header Host $host;
17              proxy_set_header X-Real-IP $remote_addr;
18              proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for
                    ;
19              proxy_set_header X-Forwarded-Proto $scheme;
20
21              # Timeouts
22              proxy_connect_timeout 60s;
23              proxy_send_timeout 60s;
24              proxy_read_timeout 60s;
25          }
26
27          location /actuator/health {
28              proxy_pass http://bijouxshop_backend/actuator/health;
29              access_log off;
30          }
31      }
32  }
```

Commandes de gestion :

```
1  # Demarrage de tous les services
2  docker-compose up -d
3
4  # Verification de l'etat
5  docker-compose ps
6
7  # Logs d'un service specifique
8  docker-compose logs -f app
9
10 # Redemarrage d'un service
11 docker-compose restart app
12
13 # Arret et suppression
14 docker-compose down
15
16 # Arret avec suppression des volumes
17 docker-compose down -v
18
19 # Mise a l'echelle de l'application
20 docker-compose up -d --scale app=3
```

0.3 Exercices Kubernetes

0.3.1 Exercice 5 : Déploiement BijouxShop sur Kubernetes

Énoncé :

Déployez BijouxShop sur Kubernetes avec haute disponibilité, incluant :

— 3 replicas de l'application

— ConfigMap pour la configuration
— Secrets pour les mots de passe
— Service avec LoadBalancer
— Ingress pour le routage HTTP
— HorizontalPodAutoscaler

Solution :

1. Namespace :

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: bijouxshop
  labels:
    app: bijouxshop
    environment: production
```

2. ConfigMap :

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: bijouxshop-config
  namespace: bijouxshop
data:
  application.properties: |
    spring.application.name=BijouxShop
    spring.profiles.active=production
    server.port=8080

    # Database configuration
    spring.datasource.url=jdbc:postgresql://postgres-service:5432/
       bijouxshop
    spring.datasource.driver-class-name=org.postgresql.Driver
    spring.jpa.hibernate.ddl-auto=validate
    spring.jpa.show-sql=false

    # Redis configuration
    spring.redis.host=redis-service
    spring.redis.port=6379

    # Actuator endpoints
    management.endpoints.web.exposure.include=health,metrics,prometheus
    management.endpoint.health.show-details=always

    # Logging
    logging.level.root=INFO
    logging.level.com.bijouxshop=DEBUG
```

3. Secrets :

```bash
# Creation des secrets
kubectl create secret generic bijouxshop-secrets \
  --from-literal=db-password=SecureDbP@ssw0rd \
  --from-literal=redis-password=SecureRedisP@ss \
```

```
  --namespace=bijouxshop

# Verification
kubectl get secrets -n bijouxshop
```

4. Deployment :

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: bijouxshop-app
  namespace: bijouxshop
  labels:
    app: bijouxshop
    version: v1.0.0
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 1
  selector:
    matchLabels:
      app: bijouxshop
  template:
    metadata:
      labels:
        app: bijouxshop
        version: v1.0.0
      annotations:
        prometheus.io/scrape: "true"
        prometheus.io/port: "8080"
        prometheus.io/path: "/actuator/prometheus"
    spec:
      containers:
      - name: bijouxshop
        image: registry.gitlab.com/bijouxshop/app:1.0.0
        imagePullPolicy: Always
        ports:
        - containerPort: 8080
          name: http
          protocol: TCP
        env:
        - name: SPRING_DATASOURCE_PASSWORD
          valueFrom:
            secretKeyRef:
              name: bijouxshop-secrets
              key: db-password
        - name: SPRING_REDIS_PASSWORD
          valueFrom:
            secretKeyRef:
              name: bijouxshop-secrets
              key: redis-password
```

```
        - name: JAVA_OPTS
          value: "-Xmx512m -Xms256m -XX:+UseG1GC"
        volumeMounts:
        - name: config
          mountPath: /app/config
          readOnly: true
        livenessProbe:
          httpGet:
            path: /actuator/health/liveness
            port: 8080
          initialDelaySeconds: 60
          periodSeconds: 10
          timeoutSeconds: 5
          failureThreshold: 3
        readinessProbe:
          httpGet:
            path: /actuator/health/readiness
            port: 8080
          initialDelaySeconds: 30
          periodSeconds: 5
          timeoutSeconds: 3
          failureThreshold: 3
        resources:
          requests:
            memory: "512Mi"
            cpu: "250m"
          limits:
            memory: "1Gi"
            cpu: "500m"
      volumes:
      - name: config
        configMap:
          name: bijouxshop-config
      imagePullSecrets:
      - name: gitlab-registry-secret
      affinity:
        podAntiAffinity:
          preferredDuringSchedulingIgnoredDuringExecution:
          - weight: 100
            podAffinityTerm:
              labelSelector:
                matchExpressions:
                - key: app
                  operator: In
                  values:
                  - bijouxshop
              topologyKey: kubernetes.io/hostname
```

5. Service :

```
apiVersion: v1
kind: Service
metadata:
  name: bijouxshop-service
```

```
  namespace: bijouxshop
  labels:
    app: bijouxshop
spec:
  type: ClusterIP
  selector:
    app: bijouxshop
  ports:
  - name: http
    port: 80
    targetPort: 8080
    protocol: TCP
  sessionAffinity: ClientIP
  sessionAffinityConfig:
    clientIP:
      timeoutSeconds: 3600
```

6. Ingress :

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: bijouxshop-ingress
  namespace: bijouxshop
  annotations:
    kubernetes.io/ingress.class: nginx
    cert-manager.io/cluster-issuer: letsencrypt-prod
    nginx.ingress.kubernetes.io/ssl-redirect: "true"
    nginx.ingress.kubernetes.io/force-ssl-redirect: "true"
    nginx.ingress.kubernetes.io/proxy-body-size: "10m"
    nginx.ingress.kubernetes.io/rate-limit: "100"
spec:
  tls:
  - hosts:
    - bijouxshop.example.com
    secretName: bijouxshop-tls
  rules:
  - host: bijouxshop.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: bijouxshop-service
            port:
              number: 80
```

7. HorizontalPodAutoscaler :

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: bijouxshop-hpa
  namespace: bijouxshop
```

```
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: bijouxshop-app
  minReplicas: 3
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
  - type: Resource
    resource:
      name: memory
      target:
        type: Utilization
        averageUtilization: 80
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 300
      policies:
      - type: Percent
        value: 50
        periodSeconds: 60
    scaleUp:
      stabilizationWindowSeconds: 0
      policies:
      - type: Percent
        value: 100
        periodSeconds: 30
      - type: Pods
        value: 2
        periodSeconds: 30
      selectPolicy: Max
```

Déploiement complet :

```
# Application de tous les manifestes
kubectl apply -f namespace.yaml
kubectl apply -f configmap.yaml
kubectl apply -f secrets.yaml
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
kubectl apply -f ingress.yaml
kubectl apply -f hpa.yaml

# Verification du deploiement
kubectl get all -n bijouxshop

# Surveillance des pods
kubectl get pods -n bijouxshop -w
```

```
15
16 # Verification de l'autoscaling
17 kubectl get hpa -n bijouxshop
18
19 # Logs d'un pod
20 kubectl logs -f deployment/bijouxshop-app -n bijouxshop
21
22 # Test de montee en charge
23 kubectl run -it --rm load-generator \
24   --image=busybox --restart=Never -- \
25   /bin/sh -c "while true; do wget -q -O- \
26   http://bijouxshop-service.bijouxshop.svc.cluster.local; done"
```

Partie VIII : Glossaire Complet DevOps (G-Z)

0.4 G

Git Flow : Modèle de branche Git populaire qui définit une structure de branchement stricte conçue autour des releases. Créé par Vincent Driessen, il utilise des branches feature, develop, release, hotfix et main pour organiser le développement.

GitHub Actions : Service d'intégration continue et de livraison continue (CI/CD) intégré à GitHub qui permet d'automatiser les workflows directement depuis les repositories GitHub. Utilise des fichiers YAML pour définir les pipelines.

GitLab CI/CD : Système CI/CD intégré dans GitLab qui permet de construire, tester et déployer du code. Utilise des runners pour exécuter les jobs définis dans le fichier `.gitlab-ci.yml`.

GitOps : Paradigme opérationnel qui utilise Git comme source de vérité unique pour l'infrastructure déclarative et les applications. Les changements d'infrastructure sont versionnés dans Git et automatiquement appliqués.

Grafana : Plateforme open source d'analyse et de visualisation de métriques. Permet de créer des tableaux de bord interactifs pour monitorer les applications et l'infrastructure en temps réel.

Graylog : Plateforme de gestion de logs open source qui permet de collecter, indexer et analyser des logs en temps réel. Alternative à la stack ELK.

Green/Blue Deployment : Voir Blue-Green Deployment. Technique de déploiement qui maintient deux environnements de production identiques, permettant des bascules rapides et des rollbacks sans temps d'arrêt.

0.5 H

Helm : Gestionnaire de packages pour Kubernetes, souvent appelé "le apt/yum de Kubernetes". Utilise des Charts (packages) pour définir, installer et mettre à jour des applications Kubernetes complexes.

Horizontal Pod Autoscaler (HPA) : Contrôleur Kubernetes qui ajuste automatiquement le nombre de pods dans un deployment, replica set ou stateful set basé sur l'utilisation CPU, mémoire ou métriques custom.

Hotfix : Correctif urgent appliqué directement en production pour résoudre un bug critique. Dans Git Flow, c'est une branche créée depuis main pour un correctif rapide.

HTTP/2 : Version majeure du protocole HTTP qui améliore les performances via multiplexing, compression des headers et server push. Largement utilisé dans les architectures microservices modernes.

0.6 I

Idempotence : Propriété d'une opération qui peut être appliquée plusieurs fois sans changer le résultat au-delà de l'application initiale. Crucial dans IaC et les API REST.

Image Registry : Dépôt centralisé pour stocker et distribuer des images de conteneurs. Exemples : Docker Hub, GitLab Container Registry, Amazon ECR, Google Container Registry.

Immutable Infrastructure : Approche où les serveurs ne sont jamais modifiés après déploiement. Toute mise à jour implique le remplacement complet du serveur par une nouvelle version.

Incident Management : Processus de restauration du service normal aussi rapidement que possible suite à un incident, minimisant l'impact négatif sur les opérations business.

Infrastructure as Code (IaC) : Gestion et provisionnement de l'infrastructure via du code et des fichiers de configuration plutôt que par configuration manuelle. Exemples : Terraform, Ansible, CloudFormation.

Ingress : Objet Kubernetes qui gère l'accès externe aux services dans un cluster, typiquement HTTP/HTTPS. Fournit load balancing, SSL termination et name-based virtual hosting.

Integration Testing : Tests qui vérifient que différents modules ou services fonctionnent correctement ensemble. Plus large que les tests unitaires, mais moins que les tests end-to-end.

Istio : Service mesh open source qui fournit un moyen uniforme de sécuriser, connecter et observer les microservices. Offre traffic management, security, observability.

0.7 J

Jaeger : Système de tracing distribué open source développé par Uber. Utilisé pour monitorer et troubleshooter les transactions dans les architectures microservices complexes.

Jenkins : Serveur d'automatisation open source le plus populaire. Permet de construire, tester et déployer des applications via des pipelines définis comme code (Jenkinsfile).

Jenkins X : Version cloud-native de Jenkins spécialement conçue pour Kubernetes. Fournit des pipelines CI/CD automatisés, preview environments et promotion entre environnements.

JSON Web Token (JWT) : Standard ouvert (RFC 7519) pour créer des tokens d'accès basés sur JSON. Largement utilisé pour l'authentification et l'échange sécurisé d'informations.

JUnit : Framework de tests unitaires pour Java. Standard de facto pour les tests en Java, supporte les annotations et assertions pour écrire des tests clairs et maintenables.

0.8 K

Kanban : Méthode de gestion de flux de travail visualisant le travail et limitant le travail en cours. Utilisé dans DevOps pour optimiser le flux de déploiements.

Kibana : Interface utilisateur pour Elasticsearch, permettant de visualiser et explorer les données indexées. Partie de la stack ELK (Elasticsearch, Logstash, Kibana).

Knative : Plateforme Kubernetes pour construire, déployer et gérer des applications serverless modernes. Fournit building blocks pour patterns cloud-native communs.

Kops : Outil pour installer, mettre à jour et maintenir des clusters Kubernetes en production. Automatise le provisionnement sur AWS, GCE, et autres clouds.

kubectl : Outil en ligne de commande pour interagir avec les clusters Kubernetes. Permet de déployer des applications, inspecter et gérer les ressources du cluster.

Kubelet : Agent qui s'exécute sur chaque nœud Kubernetes. Assure que les conteneurs fonctionnent dans un Pod conformément aux spécifications.

Kubernetes (K8s) : Système open source d'orchestration de conteneurs pour automatiser le déploiement, la mise à l'échelle et la gestion d'applications conteneurisées.

Kustomize : Outil de personnalisation de configurations Kubernetes sans templates. Permet de créer des variants de manifestes YAML pour différents environnements.

0.9 L

Label : Dans Kubernetes, paire clé-valeur attachée aux objets pour identification. Utilisé pour organiser et sélectionner des sous-ensembles d'objets.

Lead Time for Changes : Métrique DORA mesurant le temps entre un commit et son déploiement en production. Indicateur clé de performance DevOps.

Linting : Analyse statique du code pour détecter les erreurs de programmation, bugs, stylistic errors et constructions suspectes. Exemples : ESLint, Pylint, SonarQube.

Load Balancer : Dispositif qui distribue le trafic réseau ou applicatif entre plusieurs serveurs. Améliore la disponibilité et la fiabilité des applications.

Log Aggregation : Collecte, centralisation et indexation des logs depuis plusieurs sources. Facilite le debugging et le monitoring. Outils : ELK, Splunk, Graylog.

Logstash : Outil open source de collecte, transformation et transport de données. Partie de la stack ELK, traite les logs avant l'indexation dans Elasticsearch.

0.10 M

Mean Time to Recovery (MTTR) : Temps moyen pour restaurer un service après un incident. L'une des quatre métriques clés DORA pour mesurer les performances DevOps.

Microservices : Architecture découpant une application en services indépendants communiquant via APIs. Chaque service est déployable indépendamment et possède sa propre base de données.

Monitoring : Surveillance continue des systèmes et applications pour détecter les problèmes, analyser les performances et assurer la disponibilité. Outils : Prometheus, Datadog, New Relic.

Multi-stage Build : Fonctionnalité Docker permettant d'utiliser plusieurs FROM dans un Dockerfile. Réduit la taille des images finales en séparant build et runtime.

0.11 N

Namespace : Dans Kubernetes, mécanisme d'isolation virtuelle pour grouper et séparer les ressources. Permet le multi-tenancy et l'organisation logique.

Netflix Simian Army : Suite d'outils de chaos engineering développés par Netflix, incluant Chaos Monkey, Latency Monkey, et Chaos Kong pour tester la résilience.

Network Policy : Spécification Kubernetes définissant comment les pods peuvent communiquer entre eux et avec d'autres endpoints réseau. Améliore la sécurité.

Nginx : Serveur web et reverse proxy haute performance. Utilisé comme load balancer, API gateway et ingress controller dans les architectures modernes.

Node : Machine (physique ou virtuelle) dans un cluster Kubernetes. Exécute les pods et est géré par le control plane.

0.12 O

Observability : Capacité à comprendre l'état interne d'un système basé sur ses outputs externes. Les trois piliers : logs, metrics, traces.

OpenTelemetry : Ensemble d'outils, APIs et SDKs pour instrumenter, générer, collecter et exporter des données de télémétrie (metrics, logs, traces).

Operator Pattern : Pattern Kubernetes étendant l'API pour créer, configurer et gérer des applications complexes. Encode la connaissance opérationnelle en code.

Orchestration : Coordination automatisée de systèmes, middleware et services complexes. Kubernetes est le standard pour l'orchestration de conteneurs.

0.13 P

Pod : Plus petite unité déployable dans Kubernetes. Groupe un ou plusieurs conteneurs partageant stockage et réseau, avec spécifications sur leur exécution.

Post-mortem : Analyse rétrospective d'un incident majeur. Document détaillant ce qui s'est passé, pourquoi, et les actions pour éviter la récurrence.

Prometheus : Système open source de monitoring et d'alerting. Collecte et stocke des métriques time-series, supporte le querying via PromQL.

Pull Request (PR) / Merge Request (MR) : Mécanisme pour proposer des changements de code. Permet review, discussion et tests automatisés avant fusion.

0.14 Q

Quality Gate : Ensemble de conditions que le code doit satisfaire avant de progresser dans le pipeline. Peut inclure coverage, bugs, vulnérabilités.

Queue : Structure de données FIFO utilisée pour découpler les services et gérer les charges de travail asynchrones. Exemples : RabbitMQ, Kafka, Redis.

0.15 R

Rancher : Plateforme de gestion multi-cluster Kubernetes. Simplifie le déploiement et la gestion de K8s sur n'importe quelle infrastructure.

Redis : Store de structures de données en mémoire open source. Utilisé comme base de données, cache et message broker. Performances extrêmement élevées.

Registry : Voir Image Registry. Système de stockage et distribution pour images Docker et autres artefacts de conteneurs.

Replica Set : Objet Kubernetes maintenant un ensemble stable de pods répliqués. Garantit qu'un nombre spécifié de replicas tourne à tout moment.

Rolling Update : Stratégie de déploiement Kubernetes remplaçant progressivement les anciennes versions par les nouvelles. Permet zero-downtime deployments.

Runbook : Documentation des procédures opérationnelles pour gérer les incidents courants. Automatisable pour self-healing systems.

0.16 S

Secrets Management : Pratiques et outils pour stocker et accéder de manière sécurisée aux informations sensibles (passwords, tokens, clés). Outils : Vault, Sealed Secrets.

Service Discovery : Mécanisme permettant aux services de se trouver automatiquement dans un environnement distribué. Exemples : Consul, etcd, Kubernetes DNS.

Service Mesh : Couche d'infrastructure dédiée contrôlant la communication service-à-service. Fournit observability, traffic management, security. Exemples : Istio, Linkerd.

Shift Left : Pratique de déplacer les tests et la sécurité plus tôt dans le cycle de développement. Réduit les coûts de correction des bugs.

Site Reliability Engineering (SRE) : Discipline appliquant les principes de l'ingénierie logicielle aux problèmes d'opérations. Créée par Google, complète DevOps.

Smoke Test : Tests légers vérifiant que les fonctionnalités critiques fonctionnent après un déploiement. Premiers tests à exécuter.

SonarQube : Plateforme d'inspection continue de la qualité du code. Détecte bugs, code smells, vulnérabilités de sécurité et calcule la dette technique.

Splunk : Plateforme de recherche, monitoring et analyse de big data machine-generated. Collecte et indexe logs pour insights opérationnels.

StatefulSet : Workload API Kubernetes pour gérer des applications stateful. Maintient l'identité sticky et le stockage persistant des pods.

0.17 T

Terraform : Outil Infrastructure as Code permettant de définir et provisionner l'infrastructure datacenter via langage déclaratif HCL. Multi-cloud.

Test-Driven Development (TDD) : Méthodologie où les tests sont écrits avant le code. Cycle : Red (test fails) → Green (code passes) → Refactor.

Threat Modeling : Processus d'identification des menaces de sécurité potentielles et des vulnérabilités. Partie intégrante du DevSecOps.

Three Ways : Principes fondamentaux DevOps : 1) Flow (Dev vers Ops), 2) Feedback (Ops vers Dev), 3) Continuous Experimentation and Learning.

Tracing : Suivi du chemin d'une requête à travers un système distribué. Aide à identifier les bottlenecks et comprendre les dépendances.

Trunk-Based Development : Pratique où les développeurs fusionnent fréquemment de petits changements dans une branche principale (trunk). Facilite l'intégration continue.

0.18 U

Unit Testing : Tests vérifiant le comportement de petites unités de code (fonctions, méthodes) isolément. Base de la pyramide des tests.

Uptime : Pourcentage de temps qu'un système est opérationnel et accessible. Les SLAs définissent souvent des niveaux d'uptime (99.9%, 99.99%).

0.19 V

Vault (HashiCorp) : Outil de gestion de secrets fournissant une interface unifiée pour accéder à tout secret. Contrôle d'accès strict, audit logging, encryption.

Version Control System (VCS) : Système gérant les changements au code source. Git est le VCS décentralisé dominant. Fondation du DevOps.

Vertical Pod Autoscaler (VPA) : Contrôleur Kubernetes ajustant automatiquement les requests et limits CPU/mémoire des conteneurs basé sur l'utilisation.

Virtual Machine (VM) : Émulation d'un système informatique complet. Les conteneurs sont plus légers mais les VMs fournissent une isolation plus forte.

0.20 W

Webhook : Mécanisme permettant à une application d'envoyer des notifications en temps réel à d'autres applications via HTTP callbacks. Déclenche des actions automatiques.

White-box Monitoring : Monitoring basé sur les métriques internes d'un système (logs, traces). Complète le black-box monitoring (tests externes).

Workflow : Séquence automatisée d'étapes dans un pipeline CI/CD. Défini comme code dans GitHub Actions, GitLab CI, Jenkins.

0.21 X

X as Code : Paradigme appliquant l'approche "as code" à tout : Infrastructure as Code (IaC), Configuration as Code, Policy as Code. Versionnable, testable, reproductible.

0.22 Y

YAML (YAML Ain't Markup Language) : Format de sérialisation de données lisible par l'homme. Standard pour les fichiers de configuration Kubernetes, Docker Compose, CI/CD.

Yum : Gestionnaire de packages pour distributions Linux basées sur RPM (Red Hat, CentOS). Équivalent d'apt pour Debian/Ubuntu.

0.23 Z

Zero Downtime Deployment : Stratégie de déploiement permettant de mettre à jour une application sans interruption de service. Utilise blue-green, rolling updates, canary.

Zero Trust Security : Modèle de sécurité ne faisant confiance à aucun utilisateur ou système par défaut. Vérifie explicitement chaque requête. "Never trust, always verify."

Partie IX : Guide de Dépannage Avancé

0.24 Dépannage Git

0.24.1 Problème : Push Rejected (Non-fast-forward)

Symptôme :

```
! [rejected] main -> main (non-fast-forward)
error: failed to push some refs to 'origin'
```

Cause : Des commits existent sur la branche distante qui ne sont pas dans votre historique local.

Solutions :

Solution 1 : Pull avec rebase (recommandé)

```
git pull --rebase origin main
git push origin main
```

Solution 2 : Merge classique

```
git pull origin main
# Resoudre les conflits si necessaire
git push origin main
```

Solution 3 : Force push (DANGER - seulement si vous êtes sûr)

```
# ATTENTION: Ecrase l'historique distant
git push --force-with-lease origin main
```

0.24.2 Problème : Détaché HEAD (Detached HEAD)

Symptôme :

```
You are in 'detached HEAD' state. You can look around, make
experimental changes and commit them...
```

Cause : Vous avez checkout un commit spécifique plutôt qu'une branche.

Solution :

Si vous voulez garder vos changements :

```
# Creer une nouvelle branche depuis cet etat
git checkout -b nouvelle-branche-sauvegarde

# Ou attacher a une branche existante
git branch -f main HEAD
git checkout main
```

Si vous voulez revenir à la branche :

```
git checkout main
# Les commits en detached HEAD seront perdus
```

0.24.3 Problème : Gros Fichiers Binaires Commités

Symptôme : Le repository devient très lourd, cloning/push lents.
Solution : Utiliser BFG Repo-Cleaner

```
# Installation BFG
wget https://repo1.maven.org/maven2/com/madgag/bfg/1.14.0/bfg-1.14.0.
    jar

# Cloner le repo en miroir
git clone --mirror git@gitlab.com:user/repo.git

# Supprimer les fichiers > 50M
java -jar bfg-1.14.0.jar --strip-blobs-bigger-than 50M repo.git

# Cleanup et push
cd repo.git
git reflog expire --expire=now --all
git gc --prune=now --aggressive
git push
```

0.25 Dépannage Docker

0.25.1 Problème : Conteneur N'arrête Pas de Redémarrer

Symptôme :

```
docker ps
# CONTAINER ID   STATUS
# abc123def456   Restarting (1) 5 seconds ago
```

Diagnostic :

```
# Voir les logs
docker logs container_name --tail 50

# Voir la raison du dernier exit
docker inspect container_name | grep -A 5 "State"

# Desactiver le restart pour debugger
docker update --restart=no container_name
```

Causes communes et solutions :
1. Application crash au démarrage :

```
# Tester l'application en mode interactif
docker run -it --entrypoint /bin/sh image_name

# Verifier les variables d'environnement
docker inspect container_name | grep -A 20 "Env"
```

2. Health check échoue :

```
# Simplifier le health check dans Dockerfile
HEALTHCHECK --interval=60s --timeout=5s --start-period=120s \
  CMD curl -f http://localhost:8080/health || exit 1
```

3. Dépendance non disponible :

```
# docker-compose.yml - Ajouter depends_on avec condition
services:
  app:
    depends_on:
      db:
        condition: service_healthy
  db:
    healthcheck:
      test: ["CMD", "pg_isready"]
      interval: 5s
      timeout: 3s
      retries: 5
```

0.25.2 Problème : Out of Disk Space

Symptôme :

```
Error response from daemon: no space left on device
```

Diagnostic :

```
# Voir l'utilisation globale
docker system df

# Details par type
docker system df -v

# Voir les plus gros conteneurs
docker ps -as

# Voir les plus grosses images
docker images --format "table {{.Repository}}\t{{.Tag}}\t{{.Size}}" |
   sort -k 3 -h
```

Solutions :

```
# Nettoyage des ressources non utilisees (RECOMMANDE)
docker system prune -a --volumes

# Nettoyage selectif - images non utilisees
docker image prune -a

# Nettoyage - conteneurs stoppes
docker container prune

# Nettoyage - volumes non utilises
docker volume prune

```

```
# Nettoyage - build cache
docker builder prune -a

# Nettoyage complet avec force (ATTENTION)
docker system prune -a --volumes --force
```

Prévention :

```
# Configuration daemon.json pour rotation logs
sudo cat > /etc/docker/daemon.json << 'EOF'
{
  "log-driver": "json-file",
  "log-opts": {
    "max-size": "10m",
    "max-file": "3"
  }
}
EOF

sudo systemctl restart docker
```

0.25.3 Problème : Réseau Docker Ne Fonctionne Pas

Symptôme : Conteneurs ne peuvent pas communiquer entre eux.
Diagnostic :

```
# Lister les reseaux
docker network ls

# Inspecter un reseau
docker network inspect bridge

# Verifier la connectivite entre conteneurs
docker exec container1 ping container2

# Verifier DNS
docker exec container1 nslookup container2
```

Solutions :
1. Créer un réseau custom :

```
# Creer un reseau bridge custom
docker network create --driver bridge bijouxshop-net

# Connecter les conteneurs
docker network connect bijouxshop-net container1
docker network connect bijouxshop-net container2

# Test
docker exec container1 ping container2
```

2. Réseau Docker Compose :

```
version: '3.9'

```

```
services:
  app:
    networks:
      - bijouxshop-network
  db:
    networks:
      - bijouxshop-network

networks:
  bijouxshop-network:
    driver: bridge
    ipam:
      config:
        - subnet: 172.25.0.0/16
```

0.26 Dépannage Kubernetes

0.26.1 Problème : Pod en CrashLoopBackOff

Symptôme :

```
NAME                       READY   STATUS             RESTARTS
bijouxshop-app-abc123      0/1     CrashLoopBackOff   5
```

Diagnostic complet :

```
# Voir les evenements du pod
kubectl describe pod bijouxshop-app-abc123

# Logs du conteneur actuel
kubectl logs bijouxshop-app-abc123

# Logs du conteneur precedent (crash)
kubectl logs bijouxshop-app-abc123 --previous

# Logs en temps reel
kubectl logs -f bijouxshop-app-abc123

# Entrer dans le pod pour debugger (si possible)
kubectl exec -it bijouxshop-app-abc123 -- /bin/sh

# Verifier les ressources
kubectl top pod bijouxshop-app-abc123
```

Causes communes et corrections :
1. Liveness probe trop agressive :

```
livenessProbe:
  httpGet:
    path: /actuator/health
    port: 8080
  initialDelaySeconds: 120  # Augmenter si app lente
  periodSeconds: 30
  timeoutSeconds: 10
```

```
  failureThreshold: 5       # Plus tolerant
```

2. Ressources insuffisantes :

```
resources:
  requests:
    memory: "512Mi"  # Augmenter si OOMKilled
    cpu: "250m"
  limits:
    memory: "2Gi"    # Plus de marge
    cpu: "1000m"
```

3. Variables d'environnement manquantes :

```
# Verifier les variables
kubectl exec pod-name -- env

# Verifier les secrets
kubectl get secret secret-name -o yaml
kubectl describe secret secret-name
```

0.26.2 Problème : Service Non Accessible

Symptôme : Service existe mais n'est pas joignable.
Diagnostic :

```
# Verifier le service
kubectl get svc bijouxshop-service
kubectl describe svc bijouxshop-service

# Verifier les endpoints
kubectl get endpoints bijouxshop-service

# Tester depuis un pod de debug
kubectl run debug-pod --image=curlimages/curl:latest \
  --rm -it --restart=Never -- \
  curl http://bijouxshop-service:80

# Verifier les labels des pods
kubectl get pods --show-labels

# Verifier le selector du service
kubectl get svc bijouxshop-service -o yaml | grep selector -A 5
```

Solutions :
1. Labels ne correspondent pas :

```
# Corriger les labels du deployment
kubectl label pod pod-name app=bijouxshop --overwrite
```

2. Port mapping incorrect :

```
# Service
apiVersion: v1
kind: Service
spec:
```

```
  ports:
  - port: 80          # Port du service
    targetPort: 8080  # Port du conteneur (doit correspondre)
```

3. Network Policy bloquante :

```
# Lister les network policies
kubectl get networkpolicies

# Supprimer temporairement pour tester
kubectl delete networkpolicy policy-name

# Verifier la connectivite
```

0.26.3 Problème : Persistent Volume Claim Pending

Symptôme :

```
NAME          STATUS    VOLUME   CAPACITY
data-pvc      Pending
```

Diagnostic :

```
# Details du PVC
kubectl describe pvc data-pvc

# Verifier les StorageClasses disponibles
kubectl get storageclass

# Verifier les PV disponibles
kubectl get pv
```

Solutions :

1. Pas de StorageClass par défaut :

```
# Definir une StorageClass par defaut
kubectl patch storageclass standard \
  -p '{"metadata": {"annotations":{"storageclass.kubernetes.io/is-
     default-class":"true"}}}'
```

2. Spécifier la StorageClass dans le PVC :

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: data-pvc
spec:
  storageClassName: standard  # Specifier explicitement
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
```

3. Créer un PV manuellement (environnement local) :

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: manual-pv
spec:
  capacity:
    storage: 10Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  storageClassName: manual
  hostPath:
    path: /data/bijouxshop
```

0.27 Dépannage CI/CD

0.27.1 Problème : Pipeline GitLab CI Échoue sur Tests

Diagnostic :

```bash
# Reproduire localement avec gitlab-runner
gitlab-runner exec docker test-job

# Ou avec Docker directement
docker run --rm -v $(pwd):/app -w /app maven:3.9-eclipse-temurin-17 \
  mvn clean test
```

Solutions communes :

1. Tests flaky (instables) :

```yaml
# .gitlab-ci.yml - Retry pour tests flaky
test:
  stage: test
  script:
    - mvn test
  retry:
    max: 2
    when:
      - script_failure
```

2. Timeout sur tests longs :

```yaml
test:
  stage: test
  script:
    - mvn test -Dmaven.test.failure.ignore=false
  timeout: 30m  # Augmenter le timeout
```

3. Dépendances manquantes :

```yaml
test:
  stage: test
  services:
    - postgres:15
```

```
    - redis:7
  variables:
    POSTGRES_DB: bijouxshop_test
    POSTGRES_USER: test_user
    POSTGRES_PASSWORD: test_pass
  script:
    - mvn test
```

Partie X : Meilleures Pratiques DevOps en Profondeur

0.1 Culture DevOps : Principes Fondamentaux

0.1.1 Communication et Collaboration

La culture DevOps repose avant tout sur la communication et la collaboration entre les équipes. Voici les pratiques essentielles pour établir cette culture :

1. Élimination des Silos Organisationnels

Les silos traditionnels entre développement et opérations créent des frictions et ralentissent la livraison. Pour les éliminer :

- **Équipes pluridisciplinaires :** Constituez des équipes incluant développeurs, ops, testeurs et product owners travaillant ensemble sur les mêmes objectifs
- **Rotation des rôles :** Permettez aux développeurs de participer aux astreintes et aux ops de contribuer au code
- **Espaces partagés :** Créez des espaces physiques ou virtuels où les équipes se côtoient quotidiennement
- **Objectifs communs :** Alignez les KPIs sur la livraison de valeur plutôt que sur les métriques techniques isolées

2. Documentation Collaborative

La documentation doit être vivante et accessible :

- **Documentation as Code :** Stockez la documentation dans le même repository que le code, en Markdown ou AsciiDoc
- **Wikis internes :** Maintenez des wikis d'équipe (Confluence, Notion) avec architecture, runbooks, post-mortems
- **ADR (Architecture Decision Records) :** Documentez les décisions architecturales importantes avec leur contexte et rationale
- **Diagrammes as Code :** Utilisez PlantUML, Mermaid ou Diagrams pour versionner les schémas d'architecture

3. Cérémonies et Rituels DevOps

Instaurez des rituels réguliers pour maintenir l'alignement :

- **Stand-up quotidien :** 15 minutes max, focus sur les blocages et dépendances
- **Post-mortem sans blâme :** Après chaque incident majeur, analysez les causes systémiques
- **Démos hebdomadaires :** Montrez les nouvelles fonctionnalités et améliorations de plateforme

— **Retrospectives bi-hebdomadaires :** Identifiez ce qui fonctionne bien et ce qui doit s'améliorer

— **Architecture reviews :** Sessions trimestrielles pour valider les évolutions architecturales

0.1.2 Automatisation : Le Coeur du DevOps

L'automatisation est le pilier technique permettant la vélocité DevOps. Voici comment l'implémenter efficacement :

1. Automatisation des Tests

Une stratégie de tests robuste est essentielle pour déployer en confiance :

Pyramide des Tests :

— **Tests unitaires (70% des tests) :** Rapides, isolés, couvrent la logique métier
  — Coverage cible : 80%+
  — Exécution : $<$ 5 minutes pour toute la suite
  — Framework : JUnit 5, Mockito pour BijouxShop

— **Tests d'intégration (20% des tests) :** Vérifient les interactions entre composants
  — Testent les APIs REST, accès base de données, cache Redis
  — Utilisent Testcontainers pour PostgreSQL et Redis
  — Exécution : $<$ 15 minutes

— **Tests end-to-end (10% des tests) :** Valident les parcours utilisateur complets
  — Selenium ou Cypress pour les tests UI
  — Environnement dédié similaire à production
  — Exécution : $<$ 30 minutes

Tests de Performance :

— **Load testing :** JMeter ou Gatling, simuler 1000+ utilisateurs concurrents

— **Stress testing :** Identifier les limites du système

— **Soak testing :** Tests de durée (24-72h) pour détecter les memory leaks

— Critères d'acceptation : P95 latency $<$ 500ms, throughput $>$ 1000 req/s

2. Automatisation de l'Infrastructure

Infrastructure as Code (IaC) permet la reproductibilité et la scalabilité :

Terraform pour le Provisionnement :

Exemple de structure de projet Terraform pour BijouxShop :

```
terraform/
          environments/
                dev/
                      main.tf
                      variables.tf
                      terraform.tfvars
                staging/
                      main.tf
                      variables.tf
                      terraform.tfvars
                production/
                    main.tf
                    variables.tf
                    terraform.tfvars
          modules/
```

```
                vpc/
                      main.tf
                      variables.tf
                      outputs.tf
                eks/
                      main.tf
                      variables.tf
                      outputs.tf
                rds/
                      main.tf
                      variables.tf
                      outputs.tf
                elasticache/
                    main.tf
                    variables.tf
                    outputs.tf
          README.md
```

Principes IaC Essentiels :

- **Idempotence :** Les scripts doivent produire le même résultat quelle que soit le nombre d'exécutions
- **Immutabilité :** Ne jamais modifier l'infrastructure existante, toujours recréer
- **Versioning :** Tout le code IaC dans Git avec tags pour chaque release
- **State management :** Backend distant (S3 + DynamoDB) avec locking pour éviter les conflits
- **Modules réutilisables :** DRY principle - créez des modules pour les patterns communs

3. Automatisation des Déploiements

Pipeline GitLab CI/CD complet pour BijouxShop avec toutes les étapes :

```yaml
# .gitlab-ci.yml - Pipeline complet production-ready

variables:
  MAVEN_OPTS: "-Dmaven.repo.local=$CI_PROJECT_DIR/.m2/repository"
  DOCKER_DRIVER: overlay2
  DOCKER_TLS_CERTDIR: "/certs"
  SONAR_TOKEN: $SONAR_TOKEN
  KUBECONFIG: /tmp/kubeconfig

stages:
  - validate
  - build
  - test
  - security
  - package
  - deploy-dev
  - integration-test
  - deploy-staging
  - smoke-test
  - deploy-production
  - rollback

# Cache Maven dependencies
```

```yaml
cache:
  paths:
    - .m2/repository/
    - target/

# Stage 1: Validation du code
code-quality:
  stage: validate
  image: maven:3.9-eclipse-temurin-17
  script:
    - mvn checkstyle:check
    - mvn spotbugs:check
  allow_failure: false

lint-dockerfile:
  stage: validate
  image: hadolint/hadolint:latest-alpine
  script:
    - hadolint Dockerfile
  allow_failure: true

# Stage 2: Build
build:
  stage: build
  image: maven:3.9-eclipse-temurin-17
  script:
    - mvn clean compile
    - mvn dependency:tree > dependency-tree.txt
  artifacts:
    paths:
      - target/
      - dependency-tree.txt
    expire_in: 1 day

# Stage 3: Tests
unit-tests:
  stage: test
  image: maven:3.9-eclipse-temurin-17
  services:
    - postgres:15
    - redis:7
  variables:
    POSTGRES_DB: bijouxshop_test
    POSTGRES_USER: test_user
    POSTGRES_PASSWORD: test_pass
    SPRING_PROFILES_ACTIVE: test
  script:
    - mvn test
    - mvn jacoco:report
  coverage: '/Total.*?([0-9]{1,3})%/'
  artifacts:
    reports:
      junit: target/surefire-reports/TEST-*.xml
```

```
      coverage_report:
        coverage_format: cobertura
        path: target/site/jacoco/jacoco.xml
    paths:
      - target/site/jacoco/
    expire_in: 30 days

integration-tests:
  stage: test
  image: maven:3.9-eclipse-temurin-17
  services:
    - postgres:15
    - redis:7
  variables:
    POSTGRES_DB: bijouxshop_integration
    POSTGRES_USER: integration_user
    POSTGRES_PASSWORD: integration_pass
  script:
    - mvn verify -Pintegration-tests
  artifacts:
    reports:
      junit: target/failsafe-reports/TEST-*.xml

# Stage 4: Security Scanning
dependency-check:
  stage: security
  image: maven:3.9-eclipse-temurin-17
  script:
    - mvn org.owasp:dependency-check-maven:check
  artifacts:
    paths:
      - target/dependency-check-report.html
    expire_in: 30 days
  allow_failure: true

sonarqube-scan:
  stage: security
  image: maven:3.9-eclipse-temurin-17
  script:
    - mvn sonar:sonar
      -Dsonar.projectKey=bijouxshop
      -Dsonar.host.url=$SONAR_HOST_URL
      -Dsonar.login=$SONAR_TOKEN
      -Dsonar.qualitygate.wait=true
  only:
    - main
    - develop

trivy-scan:
  stage: security
  image: aquasec/trivy:latest
  script:
    - trivy fs --security-checks vuln,config .
```

```
    - trivy fs --format sarif --output trivy-results.sarif .
  artifacts:
    paths:
      - trivy-results.sarif
    expire_in: 30 days

# Stage 5: Package
build-docker-image:
  stage: package
  image: docker:24
  services:
    - docker:24-dind
  before_script:
    - echo $CI_REGISTRY_PASSWORD | docker login -u $CI_REGISTRY_USER
      --password-stdin $CI_REGISTRY
  script:
    - docker build -t $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA .
    - docker build -t $CI_REGISTRY_IMAGE:latest .
    - docker push $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA
    - docker push $CI_REGISTRY_IMAGE:latest
  only:
    - main
    - develop

scan-docker-image:
  stage: package
  image: aquasec/trivy:latest
  script:
    - trivy image --severity HIGH,CRITICAL
      $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA
  dependencies:
    - build-docker-image
  allow_failure: true

# Stage 6: Deploy to Development
deploy-dev:
  stage: deploy-dev
  image: bitnami/kubectl:latest
  environment:
    name: development
    url: https://dev.bijouxshop.internal
    on_stop: stop-dev
  before_script:
    - echo $KUBE_CONFIG_DEV | base64 -d > $KUBECONFIG
  script:
    - kubectl config use-context dev-cluster
    - kubectl set image deployment/bijouxshop-app
      bijouxshop=$CI_REGISTRY_IMAGE:$CI_COMMIT_SHA
      -n bijouxshop-dev
    - kubectl rollout status deployment/bijouxshop-app
      -n bijouxshop-dev --timeout=5m
  only:
    - develop
```

```
183
184 stop-dev:
185   stage: deploy-dev
186   image: bitnami/kubectl:latest
187   environment:
188     name: development
189     action: stop
190   script:
191     - echo $KUBE_CONFIG_DEV | base64 -d > $KUBECONFIG
192     - kubectl scale deployment/bijouxshop-app --replicas=0
193       -n bijouxshop-dev
194   when: manual
195
196 # Stage 7: Integration Tests on Dev
197 e2e-tests-dev:
198   stage: integration-test
199   image: cypress/included:12.17.0
200   services:
201     - selenium/standalone-chrome:latest
202   variables:
203     BASE_URL: https://dev.bijouxshop.internal
204   script:
205     - npm install
206     - npm run test:e2e
207   artifacts:
208     when: always
209     paths:
210       - cypress/videos/
211       - cypress/screenshots/
212     expire_in: 7 days
213   dependencies:
214     - deploy-dev
215
216 performance-tests-dev:
217   stage: integration-test
218   image: grafana/k6:latest
219   variables:
220     BASE_URL: https://dev.bijouxshop.internal
221   script:
222     - k6 run --vus 100 --duration 5m performance-tests/load-test.js
223   artifacts:
224     paths:
225       - k6-results.json
226     expire_in: 7 days
227   dependencies:
228     - deploy-dev
229
230 # Stage 8: Deploy to Staging
231 deploy-staging:
232   stage: deploy-staging
233   image: bitnami/kubectl:latest
234   environment:
235     name: staging
```

```
    url: https://staging.bijouxshop.com
  before_script:
    - echo $KUBE_CONFIG_STAGING | base64 -d > $KUBECONFIG
  script:
    - kubectl config use-context staging-cluster
    - kubectl set image deployment/bijouxshop-app
      bijouxshop=$CI_REGISTRY_IMAGE:$CI_COMMIT_SHA
      -n bijouxshop-staging
    - kubectl rollout status deployment/bijouxshop-app
      -n bijouxshop-staging --timeout=10m
  only:
    - main
  when: manual

# Stage 9: Smoke Tests on Staging
smoke-tests-staging:
  stage: smoke-test
  image: curlimages/curl:latest
  variables:
    BASE_URL: https://staging.bijouxshop.com
  script:
    - curl -f $BASE_URL/actuator/health || exit 1
    - curl -f $BASE_URL/api/products | grep -q "products" || exit 1
    - curl -f $BASE_URL/api/users/login -X POST
      -d '{"username":"test","password":"test"}' || exit 1
  dependencies:
    - deploy-staging

# Stage 10: Deploy to Production (Blue-Green)
deploy-production-blue:
  stage: deploy-production
  image: bitnami/kubectl:latest
  environment:
    name: production
    url: https://www.bijouxshop.com
  before_script:
    - echo $KUBE_CONFIG_PROD | base64 -d > $KUBECONFIG
  script:
    - kubectl config use-context prod-cluster
    # Deploy to blue environment
    - kubectl set image deployment/bijouxshop-app-blue
      bijouxshop=$CI_REGISTRY_IMAGE:$CI_COMMIT_SHA
      -n bijouxshop-prod
    - kubectl rollout status deployment/bijouxshop-app-blue
      -n bijouxshop-prod --timeout=15m
    # Wait for health checks
    - sleep 30
    # Switch traffic to blue
    - kubectl patch service bijouxshop-service -n bijouxshop-prod
      -p '{"spec":{"selector":{"version":"blue"}}}'
    # Keep green for rollback
    - echo "Green deployment kept for potential rollback"
  only:
```

```
    - main
  when: manual
  allow_failure: false

# Stage 11: Rollback if needed
rollback-production:
  stage: rollback
  image: bitnami/kubectl:latest
  environment:
    name: production
    url: https://www.bijouxshop.com
  before_script:
    - echo $KUBE_CONFIG_PROD | base64 -d > $KUBECONFIG
  script:
    - kubectl config use-context prod-cluster
    # Switch back to green (previous version)
    - kubectl patch service bijouxshop-service -n bijouxshop-prod
      -p '{"spec":{"selector":{"version":"green"}}}'
    - echo "Traffic switched back to green (previous version)"
  only:
    - main
  when: manual
```

0.1.3 Sécurité DevSecOps

La sécurité doit être intégrée à chaque étape du pipeline DevOps. Voici les pratiques essentielles :

1. Shift Left Security

Détecter les vulnérabilités le plus tôt possible dans le cycle de développement :

- **IDE Security Plugins :** SonarLint dans IntelliJ/VS Code pour feedback instantané
- **Pre-commit hooks :** Empêcher le commit de secrets ou code vulnérable
- **SAST (Static Application Security Testing) :** SonarQube, Checkmarx dans le pipeline
- **Dependency Scanning :** OWASP Dependency Check, Snyk pour détecter les CVEs
- **Secret Detection :** GitGuardian, TruffleHog pour éviter les fuites de credentials

2. Container Security

Les conteneurs introduisent de nouveaux vecteurs d'attaque :

- **Image Scanning :** Trivy, Clair, Anchore pour scanner les vulnérabilités
- **Image Signing :** Docker Content Trust (Notary) pour vérifier la provenance
- **Base Images Minimales :** Utiliser Alpine, Distroless pour réduire la surface d'attaque
- **Non-root Containers :** Toujours exécuter avec un utilisateur non privilégié
- **Read-only Root Filesystem :** Monter le filesystem en read-only quand possible

Exemple Dockerfile sécurisé :

```
FROM eclipse-temurin:17-jre-alpine AS runtime

# Installation de dumb-init pour gestion signaux
RUN apk add --no-cache dumb-init=1.2.5-r2
```

```
5
6  # Creation utilisateur non-root
7  RUN addgroup -g 10001 -S appgroup && \
8      adduser -u 10001 -S appuser -G appgroup
9
10 # Repertoire application
11 WORKDIR /app
12
13 # Copie des fichiers avec droits corrects
14 COPY --chown=appuser:appgroup target/*.jar app.jar
15
16 # Pas de shell dans l'image finale (security)
17 USER appuser
18
19 # Healthcheck
20 HEALTHCHECK --interval=30s --timeout=3s --start-period=40s \
21   CMD wget --no-verbose --tries=1 --spider \
22   http://localhost:8080/actuator/health || exit 1
23
24 # Exposition port non privilegie
25 EXPOSE 8080
26
27 # Variables d'environnement
28 ENV JAVA_OPTS="-XX:+UseContainerSupport \
29                -XX:MaxRAMPercentage=75.0 \
30                -Djava.security.egd=file:/dev/./urandom"
31
32 # Utilisation de dumb-init comme PID 1
33 ENTRYPOINT ["/usr/bin/dumb-init", "--"]
34
35 # Commande sans shell (exec form)
36 CMD ["java", "$JAVA_OPTS", "-jar", "app.jar"]
```

3. Kubernetes Security

Sécurisez votre cluster Kubernetes avec ces pratiques :

— **RBAC (Role-Based Access Control) :** Principe du moindre privilège pour tous les accès

— **Network Policies :** Isoler les pods et contrôler les flux réseau

— **Pod Security Policies/Standards :** Empêcher les pods privilégiés, hostNetwork, etc.

— **Secrets Encryption :** Chiffrer les secrets at rest avec KMS

— **Admission Controllers :** OPA/Gatekeeper pour policies as code

— **Service Mesh :** Istio/Linkerd pour mTLS automatique entre services

4. Secrets Management

Ne jamais stocker de secrets en clair dans Git ou images Docker :

— **HashiCorp Vault :** Gestion centralisée des secrets avec rotation automatique

— **Kubernetes Secrets :** Avec encryption at rest activée

— **Sealed Secrets :** Bitnami Sealed Secrets pour secrets chiffrés dans Git

— **External Secrets Operator :** Synchronisation depuis Vault, AWS Secrets Manager

— **Environment Variables :** Jamais de secrets hardcodés, toujours via env vars

0.2 Monitoring et Observabilité : Les Trois Piliers

L'observabilité est cruciale pour comprendre le comportement des systèmes en production.

0.2.1 Premier Pilier : Métriques

Les métriques fournissent des données quantitatives sur les performances :

1. Métriques d'Infrastructure

Surveillez la santé de l'infrastructure sous-jacente :

- **CPU :** Utilisation, load average, context switches
- **Mémoire :** Utilisation RAM, swap, cache/buffers
- **Disque :** IOPS, latence, espace disponible, inodes
- **Réseau :** Throughput, packets/s, erreurs, retransmissions
- **Conteneurs :** Ressources par conteneur, restarts, OOMKills

2. Métriques Applicatives

Exposez les métriques métier et techniques :

- **RED Method (pour requêtes) :**
  - Rate : Nombre de requêtes par seconde
  - Errors : Taux d'erreurs (4xx, 5xx)
  - Duration : Latence (P50, P95, P99)
- **USE Method (pour ressources) :**
  - Utilization : Pourcentage d'utilisation
  - Saturation : Travail en attente (queue depth)
  - Errors : Erreurs rencontrées
- **Métriques Métier :**
  - Commandes créées/heure
  - Panier moyen
  - Taux de conversion
  - Utilisateurs actifs

Configuration Prometheus pour BijouxShop :

```
# prometheus.yml
global:
  scrape_interval: 15s
  evaluation_interval: 15s
  external_labels:
    cluster: 'bijouxshop-prod'
    environment: 'production'

# Alertmanager configuration
alerting:
  alertmanagers:
    - static_configs:
        - targets: ['alertmanager:9093']

# Load rules
rule_files:
```

```
  - /etc/prometheus/alerts/*.yml

# Scrape configs
scrape_configs:
  # Spring Boot Actuator metrics
  - job_name: 'bijouxshop-app'
    kubernetes_sd_configs:
      - role: pod
        namespaces:
          names: ['bijouxshop-prod']
    relabel_configs:
      - source_labels: [
         __meta_kubernetes_pod_annotation_prometheus_io_scrape]
        action: keep
        regex: true
      - source_labels: [
         __meta_kubernetes_pod_annotation_prometheus_io_path]
        action: replace
        target_label: __metrics_path__
        regex: (.+)
      - source_labels: [__address__,
         __meta_kubernetes_pod_annotation_prometheus_io_port]
        action: replace
        regex: ([^:]+)(?::\d+)?;(\d+)
        replacement: $1:$2
        target_label: __address__

  # Node Exporter for infrastructure metrics
  - job_name: 'node-exporter'
    kubernetes_sd_configs:
      - role: node
    relabel_configs:
      - source_labels: [__address__]
        regex: '(.*):10250'
        replacement: '${1}:9100'
        target_label: __address__

  # PostgreSQL Exporter
  - job_name: 'postgres'
    static_configs:
      - targets: ['postgres-exporter:9187']
        labels:
          database: 'bijouxshop'

  # Redis Exporter
  - job_name: 'redis'
    static_configs:
      - targets: ['redis-exporter:9121']
```

Règles d'Alerting Prometheus :

```
# alerts/app-alerts.yml
groups:
  - name: bijouxshop_app_alerts
```

```
    interval: 30s
    rules:
      # High Error Rate
      - alert: HighErrorRate
        expr: |
          sum(rate(http_server_requests_seconds_count{status=~"5.."}[5m
             ]))
          /
          sum(rate(http_server_requests_seconds_count[5m]))
          > 0.05
        for: 5m
        labels:
          severity: critical
          component: application
        annotations:
          summary: "High error rate detected"
          description: "Error rate is {{ $value | humanizePercentage }}
                        (threshold: 5%)"

      # High Latency
      - alert: HighLatency
        expr: |
          histogram_quantile(0.95,
            sum(rate(http_server_requests_seconds_bucket[5m]))
            by (le, uri)
          ) > 1
        for: 10m
        labels:
          severity: warning
          component: application
        annotations:
          summary: "High latency on {{ $labels.uri }}"
          description: "P95 latency is {{ $value }}s (threshold: 1s)"

      # Pod Restart Loop
      - alert: PodRestartLoop
        expr: |
          rate(kube_pod_container_status_restarts_total{
            namespace="bijouxshop-prod"
          }[15m]) > 0
        for: 5m
        labels:
          severity: warning
          component: kubernetes
        annotations:
          summary: "Pod {{ $labels.pod }} is restarting"
          description: "Pod has restarted {{ $value }} times in 15
             minutes"

      # High Memory Usage
      - alert: HighMemoryUsage
        expr: |
          sum(container_memory_working_set_bytes{
```

```
            namespace="bijouxshop-prod",
            container="bijouxshop"
          }) by (pod)
          /
          sum(container_spec_memory_limit_bytes{
            namespace="bijouxshop-prod",
            container="bijouxshop"
          }) by (pod)
          > 0.9
        for: 5m
        labels:
          severity: warning
          component: resources
        annotations:
          summary: "High memory usage on {{ $labels.pod }}"
          description: "Memory usage is {{ $value | humanizePercentage
             }}"

      # Database Connection Pool Exhaustion
      - alert: DatabaseConnectionPoolExhaustion
        expr: |
          hikaricp_connections_active / hikaricp_connections_max > 0.9
        for: 2m
        labels:
          severity: critical
          component: database
        annotations:
          summary: "Database connection pool nearly exhausted"
          description: "Active connections: {{ $value |
             humanizePercentage }}"
```

0.2.2 Deuxième Pilier : Logs

Les logs fournissent un contexte détaillé sur les événements système :

1. Structured Logging

Utilisez des logs structurés (JSON) plutôt que du texte libre :

```
// Configuration Logback pour Spring Boot
// src/main/resources/logback-spring.xml
<configuration>
    <springProfile name="production">
        <appender name="CONSOLE" class="ch.qos.logback.core.
           ConsoleAppender">
            <encoder class="net.logstash.logback.encoder.
               LogstashEncoder">
                <customFields>
                    {"application":"bijouxshop","environment":"
                       production"}
                </customFields>
                <includeMdcKeyNames>
                    traceId,spanId,userId,requestId
                </includeMdcKeyNames>
            </encoder>
```

```
        </appender>

        <root level="INFO">
            <appender-ref ref="CONSOLE"/>
        </root>

        <logger name="com.bijouxshop" level="DEBUG"/>
        <logger name="org.springframework.web" level="INFO"/>
        <logger name="org.hibernate.SQL" level="DEBUG"/>
    </springProfile>
</configuration>
```

Exemple de log structuré produit :

```
{
  "@timestamp": "2026-02-04T10:15:30.123Z",
  "level": "INFO",
  "logger_name": "com.bijouxshop.service.OrderService",
  "message": "Order created successfully",
  "application": "bijouxshop",
  "environment": "production",
  "thread_name": "http-nio-8080-exec-1",
  "traceId": "abc123def456",
  "spanId": "789ghi012jkl",
  "userId": "user-42",
  "orderId": "ORDER-2026-001234",
  "orderAmount": 299.99,
  "orderItems": 3,
  "duration_ms": 145
}
```

2. Log Levels Best Practices

Utilisez les niveaux de log appropriés :

- **ERROR :** Erreurs nécessitant une intervention immédiate (exceptions non gérées)
- **WARN :** Situations anormales mais gérées (retry après échec, deprecated API)
- **INFO :** Événements métier importants (commande créée, paiement validé)
- **DEBUG :** Détails techniques pour le debugging (paramètres de requête, état interne)
- **TRACE :** Informations très détaillées (entrée/sortie de chaque méthode)

3. Log Aggregation avec ELK Stack

Configuration complète pour centraliser les logs :

```
# Filebeat configuration pour collecter les logs Kubernetes
# filebeat-configmap.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: filebeat-config
  namespace: logging
data:
  filebeat.yml: |
    filebeat.inputs:
    - type: container
      paths:
```

```
        - /var/log/containers/*bijouxshop*.log
      processors:
        - add_kubernetes_metadata:
            host: ${NODE_NAME}
            matchers:
            - logs_path:
                logs_path: "/var/log/containers/"

    output.elasticsearch:
      hosts: ["elasticsearch:9200"]
      index: "bijouxshop-logs-%{+yyyy.MM.dd}"

    setup.ilm.enabled: false
    setup.template.name: "bijouxshop"
    setup.template.pattern: "bijouxshop-*"
```

0.2.3 Troisième Pilier : Traces

Le tracing distribué permet de suivre une requête à travers tous les microservices :

1. OpenTelemetry Integration

Configuration Spring Boot pour tracing automatique :

```
<!-- pom.xml - Dependances OpenTelemetry -->
<dependency>
    <groupId>io.opentelemetry</groupId>
    <artifactId>opentelemetry-api</artifactId>
    <version>1.32.0</version>
</dependency>
<dependency>
    <groupId>io.opentelemetry.instrumentation</groupId>
    <artifactId>opentelemetry-spring-boot-starter</artifactId>
    <version>1.32.0-alpha</version>
</dependency>
<dependency>
    <groupId>io.opentelemetry.instrumentation</groupId>
    <artifactId>opentelemetry-jdbc</artifactId>
    <version>1.32.0-alpha</version>
</dependency>
```

```
# application.yml - Configuration tracing
spring:
  application:
    name: bijouxshop

otel:
  service:
    name: bijouxshop
  traces:
    exporter: jaeger
  metrics:
    exporter: prometheus
  exporter:
    jaeger:
```

```
      endpoint: http://jaeger-collector:14250
  resource:
    attributes:
      environment: production
      version: 1.0.0
```

2. Custom Spans pour Opérations Métier

Ajoutez des spans personnalisés pour tracer la logique métier :

```
import io.opentelemetry.api.trace.Span;
import io.opentelemetry.api.trace.Tracer;
import io.opentelemetry.context.Scope;

@Service
public class OrderService {

    private final Tracer tracer;

    public OrderService(Tracer tracer) {
        this.tracer = tracer;
    }

    public Order createOrder(OrderRequest request) {
        Span span = tracer.spanBuilder("createOrder")
                          .setAttribute("user.id", request.getUserId())
                          .setAttribute("order.items", request.getItems
                             ().size())
                          .startSpan();

        try (Scope scope = span.makeCurrent()) {
            // Validate order
            Span validateSpan = tracer.spanBuilder("validateOrder")
                                      .startSpan();
            try (Scope validateScope = validateSpan.makeCurrent()) {
                validateOrder(request);
            } finally {
                validateSpan.end();
            }

            // Calculate total
            Span calculateSpan = tracer.spanBuilder("calculateTotal")
                                       .startSpan();
            try (Scope calculateScope = calculateSpan.makeCurrent()) {
                BigDecimal total = calculateTotal(request);
                calculateSpan.setAttribute("order.total", total.
                   toString());
            } finally {
                calculateSpan.end();
            }

            // Save to database
            Span saveSpan = tracer.spanBuilder("saveOrder").startSpan()
               ;
            try (Scope saveScope = saveSpan.makeCurrent()) {
```

```
                Order order = orderRepository.save(toEntity(request));
                span.setAttribute("order.id", order.getId());
                return order;
            } finally {
                saveSpan.end();
            }

        } catch (Exception e) {
            span.recordException(e);
            span.setStatus(StatusCode.ERROR, e.getMessage());
            throw e;
        } finally {
            span.end();
        }
    }
}
```

0.3 Métriques DORA : Mesurer les Performances DevOps

Les quatre métriques DORA (DevOps Research and Assessment) sont les indicateurs clés pour mesurer les performances DevOps.

0.3.1 1. Deployment Frequency (Fréquence de Déploiement)

Définition : Nombre de déploiements en production par période.

Niveaux de performance :

- **Elite :** Plusieurs déploiements par jour
- **High :** Entre 1 fois par jour et 1 fois par semaine
- **Medium :** Entre 1 fois par semaine et 1 fois par mois
- **Low :** Moins d'une fois par mois

Comment améliorer :

- Automatiser complètement le pipeline CI/CD
- Réduire la taille des batches de changements
- Implémenter feature flags pour découpler déploiement et release
- Adopter trunk-based development
- Automatiser les tests pour réduire le temps de validation

Mesure avec Prometheus :

```
# Counter increment     chaque deploiement
deployment_total{
  application="bijouxshop",
  environment="production"
}

# Query PromQL pour deployments par jour
sum(increase(deployment_total{
  environment="production"
}[24h]))
```

0.3.2 2. Lead Time for Changes

Définition : Temps entre le commit de code et son déploiement en production.
Niveaux de performance :

— **Elite :** Moins d'une heure

— **High :** Entre 1 jour et 1 semaine

— **Medium :** Entre 1 semaine et 1 mois

— **Low :** Plus d'un mois

Comment améliorer :

— Optimiser le temps de build (cache, builds parallèles)

— Réduire la suite de tests (tests parallèles, flaky tests)

— Automatiser les approbations et les déploiements

— Minimiser les processus manuels et les handoffs

— Utiliser des review rapides et efficaces

Calcul avec données Git et CI/CD :

```
# Script Python pour calculer le Lead Time
import requests
from datetime import datetime

def calculate_lead_time():
    # Recuperer les commits depuis Git
    commits = get_commits_from_gitlab()

    # Recuperer les deploiements depuis Kubernetes
    deployments = get_deployments_from_k8s()

    lead_times = []
    for deployment in deployments:
        commit_time = get_commit_time(deployment['sha'])
        deploy_time = deployment['timestamp']

        lead_time = (deploy_time - commit_time).total_seconds() / 3600
        lead_times.append(lead_time)

    # Calculer P50, P95, P99
    p50 = percentile(lead_times, 50)
    p95 = percentile(lead_times, 95)
    p99 = percentile(lead_times, 99)

    print(f"Lead Time P50: {p50:.2f} hours")
    print(f"Lead Time P95: {p95:.2f} hours")
    print(f"Lead Time P99: {p99:.2f} hours")
```

0.3.3 3. Change Failure Rate

Définition : Pourcentage de déploiements causant une dégradation du service en production.

Niveaux de performance :

— **Elite :** 0-15%
— **High :** 16-30%
— **Medium :** 31-45%
— **Low :** Plus de 45%

Comment améliorer :

— Augmenter la couverture de tests (unit, integration, e2e)
— Implémenter des tests de performance automatisés
— Utiliser des déploiements progressifs (canary, blue-green)
— Effectuer des code reviews rigoureuses
— Monitorer proactivement avec des alertes
— Faire des post-mortems systématiques

Mesure :

```
# Ratio deploiements echoues / deploiements totaux
sum(deployment_failed_total{environment="production"})
/
sum(deployment_total{environment="production"})
* 100
```

0.3.4 4. Mean Time to Recovery (MTTR)

Définition : Temps moyen pour restaurer le service après un incident en production.
Niveaux de performance :

— **Elite :** Moins d'une heure
— **High :** Moins d'un jour
— **Medium :** Entre 1 jour et 1 semaine
— **Low :** Plus d'une semaine

Comment améliorer :

— Automatiser les rollbacks
— Implémenter des health checks robustes
— Créer des runbooks détaillés pour incidents communs
— Pratiquer game days / chaos engineering
— Améliorer l'observabilité pour diagnostic rapide
— Maintenir des environnements de test réalistes

Mesure avec données d'incidents :

```
# Temps moyen de resolution des incidents
avg(incident_resolution_duration_seconds{
  severity="critical",
  environment="production"
}) / 3600
```

Partie XI : Cas Pratiques Avancés

0.4 Cas Pratique 1 : Migration vers Microservices

0.4.1 Contexte

BijouxShop est actuellement une application monolithique Spring Boot. L'entreprise souhaite migrer progressivement vers une architecture microservices pour améliorer la scalabilité et permettre aux équipes de déployer indépendamment.

0.4.2 Stratégie de Migration : Strangler Fig Pattern

Le pattern Strangler Fig permet de migrer progressivement sans big bang :
Phase 1 : Identification des Bounded Contexts
Analysez le monolithe pour identifier les domaines métier :

- **User Service :** Gestion utilisateurs, authentification
- **Product Service :** Catalogue produits, inventory
- **Order Service :** Commandes, panier
- **Payment Service :** Paiements, facturation
- **Notification Service :** Emails, SMS, notifications push

Phase 2 : Extraction du Premier Microservice
Commencez par un service à faible couplage, par exemple le Notification Service :
Architecture :

```
bijouxshop-notifications/
          src/
                main/
                      java/
                            com/bijouxshop/notifications/
                                NotificationApplication.java
                                controller/
                                      NotificationController.java
                                service/
                                      EmailService.java
                                      SmsService.java
                                model/
                                      Notification.java
                                config/
                                    MessagingConfig.java
                      resources/
                          application.yml
                          templates/
                              order-confirmation.html
```

```
                                payment-receipt.html
                test/
                    java/
          Dockerfile
          pom.xml
          README.md
```

Communication Asynchrone avec RabbitMQ :

```
// Configuration RabbitMQ dans le monolithe
@Configuration
public class RabbitMQConfig {

    @Bean
    public Queue notificationQueue() {
        return new Queue("notifications.queue", true);
    }

    @Bean
    public TopicExchange notificationExchange() {
        return new TopicExchange("notifications.exchange");
    }

    @Bean
    public Binding binding(Queue queue, TopicExchange exchange) {
        return BindingBuilder.bind(queue)
                             .to(exchange)
                             .with("notification.*");
    }
}

// Publisher dans le monolithe (OrderService)
@Service
public class OrderService {

    private final RabbitTemplate rabbitTemplate;

    public void createOrder(OrderRequest request) {
        Order order = saveOrder(request);

        // Publier evenement pour notification
        NotificationEvent event = new NotificationEvent(
            "order.created",
            order.getUserEmail(),
            "Order Confirmation",
            Map.of("orderId", order.getId(),
                   "amount", order.getTotal())
        );

        rabbitTemplate.convertAndSend(
            "notifications.exchange",
            "notification.email",
            event
        );
```

```
    }
}
```

```
// Consumer dans le microservice Notification
@Service
public class NotificationConsumer {

    private final EmailService emailService;
    private final SmsService smsService;

    @RabbitListener(queues = "notifications.queue")
    public void handleNotification(NotificationEvent event) {
        log.info("Received notification event: {}", event);

        switch (event.getType()) {
            case "order.created":
                emailService.sendOrderConfirmation(
                    event.getRecipient(),
                    event.getData()
                );
                break;
            case "payment.processed":
                emailService.sendPaymentReceipt(
                    event.getRecipient(),
                    event.getData()
                );
                smsService.sendPaymentNotification(
                    event.getRecipient()
                );
                break;
            default:
                log.warn("Unknown notification type: {}", event.getType
                   ());
        }
    }
}
```

Phase 3 : API Gateway

Introduisez un API Gateway pour router les requêtes :

```
# Configuration Spring Cloud Gateway
spring:
  cloud:
    gateway:
      routes:
        # Routes vers le monolithe
        - id: monolith_users
          uri: http://bijouxshop-monolith:8080
          predicates:
            - Path=/api/users/**
          filters:
            - RewritePath=/api/(?<segment>.*), /$\{segment}

        - id: monolith_products
```

```
          uri: http://bijouxshop-monolith:8080
          predicates:
            - Path=/api/products/**

        - id: monolith_orders
          uri: http://bijouxshop-monolith:8080
          predicates:
            - Path=/api/orders/**

        # Route vers le microservice Notifications
        - id: notifications_service
          uri: http://bijouxshop-notifications:8081
          predicates:
            - Path=/api/notifications/**
          filters:
            - name: CircuitBreaker
              args:
                name: notificationsCB
                fallbackUri: forward:/fallback/notifications
            - name: Retry
              args:
                retries: 3
                statuses: BAD_GATEWAY,SERVICE_UNAVAILABLE
                methods: GET,POST
                backoff:
                  firstBackoff: 100ms
                  maxBackoff: 500ms
                  factor: 2
```

Phase 4 : Extraction Progressive

Répétez le processus pour chaque bounded context, en commençant par les domaines à plus faible couplage.

Ordre recommandé pour BijouxShop :

1. Notification Service (faible couplage)
2. Payment Service (domaine distinct)
3. Product Service (peut être isolé)
4. Order Service (couplage moyen)
5. User Service (couplage fort, à la fin)

0.4.3 Gestion des Données dans les Microservices

Pattern : Database per Service

Chaque microservice a sa propre base de données :

```
# docker-compose.yml - Bases separees
services:
  postgres-users:
    image: postgres:15
    environment:
      POSTGRES_DB: users_db
      POSTGRES_USER: users_service
      POSTGRES_PASSWORD: secure_pass_1
```

```
9
10   postgres-products:
11     image: postgres:15
12     environment:
13       POSTGRES_DB: products_db
14       POSTGRES_USER: products_service
15       POSTGRES_PASSWORD: secure_pass_2
16
17   postgres-orders:
18     image: postgres:15
19     environment:
20       POSTGRES_DB: orders_db
21       POSTGRES_USER: orders_service
22       POSTGRES_PASSWORD: secure_pass_3
```

Gestion de la Cohérence : Saga Pattern

Pour les transactions distribuées, utilisez le Saga pattern :

```
1  // Orchestration-based Saga pour creation de commande
2  @Service
3  public class OrderSagaOrchestrator {
4
5      private final ProductService productService;
6      private final PaymentService paymentService;
7      private final NotificationService notificationService;
8
9      @Transactional
10     public OrderResult createOrder(OrderRequest request) {
11         SagaContext context = new SagaContext();
12
13         try {
14             // Step 1: Reserve products
15             context.reservationId = productService.reserveProducts(
16                 request.getItems()
17             );
18
19             // Step 2: Process payment
20             context.paymentId = paymentService.processPayment(
21                 request.getPaymentInfo(),
22                 request.getTotal()
23             );
24
25             // Step 3: Create order
26             context.orderId = createOrderRecord(request, context);
27
28             // Step 4: Send notification
29             notificationService.sendOrderConfirmation(
30                 context.orderId
31             );
32
33             return OrderResult.success(context.orderId);
34
35         } catch (Exception e) {
36             // Compensation: rollback in reverse order
```

```
            compensate(context, e);
            return OrderResult.failure(e.getMessage());
        }
    }

    private void compensate(SagaContext context, Exception error) {
        log.error("Saga failed, compensating: {}", error.getMessage());

        // Rollback notification (if sent)
        // Usually not needed for notifications

        // Rollback order creation
        if (context.orderId != null) {
            cancelOrder(context.orderId);
        }

        // Rollback payment
        if (context.paymentId != null) {
            paymentService.refundPayment(context.paymentId);
        }

        // Rollback product reservation
        if (context.reservationId != null) {
            productService.cancelReservation(context.reservationId);
        }
    }
}
```

0.5 Cas Pratique 2 : Mise en Place du Chaos Engineering

0.5.1 Objectif

Tester la résilience de BijouxShop en production en introduisant des pannes contrôlées.

0.5.2 Phase 1 : Préparation

Avant de commencer le chaos engineering, assurez-vous que :

— **Monitoring robuste :** Métriques, logs, traces en place
— **Alerting configuré :** Notifications sur incidents
— **Rollback rapide :** Capacité à annuler les changements
— **Hypothèses définies :** Comportements attendus du système

0.5.3 Phase 2 : Installation de Chaos Mesh

Chaos Mesh est un outil Kubernetes pour chaos engineering :

```
# Installation via Helm
helm repo add chaos-mesh https://charts.chaos-mesh.org
helm install chaos-mesh chaos-mesh/chaos-mesh \
  --namespace=chaos-mesh \
  --create-namespace \
```

```
  --set chaosDaemon.runtime=containerd \
  --set chaosDaemon.socketPath=/run/containerd/containerd.sock

# Verification
kubectl get pods -n chaos-mesh
```

0.5.4 Phase 3 : Expériences de Chaos

Expérience 1 : Pod Failure
Simulez la défaillance aléatoire de pods :

```
# pod-kill-chaos.yaml
apiVersion: chaos-mesh.org/v1alpha1
kind: PodChaos
metadata:
  name: pod-kill-bijouxshop
  namespace: bijouxshop-prod
spec:
  action: pod-kill
  mode: one
  selector:
    namespaces:
      - bijouxshop-prod
    labelSelectors:
      app: bijouxshop
  scheduler:
    cron: "@every 10m"
  duration: "10s"
```

Hypothèse : Le système doit continuer à fonctionner grâce aux 3 replicas. Les requêtes doivent être automatiquement routées vers les pods sains.

Validation :

— Aucune erreur 5xx visible par les utilisateurs

— Latence P99 reste $<$ 1s

— Kubernetes recrée automatiquement le pod

— Aucune alerte critique

Expérience 2 : Network Latency
Introduisez de la latence réseau :

```
# network-latency-chaos.yaml
apiVersion: chaos-mesh.org/v1alpha1
kind: NetworkChaos
metadata:
  name: network-delay-bijouxshop
  namespace: bijouxshop-prod
spec:
  action: delay
  mode: all
  selector:
    namespaces:
      - bijouxshop-prod
    labelSelectors:
```

```
      app: bijouxshop
  delay:
    latency: "500ms"
    correlation: "50"
    jitter: "100ms"
  duration: "5m"
  scheduler:
    cron: "@every 1h"
  direction: to
  target:
    selector:
      namespaces:
        - bijouxshop-prod
      labelSelectors:
        app: postgres
    mode: all
```

Hypothèse : Les timeouts et circuit breakers doivent protéger l'application. Les requêtes ne doivent pas bloquer indéfiniment.

Validation :

— Circuit breaker s'ouvre après X échecs

— Messages d'erreur appropriés aux utilisateurs

— Pas d'accumulation de threads bloqués

— Recovery automatique après fin de l'expérience

Expérience 3 : Resource Stress

Stressez CPU et mémoire :

```
# stress-chaos.yaml
apiVersion: chaos-mesh.org/v1alpha1
kind: StressChaos
metadata:
  name: memory-stress-bijouxshop
  namespace: bijouxshop-prod
spec:
  mode: one
  selector:
    namespaces:
      - bijouxshop-prod
    labelSelectors:
      app: bijouxshop
  stressors:
    memory:
      workers: 4
      size: "512MB"
  duration: "3m"
```

Hypothèse : HorizontalPodAutoscaler doit scale out avant OOMKill. Les limits de ressources doivent protéger le noeud.

Validation :

— HPA augmente le nombre de replicas

— Aucun OOMKill observé

— Load balancer distribue la charge

— Scale down automatique après fin du stress

0.5.5 Phase 4 : GameDay - Exercice de Crise

Organisez un GameDay mensuel pour tester la réponse aux incidents :
Scénario GameDay Exemple :

1. **T+0min :** Base de données PostgreSQL primaire tombe en panne
2. **Objectif :** Équipe doit détecter, diagnostiquer et basculer sur le replica en moins de 15 minutes
3. **Participants :** Dev, Ops, Product Owner
4. **Observateurs :** Management, autres équipes

Déroulement :

— Injecter la panne via Chaos Mesh
— Chronométrer la détection (alertes)
— Observer la communication entre équipes
— Valider les actions de remediation
— Documenter les leçons apprises

Métriques de Succès :

— **MTTD (Mean Time To Detect) :** $<$ 2 minutes
— **MTTR (Mean Time To Recovery) :** $<$ 15 minutes
— **Communication :** War room créé en $<$ 5 minutes
— **Documentation :** Post-mortem publié sous 24h

Partie XII : Références Bibliographiques et Ressources

0.1 Ouvrages Fondamentaux

0.1.1 DevOps et Culture

1. The Phoenix Project : A Novel about IT, DevOps, and Helping Your Business Win

- **Auteurs :** Gene Kim, Kevin Behr, George Spafford
- **Année :** 2013
- **ISBN :** 978-0988262508
- **Résumé :** Roman business illustrant la transformation DevOps d'une entreprise fictive. Introduit les Three Ways : Flow, Feedback, et Continuous Learning. Référence incontournable pour comprendre la culture DevOps.
- **Concepts clés :** Théorie des contraintes, Work In Progress (WIP) limits, feedback loops

2. The DevOps Handbook : How to Create World-Class Agility, Reliability, and Security

- **Auteurs :** Gene Kim, Jez Humble, Patrick Debois, John Willis
- **Année :** 2016
- **ISBN :** 978-1942788003
- **Résumé :** Guide pratique pour implémenter DevOps. Couvre les patterns techniques et organisationnels avec des cas d'usage réels de Google, Amazon, Netflix.
- **Chapitres essentiels :** Deployment pipelines, Testing strategies, Architecture patterns

3. Accelerate : The Science of Lean Software and DevOps

- **Auteurs :** Nicole Forsgren, Jez Humble, Gene Kim
- **Année :** 2018
- **ISBN :** 978-1942788331
- **Résumé :** Basé sur 4 ans de recherche DORA, démontre scientifiquement l'impact de DevOps sur les performances. Introduit les 4 métriques clés : Deployment Frequency, Lead Time, MTTR, Change Failure Rate.
- **Impact :** Établit les métriques standard de l'industrie DevOps

4. Continuous Delivery : Reliable Software Releases through Build, Test, and Deployment Automation

- **Auteurs :** Jez Humble, David Farley
- **Année :** 2010

— **ISBN :** 978-0321601919

— **Résumé :** Bible de la livraison continue. Décrit les deployment pipelines, strategies de tests, configuration management.

— **Patterns clés :** Blue-Green Deployment, Canary Release, Feature Toggles

0.1.2 Site Reliability Engineering

5. Site Reliability Engineering : How Google Runs Production Systems

— **Éditeurs :** Betsy Beyer, Chris Jones, Jennifer Petoff, Niall Richard Murphy

— **Année :** 2016

— **ISBN :** 978-1491929124

— **Résumé :** Compilation d'articles de SREs Google. Couvre monitoring, incident response, capacity planning, automation.

— **Concepts innovants :** Error budgets, SLO/SLI/SLA, Toil reduction

— **Disponible gratuitement :** https ://sre.google/books/

6. The Site Reliability Workbook : Practical Ways to Implement SRE

— **Éditeurs :** Betsy Beyer, Niall Richard Murphy, David K. Rensin, Kent Kawahara, Stephen Thorne

— **Année :** 2018

— **ISBN :** 978-1492029502

— **Résumé :** Companion pratique du premier livre SRE. Études de cas de diverses entreprises (Evernote, The Home Depot, etc.)

0.1.3 Architecture et Microservices

7. Building Microservices : Designing Fine-Grained Systems

— **Auteur :** Sam Newman

— **Année :** 2015 (2ème édition 2021)

— **ISBN :** 978-1492034025

— **Résumé :** Guide complet sur l'architecture microservices. Couvre découpage, communication, données, sécurité, testing.

— **Patterns :** Service mesh, API gateway, Saga pattern, CQRS

8. Domain-Driven Design : Tackling Complexity in the Heart of Software

— **Auteur :** Eric Evans

— **Année :** 2003

— **ISBN :** 978-0321125217

— **Résumé :** Fondation du DDD. Essential pour identifier les bounded contexts dans une migration microservices.

— **Concepts :** Ubiquitous Language, Bounded Context, Aggregates

0.1.4 Kubernetes et Orchestration

9. Kubernetes in Action

— **Auteur :** Marko Luksa

— **Année :** 2018 (2ème édition 2020)

— **ISBN :** 978-1617297618

— **Résumé :** Introduction complète à Kubernetes avec exemples pratiques. Du déploiement basique aux patterns avancés.

10. Kubernetes Patterns : Reusable Elements for Designing Cloud-Native Applications

— **Auteurs :** Bilgin Ibryam, Roland Huß

— **Année :** 2019

— **ISBN :** 978-1492050285

— **Résumé :** Catalogue de patterns pour applications cloud-native. Sidecar, Ambassador, Adapter, etc.

0.1.5 Sécurité DevSecOps

11. DevOpsSec : Delivering Secure Software through Continuous Delivery

— **Auteurs :** Jim Bird

— **Année :** 2016

— **ISBN :** 978-1491971413

— **Résumé :** Intégration de la sécurité dans les pipelines DevOps. Threat modeling, security testing, compliance as code.

0.1.6 Infrastructure as Code

12. Infrastructure as Code : Managing Servers in the Cloud

— **Auteur :** Kief Morris

— **Année :** 2016 (2ème édition 2020)

— **ISBN :** 978-1098114671

— **Résumé :** Principes et patterns pour gérer l'infrastructure avec du code. Couvre Terraform, Ansible, CloudFormation.

13. Terraform : Up and Running

— **Auteur :** Yevgeniy Brikman

— **Année :** 2019 (3ème édition 2022)

— **ISBN :** 978-1098116743

— **Résumé :** Guide pratique Terraform avec exemples réels. Module design, state management, testing.

0.2 Articles et Papers Académiques

0.2.1 Papers Fondateurs DevOps

1. State of DevOps Report (Annual)

— **Organisation :** DORA (DevOps Research and Assessment), acquis par Google Cloud

— **Période :** 2014-présent

— **URL :** https ://dora.dev/

— **Contenu :** Recherche annuelle sur les pratiques DevOps, corrélation entre pratiques techniques et performances business

— **Méthodologie :** Survey de milliers d'organisations, analyse statistique rigoureuse

2. "Continuous Delivery : Huge Benefits, but Challenges Too"

— **Auteurs :** Lianping Chen

— **Venue :** IEEE Software, 2015

— **DOI :** 10.1109/MS.2015.27

— **Résumé :** Étude empirique sur l'adoption de CD. Identifie bénéfices et obstacles dans organisations réelles.

3. "The Role of Continuous Integration in Software Development"

— **Auteurs :** Michael Hilton et al.

— **Venue :** ACM/IEEE International Conference on Software Engineering, 2016

— **DOI :** 10.1145/2884781.2884840

— **Résumé :** Analyse de 246 projets GitHub. Impact mesurable de CI sur qualité et productivité.

0.2.2 Chaos Engineering

4. "Principles of Chaos Engineering"

— **Auteurs :** Casey Rosenthal, Nora Jones (Netflix)

— **Année :** 2015

— **URL :** https ://principlesofchaos.org/

— **Contenu :** Définition formelle du Chaos Engineering. Méthodologie pour tester la résilience en production.

0.2.3 Site Reliability Engineering

5. "Large-Scale Cluster Management at Google with Borg"

— **Auteurs :** Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, et al.

— **Venue :** EuroSys, 2015

— **Résumé :** Description de Borg, précurseur de Kubernetes. Gestion de clusters à l'échelle de Google.

0.3 Ressources en Ligne

0.3.1 Blogs Techniques

1. Netflix Tech Blog

— **URL :** https ://netflixtechblog.com/

— **Contenu :** Articles techniques sur microservices, chaos engineering, performance optimization

— **Articles recommandés :**

  — "Chaos Engineering Upgraded" (2016)
  — "Adopting Microservices at Netflix : Lessons for Architectural Design" (2015)
  — "Full Cycle Developers at Netflix" (2018)

2. Google Cloud Blog - SRE

— **URL :** https ://cloud.google.com/blog/products/devops-sre

— **Contenu :** Best practices SRE, études de cas clients, annonces produits

3. AWS Architecture Blog

— **URL :** https ://aws.amazon.com/blogs/architecture/

— **Contenu :** Architectures de référence, patterns cloud-native

4. Martin Fowler's Blog

— **URL :** https ://martinfowler.com/

— **Articles essentiels :**

  — "Microservices" (2014) - Définition canonique
  — "Continuous Integration" (2006) - Pratiques CI
  — "Feature Toggles" (2017) - Gestion feature flags
  — "Strangler Fig Application" (2004) - Pattern de migration

0.3.2 Cours et Formations

1. Kubernetes Official Documentation

— **URL :** https ://kubernetes.io/docs/

— **Sections clés :** Concepts, Tasks, Tutorials, Reference

— **Certifications :** CKA (Certified Kubernetes Administrator), CKAD (Certified Kubernetes Application Developer)

2. Docker Documentation

— **URL :** https ://docs.docker.com/

— **Guides essentiels :** Dockerfile best practices, Multi-stage builds, Security

3. GitLab CI/CD Documentation

— **URL :** https ://docs.gitlab.com/ee/ci/

— **Tutoriels :** Pipeline configuration, Variables, Artifacts, Environments

0.3.3 Outils et Repositories

1. Awesome DevOps (GitHub)

— **URL :** https ://github.com/awesome-foss/awesome-devops

— **Contenu :** Liste curatée d'outils DevOps open source par catégorie

2. Kubernetes Examples (GitHub)

— **URL :** https ://github.com/kubernetes/examples

— **Contenu :** Exemples officiels de déploiements Kubernetes

Partie XIII : Commandes de Référence Complètes

0.4 Git - Commandes Avancées

0.4.1 Gestion des Branches

```
# Lister toutes les branches (locales et distantes)
git branch -a

# Supprimer une branche locale
git branch -d feature-branch

# Forcer la suppression d'une branche locale non mergee
git branch -D feature-branch

# Supprimer une branche distante
git push origin --delete feature-branch

# Renommer une branche locale
git branch -m ancien-nom nouveau-nom

# Creer et basculer sur une nouvelle branche depuis un commit
git checkout -b new-branch abc1234

# Voir les branches mergees dans main
git branch --merged main

# Voir les branches non mergees
git branch --no-merged main

# Comparer deux branches
git diff main..feature-branch

# Voir les commits dans feature-branch mais pas dans main
git log main..feature-branch

# Lister les fichiers differents entre deux branches
git diff --name-only main..feature-branch
```

0.4.2 Historique et Recherche

```
# Afficher l'historique avec graphe
git log --oneline --graph --all --decorate

# Rechercher un commit par message
git log --grep="fix bug"

# Rechercher les commits d'un auteur
git log --author="John Doe"

# Voir les commits dans une periode
git log --since="2 weeks ago"
git log --after="2026-01-01" --before="2026-02-01"

# Voir les commits modifiant un fichier specifique
git log -- path/to/file.java

# Voir qui a modifie chaque ligne d'un fichier
git blame src/main/java/UserService.java

# Rechercher quand une fonction a ete introduite
git log -S "functionName" --source --all

# Trouver le commit qui a introduit un bug (bisect)
git bisect start
git bisect bad                  # Le commit actuel est mauvais
git bisect good abc1234         # Ce commit etait bon
# Git checkout automatiquement des commits intermediaires
# Tester et marquer comme good/bad jusqu'a trouver le commit

# Voir l'historique de reflog (utile pour recuperer des commits perdus)
git reflog

# Recuperer un commit "perdu"
git checkout -b recovery-branch abc1234
```

0.4.3 Stash - Sauvegardes Temporaires

```
# Sauvegarder les modifications en cours
git stash

# Sauvegarder avec un message descriptif
git stash save "WIP: feature X incomplete"

# Inclure les fichiers non suivis
git stash -u

# Lister tous les stashes
git stash list

# Appliquer le dernier stash sans le supprimer
git stash apply

```

```
# Appliquer un stash specifique
git stash apply stash@{2}

# Appliquer et supprimer le dernier stash
git stash pop

# Voir le contenu d'un stash
git stash show -p stash@{0}

# Creer une branche depuis un stash
git stash branch new-branch-name stash@{0}

# Supprimer un stash
git stash drop stash@{0}

# Supprimer tous les stashes
git stash clear
```

0.4.4 Rebase Avancé

```
# Rebase interactif sur les 5 derniers commits
git rebase -i HEAD~5

# Commandes dans rebase interactif:
# pick   = utiliser le commit
# reword = utiliser le commit mais modifier le message
# edit   = utiliser le commit mais s'arreter pour amendments
# squash = fusionner avec le commit precedent
# fixup  = comme squash mais ignorer le message
# drop   = supprimer le commit

# Continuer apres resolution de conflits
git rebase --continue

# Annuler le rebase en cours
git rebase --abort

# Rebase d'une branche sur main
git checkout feature-branch
git rebase main

# Rebase avec autosquash (pour fixup commits)
git commit --fixup=abc1234
git rebase -i --autosquash HEAD~10
```

0.4.5 Cherry-Pick et Reset

```
# Appliquer un commit specifique sur la branche courante
git cherry-pick abc1234

# Cherry-pick sans committer (pour modifications)
```

```
git cherry-pick -n abc1234

# Cherry-pick plusieurs commits
git cherry-pick abc1234 def5678 ghi9012

# Reset soft: garde les modifications en staged
git reset --soft HEAD~1

# Reset mixed (default): garde les modifications en unstaged
git reset HEAD~1

# Reset hard: supprime toutes les modifications
git reset --hard HEAD~1

# Reset vers un commit specifique
git reset --hard abc1234

# Annuler un commit public (cree un nouveau commit inverse)
git revert abc1234

# Revert d'un merge commit
git revert -m 1 abc1234
```

0.5 Docker - Commandes Complètes

0.5.1 Images

```
# Construire une image
docker build -t bijouxshop:latest .

# Construire avec un Dockerfile specifique
docker build -f Dockerfile.prod -t bijouxshop:prod .

# Construire avec des build args
docker build --build-arg VERSION=1.0.0 -t bijouxshop:1.0.0 .

# Construire sans cache
docker build --no-cache -t bijouxshop:latest .

# Lister les images
docker images

# Supprimer une image
docker rmi bijouxshop:latest

# Supprimer toutes les images non utilisees
docker image prune -a

# Inspecter une image
docker inspect bijouxshop:latest

# Voir l'historique de construction d'une image
```

```
docker history bijouxshop:latest

# Sauvegarder une image dans un fichier tar
docker save -o bijouxshop.tar bijouxshop:latest

# Charger une image depuis un fichier tar
docker load -i bijouxshop.tar

# Tagger une image
docker tag bijouxshop:latest registry.gitlab.com/user/bijouxshop:1.0.0

# Pousser une image vers un registry
docker push registry.gitlab.com/user/bijouxshop:1.0.0

# Tirer une image depuis un registry
docker pull registry.gitlab.com/user/bijouxshop:1.0.0
```

0.5.2 Conteneurs

```
# Lancer un conteneur en arriere-plan
docker run -d --name bijouxshop-app bijouxshop:latest

# Lancer avec exposition de port
docker run -d -p 8080:8080 --name bijouxshop-app bijouxshop:latest

# Lancer avec variables d'environnement
docker run -d -e DATABASE_URL=jdbc:postgresql://db:5432/bijouxshop \
  -e SPRING_PROFILES_ACTIVE=prod \
  --name bijouxshop-app bijouxshop:latest

# Lancer avec volume monte
docker run -d -v /host/path:/container/path bijouxshop:latest

# Lancer avec limites de ressources
docker run -d --memory="512m" --cpus="0.5" bijouxshop:latest

# Lister les conteneurs en cours
docker ps

# Lister tous les conteneurs (y compris arretes)
docker ps -a

# Arreter un conteneur
docker stop bijouxshop-app

# Demarrer un conteneur arrete
docker start bijouxshop-app

# Redemarrer un conteneur
docker restart bijouxshop-app

# Supprimer un conteneur
```

```
docker rm bijouxshop-app

# Forcer la suppression d'un conteneur en cours
docker rm -f bijouxshop-app

# Voir les logs d'un conteneur
docker logs bijouxshop-app

# Suivre les logs en temps reel
docker logs -f bijouxshop-app

# Voir les dernieres 100 lignes de logs
docker logs --tail 100 bijouxshop-app

# Executer une commande dans un conteneur en cours
docker exec bijouxshop-app ls /app

# Ouvrir un shell interactif dans un conteneur
docker exec -it bijouxshop-app /bin/bash

# Copier des fichiers depuis/vers un conteneur
docker cp bijouxshop-app:/app/logs/app.log ./local-app.log
docker cp ./config.yml bijouxshop-app:/app/config/

# Voir les statistiques d'utilisation des ressources
docker stats bijouxshop-app

# Inspecter un conteneur
docker inspect bijouxshop-app

# Voir les processus dans un conteneur
docker top bijouxshop-app

# Exporter le filesystem d'un conteneur
docker export bijouxshop-app > bijouxshop-filesystem.tar
```

0.5.3 Réseaux

```
# Lister les reseaux
docker network ls

# Creer un reseau bridge
docker network create --driver bridge bijouxshop-network

# Creer un reseau avec un subnet specifique
docker network create --driver bridge \
  --subnet=172.20.0.0/16 \
  --gateway=172.20.0.1 \
  bijouxshop-network

# Inspecter un reseau
docker network inspect bijouxshop-network
```

```
15
16 # Connecter un conteneur a un reseau
17 docker network connect bijouxshop-network bijouxshop-app
18
19 # Deconnecter un conteneur d'un reseau
20 docker network disconnect bijouxshop-network bijouxshop-app
21
22 # Supprimer un reseau
23 docker network rm bijouxshop-network
24
25 # Supprimer tous les reseaux non utilises
26 docker network prune
```

0.5.4 Volumes

```
1  # Lister les volumes
2  docker volume ls
3
4  # Creer un volume
5  docker volume create bijouxshop-data
6
7  # Inspecter un volume
8  docker volume inspect bijouxshop-data
9
10 # Supprimer un volume
11 docker volume rm bijouxshop-data
12
13 # Supprimer tous les volumes non utilises
14 docker volume prune
15
16 # Utiliser un volume dans un conteneur
17 docker run -d -v bijouxshop-data:/app/data bijouxshop:latest
```

0.6 Kubernetes - Commandes kubectl Complètes

0.6.1 Gestion des Ressources

```
1  # Appliquer un fichier de configuration
2  kubectl apply -f deployment.yaml
3
4  # Appliquer tous les fichiers d'un repertoire
5  kubectl apply -f ./k8s/
6
7  # Supprimer une ressource
8  kubectl delete -f deployment.yaml
9
10 # Supprimer par type et nom
11 kubectl delete deployment bijouxshop-app
12
13 # Lister les pods
```

```
kubectl get pods

# Lister avec plus de details
kubectl get pods -o wide

# Lister tous les namespaces
kubectl get pods --all-namespaces

# Lister avec labels
kubectl get pods --show-labels

# Filtrer par label
kubectl get pods -l app=bijouxshop

# Decrire une ressource en detail
kubectl describe pod bijouxshop-app-abc123

# Editer une ressource en place
kubectl edit deployment bijouxshop-app

# Afficher les ressources en YAML
kubectl get deployment bijouxshop-app -o yaml

# Afficher en JSON
kubectl get deployment bijouxshop-app -o json

# Afficher avec JSONPath custom
kubectl get pods -o jsonpath='{.items[*].metadata.name}'
```

0.6.2 Déploiements et Rollouts

```
# Mettre a jour l'image d'un deployment
kubectl set image deployment/bijouxshop-app \
  bijouxshop=bijouxshop:2.0.0

# Voir le statut du rollout
kubectl rollout status deployment/bijouxshop-app

# Voir l'historique des rollouts
kubectl rollout history deployment/bijouxshop-app

# Revenir a la version precedente
kubectl rollout undo deployment/bijouxshop-app

# Revenir a une version specifique
kubectl rollout undo deployment/bijouxshop-app --to-revision=2

# Mettre en pause un rollout
kubectl rollout pause deployment/bijouxshop-app

# Reprendre un rollout
kubectl rollout resume deployment/bijouxshop-app
```

```
22
23 # Redemarrer un deployment (recreer tous les pods)
24 kubectl rollout restart deployment/bijouxshop-app
```

0.6.3 Logs et Debug

```
1  # Voir les logs d'un pod
2  kubectl logs bijouxshop-app-abc123
3
4  # Suivre les logs en temps reel
5  kubectl logs -f bijouxshop-app-abc123
6
7  # Logs d'un conteneur specifique dans un pod multi-conteneurs
8  kubectl logs bijouxshop-app-abc123 -c init-container
9
10 # Logs du conteneur precedent (apres crash)
11 kubectl logs bijouxshop-app-abc123 --previous
12
13 # Logs de tous les pods d'un deployment
14 kubectl logs -l app=bijouxshop --all-containers=true
15
16 # Executer une commande dans un pod
17 kubectl exec bijouxshop-app-abc123 -- ls /app
18
19 # Ouvrir un shell dans un pod
20 kubectl exec -it bijouxshop-app-abc123 -- /bin/sh
21
22 # Port-forward pour acceder a un pod localement
23 kubectl port-forward bijouxshop-app-abc123 8080:8080
24
25 # Port-forward vers un service
26 kubectl port-forward service/bijouxshop-service 8080:80
27
28 # Copier des fichiers depuis/vers un pod
29 kubectl cp bijouxshop-app-abc123:/app/logs/app.log ./local-app.log
30 kubectl cp ./config.yml bijouxshop-app-abc123:/app/config/
31
32 # Voir les evenements du cluster
33 kubectl get events --sort-by='.lastTimestamp'
34
35 # Voir les evenements d'un namespace specifique
36 kubectl get events -n bijouxshop-prod
37
38 # Debugger un pod avec un conteneur ephemere
39 kubectl debug bijouxshop-app-abc123 -it --image=busybox
```

0.6.4 Scaling et Autoscaling

```
1  # Scaler manuellement un deployment
2  kubectl scale deployment bijouxshop-app --replicas=5
3
```

```
# Scaler avec condition
kubectl scale deployment bijouxshop-app --replicas=3 \
  --current-replicas=5

# Autoscaling horizontal
kubectl autoscale deployment bijouxshop-app \
  --min=3 --max=10 --cpu-percent=70

# Voir les HPA
kubectl get hpa

# Decrire un HPA
kubectl describe hpa bijouxshop-app

# Supprimer un HPA
kubectl delete hpa bijouxshop-app
```

0.6.5 Namespaces et Contexts

```
# Lister les namespaces
kubectl get namespaces

# Creer un namespace
kubectl create namespace bijouxshop-staging

# Definir le namespace par defaut
kubectl config set-context --current --namespace=bijouxshop-prod

# Voir le context actuel
kubectl config current-context

# Lister tous les contexts
kubectl config get-contexts

# Basculer vers un autre context
kubectl config use-context prod-cluster
```

Partie XIV : Exercices Supplémentaires et Solutions

0.7 Exercices CI/CD Avancés

0.7.1 Exercice : Pipeline Multi-Environnement avec Approbation Manuelle

Objectif : Créer un pipeline GitLab CI/CD complet déployant sur dev automatiquement, staging avec approbation, et production avec approbation + vérifications.

Contraintes :

— Tests automatiques à chaque étape

— Rollback automatique si les smoke tests échouent

— Notifications Slack à chaque déploiement

— Métriques de déploiement envoyées à Prometheus

Solution :

Le pipeline complet a été fourni dans la Partie X (section Automatisation des Déploiements) avec toutes les étapes demandées incluant :

— Validation du code (checkstyle, spotbugs, hadolint)

— Build et tests (unit, integration)

— Security scanning (dependency-check, SonarQube, Trivy)

— Déploiement multi-environnement (dev automatique, staging/prod manuel)

— Tests post-déploiement (e2e, performance, smoke tests)

— Rollback capability (blue-green deployment)

0.7.2 Exercice : Optimisation du Temps de Build

Problème : Le pipeline prend actuellement 25 minutes pour s'exécuter complètement. L'objectif est de réduire ce temps à moins de 10 minutes.

Analyse des goulets d'étranglement :

— Maven build : 8 minutes

— Tests unitaires : 5 minutes

— Tests d'intégration : 7 minutes

— Docker build : 3 minutes

— Security scans : 2 minutes

Solutions proposées :

1. Parallélisation des jobs

```yaml
# .gitlab-ci.yml - Jobs parallèles
unit-tests:
  stage: test
  script:
    - mvn test -DfailIfNoTests=false
  parallel: 4  # Split tests en 4 jobs parallèles

integration-tests:
  stage: test
  script:
    - mvn verify -Pintegration-tests
  parallel: 2
```

Gain estimé : Tests passent de 12 à 4 minutes (reduction 67%)

2. Cache Maven optimisé

```yaml
# Cache agressif des dépendances Maven
cache:
  key: ${CI_COMMIT_REF_SLUG}
  paths:
    - .m2/repository/
    - target/
  policy: pull-push

variables:
  MAVEN_OPTS: "-Dmaven.repo.local=$CI_PROJECT_DIR/.m2/repository"
```

Gain estimé : Build passe de 8 à 3 minutes (reduction 62%)

3. Docker Layer Caching

```yaml
# Utilisation du Docker layer caching
build-docker:
  stage: package
  script:
    - docker build --cache-from $CI_REGISTRY_IMAGE:latest \
      -t $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA .
```

Gain estimé : Docker build passe de 3 à 1 minute (reduction 67%)

Résultat final :

— Temps original : 25 minutes

— Temps optimisé : 8 minutes

— Réduction : 68%

0.8 Exercices Monitoring et Alerting

0.8.1 Exercice : Créer un Dashboard Grafana Complet

Objectif : Créer un dashboard Grafana pour monitorer BijouxShop avec :

— Métriques application (requêtes, latence, erreurs)

— Métriques infrastructure (CPU, mémoire, disque)

— Métriques business (commandes/heure, revenus)

— Alertes configurées sur seuils critiques

Solution - Dashboard JSON :
Le dashboard complet inclurait des panels pour :
Row 1 : Aperçu Global

— Stat Panel : Request Rate (req/s)

— Stat Panel : Error Rate (%)

— Stat Panel : P95 Latency (ms)

— Stat Panel : Active Users

Row 2 : Métriques HTTP

— Graph : Requests par endpoint

— Graph : Latency (P50, P95, P99)

— Heatmap : Distribution de latence

Query PromQL pour requests rate :

```
sum(rate(http_server_requests_seconds_count{
  application="bijouxshop"
}[5m])) by (uri)
```

Query pour P95 latency :

```
histogram_quantile(0.95,
  sum(rate(http_server_requests_seconds_bucket{
    application="bijouxshop"
  }[5m])) by (le, uri)
)
```

0.8.2 Exercice : Configurer Alertmanager

Objectif : Router les alertes vers différents canaux selon la sévérité.
Solution - Configuration Alertmanager :

```
# alertmanager.yml
global:
  resolve_timeout: 5m
  slack_api_url: 'https://hooks.slack.com/services/XXX/YYY/ZZZ'

route:
  group_by: ['alertname', 'cluster', 'service']
  group_wait: 10s
  group_interval: 10s
  repeat_interval: 12h
  receiver: 'default'
  routes:
    # Alertes critiques => PagerDuty + Slack
    - match:
        severity: critical
      receiver: 'pagerduty-critical'
      continue: true
    - match:
        severity: critical
      receiver: 'slack-critical'
```

```
21
22      # Alertes warning => Slack seulement
23      - match:
24          severity: warning
25        receiver: 'slack-warning'
26
27  receivers:
28    - name: 'default'
29      slack_configs:
30        - channel: '#devops-alerts'
31          title: 'Alert: {{ .GroupLabels.alertname }}'
32          text: '{{ range .Alerts }}{{ .Annotations.description }}{{ end
              }}'
33
34    - name: 'pagerduty-critical'
35      pagerduty_configs:
36        - service_key: 'YOUR_PAGERDUTY_KEY'
37          description: '{{ .GroupLabels.alertname }}'
38
39    - name: 'slack-critical'
40      slack_configs:
41        - channel: '#incidents'
42          title: 'CRITICAL: {{ .GroupLabels.alertname }}'
43          text: '{{ range .Alerts }}{{ .Annotations.description }}{{ end
              }}'
44          color: 'danger'
45
46    - name: 'slack-warning'
47      slack_configs:
48        - channel: '#devops-alerts'
49          title: 'WARNING: {{ .GroupLabels.alertname }}'
50          text: '{{ range .Alerts }}{{ .Annotations.description }}{{ end
              }}'
51          color: 'warning'
```

0.9 Exercices Kubernetes Avancés

0.9.1 Exercice : Implémenter un Canary Deployment

Objectif : Déployer une nouvelle version de BijouxShop progressivement en dirigeant 10% du trafic vers la nouvelle version.

Solution avec Istio :

1. Virtual Service pour split de trafic

```
1  apiVersion: networking.istio.io/v1beta1
2  kind: VirtualService
3  metadata:
4    name: bijouxshop-vs
5  spec:
6    hosts:
7      - bijouxshop-service
8    http:
```

```
    - match:
        - headers:
            canary:
              exact: "true"
      route:
        - destination:
            host: bijouxshop-service
            subset: v2
    - route:
        - destination:
            host: bijouxshop-service
            subset: v1
          weight: 90
        - destination:
            host: bijouxshop-service
            subset: v2
          weight: 10
```

2. Destination Rule

```
apiVersion: networking.istio.io/v1beta1
kind: DestinationRule
metadata:
  name: bijouxshop-dr
spec:
  host: bijouxshop-service
  subsets:
    - name: v1
      labels:
        version: v1.0.0
    - name: v2
      labels:
        version: v2.0.0
```

3. Script de progression automatique

```
#!/bin/bash
# Script pour augmenter progressivement le trafic canary

WEIGHTS=(10 25 50 75 100)
WAIT_TIME=300  # 5 minutes entre chaque etape

for WEIGHT in "${WEIGHTS[@]}"; do
  echo "Routing $WEIGHT% traffic to v2..."

  V1_WEIGHT=$((100 - WEIGHT))

  kubectl patch virtualservice bijouxshop-vs --type=json \
    -p="[{
      'op': 'replace',
      'path': '/spec/http/1/route/0/weight',
      'value': $V1_WEIGHT
    }, {
      'op': 'replace',
      'path': '/spec/http/1/route/1/weight',
```

```
      'value': $WEIGHT
    }]"

  # Attendre et verifier les metriques
  sleep $WAIT_TIME

  # Verifier le taux d'erreur
  ERROR_RATE=$(prometheus_query_error_rate)

  if (( $(echo "$ERROR_RATE > 0.05" | bc -l) )); then
    echo "Error rate too high! Rolling back..."
    kubectl patch virtualservice bijouxshop-vs --type=json \
      -p="[{
        'op': 'replace',
        'path': '/spec/http/1/route/0/weight',
        'value': 100
      }, {
        'op': 'replace',
        'path': '/spec/http/1/route/1/weight',
        'value': 0
      }]"
    exit 1
  fi
done

echo "Canary deployment successful!"
```

0.10 Conclusion

Ce cours DevOps représente une synthèse complète des pratiques modernes de développement et d'opérations. Les concepts, outils et méthodologies présentés sont le fruit d'une recherche approfondie dans le cadre de l'état de l'art de ma thèse doctorale, combinant les meilleures pratiques de l'industrie avec des études de cas réels d'entreprises leaders comme Netflix, Amazon, Google, Spotify et Etsy.

Points clés à retenir :

— **Culture d'abord :** DevOps est avant tout une transformation culturelle

— **Automatisation totale :** Du build au déploiement en passant par les tests

— **Mesure continue :** Les 4 métriques DORA comme boussole

— **Sécurité intégrée :** DevSecOps dès le début du cycle

— **Observabilité complète :** Logs, métriques et traces

— **Amélioration continue :** Post-mortems et feedback loops

L'objectif de ce cours est de vous fournir les connaissances et compétences nécessaires pour implémenter DevOps dans vos organisations, en s'appuyant sur des outils matures et des patterns éprouvés.

Dr. BADR EL KHALYLY
Cours Master/Ingénierie - 2026

Partie XV : Guide de Référence Rapide

0.1 Checklist de Démarrage Projet DevOps

0.1.1 Phase 1 : Préparation (Semaine 1-2)

Infrastructure et Outils

- Choisir le système de contrôle de version (Git recommandé)
- Configurer le repository Git (GitHub, GitLab, Bitbucket)
- Définir la stratégie de branche (Git Flow, Trunk-based, GitHub Flow)
- Installer les hooks Git pre-commit pour linting et tests
- Configurer le serveur CI/CD (GitLab CI, Jenkins, GitHub Actions)
- Provisionner les environnements (dev, staging, production)
- Configurer le registry Docker privé
- Installer Kubernetes ou container orchestrator
- Configurer le monitoring (Prometheus + Grafana)
- Configurer l'aggregation de logs (ELK Stack ou Loki)

Sécurité

- Configurer le secrets management (Vault, Sealed Secrets)
- Activer 2FA sur tous les comptes critiques
- Configurer RBAC sur Kubernetes
- Activer le scanning de vulnerabilités (Trivy, Snyk)
- Configurer les network policies Kubernetes
- Activer l'encryption at rest pour les secrets

0.1.2 Phase 2 : Implémentation (Semaine 3-6)

Pipeline CI/CD

- Créer le Dockerfile optimisé (multi-stage build)
- Configurer le build automatique sur commit
- Ajouter les tests unitaires au pipeline
- Ajouter les tests d'intégration
- Configurer le quality gate (SonarQube)
- Ajouter le security scanning (SAST, DAST)
- Configurer le deployment automatique vers dev
- Configurer le deployment manuel vers staging

Configurer le deployment manuel vers production

Implémenter la stratégie de rollback

Monitoring et Alerting

Exposer les métriques applicatives (Prometheus)

Créer les dashboards Grafana de base

Configurer les alertes critiques (Alertmanager)

Configurer les health checks Kubernetes

Implémenter le distributed tracing (Jaeger/Zipkin)

Configurer l'aggregation de logs

Créer les runbooks pour incidents communs

0.1.3 Phase 3 : Optimisation (Semaine 7-8)

Performance et Scalabilité

Configurer HorizontalPodAutoscaler

Optimiser les resource requests/limits

Implémenter le caching (Redis)

Optimiser les images Docker (taille)

Configurer le CDN pour assets statiques

Effectuer les tests de charge (K6, JMeter)

Optimiser les queries base de données

Haute Disponibilité

Configurer les replicas (minimum 3)

Implémenter le blue-green ou canary deployment

Configurer le backup automatique des données

Tester le disaster recovery

Configurer le circuit breaker

Implémenter les retry policies

0.2 Matrice de Décision : Choix d'Outils

0.2.1 Systèmes CI/CD

Outil	Points Forts	Points Faibles	Recommandé pour
GitLab CI/CD	Intégré, facile à démarrer, bon pour K8s	Moins de plugins que Jenkins	Projets nouveaux, équipes moyennes
Jenkins	Très extensible, énorme écosystème de plugins	Configuration complexe, maintenance	Projets legacy, besoins spécifiques

Outil	Points Forts	Points Faibles	Recommandé pour
GitHub Actions	Intégration GitHub, marketplace riche	Limité hors GitHub	Projets open source, startups
CircleCI	Performance excellente, config simple	Coût élevé à grande échelle	Projets moyens, focus performance
Azure DevOps	Intégration Microsoft, complet	Lock-in Azure	Environnements Microsoft

0.2.2 Container Orchestration

Outil	Points Forts	Points Faibles	Recommandé pour
Kubernetes	Standard industrie, ecosystème riche	Courbe apprentissage élevée	Production, toutes tailles
Docker Swarm	Simple, natif Docker	Moins de fonctionnalités	Petits projets, POCs
Amazon ECS	Intégration AWS native, simple	Lock-in AWS	Applications AWS-centric
Nomad	Léger, simple, polyvalent	Écosystème plus petit	Multi-workload (containers + VMs)

0.2.3 Monitoring et Observabilité

Outil	Points Forts	Points Faibles	Recommandé pour
Prometheus + Grafana	Open source, standard K8s, puissant	Configuration initiale	Tous projets cloud-native
Datadog	Tout-en-un, UI excellente	Coût élevé	Entreprises, budget confortable
New Relic	APM complet, facile	Coût par host	Applications critiques
ELK Stack	Logs + search, gratuit	Ressources intensives	Logs centralisés, recherche
Jaeger	Tracing distribué, open source	Setup complexe	Microservices, debugging

0.3 Patterns DevOps Essentiels

0.3.1 Deployment Patterns

1. Blue-Green Deployment
Quand utiliser :

— Besoin de rollback instantané

— Zero-downtime requirement strict

— Budget infrastructure permettant double environnement

Avantages :

— Rollback instantané (switch du load balancer)

— Tests en conditions réelles avant bascule

— Zero downtime garanti

Inconvénients :

— Double coût infrastructure temporaire

— Complexité des migrations de données

— Gestion des sessions utilisateur

2. Canary Deployment
Quand utiliser :

— Changements à risque élevé

— Besoin de validation progressive

— Monitoring robuste en place

Avantages :

— Risque limité (impact sur subset d'utilisateurs)

— Détection précoce des problèmes

— Rollout progressif contrôlé

Inconvénients :

— Durée de déploiement plus longue

— Nécessite monitoring sophistiqué

— Complexité de routing du trafic

3. Rolling Update
Quand utiliser :

— Déploiements fréquents

— Ressources limitées

— Changements à faible risque

Avantages :

— Pas de coût infrastructure supplémentaire

— Supporté nativement par Kubernetes

— Simple à implémenter

Inconvénients :

— Rollback plus lent

— Versions mixtes temporairement

— Peut causer des problèmes de compatibilité

0.3.2 Resilience Patterns

1. Circuit Breaker
Principe : Arrêter d'appeler un service défaillant pour éviter la propagation de la panne.
États :

— **Closed :** Fonctionnement normal, requêtes passent

— **Open :** Service considéré down, requêtes échouent immédiatement

— **Half-Open :** Test périodique si le service est revenu

Configuration typique :

— Seuil d'erreurs : 50% sur 10 requêtes

— Timeout : 5 secondes

— Durée Open : 30 secondes avant Half-Open

2. Retry with Exponential Backoff
Principe : Réessayer les requêtes échouées avec délai croissant.
Configuration typique :

— Nombre de retries : 3

— Délai initial : 100ms

— Multiplicateur : 2x

— Jitter : ±25% pour éviter thundering herd

3. Bulkhead
Principe : Isoler les ressources pour qu'une panne ne consomme pas toutes les ressources.
Exemple :

— Pool de threads dédié par service downstream

— Connection pool séparé par base de données

— Rate limiting par tenant/user

0.4 Métriques et KPIs DevOps

0.4.1 Métriques DORA (Rappel)

Métrique	Elite	High	Medium
Deployment Frequency	Plusieurs/jour	1/jour - 1/semaine	1/semaine - 1/mois
Lead Time for Changes	$<$ 1 heure	1 jour - 1 semaine	1 semaine - 1 mois
Change Failure Rate	0-15%	16-30%	31-45%
MTTR	$<$ 1 heure	$<$ 1 jour	1 jour - 1 semaine

0.4.2 Métriques Additionnelles

Métriques de Pipeline :

— **Build Success Rate :** % de builds réussis (cible : $>$ 95%)

— **Pipeline Duration :** Temps moyen du pipeline (cible : $<$ 10 min)

— **Test Coverage :** % de code couvert (cible : $> 80\%$)
— **Flaky Tests Rate :** % de tests instables (cible : $< 1\%$)

Métriques d'Infrastructure :

— **Uptime :** Disponibilité du service (cible : $> 99.9\%$)
— **Resource Utilization :** CPU/Mémoire moyenne (cible : 60-70%)
— **Pod Restart Rate :** Redémarrages/jour (cible : < 1)
— **Node Health :** Noeuds sains (cible : 100%)

Métriques de Sécurité :

— **Vulnerabilities :** CVEs critiques non patchés (cible : 0)
— **Security Scan Failures :** Échecs de scan (cible : 0)
— **Secret Leaks :** Détections de secrets (cible : 0)
— **Compliance Score :** Score de conformité (cible : 100%)

Partie XVI : Tendances et Futur du DevOps

0.5 Tendances Émergentes 2025-2026

0.5.1 1. Platform Engineering

Concept : Construire des Internal Developer Platforms (IDP) pour abstraire la complexité infrastructure.

Caractéristiques :

— Self-service pour les développeurs
— Golden paths (chemins recommandés)
— Templates et scaffolding automatisé
— Portal unique pour toutes les opérations

Outils :

— **Backstage (Spotify) :** Developer portal open source
— **Humanitec :** Platform orchestration
— **Port :** Internal developer portal
— **Kratix :** Platform-as-a-product framework

Bénéfices :

— Réduction du temps de onboarding
— Standardisation des pratiques
— Autonomie des développeurs
— Réduction du toil pour les ops

0.5.2 2. GitOps Généralisé

Évolution : GitOps s'étend au-delà de Kubernetes vers toute l'infrastructure.

Pratiques :

— Infrastructure as Code dans Git comme source de vérité unique
— Continuous reconciliation (état désiré vs réel)
— Auditing automatique via Git history
— Pull-based deployments (agents dans clusters)

Outils majeurs :

— **Flux CD :** GitOps operator pour Kubernetes
— **Argo CD :** Continuous delivery déclaratif
— **Crossplane :** Infrastructure GitOps multi-cloud
— **Terraform Cloud :** GitOps pour Terraform

0.5.3 3. FinOps - Optimisation des Coûts Cloud

Principe : Apporter la responsabilité financière aux équipes techniques.
Pratiques clés :
— Tagging systématique des ressources
— Showback/Chargeback par équipe
— Right-sizing automatisé
— Utilisation de spot instances/preemptible VMs
— Automatic shutdown environnements non-prod
Outils :
— **Kubecost :** Coûts Kubernetes détaillés
— **CloudHealth :** Multi-cloud cost management
— **Infracost :** Estimation coûts IaC
— **OpenCost :** Standard open source monitoring coûts

0.5.4 4. AI/ML for Operations (AIOps)

Application : Intelligence artificielle pour automatiser les opérations.
Use cases :
— **Anomaly Detection :** Détection automatique de comportements anormaux
— **Root Cause Analysis :** Identification des causes d'incidents
— **Predictive Scaling :** Scaling prédictif basé sur patterns
— **Auto-remediation :** Correction automatique de problèmes connus
— **Chatops amélioré :** Assistants IA pour opérations
Technologies :
— ML models pour time-series forecasting
— NLP pour analyse de logs
— Reinforcement learning pour optimisation

0.5.5 5. Edge Computing et DevOps

Défi : Gérer des milliers de locations edge avec DevOps practices.
Solutions émergentes :
— **K3s :** Kubernetes léger pour edge
— **KubeEdge :** Extension Kubernetes pour edge
— **Azure Arc / AWS Outposts :** Contrôle centralisé edge
— **GitOps at Edge :** Deployments vers edge locations

0.5.6 6. Security Shift Further Left

Évolution : Sécurité encore plus tôt dans le cycle.
Nouvelles pratiques :
— **Threat Modeling as Code :** Modèles de menaces versionnés
— **Policy as Code :** OPA, Kyverno pour policies K8s
— **SBOM (Software Bill of Materials) :** Transparence supply chain
— **Sigstore :** Signature et vérification artifacts
— **Zero Trust Architecture :** Vérification systématique

0.6 Compétences DevOps du Futur

0.6.1 Compétences Techniques

Must-have 2026+ :

1. **Kubernetes avancé :** CRDs, Operators, Service Mesh
2. **Observabilité :** OpenTelemetry, distributed tracing
3. **Security :** DevSecOps, compliance as code
4. **IaC multi-cloud :** Terraform, Crossplane, Pulumi
5. **GitOps :** Flux, Argo CD
6. **FinOps :** Optimisation coûts cloud
7. **Platform Engineering :** Backstage, golden paths

Nice-to-have :

1. **Service Mesh :** Istio, Linkerd
2. **Chaos Engineering :** Chaos Mesh, Litmus
3. **WebAssembly :** WASM pour edge/serverless
4. **eBPF :** Observabilité système avancée

0.6.2 Compétences Soft Skills

Essentielles :

— **Communication :** Expliquer concepts techniques clairement
— **Collaboration :** Travailler avec Dev, Sec, Product
— **Empathie :** Comprendre les besoins des développeurs
— **Pédagogie :** Former et mentorer
— **Résolution de problèmes :** Approche systématique
— **Continuous Learning :** Veille technologique constante

0.7 Certification et Parcours d'Apprentissage

0.7.1 Parcours DevOps Junior (0-2 ans)

Étape 1 : Fondamentaux (3-6 mois)

— Linux basics et scripting (Bash, Python)
— Git et GitHub/GitLab
— Docker fondamentaux
— CI/CD concepts

Certifications :

— Linux Foundation Certified System Administrator (LFCS)
— Docker Certified Associate (DCA)

Étape 2 : CI/CD et Cloud (6-12 mois)

— Jenkins ou GitLab CI/CD
— AWS/Azure/GCP fondamentaux

— Terraform basics
— Monitoring avec Prometheus

Certifications :

— AWS Certified Solutions Architect Associate
— HashiCorp Certified : Terraform Associate

0.7.2 Parcours DevOps Senior (2-5 ans)

Compétences avancées :

— Kubernetes administration (CKA)
— Architecture microservices
— Security (DevSecOps)
— Observabilité complète (traces, metrics, logs)

Certifications :

— Certified Kubernetes Administrator (CKA)
— Certified Kubernetes Security Specialist (CKS)
— AWS Certified DevOps Engineer Professional

0.7.3 Parcours Platform Engineer (5+ ans)

Compétences expertise :

— Platform design et architecture
— Multi-tenancy et isolation
— Developer experience (DX)
— FinOps et cost optimization

Certifications :

— Certified Kubernetes Application Developer (CKAD)
— AWS/Azure/GCP Professional/Expert certifications
— CNCF certifications spécialisées

0.8 Conclusion du Cours

0.8.1 Récapitulatif des Concepts Clés

Ce cours de DevOps, fruit de l'état de l'art de ma thèse doctorale, a couvert l'ensemble du spectre DevOps :

Fondamentaux :

— Culture DevOps et transformation organisationnelle
— Les Three Ways : Flow, Feedback, Continuous Learning
— Métriques DORA comme boussole de performance

Pratiques Techniques :

— Continuous Integration et Continuous Delivery
— Infrastructure as Code avec Terraform et Ansible

— Containerisation avec Docker
— Orchestration avec Kubernetes
— Monitoring et Observabilité (Prometheus, Grafana, Jaeger)

Sécurité :

— DevSecOps et Shift Left Security
— Scanning de vulnérabilités
— Secrets management
— Compliance as Code

Cas d'Usage Réels :

— Netflix : Chaos Engineering et résilience
— Amazon : Vitesse de déploiement à l'échelle
— Google : Site Reliability Engineering
— Spotify : Squads et autonomie

0.8.2 Prochaines Étapes

Pour approfondir vos connaissances :

1. **Pratiquer :** Mettez en œuvre les concepts sur BijouxShop ou vos projets personnels
2. **Contribuer :** Participez à des projets open source DevOps
3. **Certifier :** Obtenez des certifications reconnues (CKA, AWS DevOps)
4. **Communauté :** Rejoignez les meetups DevOps locaux et conférences
5. **Veille :** Suivez les blogs techniques des leaders (Netflix, Google, etc.)
6. **Expérimenter :** Testez de nouveaux outils et patterns
7. **Partager :** Écrivez des articles, donnez des talks sur vos expériences

0.8.3 Ressources pour Aller Plus Loin

Livres recommandés :

— *The DevOps Handbook* - Gene Kim et al.
— *Site Reliability Engineering* - Google
— *Accelerate* - Nicole Forsgren et al.
— *Building Microservices* - Sam Newman

Conférences majeures :

— KubeCon + CloudNativeCon (CNCF)
— DevOps Enterprise Summit
— DockerCon
— HashiConf
— AWS re :Invent

Communautés en ligne :

— CNCF Slack
— DevOps subreddit
— Kubernetes Slack

— HashiCorp Community Forum

Ce cours représente une base solide pour votre parcours DevOps. Le domaine évolue rapidement - continuez à apprendre, expérimenter et partager vos connaissances. Le DevOps est autant un état d'esprit qu'un ensemble de pratiques techniques.

Bonne continuation dans votre parcours DevOps !

Dr. BADR EL KHALYLY
Cours Master/Ingénierie
Février 2026

Remerciements

Ce cours sur DevOps est le résultat d'un travail de recherche approfondi mené dans le cadre de l'état de l'art de ma thèse doctorale. Il représente une synthèse des meilleures pratiques de l'industrie, enrichie par l'étude détaillée des approches adoptées par les entreprises leaders du domaine.

Contributions Académiques

Je tiens à exprimer ma profonde gratitude envers la communauté académique qui a contribué à l'avancement des connaissances dans le domaine DevOps. Les travaux de recherche de l'équipe DORA (DevOps Research and Assessment), notamment ceux de Nicole Forsgren, Jez Humble et Gene Kim, ont fourni une base scientifique solide pour comprendre l'impact des pratiques DevOps sur les performances organisationnelles.

Les publications académiques et les conférences internationales comme ICSE, SEAA, et DevOps Enterprise Summit ont été des sources d'inspiration essentielles pour structurer ce cours et identifier les tendances émergentes du domaine.

Contributions Industrielles

Une reconnaissance particulière est due aux ingénieurs et architectes des grandes entreprises technologiques qui partagent généreusement leurs expériences et apprentissages :

Netflix pour ses innovations en matière de Chaos Engineering et d'architecture microservices, avec des outils comme Spinnaker, Hystrix, et Chaos Monkey qui ont révolutionné les pratiques de résilience.

Google pour la formalisation du Site Reliability Engineering (SRE) et le partage de leurs pratiques à travers les livres SRE et la création de Kubernetes, qui est devenu le standard de l'orchestration de conteneurs.

Amazon Web Services pour la démocratisation du cloud computing et l'établissement des pratiques DevOps à l'échelle, inspirant des millions d'organisations dans leur transformation digitale.

Spotify pour le modèle organisationnel des Squads, Tribes, Chapters et Guilds, qui montre comment structurer les équipes pour maximiser l'autonomie et la vélocité.

Microsoft, HashiCorp, Docker, GitLab, et la Cloud Native Computing Foundation (CNCF) pour leurs contributions majeures à l'écosystème d'outils DevOps et leur engagement envers l'open source.

Communauté Open Source

Le mouvement open source mérite une mention spéciale. Sans les contributions de milliers de développeurs bénévoles aux projets Kubernetes, Prometheus, Grafana, Terraform, Ansible, Jenkins, et tant d'autres, l'écosystème DevOps ne serait pas ce qu'il est aujourd'hui.

Étudiants et Praticiens

Merci aux étudiants des promotions précédentes dont les questions, retours et projets ont contribué à affiner le contenu de ce cours. Leurs expériences pratiques et leurs défis rencontrés sur le terrain ont permis d'enrichir les exemples et cas d'usage présentés.

À propos du Projet BijouxShop

Le projet BijouxShop utilisé tout au long de ce cours comme fil conducteur a été conçu pour illustrer de manière concrète l'application des principes DevOps sur une application e-commerce réaliste. Tous les exemples de code, configurations et architectures présentés sont inspirés de patterns de production éprouvés et peuvent être directement appliqués dans des environnements réels.

Évolution Continue

DevOps étant un domaine en constante évolution, ce cours sera régulièrement mis à jour pour refléter les nouvelles pratiques, outils et tendances. Les retours d'expérience des praticiens et les avancées de la recherche académique continueront d'enrichir ce matériel pédagogique.

Ce cours est dédié à tous ceux qui œuvrent chaque jour pour améliorer la qualité, la fiabilité et la rapidité de livraison des logiciels, contribuant ainsi à la transformation numérique de nos sociétés.

Dr. BADR EL KHALYLY
Cours Master/Ingénierie
Février 2026